arXiv:2411.18279v3 [cs.AI] 3 Dec 2024

Large Language Model-Brained GUI Agents: A Survey

Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, Qi Zhang

Abstract—Graphical User Interfaces (GUIs) have long been central to human-computer interaction, providing an intuitive and visually-driven way to access and interact with digital systems. Traditionally, automating GUI interactions relied on script-based or rule-based approaches, which, while effective for fixed workflows, lacked the flexibility and adaptability required for dynamic, real-world applications. The advent of Large Language Models (LLMs), particularly multimodal models, has ushered in a new era of GUI automation. They have demonstrated exceptional capabilities in natural language understanding, code generation, task generalization, and visual processing. This has paved the way for a new generation of "LLM-brained" GUI agents capable of interpreting complex GUI elements and autonomously executing actions based on natural language instructions. These agents represent a paradigm shift, enabling users to perform intricate, multi-step tasks through simple conversational commands. Their applications span across web navigation, mobile app interactions, and desktop automation, offering a transformative user experience that revolutionizes how individuals interact with software. This emerging field is rapidly advancing, with significant progress in both research and industry.

To provide a structured understanding of this trend, this paper presents a comprehensive survey of LLM-brained GUI agents, exploring their historical evolution, core components, and advanced techniques. We address critical research questions such as existing GUI agent frameworks, the collection and utilization of data for training specialized GUI agents, the development of large action models tailored for GUI tasks, and the evaluation metrics and benchmarks necessary to assess their effectiveness. Additionally, we examine emerging applications powered by these agents. Through a detailed analysis, this survey identifies key research gaps and outlines a roadmap for future advancements in the field. By consolidating foundational knowledge and state-of-the-art developments, this work aims to guide both researchers and practitioners in overcoming challenges and unlocking the full potential of LLM-brained GUI agents. We anticipate that this survey will serve both as a practical cookbook for constructing LLM-powered GUI agents, and as a definitive reference for advancing research in this rapidly evolving domain.

The collection of papers reviewed in this survey will be hosted and regularly updated on the GitHub repository: https://github.com/vyokky/LLM-Brained-GUI-Agents-Survey. Additionally, a searchable webpage is available at https://aka.ms/gui-agent for easier access and exploration.

Index Terms—Large Language Model, Graphical User Interface, AI Agent, Automation, Human-Computer Interaction

1 Introduction

Graphical User Interfaces (GUIs) have been a cornerstone of human-computer interaction, fundamentally transforming how users navigate and operate within digital systems [1]. Designed to make computing more intuitive and accessible, GUIs replaced command-line interfaces (CLIs) [2] with visually driven, user-friendly environments. Through the use of icons, buttons, windows, and menus, GUIs empowered a broader range of users to interact with computers using simple actions such as clicks, typing, and gestures. This shift democratized access to computing, allowing even non-technical users to effectively engage with complex systems. However, GUIs often sacrifice efficiency for usability, particularly in workflows requiring repetitive or multi-step interactions, where CLIs can remain more streamlined [3].

While GUIs revolutionized usability, their design, primarily tailored for human visual interaction, poses significant challenges for automation. The diversity, dynamism, and platform-specific nature of GUI layouts make it difficult to develop flexible and intelligent automation tools capable of adapting to various environments. Early efforts to automate GUI interactions predominantly relied on script-based or rule-based methods [4], [5]. Although effective for predefined workflows, these methods were inherently narrow in scope, focusing primarily on tasks such as software testing and robotic process automation (RPA) [6]. Their rigidity required frequent manual updates to accommodate new tasks, changes in GUI layouts, or evolving workflows, limiting their scalability and versatility. Moreover, these approaches lacked the sophistication needed to support dynamic, human-like interactions, thereby constraining their applicability in complex or unpredictable scenarios.

The rise of Large Language Models (LLMs)[1] [8], [9],

Version: v1 (major update on November 27, 2024)
Chaoyun Zhang, Shilin He, Jiaxu Qian, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang and Qi Zhang are with Microsoft. e-mail: {chaoyun.zhang, shilin.he, v-jiaxuqian, liqun.li, si.qin, yu.kang, minghuama, qlin, saravan.rajmohan, dongmeiz, zhang.qi}@microsoft.com.
Bowen Li is with Shanghai Artificial Intelligence Laboratory, China. e-mail: libowen.ne@gmail.com.
Guyue Liu is with Peking University, China. e-mail: guyue.liu@gmail.com.
For any inquiries or discussions, please contact Chaoyun Zhang and Shilin He.

1. By LLMs, we refer to the general concept of foundation models capable of accepting various input modalities (*e.g.*, visual language models (VLMs), multimodal LLMs (MLLMs)) while producing output exclusively in textual sequences [7].

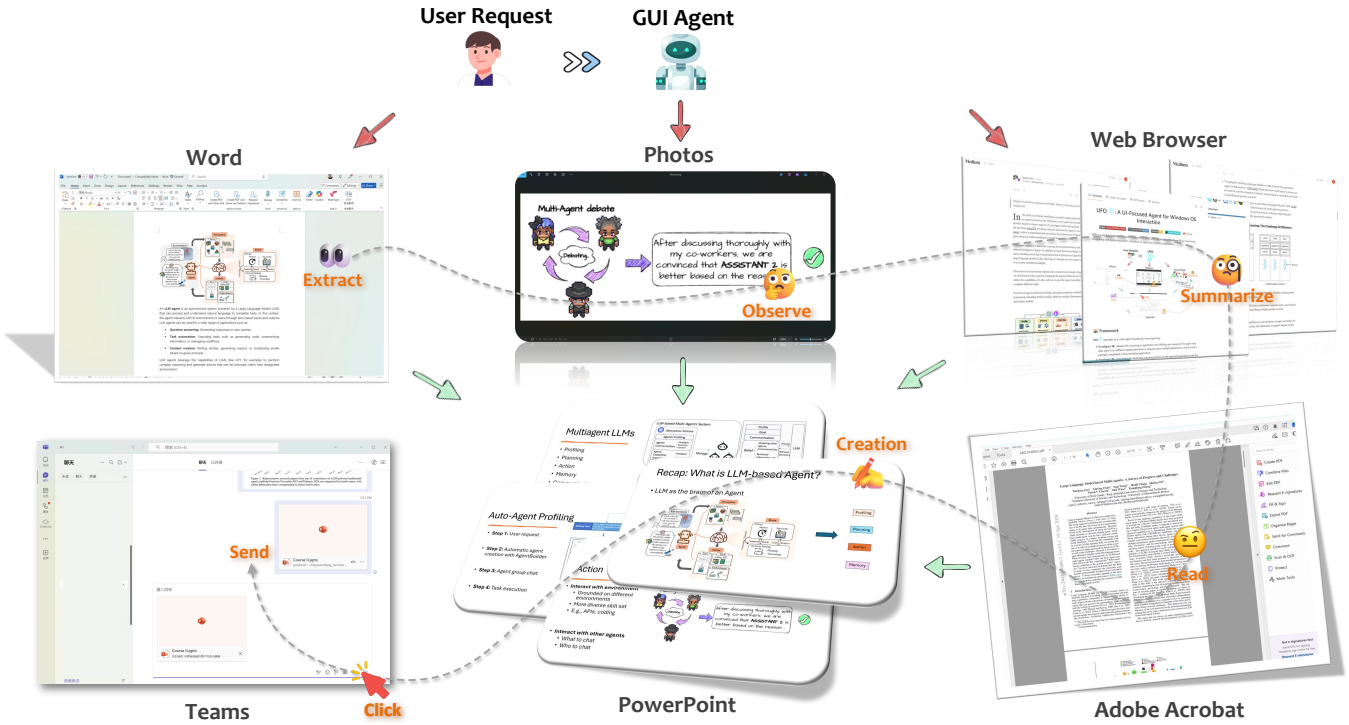


Fig. 1: Illustration of the high-level concept of an LLM-powered GUI agent. The agent receives a user's natural language request and orchestrates actions seamlessly across multiple applications. It extracts information from Word documents, observes content in Photos, summarizes web pages in the browser, reads PDFs in Adobe Acrobat, and creates slides in PowerPoint before sending them through Teams.

especially those augmented with multimodal capabilities [10], has drastically redefined the possibilities for GUI automation. Beginning with models like ChatGPT [11], LLMs have demonstrated extraordinary proficiency in natural language understanding, code generation, and generalization across diverse tasks [8], [12], [13]. The integration of visual language models (VLMs) has further extended these capabilities, enabling these models to process visual data, such as the intricate layouts of GUIs [14]. This evolution bridges the gap between linguistic and visual comprehension, empowering intelligent agents to interact with GUIs in a more human-like and adaptive manner. By leveraging these advancements, LLMs and VLMs offer transformative potential, enabling agents to navigate complex digital environments, execute tasks dynamically, and revolutionize the field of GUI automation.

1.1 Motivation for LLM-Brained GUI agents

With an LLM serving as its **"brain"**, LLM-powered GUI automation introduces a new class of intelligent agents capable of interpreting a user's natural language requests, analyzing GUI screens and their elements, and autonomously executing appropriate actions. Importantly, these capabilities are achieved without reliance on complex, platform-specific scripts or predefined workflows. These agents, referred to as **"LLM-brained GUI agents"**, can be formally defined as:

> *Intelligent agents that operate within GUI environments, leveraging LLMs as their core inference and cognitive engine to generate, plan, and execute actions in a flexible and adaptive manner.*

This paradigm represents a transformative leap in GUI automation, fostering dynamic, human-like interactions across diverse platforms. It enables the creation of intelligent, adaptive systems that can reason, make decisions in real-time, and respond flexibly to evolving tasks and environments. We illustrate this high-level concept in Figure 1.

Traditional GUI automation are often limited by predefined rules or narrowly focused on specific tasks, constraining their ability to adapt to dynamic environments and diverse applications. In contrast, LLM-powered GUI agents bring a paradigm shift by integrating natural language understanding, visual recognition, and decision-making into a unified framework. This enables them to generalize across a wide range of use cases, transforming task automation and significantly enhancing the intuitiveness and efficiency of human-computer interaction. Moreover, unlike the emerging trend of pure Application Programming Interface (API)-based agents—which depend on APIs that may not always be exposed or accessible—GUI agents leverage the universal nature of graphical interfaces. GUIs offer a general mechanism to control most software applications, enabling agents to operate in a non-intrusive manner without requiring internal API access. This capability not only broadens the applicability of GUI agents but also empowers external developers to build advanced functionality on top of existing software across diverse platforms and ecosystems. Together, these innovations position GUI agents as a versatile and transformative technology for the future of intelligent automation.

This new paradigm enables users to control general software systems with conversational commands. By reducing

the cognitive load of multi-step GUI operations, LLM-powered agents make complex systems accessible to non-technical users and streamline workflows across diverse domains. Notable examples include SeeAct [15] for web navigation, AppAgent [16] for mobile interactions, and UFO [17] for Windows OS applications. These agents resemble a "virtual assistant" [18] akin to J.A.R.V.I.S. from Iron Man—an intuitive, adaptive system capable of understanding user goals and autonomously performing actions across applications. The futuristic concept of an AI-powered operating system that executes cross-application tasks with fluidity and precision is rapidly becoming a reality [19], [20].

Real-world applications of LLM-powered GUI agents are already emerging. For example, Microsoft Power Automate utilizes LLMs to streamline low-code/no-code automation[2], allowing users to design workflows across Microsoft applications with minimal technical expertise. Integrated AI assistants in productivity software, like Microsoft Copilot[3], are bridging the gap between natural language instructions and operations on application. Additionally, LLM-powered agents show promise for enhancing accessibility [21], potentially allowing visually impaired users to navigate GUIs more effectively by converting natural language commands into executable steps. These developments underscore the timeliness and transformative potential of LLM-powered GUI agents across diverse applications.

The convergence of LLMs and GUI automation addresses longstanding challenges in human-computer interaction and introduces new opportunities for intelligent GUI control [22]. This integration has catalyzed a surge in research activity, spanning application frameworks [17], data collection [23], model optimization [14], and evaluation benchmarks [24]. Despite these advancements, key challenges and limitations persist, and many foundational questions remain unexplored. However, a systematic review of this rapidly evolving area is notably absent, leaving a critical gap in understanding.

1.2 Scope of the Survey

To address this gap, this paper provides a pioneering, comprehensive survey of LLM-brained GUI agents. We cover the historical evolution of GUI agents, provide a step-by-step guide to building these agents, summarize essential and advanced techniques, review notable tools and research related to frameworks, data and models, showcase representative applications, and outline future directions. Specifically, this survey aims to answer the following research questions (RQs):

1) **RQ1:** What is the historical development trajectory of LLM-powered GUI agents? (Section 4)
2) **RQ2:** What are the essential components and advanced technologies that form the foundation of LLM-brained GUI agents? (Section 5)
3) **RQ3:** What are the principal frameworks for LLM GUI agents, and what are their defining characteristics? (Section 6)
4) **RQ4:** What are the existing datasets, and how can comprehensive datasets be collected to train optimized LLMs for GUI agents? (Section 7)
5) **RQ5:** How can the collected data be used to train purpose-built Large Action Models (LAMs) for GUI agents, and what are the current leading models in the field? (Section 8)
6) **RQ6:** What metrics and benchmarks are used to evaluate the capability and performance of GUI agents? (Section 9)
7) **RQ7:** What are the most significant real-world applications of LLM-powered GUI agents, and how have they been adapted for practical use? (Section 10)
8) **RQ8:** What are the major challenges, limitations, and future research directions for developing robust and intelligent GUI agents? (Section 11)

Through these questions, this survey aims to provide a comprehensive overview of the current state of the field, offer a guide for building LLM-brained GUI agents, identify key research gaps, and propose directions for future work. This survey is one of the pioneers to systematically examine the domain of LLM-brained GUI agents, integrating perspectives from LLM advancements, GUI automation, and human-computer interaction.

1.3 Survey Structure

The survey is organized as follows, with a structural illustration provided in Figure 2. Section 2 reviews related survey and review literature on LLM agents and GUI automation. Section 3 provides preliminary background on LLMs, LLM agents, and GUI automation. Section 2 traces the evolution of LLM-powered GUI agents. Section 5 introduces key components and advanced technologies within LLM-powered GUI agents, serving as a comprehensive guide. Section 6 presents representative frameworks for LLM-powered GUI agents. Section 7 discusses dataset collection and related data-centric research for optimizing LLMs in GUI agent. Section 8 covers foundational and optimized models for GUI agents. Section 9 outlines evaluation metrics and benchmarks. Section 10 explores real-world applications and use cases. Finally, Section 11 examines current limitations, challenges, and potential future directions, and section 12 conclude this survey. For clarity, a list of abbreviations is provided in Table 1.

2 Related Work

The integration of LLMs with GUI agents is an emerging and rapidly growing field of research. Several related surveys and tutorials provide foundational insights and guidance. We provide a brief review of existing overview articles on GUI automation and LLM agents, as these topics closely relate to and inform our research focus. To begin, we provide an overview of representative surveys and books on GUI automation, LLM agents, and their integration, as summarized in Table 2. These works either directly tackle one or two core areas in GUI automation and LLM-driven agents, or provide valuable insights that, while not directly addressing the topic, contribute indirectly to advancing the field.

2. https://www.microsoft.com/en-us/power-platform/blog/power-automate/revolutionize-the-way-you-work-with-automation-and-ai/
3. https://copilot.microsoft.com/

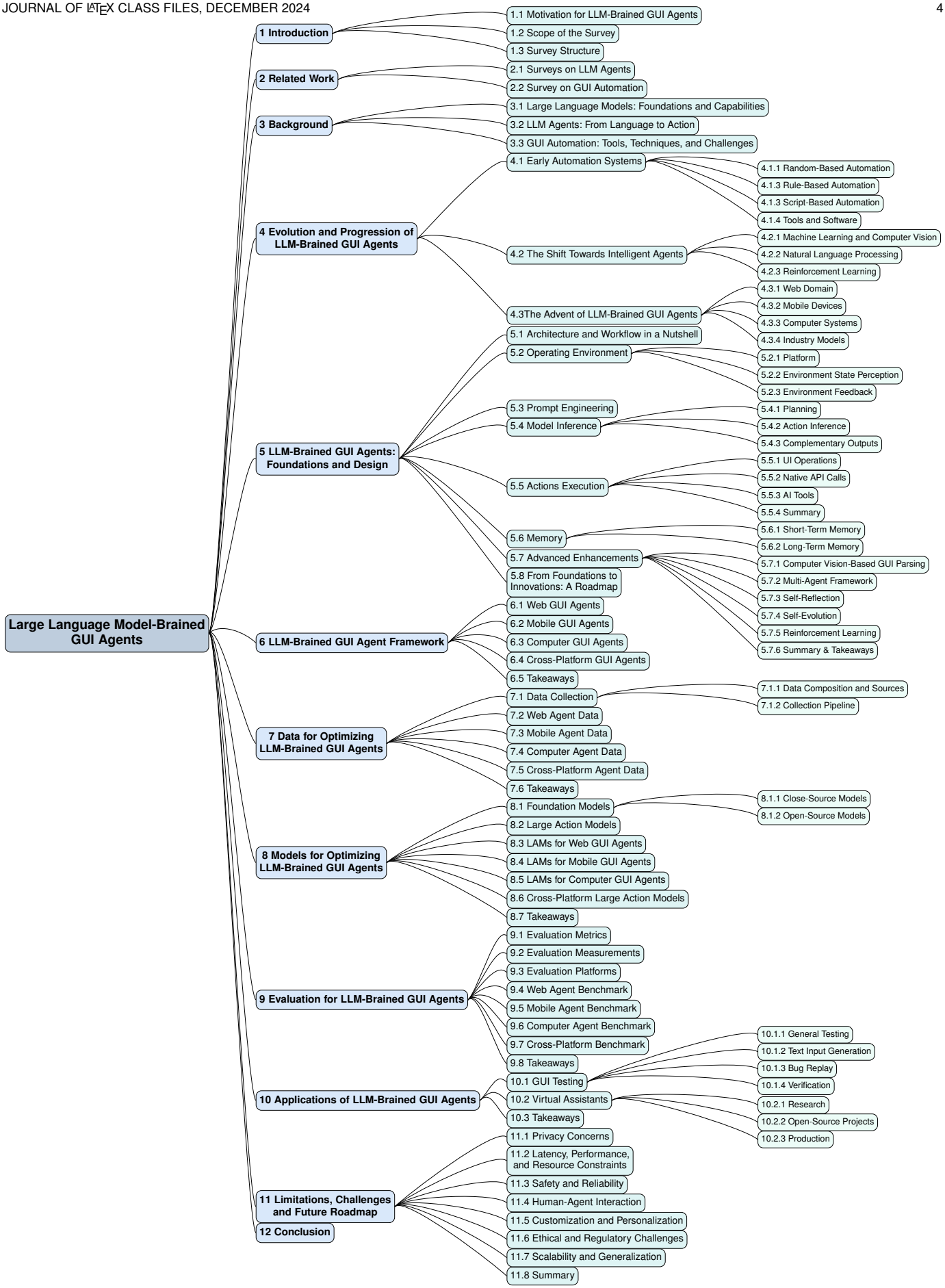


Fig. 2: The structure of the survey on LLM-brained GUI agents.

TABLE 1: List of abbreviations in alphabetical order.

Acronym	Explanation
AI	Artificial Intelligence
AITW	Android in the Wild
AITZ	Android in The Zoo
API	Application Programming Interface
CLI	Command-Line Interface
CLIP	Contrastive Language-Image Pre-Training
CoT	Chain-of-Thought
CSS	Cascading Style Sheets
CuP	Completion under Policy
CV	Computer Vision
DOM	Document Object Model
DPO	Direct Preference Optimization
GCC	General Computer Control
GPT	Generative Pre-trained Transformers
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HTML	Hypertext Markup Language
ICL	In-Context Learning
IoU	Intersection over Union
LAM	Large Action Model
LLM	Large Language Model
LSTM	Long Short-Term Memory
LTM	Long-Term Memory
MCTS	Monte Carlo Tree Search
MoE	Mixture of Experts
MDP	Markov Decision Process
MLLM	Multimodal Large Language Model
OCR	Optical Character Recognition
OS	Operation System
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
RNN	Recurrent Neural Network
RPA	Robotic Process Automation
UI	User Interface
VAB	VisualAgentBench
VLM	Visual Language Models
ViT	Vision Transformer
VQA	Visual Question Answering
SAM	Segment Anything Model
SoM	Set-of-Mark
STM	Short-Trem Memory

2.1 Survey on GUI Automation

GUI automation has a long history and wide applications in industry, especially in GUI testing [25]–[27] and RPA [6] for task automation [40].

Said *et al.*, [28] provide an overview of GUI testing for mobile applications, covering objectives, approaches, and challenges within this domain. Focusing on Android applications, Li [29] narrows the scope further, while Oksanen *et al.*, [30] explore automatic testing techniques for Windows GUI applications, a key platform for agent operations. Similarly, Moura *et al.*, [59] review GUI testing for web applications, which involves diverse tools, inputs, and methodologies. Deshmukh *et al.*, [31] discuss automated GUI testing for enhancing user experience, an area where LLMs also bring new capabilities. A cornerstone of modern GUI testing is computer vision (CV), which is used to interpret UI elements and identify actionable controls [32]. Yu *et al.*, [33] survey the application of CV in mobile GUI testing, highlighting both its significance and associated challenges. In LLM-powered GUI agents, application UI screenshots are equally essential, serving as key inputs for reliable task comprehension and execution.

On the other hand, RPA, which focuses on automating repetitive human tasks, also relies heavily on GUI automation for relevant processes. Syed *et al.*, [34] review this field and highlight contemporary RPA themes, identifying key challenges for future research. Chakraborti *et al.*, [35] emphasize the importance of shifting from traditional, script-based RPA toward more intelligent, adaptive paradigms, offering a systematic overview of advancements in this direction. Given RPA's extensive industrial applications, Enriquez *et al.*, [36] and Ribeiro *et al.*, [37] survey the field from an industrial perspective, underscoring its significance and providing a comprehensive overview of RPA methods, development trends, and practical challenges.

Both GUI testing [38] and RPA [39] continue to face significant challenges in achieving greater intelligence and robustness. LLM-powered GUI agents are poised to play a transformative role in these fields, providing enhanced capabilities and adding substantial value to address these persistent issues.

2.2 Surveys on LLM Agents

The advent of LLMs has significantly enhanced the capabilities of intelligent agents [41], enabling them to tackle complex tasks previously out of reach, particularly those involving natural language understanding and code generation [42]. This advancement has spurred substantial research into LLM-based agents designed for a wide array of applications [43].

Both Xie *et al.*, [44] and Wang *et al.*, [45] offer comprehensive surveys on LLM-powered agents, covering essential background information, detailed component breakdowns, taxonomies, and various applications. These surveys serve as valuable references for a foundational understanding of LLM-driven agents, laying the groundwork for further exploration into LLM-based GUI agents. Xie *et al.*, [54] provide an extensive overview of multimodal agents, which can process images, videos, and audio in addition to text. This multimodal capability significantly broadens the scope beyond traditional text-based agents [55]. Notably, most GUI agents fall under this category, as they rely on image inputs, such as screenshots, to interpret and interact with graphical interfaces effectively. Multi-agent frameworks are frequently employed in the design of GUI agents to enhance their capabilities and scalability. Surveys by Guo *et al.*, [46] and Han *et al.*, [47] provide comprehensive overviews of the current landscape, challenges, and future directions in this area. Sun *et al.*, [48] provide an overview of recent methods that leverage reinforcement learning to strengthen multi-agent LLM systems, opening new pathways for enhancing their capabilities and adaptability. These surveys offer valuable insights and guidance for designing effective multi-agent systems within GUI agent frameworks.

In the realm of digital environments, Wu *et al.*, [56] presents a survey on LLM agents operating in mobile environments, covering key aspects of mobile GUI agents. In a boarder scope, Wang *et al.*, [57] present a survey on the integration of foundation models with GUI agents. Another survey by Gao *et al.*, provides an overview of autonomous agents operating across various digital platforms [58], highlighting their capabilities, challenges, and applications. All these surveys highlighting emerging trends in this area.

TABLE 2: Summary of representative surveys and books on GUI automation and LLM agents. A ✓symbol indicates that a publication explicitly addresses a given domain, while an ○ symbol signifies that the publication does not focus on the area but offers relevant insights. Publications covering both GUI automation and LLM agents are highlighted for emphasis.

Survey	One Sentence Summary	Scope		
		GUI Automation	LLM Agent	LLM Agent + GUI Automation
Li *et al.*, [25]	A book on how to develop an automated GUI testing tool.	✓		
Rodríguez *et al.*, [26]	A survey on automated GUI testing in 30 years.	✓		
Arnatovich *et al.*, [27]	A survey on automated techniques for mobile functional GUI testing.	✓		
Ivančić *et al.*, [6]	A literature review on RPA.	✓		
Said *et al.*, [28]	An overview on mobile GUI testing.	✓		
Li [29]	An survey on Android GUI testing.	✓		
Oksanen *et al.*, [30]	GUI testing on Windows OS.	✓		
Deshmukh *et al.*, [31]	A survey on GUI testing for improving user experience.	✓		
Bajammal *et al.*, [32]	A survey on the use of computer vision for software engineering.	✓		
Yu *et al.*, [33]	A survey on using computer for mobile app GUI testing.	✓		
Syed *et al.*, [34]	A review of contemporary themes and challenges in RPA.	✓		
Chakraborti *et al.*, [35]	A review of emerging trends of intelligent process automation.	✓		
Enriquez *et al.*, [36]	A scientific and industrial systematic mapping study of RPA.	✓		
Ribeiro *et al.*, [37]	A review of combining AI and RPA in industry 4.0.	✓		
Nass *et al.*, [38]	Discuss the chanllenges of GUI testing.	✓		
Agostinelli *et al.*, [39]	Discuss the research challenges of intelligent RPA.	✓		
Wali *et al.*, [40]	A review on task automation with intelligent agents.	✓		
Zhao *et al.*,. [8]	A comprehensive survey of LLMs.		✓	
Zhao *et al.*,. [41]	A survey of LLM-based agents.		✓	
Cheng *et al.*, [42]	An overview of LLM-based AI agent.		✓	
Li *et al.*, [43]	A survey on personal LLM agents on their capability, efficiency and security.		✓	
Xie *et al.*,. [44]	A comprehensive survey of LLM-based agents.		✓	
Wang *et al.*,. [45]	A survey on LLM-based autonomous agents.		✓	
Guo *et al.*, [46]	A survey of mult-agent LLM frameworks.		✓	
Han *et al.*, [47]	A survey on LLM multi-agent systems, with their challenges and open problems.		✓	
Sun *et al.*, [48]	A survey on LLM-based multi-agent reinforcement learning.		✓	
Huang *et al.*, [49]	A survey on planning in LLM agents.		✓	
Zhang *et al.*, [50]	A survey on the memory of LLM-based agents.		✓	
Shen [13]	A survey of the tool usage in LLM agents.		✓	
Chang *et al.*, [51]	A survey on evaluation of LLMs.		✓	
Li *et al.*, [52]	A survey on benchmarks multimodal applications.		✓	
Huang and Zhang [53]	A survey on evaluation of multimodal LLMs.		✓	
Xie *et al.*,. [54]	A survey on LLM based multimodal agent.		✓	○
Durante *et al.*, [55]	A survey of multimodal interaction with AI agents.		✓	○
Wu *et al.*, [56]	A survey of foundations and trend on multimodal mobile agents.		✓	✓
Wang *et al.*, [57]	A survey on the integration of foundation models with GUI agents.		✓	✓
Gao *et al.*, [58]	A Survey on autonomous agents across digital platforms.		✓	✓
Our work	**A comprehensive survey on LLM-brained GUI agents, on their foundations, technologies, frameworks, data, models, applications, challenges and future roadmap.**	○	✓	✓

Regarding individual components within LLM agents, several surveys provide detailed insights that are especially relevant for GUI agents. Huang *et al.*, [49] examine planning mechanisms in LLM agents, which are essential for executing long-term tasks—a frequent requirement in GUI automation. Zhang *et al.*, [50] explore memory mechanisms, which allow agents to store critical historical information, aiding in knowledge retention and decision-making. Additionally, Shen [13] surveys the use of tools by LLMs (such as APIs and code) to interact effectively with their environments, grounding actions in ways that produce tangible impacts. Further, Chang *et al.*, [51] provide a comprehensive survey on evaluation methods for LLMs, which is crucial for ensuring the robustness and safety of GUI agents. Two additional surveys, [52] and [53], provide comprehensive overviews of benchmarks and evaluation methods specifically tailored to multimodal LLMs. The evaluation also facilitates a feedback loop, allowing agents to improve iteratively based on assessment results. Together, these surveys serve as valuable resources, offering guidance on essential components of LLM agents and forming a foundational basis for LLM-based GUI agents.

Our survey distinguishes itself from existing work by providing a pioneering and comprehensive analysis of the intersections and integrations between LLMs and GUI agents. It thoroughly examines foundational components, advanced technologies, framework architectures, data and model optimization strategies, applications, key challenges, and a forward-looking roadmap—areas that have not been fully explored in previous studies.

3 BACKGROUND

The development of LLM-brained GUI agents is grounded in three major advancements: *(i)* large language models (LLMs) [8], which bring advanced capabilities in natural language understanding and code generation, forming the core intelligence of these agents; *(ii)* accompanying agent architectures and tools [45] that extend LLM capabilities, bridging the gap between language models and physical environments to enable tangible impacts; and *(iii)* GUI automation [60], which has cultivated a robust set of tools, models, and methodologies essential for GUI agent functionality. Each of these components has played a critical role in the emergence of LLM-powered GUI agents. In the following subsections, we provide a brief overview of these areas to set the stage for our discussion.

3.1 Large Language Models: Foundations and Capabilities

The study of language models has a long and rich history [61], beginning with early statistical language models [62] and smaller neural network architectures [63]. Building on these

foundational concepts, recent advancements have focused on transformer-based LLMs, such as the Generative Pre-trained Transformers (GPTs) [64]. These models are pretrained on extensive text corpora and feature significantly larger model sizes, validating scaling laws and demonstrating exceptional capabilities across a wide range of natural language tasks. Beyond their sheer size, these LLMs exhibit enhanced language understanding and generation abilities, as well as emergent properties that are absent in smaller-scale language models [65].

Early neural language models, based on architectures like recurrent neural networks (RNNs) [66] and long short-term memory networks (LSTMs) [67], were limited in both performance and generalization. The introduction of the Transformer model, built on the attention mechanism [68], marked a transformative milestone, establishing the foundational architecture now prevalent across almost all subsequent LLMs. This development led to variations in model structures, including encoder-only models (*e.g.*, BERT [69], RoBERTa [70], ALBERT [71]), decoder-only models (*e.g.*, GPT-1 [72], GPT-2 [73]), and encoder-decoder models (*e.g.*, T5 [74], BART [75]). In 2022, ChatGPT [11] based on GPT-3.5 [76] launched as a groundbreaking LLM, fundamentally shifting perceptions of what language models can achieve. Since then, numerous advanced LLMs have emerged, including GPT-4 [77], LLaMA-3 [78], and Gemini [79], propelling the field into rapid growth. Today's LLMs are highly versatile, with many of them are capable of processing multimodal data and performing a range of tasks, from question answering to code generation, making them indispensable tools in various applications [80]–[83].

The emergence of LLMs has also introduced significant advanced properties that invigorate their applications, making previously challenging tasks, such as natural language-driven GUI agents feasible. These advancements include:

1) **Few-Shot Learning [64]:** Also referred to as in-context learning [84], LLMs can acquire new tasks from a small set of demonstrated examples presented in the prompt during inference, eliminating the need for retraining. This capability is crucial for enabling GUI agents to generalize across different environments with minimal effort.
2) **Instruction Following [85]:** After undergoing instruction tuning, LLMs exhibit a remarkable ability to follow instructions for novel tasks, demonstrating strong generalization skills [76]. This allows LLMs to effectively comprehend user requests directed at GUI agents and to follow predefined objectives accurately.
3) **Long-Term Reasoning [86]:** LLMs possess the ability to plan and solve complex tasks by breaking them down into manageable steps, often employing techniques like chain-of-thought (CoT) reasoning [87], [88]. This capability is essential for GUI agents, as many tasks require multiple steps and a robust planning framework.
4) **Code Generation and Tool Utilization [89]:** LLMs excel in generating code and utilizing various tools, such as APIs [13]. This expertise is vital, as code and tools form the essential toolkit for GUI agents to interact with their environments.
5) **Multimodal Comprehension [10]:** Advanced LLMs can integrate additional data modalities, such as images, into their training processes, evolving into multimodal models. This ability is particularly important for GUI agents, which must interpret GUI screenshots presented as images in order to function effectively [90].

To further enhance the specialization of LLMs for GUI agents, researchers often fine-tune these models with domain-specific data, such as user requests, GUI screenshots, and action sequences, thereby increasing their customization and effectiveness. In Section 8, we delve into these advanced, tailored models for GUI agents, discussing their unique adaptations and improved capabilities for interacting with graphical interfaces.

3.2 LLM Agents: From Language to Action

Traditional AI agents have often focused on enhancing specific capabilities, such as symbolic reasoning or excelling in particular tasks like Go or Chess. In contrast, the emergence of LLMs has transformed AI agents by providing them with a natural language interface, enabling human-like decision-making capabilities, and equipping them to perform a wide variety of tasks and take tangible actions in diverse environments [12], [45], [91], [92]. In LLM agents, if LLMs form the "brain" of a GUI agent, then its accompanying components serve as its "eyes and hands", enabling the LLM to perceive the environment's status and translate its textual output into actionable steps that generate tangible effects [44]. These components transform LLMs from passive information sources into interactive agents that execute tasks on behalf of users, which redefine the role of LLMs from purely text-generative models to systems capable of driving actions and achieving specific goals.

In the context of GUI agents, the agent typically perceives the GUI status through screenshots and widget trees [93], then performs actions to mimic user operations (*e.g.*, mouse clicks, keyboard inputs, touch gestures on phones) within the environment. Since tasks can be long-term, effective planning and task decomposition are often required, posing unique challenges. Consequently, an LLM-powered GUI agent usually possess multimodal capabilities [54], a robust planning system [49], a memory mechanism to analyze historical interactions [50], and a specialized toolkit to interact with its environment [25]. We will discuss these tailored designs for GUI agents in detail in Section 5.

3.3 GUI Automation: Tools, Techniques, and Challenges

GUI automation has been a critical area of research and application since the early days of GUIs in computing. Initially developed to improve software testing efficiency, GUI automation focused on simulating user actions, such as clicks, text input, and navigation, across graphical applications to validate functionality [28]. Early GUI automation tools were designed to execute repetitive test cases on static interfaces [26]. These approaches streamlined quality assurance processes, ensuring consistency and reducing manual testing time. As the demand for digital solutions has grown, GUI automation has expanded beyond testing to other applications, including RPA [6] and Human-Computer Interaction (HCI) [94]. RPA leverages GUI automation to replicate human actions in business workflows, automating routine tasks to improve operational efficiency. Similarly, HCI research employs GUI

automation to simulate user behaviors, enabling usability assessments and interaction studies. In both cases, automation has significantly enhanced productivity and user experience by minimizing repetitive tasks and enabling greater system adaptability [95], [96].

Traditional GUI automation methods have primarily depended on scripting and rule-based frameworks [4], [97]. Scripting-based automation utilizes languages such as Python, Java, and JavaScript to control GUI elements programmatically. These scripts simulate a user's actions on the interface, often using tools like Selenium [98] for web-based automation or AutoIt [99] and SikuliX [100] for desktop applications. Rule-based approaches, meanwhile, operate based on predefined heuristics, using rules to detect and interact with specific GUI elements based on properties such as location, color, and text labels [4]. While effective for predictable, static workflows [101], these methods struggle to adapt to the variability of modern GUIs, where dynamic content, responsive layouts, and user-driven changes make it challenging to maintain rigid, rule-based automation [102].

CV has become essential for interpreting the visual aspects of GUIs [33], [103], [104], enabling automation tools to recognize and interact with on-screen elements even as layouts and designs change. CV techniques allow GUI automation systems to detect and classify on-screen elements, such as buttons, icons, and text fields, by analyzing screenshots and identifying regions of interest [105]–[107]. Optical Character Recognition (OCR) further enhances this capability by extracting text content from images, making it possible for automation systems to interpret labels, error messages, and form instructions accurately [108]. Object detection models add robustness, allowing automation agents to locate GUI elements even when the visual layout shifts [90]. By incorporating CV, GUI automation systems achieve greater resilience and adaptability in dynamic environments.

Despite advances, traditional GUI automation methods fall short in handling the complexity and variability of contemporary interfaces. Today's applications often feature dynamic, adaptive elements that cannot be reliably automated through rigid scripting or rule-based methods alone [109], [110]. Modern interfaces increasingly require contextual awareness [111], such as processing on-screen text, interpreting user intent, and recognizing visual cues. These demands reveal the limitations of existing automation frameworks and the need for more flexible solutions capable of real-time adaptation and context-sensitive responses.

LLMs offer a promising solution to these challenges. With their capacity to comprehend natural language, interpret context, and generate adaptive scripts, LLMs can enable more intelligent, versatile GUI automation [112]. Their ability to process complex instructions and learn from context allows them to bridge the gap between static, rule-based methods and the dynamic needs of contemporary GUIs [113]. By integrating LLMs with GUI agents, these systems gain the ability to generate scripts on-the-fly based on the current state of the interface, providing a level of adaptability and sophistication that traditional methods cannot achieve. The combination of LLMs and GUI agents paves the way for an advanced, user-centered automation paradigm, capable of responding flexibly to user requests and interacting seamlessly with complex, evolving interfaces.

4 Evolution and Progression of LLM-Brained GUI Agents

"Rome wasn't built in a day." The development of LLM-brained GUI agents has been a gradual journey, grounded in decades of research and technical progress. Beginning with simple GUI testing scripts and rule-based automation frameworks, the field has evolved significantly through the integration of machine learning techniques, creating more intelligent and adaptive systems. The introduction of LLMs, especially multimodal models, has transformed GUI automation by enabling natural language interactions and fundamentally reshaping how users interact with software applications.

As illustrated in Figure 3, prior to 2023 and the emergence of LLMs, work on GUI agents was limited in both scope and capability. Since then, the proliferation of LLM-based approaches has fostered numerous notable developments across platforms including web, mobile, and desktop environments. This surge is ongoing and continues to drive innovation in the field. This section takes you on a journey tracing the evolution of GUI agents, emphasizing key milestones that have brought the field to its present state.

4.1 Early Automation Systems

In the initial stages of GUI automation, researchers relied on random-based, rule-based, and script-based strategies. While foundational, these methods had notable limitations in terms of flexibility and adaptability.

4.1.1 Random-Based Automation

Random-based automation uses random sequences of actions within the GUI without relying on specific algorithms or structured models using monkey test [114]. This approach was widely used in GUI testing to uncover potential issues by exploring unpredictable input sequences [115]. While effective at identifying edge cases and bugs, random-based methods were often inefficient due to a high number of redundant or irrelevant trials.

4.1.2 Rule-Based Automation

Rule-based automation applies predefined rules and logic to automate tasks. In 2001, Memon *et al.*, [116] introduced a planning approach that generated GUI test cases by transforming initial states to goal states through a series of predefined operators. Hellmann *et al.*, [4] (2011) demonstrated the potential of rule-based approaches in exploratory testing, enhancing bug detection. In the RPA domain, SmartRPA [117] (2020) used rule-based processing to automate routine tasks, illustrating the utility of rules for streamlining structured processes.

4.1.3 Script-Based Automation

Script-based automation relies on detailed scripts to manage GUI interactions. Tools like jRapture [5] (2000) record and replay Java-based GUI sequences using Java binaries and the JVM, enabling consistent execution by precisely reproducing input sequences. Similarly, DART [118] (2003) automated the GUI testing lifecycle, from structural analysis to test case generation and execution, offering a comprehensive framework for regression testing.

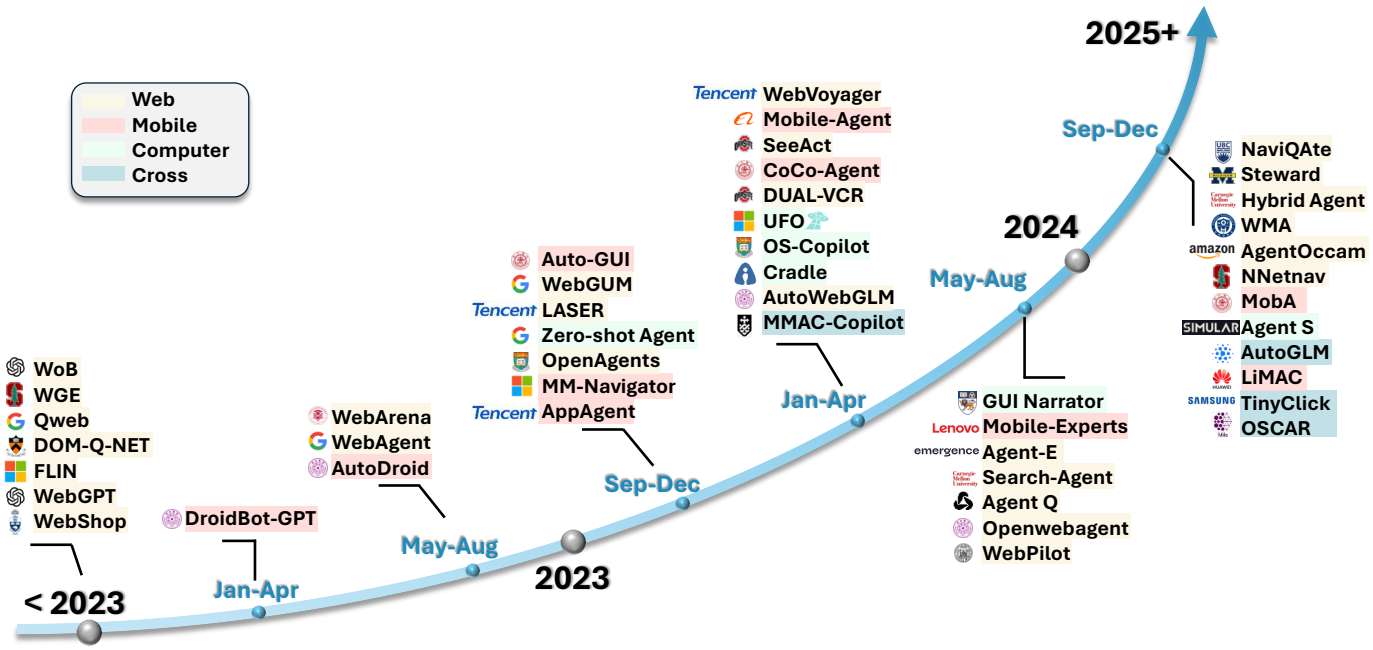


Fig. 3: An overview of GUI agents evolution over years.

4.1.4 *Tools and Software*

A range of software tools were developed for GUI testing and business process automation during this period. Microsoft Power Automate [119] (2019) provides a low-code/no-code environment for creating automated workflows within Microsoft applications. Selenium [120] (2004) supports cross-browser web testing, while Appium [121] (2012) facilitates mobile UI automation. Commercial tools like TestComplete [122] (1999), Katalon Studio [123] (2015), and Ranorex [124] (2007) allow users to create automated tests with cross-platform capabilities.

Although these early systems were effective for automating specific, predefined workflows, they lacked flexibility and required manual scripting or rule-based logic. Nonetheless, they established the foundations of GUI automation, upon which more intelligent systems were built.

4.2 The Shift Towards Intelligent Agents

The incorporation of machine learning marked a major shift towards more adaptable and capable GUI agents. Early milestones in this phase included advancements in machine learning, natural language processing, computer vision, and reinforcement learning applied to GUI tasks.

4.2.1 *Machine Learning and Computer Vision*

RoScript [97] (2020) was a pioneering system that introduced a non-intrusive robotic testing system for touchscreen applications, expanding GUI automation to diverse platforms. AppFlow [125] (2018) used machine learning to recognize common screens and UI components, enabling modular testing for broad categories of applications. Progress in computer vision also enabled significant advances in GUI testing, with frameworks [104] (2010) automating visual interaction tasks. Humanoid [126] (2019) uses a deep neural network model trained on human interaction traces within the Android system to learn how users select actions based on an app's GUI. This model is then utilized to guide test input generation, resulting in improved coverage and more human-like interaction patterns during testing. Similarly, Deep GUI [127] (2021) applies deep learning techniques to filter out irrelevant parts of the screen, thereby enhancing black-box testing effectiveness in GUI testing by focusing only on significant elements. These approaches demonstrate the potential of deep learning to make GUI testing more efficient and intuitive by aligning it closely with actual user behavior.

Widget detection, as demonstrated by White *et al.*, [90] (2019), leverages computer vision to accurately identify UI elements, serving as a supporting technique that enables more intelligent and responsive UI automation. By detecting and categorizing interface components, this approach enhances the agent's ability to interact effectively with complex and dynamic GUIs [128].

4.2.2 *Natural Language Processing*

Natural language processing capabilities introduced a new dimension to GUI automation. Systems like RUSS [129] (2021) and FLIN [130] (2020) allowed users to control GUIs through natural language commands, bridging human language and machine actions. Datasets, such as those in [131] (2020), further advanced the field by mapping natural language instructions to mobile UI actions, opening up broader applications in GUI control. However, these approaches are limited to handling simple natural commands and are not equipped to manage long-term tasks.

4.2.3 *Reinforcement Learning*

The development of environments like World of Bits (WoB) [132] (2017) enabled the training of web-based agents using reinforcement learning (RL). Workflow-guided exploration [133] (2018) improved RL efficiency and task performance. DQT [134] (2024) applied deep reinforcement learning to

automate Android GUI testing by preserving widget structures and semantics, while AndroidEnv [135] (2021) offered realistic simulations for agent training on Android. WebShop [136] (2022) illustrated the potential for large-scale web interaction, underscoring the growing sophistication of RL-driven GUI automation.

While these machine learning-based approaches were more adaptable than earlier rule-based systems [137], [138], they still struggled to generalize across diverse, unforeseen tasks. Their dependence on predefined workflows and limited adaptability required retraining or customization for new environments, and natural language control was still limited.

4.3 The Advent of LLM-Brained GUI Agents

The introduction of LLMs, particularly multimodal models like GPT-4o [80] (2023), has radically transformed GUI automation by allowing intuitive interactions through natural language. Unlike previous approaches that required integration of separate modules, LLMs provide an end-to-end solution for GUI automation, offering advanced capabilities in natural language understanding, visual recognition, and reasoning.

LLMs present several unique advantages for GUI agents, including natural language understanding, multimodal processing, planning, and generalization. These features make LLMs and GUI agents a powerful combination. While there were earlier explorations, 2023 marked a pivotal year for LLM-powered GUI agents, with significant developments across various platforms such as web, mobile, and desktop applications.

4.3.1 Web Domain

The initial application of LLMs in GUI automation was within the web domain, with early studies establishing benchmark datasets and environments [132], [136]. A key milestone was WebAgent [139] (2023), which, alongside WebGUM [140] (2023), pioneered real-world web navigation using LLMs. These advancements paved the way for further developments [15], [141], [142], utilizing more specialized LLMs to enhance web-based interactions.

4.3.2 Mobile Devices

The integration of LLMs into mobile devices began with AutoDroid [143] (2023), which combined LLMs with domain-specific knowledge for smartphone automation. Additional contributions like MM-Navigator [144] (2023), AppAgent [16] (2023), and Mobile-Agent [145] (2023) enabled refined control over smartphone applications. Research has continued to improve accuracy for mobile GUI automation through model fine-tuning [146], [147] (2024).

4.3.3 Computer Systems

For desktop applications, UFO [17] (2024) was one of the first systems to leverage GPT-4 with visual capabilities to fulfill user commands in Windows environments. Cradle [148] (2024) extended these capabilities to software applications and games, while Wu *et al.*, [149] (2024) provided interaction across diverse desktop applications, including web browsers, code terminals, and multimedia tools.

4.3.4 Industry Models

In industry, the `Claude 3.5 Sonnet` model [150] (2024) introduced a "computer use" feature capable of interacting with desktop environments through UI operations [151]. This signifies the growing recognition of LLM-powered GUI agents as a valuable application in industry, with stakeholders increasingly investing in this technology.

Undoubtedly, LLMs have introduced new paradigms and increased the intelligence of GUI agents in ways that were previously unattainable. As the field continues to evolve, we anticipate a wave of commercialization, leading to transformative changes in user interaction with GUI applications.

5 LLM-Brained GUI Agents: Foundations and Design

In essence, LLM-brained GUI agents are designed to process user instructions or requests given in natural language, interpret the current state of the GUI through screenshots or UI element trees, and execute actions that simulate human interaction across various software interfaces [17]. These agents harness the sophisticated natural language understanding, reasoning, and generative capabilities of LLMs to accurately comprehend user intent, assess the GUI context, and autonomously engage with applications across diverse environments, thereby enabling the completion of complex, multi-step tasks. This integration allows them to seamlessly interpret and respond to user requests, bringing adaptability and intelligence to GUI automation.

As a specialized type of LLM agent, most current GUI agents adopt a similar foundational framework, integrating core components such as planning, memory, tool usage, and advanced enhancements like multi-agent collaboration, among others [45]. However, each component must be tailored to meet the specific objectives of GUI agents to ensure adaptability and functionality across various application environments.

In the following sections, we provide an in-depth overview of each component, offering a practical guide and tutorial on building an LLM-powered GUI agent from the ground up. This comprehensive breakdown serves as a cookbook for creating effective and intelligent GUI automation systems that leverage the capabilities of LLMs.

5.1 Architecture and Workflow In a Nutshell

In Figure 4, we present the architecture of an LLM-brained GUI agent, showcasing the sequence of operations from user input to task completion. The architecture comprises several integrated components, each contributing to the agent's ability to interpret and execute tasks based on user-provided natural language instructions. Upon receiving a user request, the agent follows a systematic workflow that includes environment perception, prompt engineering, model inference, action execution, and continuous memory utilization until the task is fully completed.

In general, it consists of the following components:

1) **Operating Environment:** The environment defines the operational context for the agent, encompassing platforms such as mobile devices, web browsers, and desktop operating systems like Windows. To interact

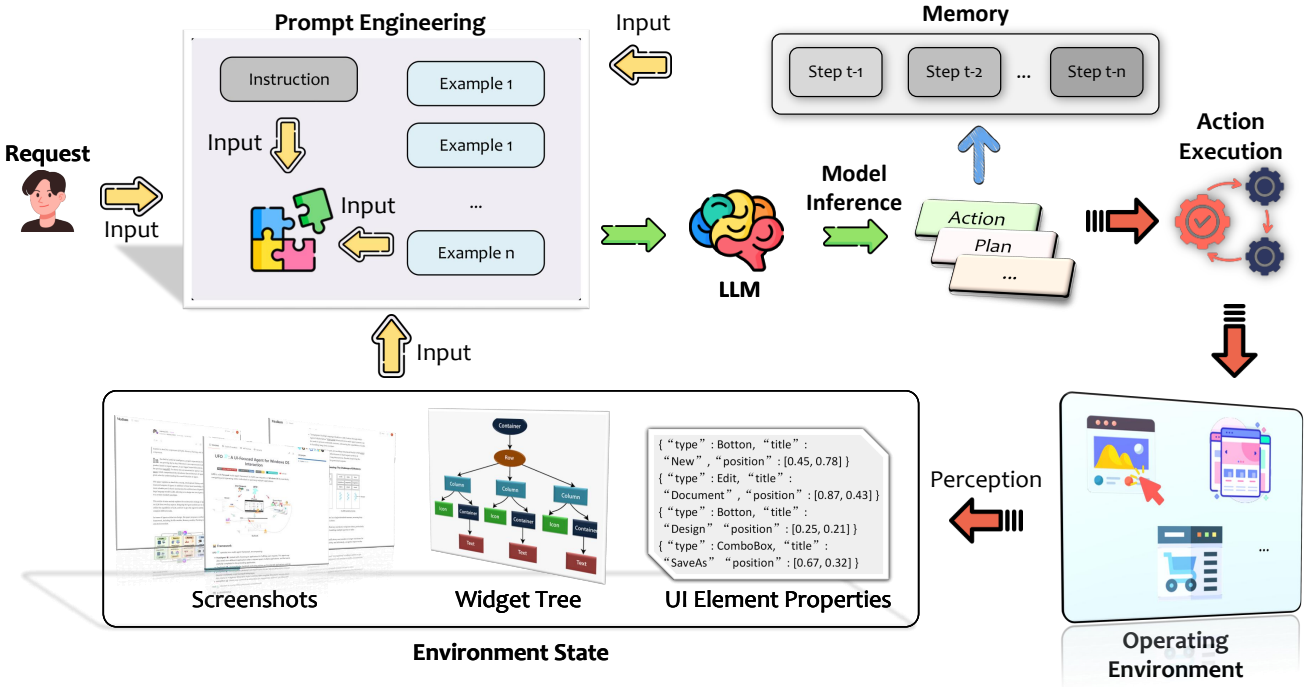


Fig. 4: An overview of the architecture and workflow of a basic LLM-powered GUI agent.

meaningfully, the agent perceives the environment's current state through screenshots, widget trees, or other methods of capturing UI structure [152]. It continuously monitors feedback on each action's impact, adjusting its strategy in real time to ensure effective task progression.

2) **Prompt Engineering:** Following environment perception, the agent constructs a detailed prompt to guide the LLM's inference [153]. This prompt incorporates user instructions, processed visual data (*e.g.*, screenshots), UI element layouts, properties, and any additional context relevant to the task. This structured input maximizes the LLM's ability to generate coherent, context-aware responses aligned with the current GUI state.
3) **Model Inference:** The constructed prompt is passed to a LLM, the agent's inference core, which produces a sequence of plans, actions and insights required to fulfill the user's request. This model may be a general-purpose LLM or a specialized model fine-tuned with GUI-specific data, enabling a more nuanced understanding of GUI interactions, user flows, and task requirements.
4) **Actions Execution:** Based on the model's inference results, the agent identifies specific actions (such as mouse clicks, keyboard inputs, touchscreen gestures, or API calls) required for task execution [13]. An executor within the agent translates these high-level instructions into actionable commands that impact the GUI directly, effectively simulating human-like interactions across diverse applications and devices.
5) **Memory:** For multi-step tasks, the agent maintains an internal memory to track prior actions, task progress, and environment states [50]. This memory ensures coherence throughout complex workflows, as the agent can reference previous steps and adapt its actions accordingly. An external memory module may also be incorporated to enable continuous learning, access external knowledge, and enhance adaptation to new environments or requirements.



Fig. 5: Examples of GUIs from web, mobile and computer platforms.

By iteratively traversing these stages and assembling the foundational components, the LLM-powered GUI agent operates intelligently, seamlessly adapting across various software interfaces and bridging the gap between language-based instruction and concrete action. Each component is critical to the agent's robustness, responsiveness, and capability to handle complex tasks in dynamic environments. In the following subsections, we detail the design and core techniques underlying each of these components, providing a comprehensive guide for constructing LLM-powered GUI agents from the ground up.

5.2 Operating Environment

The operating environment for LLM-powered GUI agents encompasses various platforms, such as mobile, web, and desktop operating systems, where these agents can interact with graphical interfaces. Each platform has distinct characteristics that impact the way GUI agents perceive, interpret, and act within it. Examples of GUIs from each platform are

shown in Figure 5. This section details the nuances of each platform, the ways agents gather environmental information, and the challenges they face in adapting to diverse operating environments.

5.2.1 Platform

GUI agents can interact with a wide range of platforms, including mobile devices, web applications, and computer operating systems like Windows. Each platform offers unique capabilities and constraints for GUI automation, requiring agents to adapt their perception and interaction strategies accordingly.

1) **Mobile Platforms:** Mobile devices operate within constrained screen real estate, rely heavily on touch interactions [154], and offer varied app architectures (*e.g.*, native vs. hybrid apps). Mobile platforms often use accessibility frameworks, such as Android's Accessibility API[4] [155] and iOS's VoiceOver Accessibility Inspector[5], to expose structured information about UI elements. However, GUI agents must handle additional complexities in mobile environments, such as gesture recognition [156], app navigation [157], and platform-specific constraints (*e.g.*, security and privacy permissions) [158], [159].
2) **Web Platforms:** Web applications provide a relatively standardized interface, typically accessible through Hypertext Markup Language (HTML) and Document Object Model (DOM) structures [160], [161]. GUI agents can leverage HTML attributes, such as element ID, class, and tag, to identify interactive components. Web environments also present dynamic content, responsive layouts, and asynchronous updates (*e.g.*, AJAX requests) [162], requiring agents to continuously assess the DOM and adapt their actions to changing interface elements.
3) **Computer Platforms:** Computer OS platforms, such as Windows, offer full control over GUI interactions. Agents can utilize system-level automation APIs, such as Windows UI Automation[6] [30], to obtain comprehensive UI element data, including type, label, position, and bounding box. These platforms often support a broader set of interaction types, mouse, keyboard, and complex multi-window operations. These enable GUI agents to execute intricate workflows. However, these systems also require sophisticated adaptation for diverse applications, ranging from simple UIs to complex, multi-layered software suites.

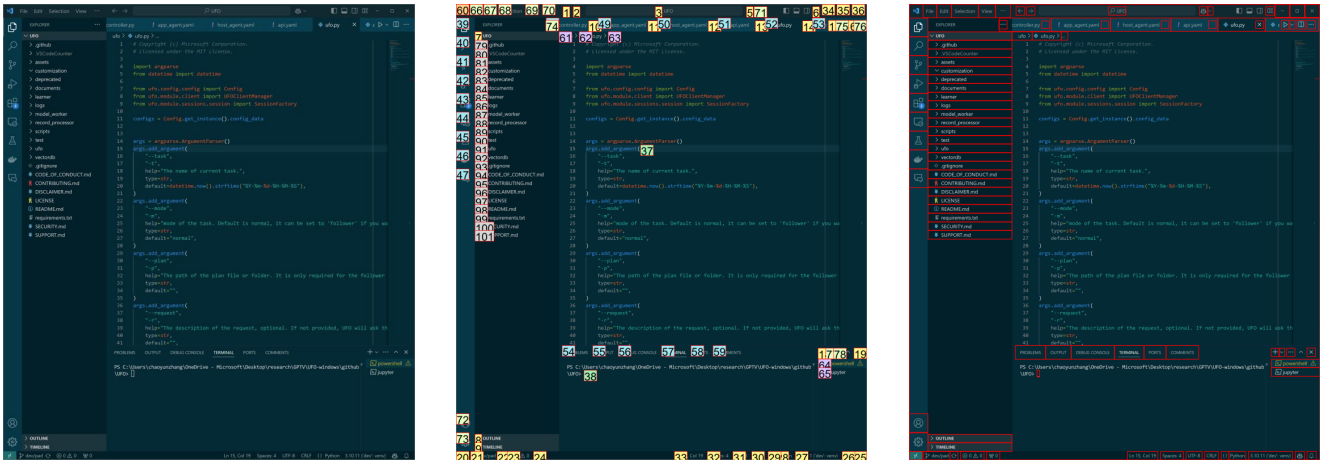
In summary, the diversity of platforms, spanning mobile, web, and desktop environments, enable GUI agents to deliver broad automation capabilities, making them a generalized solution adaptable across a unified framework. However, each platform presents unique characteristics and constraints at both the system and application levels, necessitating a tailored approach for effective integration. By considering these platform-specific features, GUI agents can be optimized to address the distinctive requirements of each environment, thus enhancing their adaptability and reliability in varied automation scenarios.

5.2.2 Environment State Perception

Accurately perceiving the current state of the environment is essential for LLM-powered GUI agents, as it directly informs their decision-making and action-planning processes. This perception is enabled by gathering a combination of structured data, such as widget trees, and unstructured data, like screenshots, to capture a complete representation of the interface and its components. In Table 3, we outline key toolkits available for collecting GUI environment data across various platforms, and below we discuss their roles in detail:

1) **GUI Screenshots:** Screenshots provide a visual snapshot of the application, capturing the entire state of the GUI at a given moment. They offer agents a reference for layout, design, and visual content, which is crucial when structural details about UI elements are either limited or unavailable. Visual elements like icons, images, and other graphical cues that may hold important context can be analyzed directly from screenshots. Many platforms have built-in tools to capture screenshots (*e.g.*, Windows Snipping Tool[7], macOS Screenshot Utility[8], and Android's MediaProjection API[9]), and screenshots can be enhanced with additional annotations, such as Set-of-Mark (SoM) highlights [163] or bounding boxes [164] around key UI components, to streamline agent decisions. Figure 6 illustrates various screenshots of the VS Code GUI, including a clean version, as well as ones with SoM and bounding boxes that highlight actionable components, helping the agent focus on the most critical areas of the interface.
2) **Widget Trees:** Widget trees present a hierarchical view of interface elements, providing structured data about the layout and relationships between components [165]. We show an example of a GUI and its widget tree in Figure 7. By accessing the widget tree, agents can identify attributes such as element type, label, role, and relationships within the interface, all of which are essential for contextual understanding. Tools like Windows UI Automation and macOS's Accessibility API[10] provide structured views for desktop applications, while Android's Accessibility API and HTML DOM structures serve mobile and web platforms, respectively. This hierarchical data is indispensable for agents to map out logical interactions and make informed choices based on the UI structure.
3) **UI Element Properties:** Each UI element in the interface contains specific properties, such as control type, label text, position, and bounding box dimensions, that help agents target the appropriate components. These properties are instrumental for agents to make decisions

4. https://developer.android.com/reference/android/accessibilityservice/AccessibilityService
5. https://developer.apple.com/documentation/accessibility/accessibility-inspector
6. https://learn.microsoft.com/en-us/dotnet/framework/ui-automation/ui-automation-overview
7. https://support.microsoft.com/en-us/windows/use-snipping-tool-to-capture%2Dscreenshots%2D00246869%2D1843%2D655f%2Df220%2D97299b865f6b
8. https://support.apple.com/guide/mac-help/take-a-screenshot-mh26782/mac
9. https://developer.android.com/reference/android/media/projection/MediaProjection
10. https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/



(a) A clean GUI screenshot.

(b) A GUI screenshot with widgets highlighted by SoM.

(c) A GUI screenshot with widgets highlighted by bounding boxes.

Fig. 6: Examples of different variants of VS Code GUI screenshots.

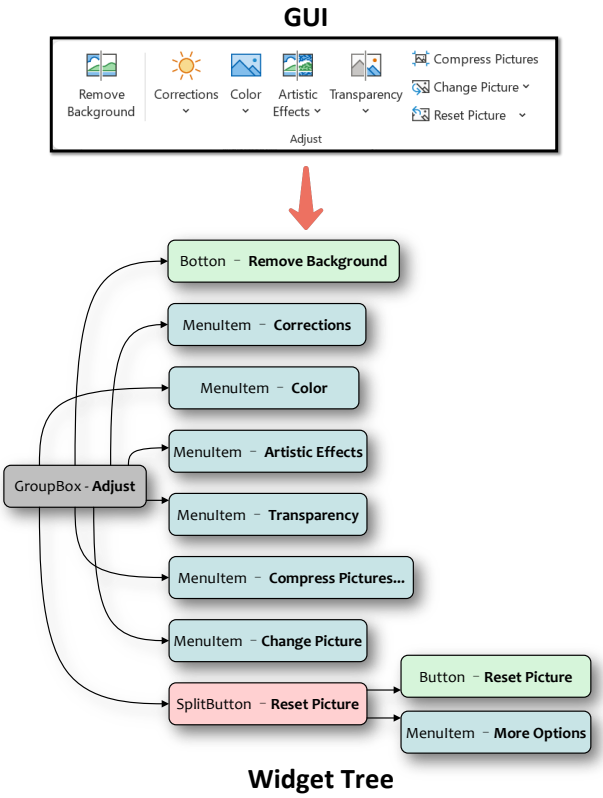


Fig. 7: An example of a GUI and its widget tree.

Widget	Widget Name	Position	Attributes
Remove Background	Button - 'Remove Background'	L-3810, T128, R-3708, B243	title='Remove Background'; auto_id='PictureBackgroundRemoval'; control_type='Button'
Corrections	MenuItem - 'Corrections'	L-3689, T128, R-3592, B243	title='Corrections'; auto_id='PictureCorrectionsMenu'; control_type='MenuItem'
Color	MenuItem - 'Color'	L-3589, T128, R-3527, B243	title='Color'; auto_id='PictureColorMenu'; control_type='MenuItem'
Artistic Effects	MenuItem - 'Artistic Effects'	L-3524, T128, R-3448, B243	title='Artistic Effects'; auto_id='PictureArtisticEffectsGallery'; control_type='MenuItem'
Transparency	MenuItem - 'Transparency'	L-3445, T128, R-3336, B243	title='Transparency'; auto_id='PictureTransparencyGallery'; control_type='MenuItem'
Compress Pictures	Button - 'Compress Pictures...'	L-3333, T128, R-3138, B164	title='Compress Pictures...'; auto_id='PicturesCompress'; control_type='Button'
Change Picture	MenuItem - 'Change Picture'	L-3333, T167, R-3149, B203	title='Change Picture'; auto_id='PictureChangeMenu'; control_type='MenuItem'
Reset Picture	SplitButton - 'Reset Picture'	L-3333, T206, R-3160, B242	title='Reset Picture'; control_type='SplitButton'

Fig. 8: Examples of UI element properties in the PowerPoint application for GUI Agent interaction.

about spatial relationships (*e.g.*, adjacent elements) and functional purposes (*e.g.*, distinguishing between buttons and text fields). For instance, web applications reveal properties like DOM attributes (id, class, name) and CSS styles that provide context and control information. These attributes assist agents in pinpointing precise elements for interaction, enhancing their ability to navigate and operate within diverse UI environments. Figure 8 illustrates examples of selected UI element properties extracted by the Windows UI Automation API, which support GUI agents in decision-making.

4) **Complementary CV Approaches:** When structured information is incomplete or unavailable, computer vision techniques can provide additional insights [166]. For instance, OCR allows agents to extract text content directly from screenshots, facilitating the reading of labels, error messages, and instructions [108]. Furthermore, advanced object detection [107] models like SAM (Segment Anything Model) [167], DINO [168] and OmniParser [169] can identify and classify UI components in various layouts, supporting the agent in dynamic environments where UI elements may frequently change. These vision-based methods ensure robustness, enabling agents to function effectively even in settings where standard UI APIs are insufficient. We illustrate an example of

TABLE 3: Key toolkits for collecting GUI environment data.

Tool	Platform	Environment	Accessible Information	Highlight	Link
Selenium	Web	Browser (Cross-platform)	DOM elements, HTML structure, CSS properties	Extensive browser support and automation capabilities	https://www.selenium.dev/
Puppeteer	Web	Browser (Chrome, Firefox)	DOM elements, HTML/CSS, network requests	Headless browser automation with rich API	https://pptr.dev/
Playwright	Web	Browser (Cross-platform)	DOM elements, HTML/CSS, network interactions	Multi-browser support with automation and testing capabilities	https://playwright.dev/
TestCafe	Web	Browser (Cross-platform)	DOM elements, HTML structure, CSS properties	Easy setup with JavaScript/-TypeScript support	https://testcafe.io/
BeautifulSoup	Web	HTML Parsing	HTML content, DOM elements	Python library for parsing HTML and XML documents	https://www.crummy.com/software/BeautifulSoup/
Protractor	Web	Browser (Angular)	DOM elements, Angular-specific attributes	Designed for Angular applications, integrates with Selenium	https://www.protractortest.org/
WebDriverIO	Web	Browser (Cross-platform)	DOM elements, HTML/CSS, network interactions	Highly extensible with a vast plugin ecosystem	https://webdriver.io/
Ghost Inspector	Web	Browser (Cross-platform)	DOM elements, screenshots, test scripts	Cloud-based automated browser testing and monitoring	https://ghostinspector.com/
Cypress	Web	Browser (Cross-platform)	DOM elements, HTML/CSS, network requests	Real-time reloads and interactive debugging	https://www.cypress.io/
UIAutomator	Mobile	Android	UI hierarchy, widget properties, screen content	Native Android UI testing framework	https://developer.android.com/training/testing/ui-automator
Espresso	Mobile	Android	UI components, view hierarchy, widget properties	Google's native Android UI testing framework	https://developer.android.com/training/testing/espresso
Android View Hierarchy	Mobile	Android	UI hierarchy, widget properties, layout information	View hierarchy accessible via developer tools	https://developer.android.com/studio/debug/layout-inspector
iOS Accessibility Inspector	Mobile	iOS	Accessibility tree, UI elements, properties	Tool for inspecting iOS app UI elements	https://developer.apple.com/documentation/accessibility/accessibility-inspector
XCUITest	Mobile	iOS	UI elements, accessibility properties, view hierarchy	Apple's iOS UI testing framework	https://developer.apple.com/documentation/xctest/user_interface_tests
Flutter Driver	Mobile	Flutter apps	Widget tree, properties, interactions	Automation for Flutter applications	https://flutter.dev/docs/testing
Android's MediaProjection API	Mobile	Android	Screenshots, screen recording	Capturing device screen content programmatically	https://developer.android.com/reference/android/media/projection/MediaProjection
Windows UI Automation	Computer	Windows	Control properties, widget trees, accessibility tree	Native Windows support with OS integration	https://docs.microsoft.com/windows/win32/winauto/entry-uiauto-win32
Sikuli	Computer	Windows, macOS, Linux	Screenshots (image recognition), UI elements	Image-based automation using computer vision	http://sikulix.com/
AutoIt	Computer	Windows	Window titles, control properties, coordinates	Scripting language for Windows GUI automation	https://www.autoitscript.com/site/autoit/
Inspect.exe	Computer	Windows	UI elements, control properties, accessibility tree	Tool for inspecting Windows UI elements	https://docs.microsoft.com/windows/win32/winauto/inspect-objects
macOS Accessibility API	Computer	macOS	Accessibility tree, UI elements, control properties	macOS support for accessibility and UI automation	https://developer.apple.com/accessibility/
Pywinauto	Computer	Windows	Control properties, UI hierarchy, window information	Python-based Windows GUI automation	https://pywinauto.readthedocs.io/
Electron Inspector	Computer	Electron apps	DOM elements, HTML/CSS, JavaScript state	Tool for Electron applications	https://www.electronjs.org/docs/latest/tutorial/automated-testing
Windows Snipping Tool	Computer	Windows	Screenshots	Tool for capturing screenshots in Windows	https://www.microsoft.com/en-us/windows/tips/snipping-tool
macOS Screenshot Utility	Computer	macOS	Screenshots, screen recording	Tool for capturing screenshots and recording screen	https://support.apple.com/guide/mac-help/take-a-screenshot-or%2Dscreen-recording%2Dmh26782/mac
AccessKit	Cross-Platform	Various OS	Accessibility tree, control properties, roles	Standardized APIs across platforms	https://github.com/AccessKit/accesskit
Appium	Cross-Platform	Android, iOS, Windows, macOS	UI elements, accessibility properties, gestures	Mobile automation framework	https://appium.io/
Robot Framework	Cross-Platform	Web, Mobile, Desktop	UI elements, DOM, screenshots	Extensible with various libraries	https://robotframework.org/
Cucumber	Cross-Platform	Web, Mobile, Desktop	Step definitions, UI interactions	BDD framework supporting automation tools	https://cucumber.io/
TestComplete	Cross-Platform	Web, Mobile, Desktop	UI elements, DOM, control properties	Tool with extensive feature set	https://smartbear.com/product/testcomplete/overview/
Katalon Studio	Cross-Platform	Web, Mobile, Desktop	UI elements, DOM, screenshots	All-in-one automation solution	https://www.katalon.com/
Ranorex	Cross-Platform	Web, Mobile, Desktop	UI elements, DOM, control properties	Tool with strong reporting features	https://www.ranorex.com/
Applitools	Cross-Platform	Web, Mobile, Desktop	Screenshots, visual checkpoints, DOM elements	AI-powered visual testing	https://applitools.com/

this complementary information in Figure 9 and further detail these advanced computer vision approaches in Section 5.7.1.

Together, these elements create a comprehensive, multimodal representation of the GUI environment's current state, delivering both structured and visual data. By incorporating this information into prompt construction, agents are empowered to make well-informed, contextually aware decisions without missing critical environmental cues.

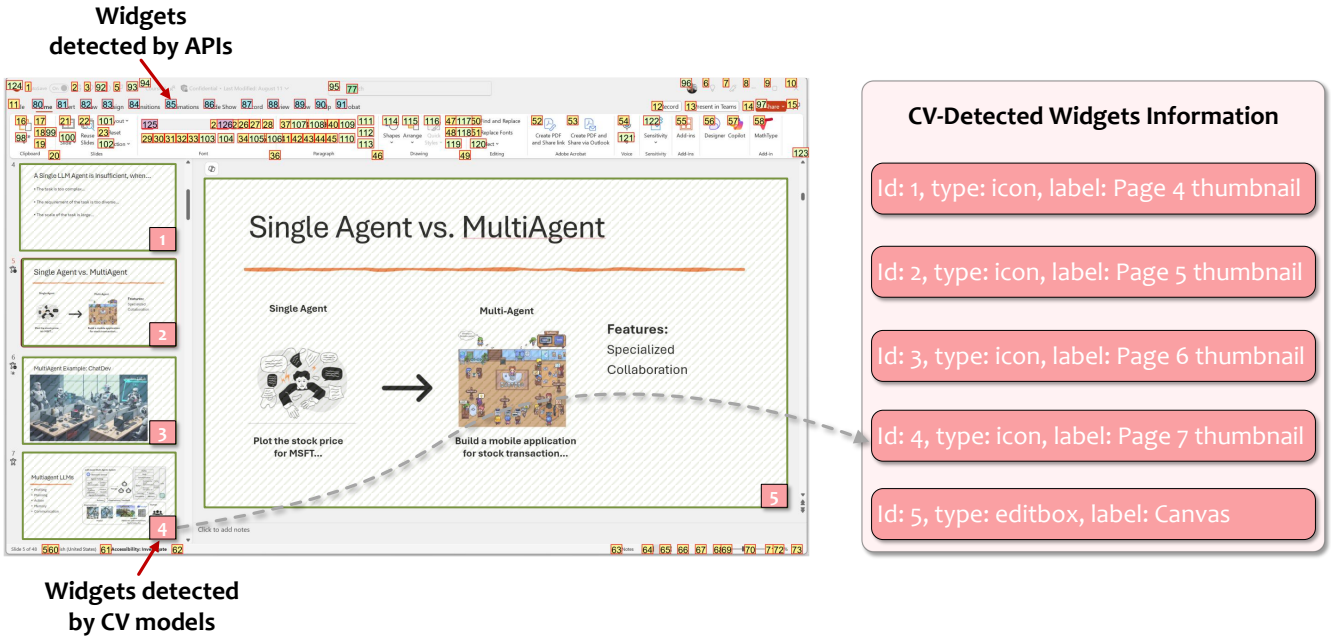


Fig. 9: An example illustrating the use of a CV approach to parse a PowerPoint GUI and detect non-standard widgets, inferring their types and labels.

5.2.3 *Environment Feedback*

Effective feedback mechanisms are essential for GUI agents to assess the success of each action and make informed decisions for subsequent steps. Feedback can take several forms, depending on the platform and interaction type. Figure 10 presents examples of various types of feedback obtained from the environment.

1) **Screenshot Update:** By comparing before-and-after screenshots, agents can identify visual differences that signify state changes in the application. Screenshot analysis can reveal subtle variations in the interface, such as the appearance of a notification, visual cues, or confirmation messages, that may not be captured by structured data [170].
2) **UI Structure Change:** After executing an action, agents can detect modifications in the widget tree structure, such as the appearance or disappearance of elements, updates to element properties, or hierarchical shifts [171]. These changes indicate successful interactions (*e.g.*, opening a dropdown or clicking a button) and help the agent determine the next steps based on the updated environment state.
3) **Function Return Values and Exceptions:** Certain platforms offer direct feedback on action outcomes through function return values or system-generated exceptions [172]. For example, API responses or JavaScript return values can confirm action success on web platforms, while exceptions or error codes can signal failed interactions, guiding the agent to retry or select an alternative approach.

These feedback provided by the environment is crucial for GUI agents to assess the outcomes of their previous actions. This real-time information enables agents to evaluate the effectiveness of their interventions and determine whether to adhere to their initial plans or pivot towards alternative strategies. Through this process of self-reflection, agents can adapt their decision-making, optimizing task execution and enhancing overall performance in dynamic and varied application environments.

5.3 Prompt Engineering

In the operation of LLM-powered GUI agents, effective prompt construction is a crucial step that encapsulates all necessary information for the agent to generate appropriate responses and execute tasks successfully [153]. After gathering the relevant data from the environment, the agent formulates a comprehensive prompt that combines various components essential for inference by the LLM. Each component serves a specific purpose, and together they enable the agent to execute the user's request efficiently. Figure 11 illustrates a basic example of prompt construction in an LLM-brained GUI agent. The key elements of the prompt are summarized as follows:

1) **User Request:** This is the original task description provided by the user, outlining the objective and desired outcome. It serves as the foundation for the agent's actions and is critical for ensuring that the LLM understands the context and scope of the task.
2) **Agent Instruction:** This section provides guidance for the agent's operation, detailing its role, rules to follow, and specific objectives. Instructions clarify what inputs the agent will receive and outline the expected outputs from the LLM, establishing a framework for the inference process.
3) **Environment States:** The agent includes perceived GUI screenshots and UI information, as introduced in Section 5.2.2. This multimodal data may consist of various versions of screenshots (*e.g.*, a clean version and

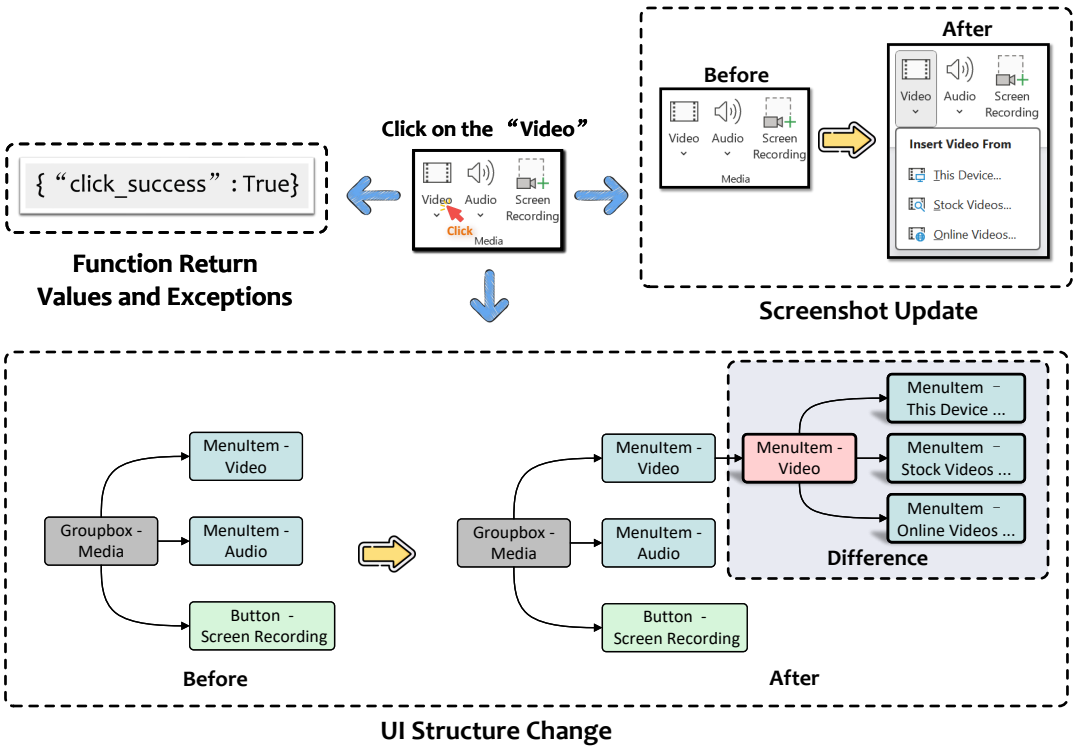


Fig. 10: Examples of various types of feedback obtained from a PowerPoint application environment.

a SoM annotated version) to ensure clarity and mitigate the risk of UI controls being obscured by annotations. This comprehensive representation of the environment is vital for accurate decision-making.

4) **Action Documents:** This component outlines the available actions the agent can take, detailing relevant documentation, function names, arguments, return values, and any other necessary parameters. Providing this information equips the LLM with the context needed to select and generate appropriate actions for the task at hand.
5) **Demonstrated Examples:** Including example input/output pairs is essential to activate the in-context learning [84] capability of the LLM. These examples help the model comprehend and generalize the task requirements, enhancing its performance in executing the GUI agent's responsibilities.
6) **Complementary Information:** Additional context that aids in planning and inference may also be included. This can consist of historical data retrieved from the agent's memory (as detailed in Section 5.6) and external knowledge sources, such as documents obtained through retrieval-augmented generation (RAG) methods [173], [174]. This supplemental information can provide valuable insights that further refine the agent's decision-making processes.

The construction of an effective prompt is foundational for the performance of LLM-powered GUI agents. By systematically incorporating aforementioned information, the agent ensures that the LLM is equipped with the necessary context and guidance to execute tasks accurately and efficiently.

5.4 Model Inference

The constructed prompt is submitted to the LLM for inference, where the LLM is tasked with generating both a plan and the specific actions required to execute the user's request. This inference process is critical as it dictates how effectively the GUI agent will perform in dynamic environments. It typically involves two main components: planning and action inference, as well as the generation of complementary outputs. Figure 12 shows an example of the LLM's inference output.

5.4.1 Planning

Successful execution of GUI tasks often necessitates a series of sequential actions, requiring the agent to engage in effective planning [175]. Analogous to human cognitive processes, thoughtful planning is essential to organize tasks, schedule actions, and ensure successful completion [49], [176]. The LLM must initially conceptualize a long-term goal while simultaneously focusing on short-term actions to initiate progress toward that goal [177].

To effectively navigate the complexity of multi-step tasks, the agent should decompose the overarching task into manageable subtasks and establish a timeline for their execution [178]. Techniques such as CoT reasoning [87] can be employed, enabling the LLM to develop a structured plan that guides the execution of actions. This plan, which can be stored for reference during future inference steps, enhances the organization and focus of the agent's activities.

The granularity of planning may vary based on the nature of the task and the role of the agent [49]. For complex tasks, a hierarchical approach that combines global planning (identifying broad subgoals) with local planning (defining detailed steps for those subgoals) can significantly improve

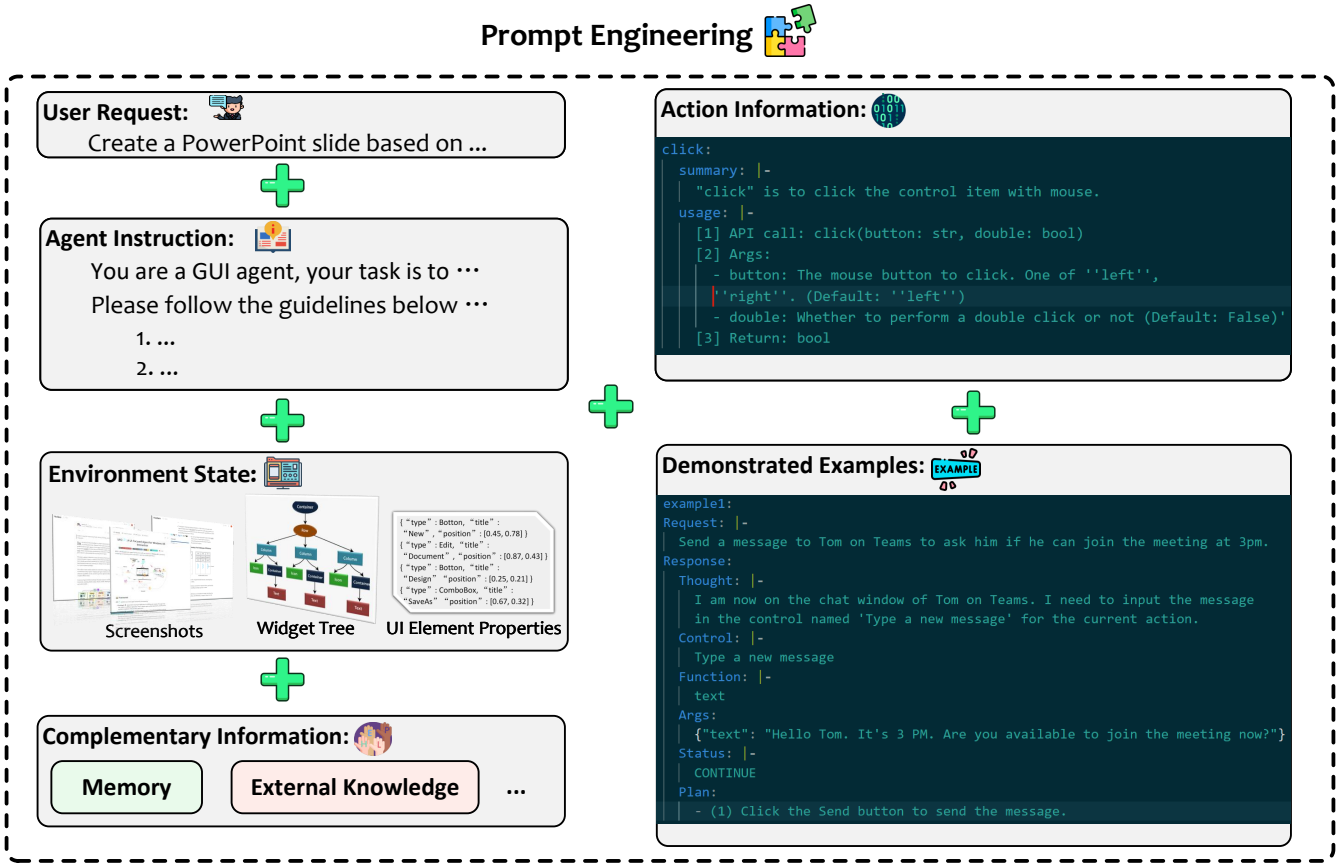


Fig. 11: A basic example of prompt construction in a LLM-brained GUI agent.

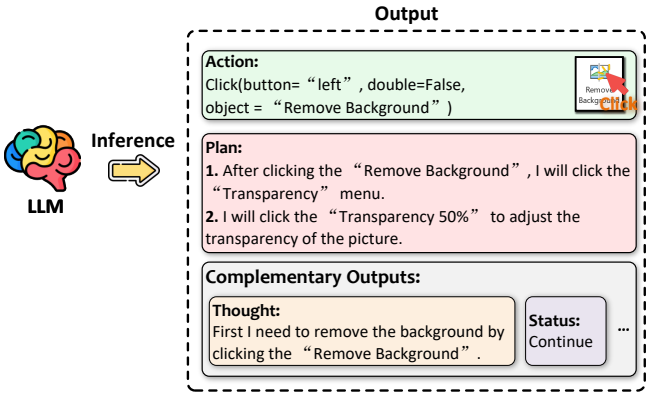


Fig. 12: An example of the LLM's inference output in a GUI agent.

the agent's ability to manage long-term objectives effectively [179].

5.4.2 *Action Inference*

Action inference is the core objective of the inference stage, as it translates the planning into executable tasks. The inferred actions are typically expressed as function call strings, encompassing the function name and relevant parameters. These strings can be readily converted into real-world interactions with the environment, such as clicks, keyboard inputs, mobile

10. https://developer.apple.com/library/archive/documentation/AppleScript/Conceptual/AppleScriptLangGuide/introduction/ASLR_intro.html
11. https://www.macosxautomation.com/automator/
12. https://docs.blender.org/manual/en/latest/sculpt_paint/sculpting/introduction/gesture_tools.html
13. https://developer.android.com/reference/android/speech/SpeechRecognizer
14. https://developer.apple.com/documentation/sirikit/
15. https://pypi.org/project/pyperclip/
16. https://clipboardjs.com/
17. https://developer.android.com/develop/sensors-and-location/sensors/sensors_overview
18. https://learn.microsoft.com/en-us/previous-versions/office/office-365-api/
19. https://developer.android.com/reference
20. https://developer.apple.com/ios/
21. https://learn.microsoft.com/en-us/windows/win32/api/
22. https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/CocoaFundamentals/WhatIsCocoa/WhatIsCocoa.html
23. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
24. https://axios-http.com/docs/api_intro
25. https://platform.openai.com/docs/overview

TABLE 4: Overview of actions for GUI agents.

Action	Category	Original Executor	Examples	Platform	Environment	Toolkit
Mouse actions	UI Operations	Mouse	Click, scroll, hover, drag	Computer	Windows	UI Automation 6, Pywinauto [180]
Mouse actions	UI Operations	Mouse	Click, scroll, hover, drag	Computer	macOS	AppleScript [10], Automator [11]
Mouse actions	UI Operations	Mouse	Click, scroll, hover, drag	Web	Browser	Selenium, Puppeteer
Keyboard actions	UI Operations	Keyboard	Typing, key presses, shortcuts	Computer	Windows	UI Automation 6, Pywinauto [180]
Keyboard actions	UI Operations	Keyboard	Typing, key presses, shortcuts	Computer	macOS	AppleScript [10], Automator [11]
Keyboard actions	UI Operations	Keyboard	Typing, key presses, shortcuts	Web	Browser	Selenium, Puppeteer
Touch actions	UI Operations	Touchscreen	Tap, swipe, pinch, zoom	Mobile	Android	Appium, UIAutomator
Touch actions	UI Operations	Touchscreen	Tap, swipe, pinch, zoom	Mobile	iOS	Appium, XCUITest
Gesture actions	UI Operations	User hand	Rotate, multi-finger gestures	Mobile	Android, iOS	Appium , GestureTools [12]
Voice commands	UI Operations	User voice	Speech input, voice commands	Mobile	Android	SpeechRecognizer [13]
Voice commands	UI Operations	User voice	Speech input, voice commands	Mobile	iOS	SiriKit [14]
Clipboard operations	UI Operations	System clipboard	Copy, paste	Cross-platform	Cross-OS	Pyperclip [15], Clipboard.js [16]
Screen interactions	UI Operations	User	Screen rotation, shake	Mobile	Android, iOS	Device sensors APIs [17]
Shell Commands	Native API Calls	Command Line Interface	File manipulation, system operations, script execution	Computer	Unix/Linux, macOS	Bash, Terminal
Application APIs	Native API Calls	Application APIs	Send email, create document, fetch data	Computer	Windows	Microsoft Office COM APIs [18]
Application APIs	Native API Calls	Application APIs	Access calendar, send messages	Mobile	Android	Android SDK APIs [19]
Application APIs	Native API Calls	Application APIs	Access calendar, send messages	Mobile	iOS	iOS SDK APIs [20]
System APIs	Native API Calls	System APIs	File operations, network requests	Computer	Windows	Win32 API [21]
System APIs	Native API Calls	System APIs	File operations, network requests	Computer	macOS	Cocoa APIs [22]
Web APIs	Native API Calls	Web Services	Fetch data, submit forms	Web	Browser	Fetch API [23] , Axios [24]
AI Models	AI Tools	AI Models	Screen understanding, summarization, image generation	Cross-platform	Cross-OS	DALL·E [181] , OpenAI APIs [26]

gestures, or API calls. A detailed discussion of these action types is presented in Section 5.5.

The input prompt must include a predefined set of actions available for the agent to select from. The agent can choose an action from this set or, if allowed, generate custom code or API calls to interact with the environment [148]. This flexibility can enhance the agent's adaptability to unforeseen circumstances; however, it may introduce reliability concerns, as the generated code may be prone to errors.

5.4.3 Complementary Outputs

In addition to planning and action inference, the LLM can also generate complementary outputs that enhance the agent's capabilities. These outputs may include reasoning processes that clarify the agent's decision-making (*e.g.*, CoT reasoning), messages for user interaction, or communication with other agents or systems, or the status of the task (*e.g.*, continue or finished). The design of these functionalities can be tailored to meet specific needs, thereby enriching the overall performance of the GUI agent.

By effectively balancing planning and action inference while incorporating complementary outputs, agents can navigate complex tasks with a higher degree of organization and adaptability.

5.5 Actions Execution

Following the inference process, a crucial next step is for the GUI agent to execute the actions derived from the inferred commands within the GUI environment and subsequently gather feedback. Although the term "GUI agent" might suggest a focus solely on user interface actions, the action space can be greatly expanded by incorporating various toolboxes that enhance the agent's versatility. Broadly, the actions available to GUI agents fall into three main categories: *(i)* UI operations [131], *(ii)* native API calls [182], and *(iii)* AI tools [183]. Each category offers unique advantages and challenges, enabling the agent to tackle a diverse range of tasks more effectively. We summarize the various actions commonly used in GUI agents, categorized into distinct types, in Table 4, and provide detailed explanations of each category below.

5.5.1 UI Operations

UI operations encompass the fundamental interactions that users typically perform with GUIs in software applications. These operations include various forms of input, such as mouse actions (clicks, drags, hovers), keyboard actions (key presses, combinations), touch actions (taps, swipes), and gestures (pinching, rotating). The specifics of these actions may differ across platforms and applications, necessitating a tailored approach for each environment.

While UI operations form the foundation of agent interactions with the GUI, they can be relatively slow due to the sequential nature of these tasks. Each operation must be executed step by step, which can lead to increased latency, especially for complex workflows that involve numerous interactions. Despite this drawback, UI operations are crucial for maintaining a broad compatibility across various applications, as they leverage standard user interface elements and interactions.

5.5.2 Native API Calls

In contrast to UI operations, some applications provide native APIs that allow GUI agents to perform actions more efficiently. These APIs offer direct access to specific functionalities within the application, enabling the agent to execute complex tasks with a single command [184]. For instance, calling the Outlook API allows an agent to send an email in one operation, whereas using UI operations would require a series of steps, such as navigating through menus and filling out forms [185].

While native APIs can significantly enhance the speed and reliability of action execution, their availability is limited. Not all applications or platforms expose APIs for external use, and developing these interfaces can require substantial effort and expertise. Consequently, while native APIs present a powerful means for efficient task completion, they may not be as generalized across different applications as UI operations.

5.5.3 AI Tools

The integration of AI tools into GUI agents represents a transformative advancement in their capabilities. These tools can assist with a wide range of tasks, including content summarization from screenshots or text, document enhancement, image or video generation (*e.g.*, calling ChatGPT [11], DALL·E [181]), and even invoking other agents or Copilot tools for collaborative assistance. The rapid development of generative AI technologies enables GUI agents to tackle complex challenges that were previously beyond their capabilities.

By incorporating AI tools, agents can extend their functionality and enhance their performance in diverse contexts. For example, a GUI agent could use an AI summarization tool to quickly extract key information from a lengthy document or leverage an image generation tool to create custom visuals for user presentations. This integration not only streamlines workflows but also empowers agents to deliver high-quality outcomes in a fraction of the time traditionally required.

5.5.4 Summary

An advanced GUI agent should adeptly leverage all three categories of actions: UI operations for broad compatibility, native APIs for efficient execution, and AI tools for enhanced capabilities. This multifaceted approach enables the agent to operate reliably across various applications while maximizing efficiency and effectiveness. By skillfully navigating these action types, GUI agents can fulfill user requests more proficiently, ultimately leading to a more seamless and productive user experience.

5.6 Memory

For a GUI agent to achieve robust performance in complex, multi-step tasks, it must retain memory, enabling it to manage states in otherwise stateless environments. Memory allows the agent to track its prior actions, their outcomes, and the task's overall status, all of which are crucial for informed decision-making in subsequent steps [186]. By establishing continuity, memory transforms the agent from a reactive system into a proactive, stateful one, capable of self-adjustment based on accumulated knowledge. The agent's memory is generally divided into two main types: Short-Term Memory [187] and Long-Term Memory [188]. We show an overview of different types of memory in GUI agents in Table 5.

5.6.1 Short-Term Memory

Short-Term Memory (STM) provides the primary, ephemeral context used by the LLM during runtime [189]. STM stores information pertinent to the current task, such as recent plans, actions, results, and environmental states, and continuously updates to reflect the task's ongoing status. This memory is particularly valuable in multi-step tasks, where each decision builds on the previous one, requiring the agent to maintain a clear understanding of the task's trajectory. As illustrated in Figure 13, during the completion of independent tasks, the task trajectory, comprising actions and plans—is stored in the STM. This allows the agent to track task progress effectively and make more informed decisions.

However, STM is constrained by the LLM's context window, limiting the amount of information it can carry forward. To manage this limitation, agents can employ selective memory management strategies, such as selectively discarding or summarizing less relevant details to prioritize the most impactful information. Despite its limited size, STM is essential for ensuring coherent, contextually aware interactions and supporting the agent's capacity to execute complex workflows with immediate, relevant feedback.

5.6.2 Long-Term Memory

Long-Term Memory (LTM) serves as an external storage repository for contextual information that extends beyond the immediate runtime [190]. Unlike STM, which is transient, LTM retains historical task data, including previously completed tasks, successful action sequences, contextual tips, and learned insights. LTM can be stored on disk or in a database, enabling it to retain larger volumes of information than what is feasible within the LLM's immediate context window. In the example shown in Figure 13, when the second task requests downloading a game related to the previous task, the agent retrieves relevant information from its LTM. This enables the agent to accurately identify the correct game, facilitating efficient task completion.

LTM contributes to the agent's self-improvement over time by preserving examples of successful task trajectories, operational guidelines, and common interaction patterns.

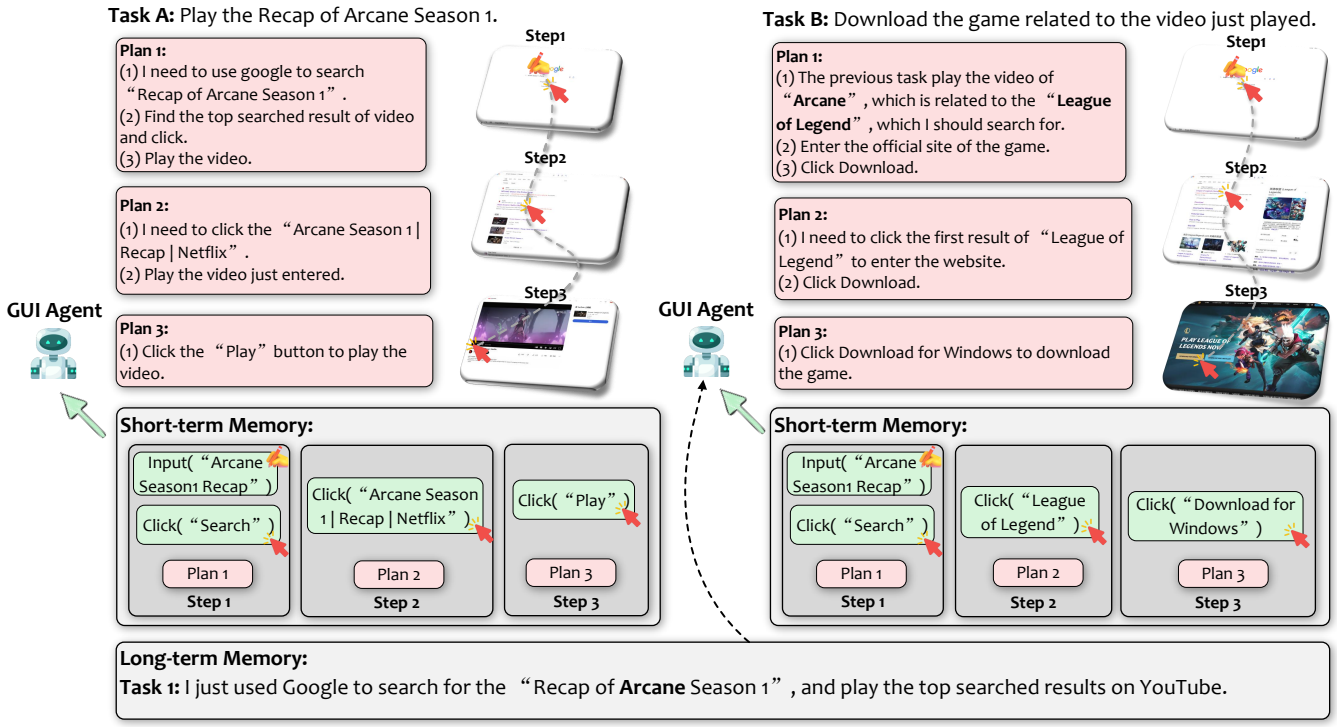


Fig. 13: Illustration of short-term memory and long-term memory in an LLM-brained GUI agent.

TABLE 5: Summary of memory in GUI agents.

Memory Element	Memory Type	Description	Storage Medium/Method
Action	Short-term	Historical actions trajectory taken in the environment	In-memory, Context window
Plan	Short-term	Plan passed from previous step	In-memory, Context window
Execution Results	Short-term	Return values, error traces, and other environmental feedback	In-memory, Context window
Environment State	Short-term	Important environment state data, e.g., UI elements	In-memory, Context window
Self-experience	Long-term	Task completion trajectories from historical tasks	Database, Disk
Self-guidance	Long-term	Guidance and rules summarized from historical trajectories	Database, Disk
External Knowledge	Long-term	Other external knowledge sources aiding task completion	External Knowledge Base
Task Success Metrics	Long-term	Metrics from task success or failure rates across sessions	Database, Disk

When approaching a new task, the agent can leverage RAG techniques to retrieve relevant historical data, which enhances its ability to adapt strategies based on prior success. This makes LTM instrumental in fostering an agent's capacity to "learn" from experience, enabling it to perform tasks with greater accuracy and efficiency as it accumulates insights across sessions. For instance, [191] provides an illustrative example of using past task trajectories stored in memory to guide and enhance future decision-making, a technique that is highly adaptable for GUI agents. It also enables better personalization by retaining information about previous tasks.

5.7 Advanced Enhancements

While most LLM-brained GUI agents incorporate fundamental components such as perception, planning, action execution, and memory, several advanced techniques have been developed to significantly improve the reasoning and overall capabilities of these agents. Here, we outline shared advancements widely adopted in research to guide the development of more specialized and capable LLM-brained GUI agents.

5.7.1 Computer Vision-Based GUI Parsing

Although various tools (Section 3) enable GUI agents to access information like widget location, captions, and properties, certain non-standard GUIs or widgets may not adhere to these tools' protocols [192], rendering their information inaccessible. Additionally, due to permission management, these tools are not always usable. Such incomplete information can present significant challenges for GUI agents, as the LLM may need to independently locate and interact with required widgets by estimating their coordinates to perform actions like clicking—a task that is inherently difficult without precise GUI data.

CV models offer a non-intrusive solution for parsing GUIs directly from screenshots, enabling the detection, localization, segmentation, and even functional estimation of widgets [90], [193]–[195]. This approach allows agents to interpret the visual structure and elements of the GUI without relying on system-level tools or internal metadata, which may be unavailable or incomplete. CV-based GUI parsing provides agents with valuable insights into interactive components, screen layout, and widget functionalities based solely on visual cues, enhancing their ability to recognize and act upon elements on the screen. Figure 9 provides an illustrative example of

how a CV-based GUI parser works. While standard API-based detection captures predefined widgets, the CV model can identify additional elements, such as thumbnails and canvases, which may not have explicit API representations in the PowerPoint interface. This enhances widget recognition, allowing the agent to detect components beyond the scope of API detection.

A notable example is OmniParser [169], which implements a multi-stage parsing technique involving a fine-tuned model for detecting interactable icons, an OCR module for extracting text, and an icon description model that generates localized semantic descriptions for each UI element. By integrating these components, OmniParser constructs a structured representation of the GUI, enhancing an agent's understanding of interactive regions and functional elements. This comprehensive parsing strategy has shown to significantly improve GPT-4V's screen comprehension and interaction accuracy.

Such CV-based GUI parsing layers provide essential information that greatly improves the agent's ability to interact intuitively and accurately with diverse GUIs, especially in cases involving custom or non-standard elements that do not conform to typical accessibility protocols. This approach paves the way for more adaptable and resilient GUI agents, able to operate effectively in a wider variety of screen environments.

5.7.2 Multi-Agent Framework

The adage "two heads are better than one" holds particular relevance for GUI automation tasks, where a single agent, though capable, can be significantly enhanced within a multi-agent framework [196], [197]. Multi-agent systems leverage the collective intelligence, specialized skills, and complementary strengths of multiple agents to tackle complex tasks more effectively than any individual agent could alone. In the context of GUI agents, multi-agent systems offer advanced capabilities through two primary mechanisms: *(i)* specialization and *(ii)* inter-agent collaboration. Figure 14 illustrates an example of how an LLM-powered multi-agent collaborates to create a desk.

1) **Specialization of Agents:** In a multi-agent framework, each agent is designed to specialize in a specific role or function, leveraging its unique capabilities to contribute to the overall task. As illustrated in the Figure 14, specialization enables distinct agents to focus on different aspects of the task pipeline. For instance, the "Document Extractor" specializes in extracting relevant content from local documents, such as PDFs, while the "Web Retriever" focuses on gathering additional information from online sources. Similarly, the "Designer" transforms the retrieved information into visually appealing slides, and the "Evaluator" provides feedback to refine and improve the output. This functional separation ensures that each agent becomes highly adept at its designated task, leading to improved efficiency and quality of results [198].
2) **Collaborative Inter-Agent Dynamics:** The multi-agent system shown in the Figure 14 exemplifies how agents collaborate dynamically to handle complex tasks. The process begins with the "Document Extractor" and "Web Retriever", which work in parallel to collect information from local and online sources. The retrieved data is communicated to the "Designer", who synthesizes it into a cohesive set of slides. Once the slides are created, the "Evaluator" reviews the output, providing feedback for refinement. These agents share information, exchange context, and operate in a coordinated manner, reflecting a human-like teamwork dynamic. For example, as depicted, the agents' roles are tightly integrated—each output feeds into the next stage, creating a streamlined workflow that mirrors real-world collaborative environments [17].

In such a system, agents can collectively engage in tasks requiring planning, discussion, and decision-making. Through these interactions, the system taps into each agent's domain expertise and latent potential for specialization, maximizing overall performance across diverse, multi-step processes.

5.7.3 Self-Reflection

"A fault confessed is half redressed". In the context of GUI multi-agent systems, self-reflection refers to the agents' capacity to introspectively assess their reasoning, actions, and decisions throughout the task execution process [199]. This capability allows agents to detect potential mistakes, adjust strategies, and refine actions, thereby improving the quality and robustness of their decisions, especially in complex or unfamiliar GUI environments. By periodically evaluating their own performance, self-reflective agents can adapt dynamically to produce more accurate and effective results [200].

Self-reflection is particularly critical for GUI agents due to the variable nature of user interfaces and the potential for errors, even in human-operated systems. GUI agents frequently encounter situations that deviate from expectations, such as clicking the wrong button, encountering unexpected advertisements, navigating unfamiliar interfaces, receiving error messages from API calls, or even responding to user feedback on task outcomes. To ensure task success, a GUI agent must quickly reflect on its actions, assess these feedback signals, and adjust its plans to better align with the desired objectives.

As illustrated in Figure 15, when the agent initially fails to locate the "Line Drawing" option in the Design menu, self-reflection enables it to reconsider and identify its correct location under Artistic Effects" in the "Picture Format" menu, thereby successfully completing the task.

In practice, self-reflection techniques for GUI agents typically involve two main approaches: *(i)* **ReAct** [201] and *(ii)* **Reflexion** [202].

1) **ReAct (Reasoning and Acting):** ReAct integrates self-reflection into the agent's action chain by having the agent evaluate each action's outcome and reason about the next best step. In this framework, the agent doesn't simply follow a linear sequence of actions; instead, it adapts dynamically, continuously reassessing its strategy in response to feedback from each action. For example, if a GUI agent attempting to fill a form realizes it has clicked the wrong field, it can adjust by backtracking and selecting the correct element. Through ReAct, the agent achieves higher consistency and accuracy, as it learns to refine its behavior with each completed step.
2) **Reflexion:** Reflexion emphasizes language-based feedback, where agents receive and process feedback from

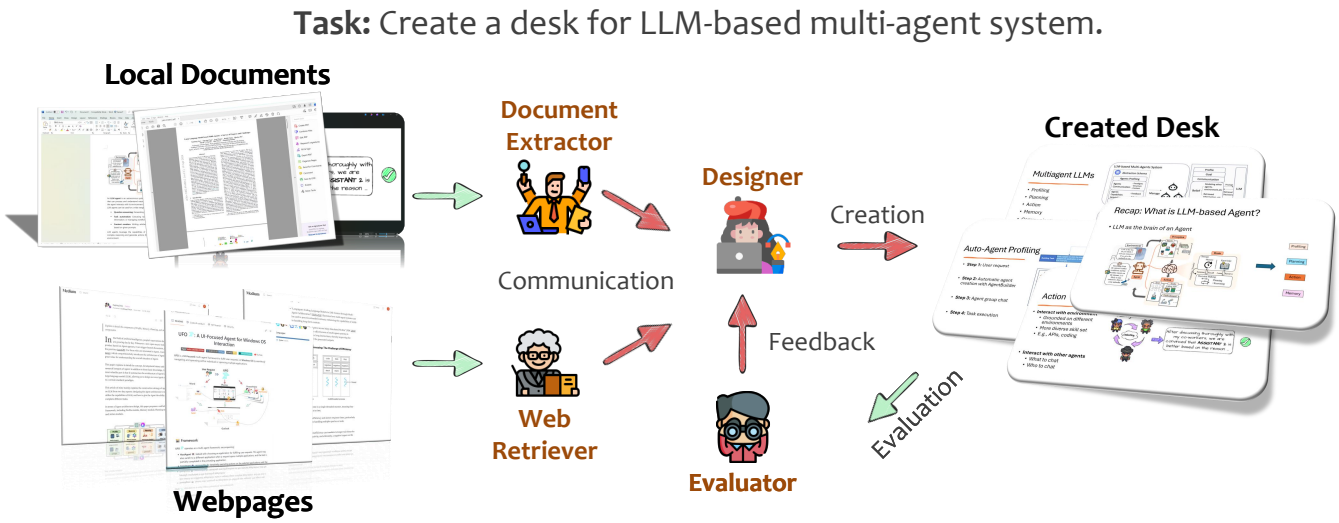


Fig. 14: An example of multi-agent system collaboration in creating a desk.

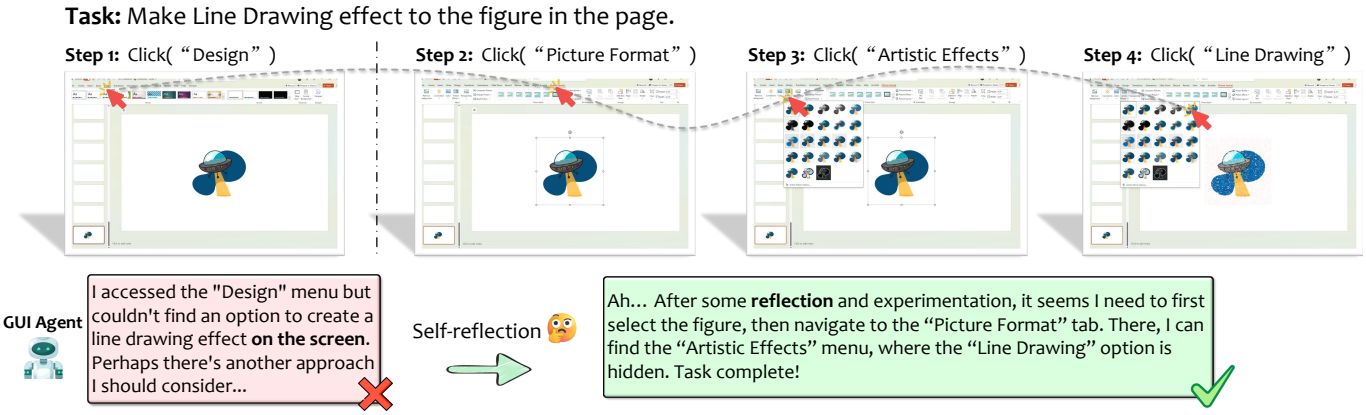


Fig. 15: An example of self-reflection in task completion of an LLM-powered GUI agent.

the environment as linguistic input, referred to as self-reflective feedback. This feedback is contextualized and used as input in subsequent interactions, helping the agent to learn rapidly from prior mistakes. For instance, if a GUI agent receives an error message from an application, Reflexion enables the agent to process this message, update its understanding of the interface, and avoid similar mistakes in future interactions. Reflexion's iterative feedback loop promotes continuous improvement and is particularly valuable for GUI agents navigating complex, multi-step tasks.

Overall, self-reflection serves as an essential enhancement in GUI multi-agent systems, enabling agents to better navigate the variability and unpredictability of GUI environments. This introspective capability not only boosts individual agent performance but also promotes resilience, adaptability, and long-term learning in a collaborative setting.

5.7.4 Self-Evolution

Self-evolution [203] is a crucial attribute that GUI agents should possess, enabling them to enhance their performance progressively through accumulated experience. In the context of GUI multi-agent systems, self-evolution allows not only individual agents to improve but also facilitates collective learning and adaptation by sharing knowledge and strategies among agents. During task execution, GUI agents generate detailed action trajectories accompanied by complementary information such as environment states, internal reasoning processes (the agent's thought processes), and evaluation results. This rich data serves as a valuable knowledge base from which GUI agents can learn and evolve. The knowledge extracted from this experience can be categorized into three main areas:

1) **Task Trajectories**: The sequences of actions executed by agents, along with the corresponding environment states, are instrumental for learning [204]. These successful trajectories can be leveraged in two significant ways. First, they can be used to fine-tune the core LLMs that underpin the agents. Fine-tuning with such domain-specific and task-relevant data enhances the model's ability to generalize and improves performance on similar tasks in the future. Second, these trajectories can be utilized as demonstration examples to activate the in-context learning capabilities of LLMs during prompt engineering. By including examples of successful task executions in the prompts, agents can better understand

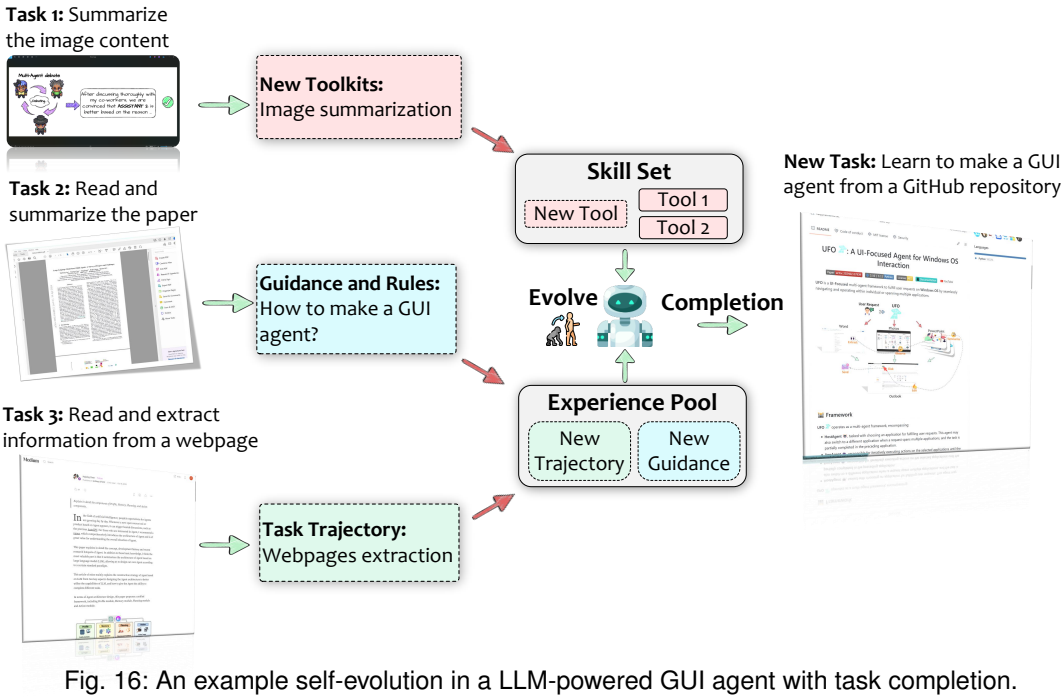


Fig. 16: An example self-evolution in a LLM-powered GUI agent with task completion.

and replicate the desired behaviors without additional model training.

For instance, suppose an agent successfully completes a complex task that involves automating data entry across multiple applications. The recorded action trajectory—comprising the steps taken, decisions made, and contextual cues—can be shared with other agents. These agents can then use this trajectory as a guide when faced with similar tasks, reducing the learning curve and improving efficiency.

2) **Guidance and Rules:** From the accumulated experiences, agents can extract high-level rules or guidelines that encapsulate best practices, successful strategies, and lessons learned from past mistakes [205], [206]. This knowledge can be formalized into policies or heuristics that agents consult during decision-making processes, thereby enhancing their reasoning capabilities.

   For example, if agents repeatedly encounter errors when attempting to perform certain actions without proper prerequisites (*e.g.*, trying to save a file before specifying a file path), they can formulate a rule to check for these prerequisites before executing the action. This proactive approach reduces the likelihood of errors and improves task success rates.

3) **New Toolkits:** Throughout their interactions, GUI agents may discover or develop more efficient methods, tools, or sequences of actions that streamline task execution [148]. These may include optimized API calls, macros, or combinations of UI operations that accomplish tasks more effectively than previous approaches. By incorporating these new tools into their repertoire, agents expand their capabilities and enhance overall efficiency.

   As an example, an agent might find that using a batch processing API can automate repetitive tasks more efficiently than performing individual UI operations in a loop. This new approach can be shared among agents within the multi-agent system, allowing all agents to benefit from the improved method and apply it to relevant tasks.

Figure 16 illustrates how a GUI agent evolves through task completion. During its operations, the agent adds new capabilities to its skill set, such as an image summarization toolkit, gains insights from reading a paper on creating GUI agents, and stores task trajectories like webpage extraction in its experience pool. When assigned a new task, such as "Learn to make a GUI agent from a GitHub repository", the agent draws on its acquired skills and past experiences to adapt and perform effectively.

This dynamic evolution highlights the agent's ability to continually learn, grow, and refine its capabilities. By leveraging past experiences, incorporating new knowledge, and expanding its toolset, GUI agents can adapt to diverse challenges, improve task execution, and significantly enhance the overall performance of the system, fostering a collaborative and ever-improving environment.

5.7.5 Reinforcement Learning

Reinforcement Learning (RL) [207] has witnessed significant advancements in aligning LLMs with desired behaviors [208], and has recently been employed in the development of LLM agents [48], [209], [209]. In the context of GUI multi-agent systems, RL offers substantial potential to enhance the performance, adaptability, and collaboration of GUI agents. GUI automation tasks naturally align with the structure of a Markov Decision Process (MDP) [210], making them particularly well-suited for solutions based on RL. In this context, the *state* corresponds to the environment perception (such as GUI screenshots, UI element properties, and layout configurations), while *actions* map directly to UI operations, including mouse clicks, keyboard inputs, and API calls. *Rewards* can be explicitly defined based on various performance metrics,

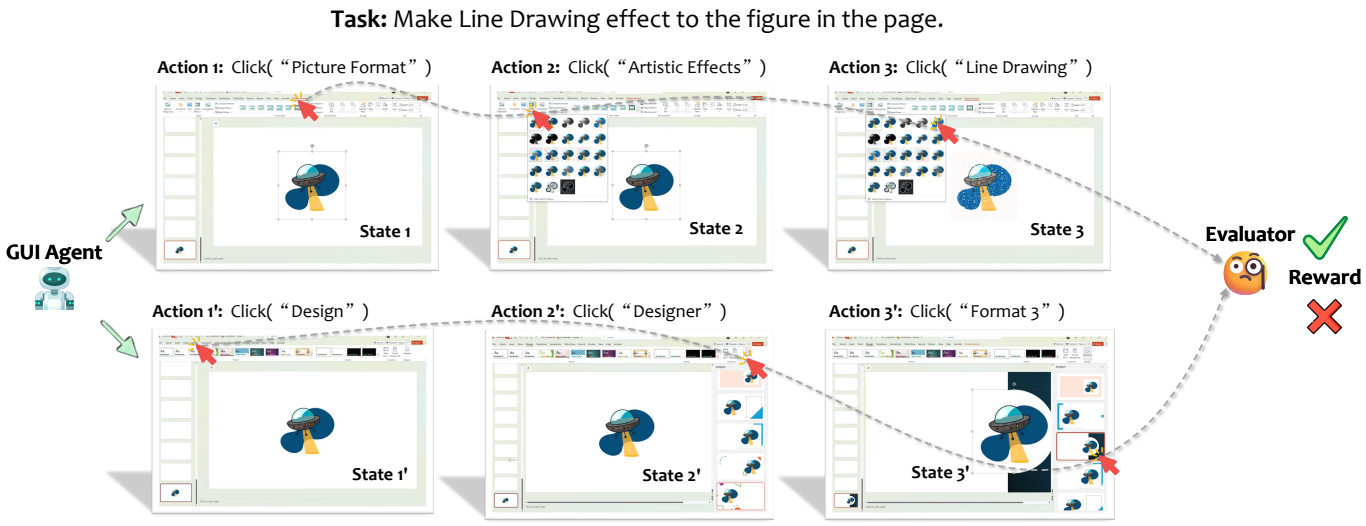


Fig. 17: An example of MDP modeling for task completion in a GUI agent.

such as task completion, efficiency, and accuracy, allowing the agent to optimize its actions for maximal effectiveness. Figure 17 illustrates an example of MDP modeling for task completion in a GUI agent, where state, action and reward are clearly defined.

By formulating GUI agent interactions as an MDP, we can leverage RL techniques to train agents that learn optimal policies for task execution through trial and error [211]. This approach enables agents to make decisions that maximize cumulative rewards over time, leading to more efficient and effective task completion. For example, an agent learning to automate form filling in a web application can use RL to discover the most efficient sequence of actions to input data and submit the form successfully, minimizing errors and redundant steps. This process helps align the agents more closely with desired behaviors in GUI automation tasks, especially in complex or ambiguous situations where predefined action sequences are insufficient.

As a representative approach, Bai *et al.*, introduce DigiRL [212], a two-phase RL framework for training GUI agents in dynamic environments. DigiRL begins with an offline RL phase that uses offline data to initialize the agent model, followed by online fine-tuning, where the model interacts directly with an environment to refine its strategies through live data within an Android learning environment using an LLM evaluator that provides reliable reward signals. This adaptive setting enables the agent to learn and respond effectively to the complexities of dynamic GUIs. Wang *et al.*, propose DistRL [213], an RL fine-tuning pipeline specifically designed for on-device mobile control agents operating within Android. DistRL employs an asynchronous architecture, deploying RL fine-tuned agents across heterogeneous worker devices and environments for decentralized data collection. By leveraging off-policy RL techniques, DistRL enables centralized training with data gathered remotely from diverse environments, significantly enhancing the scalability and robustness of the model. These representative methods illustrate the potential of RL to improve GUI agents, demonstrating how both centralized and distributed RL frameworks can enable more responsive, adaptable, and effective GUI automation models in real-world applications.

5.7.6 Summary & Takeaways

In conclusion, the advanced techniques significantly enhance the capabilities of LLM-brained GUI agents, making them more versatile, efficient, and adaptive within multi-agent frameworks. Importantly, these techniques are not mutually exclusive—many can be integrated to create more powerful agents. For instance, incorporating self-reflection within a multi-agent framework allows agents to collaboratively improve task strategies and recover from errors. By leveraging these advancements, developers can design LLM-brained GUI agents that are not only adept at automating complex, multi-step tasks but also capable of continuously improving through self-evolution, adaptability to dynamic environments, and effective inter-agent collaboration. Future research is expected to yield even more sophisticated techniques, further extending the scope and robustness of GUI automation.

5.8 From Foundations to Innovations: A Roadmap

Building robust, adaptable, and effective LLM-powered GUI agents is a multifaceted process that requires careful integration of several core components. With a solid foundation in architecture, design, environment interaction, and memory, as outlined in Section 5, we now shift our focus to the critical elements required for deploying these agents in practical scenarios. This exploration begins with an in-depth review of state-of-the-art LLM-brained GUI agent frameworks in Section 6, highlighting their advancements and unique contributions to the field. Building on this, we delve into the methodologies for optimizing LLMs for GUI agents, starting with data collection and processing strategies in Section 7, and progressing to model optimization techniques in Section 8. To ensure robust development and validation, we then examine evaluation methodologies and benchmarks in Section 9, which are essential for assessing agent performance and reliability. Finally, we explore a diverse range of practical applications in

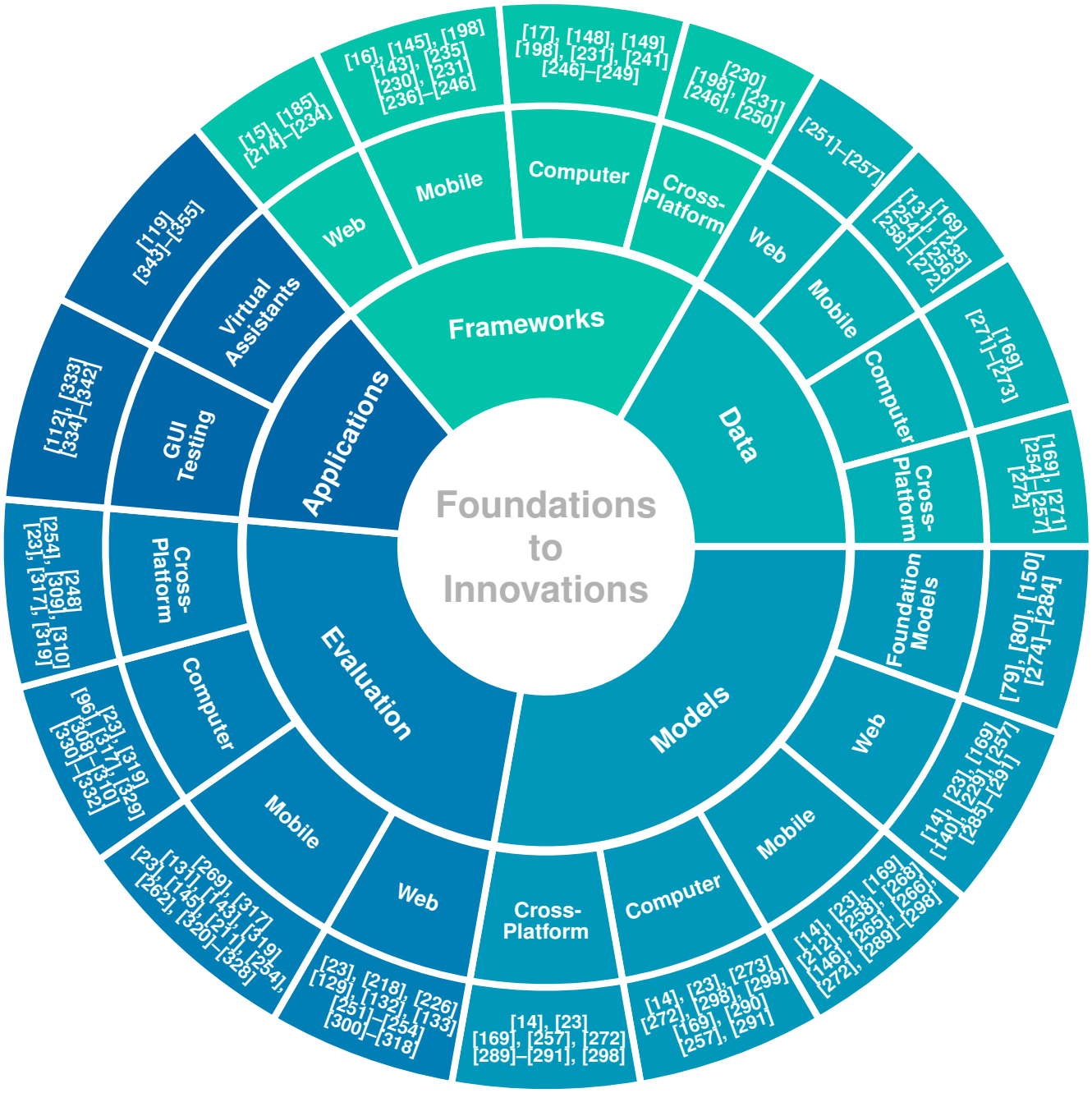


Fig. 18: A Taxonomy of frameworks, data, models, evaluations, and applications: from foundations to innovations in LLM-brained GUI agents.

Section 10, demonstrating the transformative impact of these agents across various domains.

Together, these sections provide a comprehensive roadmap for advancing LLM-brained GUI agents from foundational concepts to real-world implementation and innovation. This roadmap, spanning from foundational components to real-world deployment, encapsulates the essential pipeline required to bring an LLM-powered GUI agent concept from ideation to implementation.

To provide a comprehensive view, we first introduce a taxonomy in Figure 18, which categorizes recent work on LLM-brained GUI agents across frameworks, data, models, evaluation, and applications. This taxonomy serves as a blueprint for navigating the extensive research and development efforts within each field, while acknowledging overlaps among categories where certain models, frameworks, or datasets contribute to multiple aspects of GUI agent functionality.

6 LLM-Brained GUI Agent Framework

The integration of LLMs has unlocked new possibilities for constructing GUI agents, enabling them to interpret user requests, analyze GUI components, and autonomously perform actions across diverse environments. By equipping these models with essential components and functionalities, as

TABLE 6: Overview of LLM-brained GUI agent frameworks (Part I).

Agent	Platform	Perception	Action	Model	Architecture	Highlight	Link
WMA [214]	Web	Accessibility tree from DOM	UI operations, *e.g.*, clock, type, and hover	Llama-3.1-8B-Instruct [78] for predicting observations and GPT-4 for policy modeling	Single-agent with simulation-based observation	Uses a world model to predict state changes before committing actions, improving task success rates and minimizing unnecessary interactions with the environment	https://github.com/kyle8581/WMA-Agents
WebAgent [215]	Web	HTML structure	UI interactions	HTML-T5 for task planning and summarization and Flan-U-PaLM [356] for code generation	Two-stage architecture for planning and program synthesis	Leverages specialized LLMs to achieve HTML-based task planning and programmatic action execution	/
LASER [216]	Web	GUI structure of the web environment, with defined states	Defined per state, such as searching, selecting items, navigating pages, and finalizing a purchase	GPT-4	Single-agent	Uses a state-space exploration approach, allowing it to handle novel situations with flexible backtracking	https://github.com/Mayer123/LASER
WebVoyager [217]	Web	Screenshots with numerical labels on interactive elements	Standard UI operations	GPT-4V	Single-agent	Integrates visual and textual cues within real-world, rendered web pages, enhancing its ability to navigate complex web structures	https://github.com/MinorJerry/WebVoyager
AutoWebGLM [218]	Web	Simplified HTML and OCR for text recognition	UI operations such as clicking, typing, scrolling, and selecting, and advanced APIs like jumping to specific URLs	ChatGLM3-6B [357]	Single-agent	Its HTML simplification method for efficient webpage comprehension and its bilingual benchmark	https://github.com/THUDM/AutoWebGLM
OpenAgents [219]	Web	DOM elements	Standard UI operations, browser-based actions controlled, API calls for tool execution, and structured data manipulation	GPT-4 and Claude [150]	Multi-agent architecture, with distinct agents (Data Agent, Plugins Agent, and Web Agent)	Democratizes access to language agents by providing an open-source, multi-agent framework optimized for real-world tasks	https://github.com/xlang-ai/OpenAgents
SeeAct [15]	Web	Screenshot images and HTML structure	Standard UI operations	GPT-4V	Single-agent	Its use of GPT-4V's multimodal capabilities to integrate both visual and HTML information, allowing for more accurate task performance on dynamic web content	https://github.com/OSU-NLP-Group/SeeAct
DUAL-VCR [220]	Web	HTML elements and screenshots	Standard UI operations	Flan-T5-base [356]	Two-stage single-agent architecture	Dual-view contextualization	/
Agent-E [221]	Web	DOM structure and change observation	Standard UI operations	GPT-4 Turbo	Hierarchical multi-agent architecture, composed of a planner agent and a browser navigation agent	Hierarchical architecture and adaptive DOM perception	https://github.com/EmergenceAI/Agent-E
Search-Agent [222]	Web	Screenshot and text descriptions	Standard UI operations	GPT-4	Single-agent with search	Novel inference-time search algorithm that enhances the agent's ability to perform multi-step planning and decision-making	https://jykoh.com/search-agents

outlined in Section 5, researchers have created sophisticated frameworks tailored to various platforms and applications. These frameworks represent a rapidly evolving area of research, with each introducing innovative techniques and specialized capabilities that push the boundaries of what GUI agents can achieve.

To provide a comprehensive view of the field, we first summarize key frameworks across web, mobile, computer and cross-platform in Tables 6, 7, 8, and 9, highlighting their features, target platforms, and notable innovations. These summaries serve as an entry point to understanding the breadth of development in LLM-brained GUI agents. Following this overview, we offer a detailed discussion of each framework, examining their foundational design principles, technical advancements, and the specific challenges they address in the realm of GUI automation. By delving into these aspects, we aim to provide deeper insights into how these agents are shaping the future of human-computer interaction and task automation, and the critical role they play in advancing this transformative field.

6.1 Web GUI Agents

Advancements in web GUI agents have led to significant strides in automating complex tasks within diverse and dynamic web environments. Recent frameworks have introduced innovative approaches that leverage multimodal inputs, predictive modeling, and task-specific optimizations to enhance performance, adaptability, and efficiency. In this subsection, we delve into these frameworks, highlighting

TABLE 7: Overview of LLM-brained GUI agent frameworks (Part II).

Agent	Platform	Perception	Action	Model	Architecture	Highlight	Link
WebPilot [223]	Web	Accessibility trees (actrees) and dynamic observations	Standard UI operations	GPT-4	Multi-agent architecture, with Global Optimization and Local Optimization	Dual optimization strategy (Global and Local) with Monte Carlo Tree Search (MCTS) [358], allowing dynamic adaptation to complex, real-world web environments	https://yaoz720.github.io/WebPilot/
Hybrid Agent [185]	Web	Accessibility trees and screenshots	Standard UI operations, API calls, and generating code	GPT-4	Multi-agent system, combining both API and browsing capabilities	Hybrid Agent seamlessly integrates web browsing and API calls	https://github.com/yueqis/API-Based-Agent
AgentOccam [224]	Web	HTML	Standard UI operations	GPT-4	Single-agent	Simple design that optimizes the observation and action spaces	/
NNetnav [225]	Web	DOM	Standard UI operations	GPT-4	Single-agent	Trains web agents using synthetic demonstrations, eliminating the need for expensive human input	https://github.com/MurtyShikhar/Nnetnav
NaviQAte [226]	Web	Screenshots	Standard UI operations	GPT-4	Single-agent system	Frames web navigation as a question-and-answer task	/
OpenWeb-Agent [227]	Web	HTML and screenshots	UI operations, Web APIs, and self-generated code	GPT-4 and AutoWebGLM [218]	Modular single-agent	Modular design that allows developers to seamlessly integrate various models to automate web tasks	https://github.com/THUDM/OpenWebAgent/
Steward [228]	Web	HTML and screenshots	Standard UI operations	GPT-4	Single-agent	Ability to automate web interactions using natural language instructions	/
WebDreamer [232]	Web	Screenshots combined with SoM, and HTML	Standard UI operations and navigation actions	GPT-4o	Model-based single-agent architecture	Pioneers the use of LLMs as world models for planning in complex web environments	https://github.com/OSU-NLP-Group/WebDreamer
Agent Q [229]	Web	DOM for textual input, screenshots for visual feedback	UI interactions, querying the user for help	LLaMA-3 70B [78] for policy learning and execution, GPT-V for visual feedback	Single-agent with MCTS and RL	Combination of MCTS-guided search and self-critique mechanisms enables iterative improvement in reasoning and task execution	https://github.com/sentient-engineering/agent-q
Auto-Intent [234]	Web	HTML structure	Standard UI Operations	GPT-3.5, GPT-4, Llama-3 [78] for action inference; Mistral-7B [359] and Flan-T5XL [356] for intent prediction	Single-agent with self-exploration	Introduces a unique self-exploration strategy to generate semantically diverse intent hints	/
AdaptAgent [233]	Web	GUI screenshots with HTML/DOM structures	Standard UI Operations and Playwright scripts	GPT-4o and CogAgent [14]	Single-agent	Adapts to unseen tasks with just 1–2 multimodal human demonstrations	/
VisionTasker [236]	Android mobile devices	UI screenshots with widget detection and text extraction	UI operations such as tapping, swiping, and entering text	ERNIE Bot [360]	Single-agent with vision-based UI understanding and sequential task planning	Vision-based UI understanding approach, which allows it to interpret UI semantics directly from screenshots without view hierarchy dependencies	https://github.com/AkimotoAyako/VisionTasker
DroidBot-GPT [237]	Android mobile devices	Translates the GUI state information of Android applications into natural language prompts	UI operations, including actions like click, scroll, check, and edit	GPT	Single-agent	Automates Android applications without modifications to either the app or the model	https://github.com/MobileLLM/DroidBot-GPT
CoCo-Agent [238]	Android mobile devices	GUI screenshots, OCR layouts, and historical actions	GUI actions, such as clicking, scrolling, and typing	CLIP [361] for vision encoding and LLaMA-2-chat-7B for language processing	Single-agent	Its dual approach of Comprehensive Environment Perception and Conditional Action Prediction	https://github.com/xbmxb/CoCo-Agent
Auto-GUI [239]	Android mobile devices	GUI screenshots	GUI operations	BLIP-2 vision encoder [283] with a FLAN-Alpaca [65]	Single-agent with chain-of-action	Its direct interaction with GUI elements. Its chain-of-action mechanism enables it to leverage both past and planned actions	https://github.com/cooelf/Auto-GUI

their unique contributions and how they collectively push the boundaries of web-based GUI automation.

One prominent trend is the integration of multimodal capabilities to improve interaction with dynamic web content. For instance, **SeeAct** [15] harnesses GPT-4V's multimodal capacities to ground actions on live websites effectively. By leveraging both visual data and HTML structure, SeeAct integrates grounding techniques using image annotations, HTML attributes, and textual choices, optimizing interactions with real-time web content. This approach allows SeeAct to achieve a task success rate of 51.1% on real-time web tasks, highlighting the importance of dynamic evaluation in developing robust web agents.

Building upon the advantages of multimodal inputs, **We-**

TABLE 8: Overview of LLM-brained GUI agent frameworks (Part III).

Agent	Platform	Perception	Action	Model	Architecture	Highlight	Link
MM-Navigator [240]	Mobile iOS and Android	Smartphone screenshots with associated set-of-mark tags	Clickable UI operations	GPT-4V	Single-agent	Using set-of-mark prompting with GPT-4V for precise GUI navigation on smartphones	https://github.com/zzxslp/MM-Navigator
AppAgent [16]	Android mobile devices	Real-time screenshots and XML files detailing the interactive elements	User-like actions, like Tap, Long press, Swipe, Text input, Back and Exit	GPT-4V	Single-agent	Its ability to perform tasks on any smartphone app using a human-like interaction method	https://appagent-official.github.io/
AppAgent-V2 [241]	Android mobile devices	GUI screenshots with annotated elements, OCR for detecting text and icons, Structured XML metadata	Standard UI Operations: Tap, text input, long press, swipe, back, and stop	GPT-4	Multi-phase architecture with Exploration Phase and Deployment Phase	Enhances adaptability and precision in mobile environments by combining structured data parsing with visual features	/
ScreenAgent [273]	Linux and Windows desktop	Screenshots	Standard UI operations	ScreenAgent model	Single-agent	Integrated planning-acting-reflecting pipeline that simulates a continuous thought process	https://github.com/niuzaisheng/ScreenAgent
AutoDroid [143]	Android mobile devices	Simplified HTML-style representation	Standard UI operations	GPT-3.5, GPT-4, and Vicuna-7B [362]	Single-agent architecture	Its use of app-specific knowledge and a multi-granularity query optimization module to reduce the computational cost	https://autodroid-sys.github.io/
CoAT [235]	Android mobile devices	Screenshot-based context and semantic information	Standard UI operations	GPT-4V	Single-agent architecture	The integration of a chain-of-action-thought process, which explicitly maps each action to screen descriptions, reasoning steps, and anticipated outcomes	https://github.com/ZhangL-HKU/CoAT
Mobile-Agent [145]	Mobile Android	Screenshots with icon detection	Standard UI operations	GPT-4V with Grounding DINO [168] and CLIP [361] for icon detection	Single-agent	Vision-centric approach that eliminates dependency on system-specific data	https://github.com/X-PLUG/MobileAgent
Mobile-Agent-v2 [242]	Mobile Android OS and Harmony OS	Screenshots with text, icon recognition, and description	Standard UI operations on mobile phones	GPT-4V with Grounding DINO [168] and Qwen-VL-Int4 [363]	Multi-agent architecture with Planning Agent, Decision Agent, and Reflection Agent	Multi-agent architecture enhances task navigation for long-sequence operations	https://github.com/X-PLUG/MobileAgent
Mobile-Experts [243]	Mobile Android	Interface memory and procedural memory	Standard UI operations and code-combined tool formulation	VLMs	Multi-agent framework with double-layer planning	Code-combined tool formulation method and double-layer planning mechanism for collaborative task execution	/
LiMAC [244]	Mobile Android	Screenshots and corresponding widget trees	Standard UI operations	Lightweight transformer and fine-tuned VLMs	Single-agent	Balances computational efficiency and natural language understanding	/
MobA [245]	Mobile Android	GUI structures, screenshots with annotation	Standard UI operations and API function calls	GPT-4	Two-level agent: a Global Agent and a Local Agent	Two-level agent system that separates task planning and execution into two specialized agents	/

bVoyager [217] advances autonomous web navigation by supporting end-to-end task completion across real-world web environments. Utilizing GPT-4V for both visual (screenshots) and textual (HTML elements) inputs, WebVoyager effectively interacts with dynamic web interfaces, including those with dynamically rendered content and intricate interactive elements. This multimodal capability allows WebVoyager to manage complex interfaces with a success rate notably surpassing traditional text-only methods, setting a new benchmark in web-based task automation.

In addition to multimodal integration, some frameworks focus on parsing intricate web structures and generating executable code to navigate complex websites. **WebAgent** [215] employs a two-tiered model approach by combining HTML-T5 for parsing long, complex HTML documents with Flan-U-PaLM [356] for program synthesis. This modular design enables WebAgent to translate user instructions into executable Python code, autonomously handling complex, real-world websites through task-specific sub-instructions. WebAgent demonstrates a 50% improvement in success rates on real websites compared to traditional single-agent models, showcasing the advantages of integrating HTML-specific parsing with code generation for diverse and dynamic web environments.

To enhance decision-making in web navigation, several frameworks introduce state-space exploration and search algorithms. **LASER** [216] models web navigation as state-space exploration, allowing flexible backtracking and efficient decision-making without requiring extensive in-context examples. By associating actions with specific states and leveraging GPT-4's function-calling feature for state-based action selection, LASER minimizes errors and improves task success, particularly in e-commerce navigation tasks such as WebShop and Amazon. This state-based approach provides a scalable and efficient solution, advancing the efficiency of LLM agents in GUI navigation.

TABLE 9: Overview of LLM-brained GUI agent frameworks (Part IV).

Agent	Platform	Perception	Action	Model	Architecture	Highlight	Link
UFO [17]	Windows computer	Screenshots with annotated controls, and widget properties	Standard UI operations with additional customized operations	GPT-Vision	Dual-agent architecture, consisting of a HostAgent (for application selection and global planning) and an AppAgent (for specific task execution within applications)	Its dual-agent system that seamlessly navigates and interacts with multiple applications to fulfill complex user requests in natural language on Windows OS	https://github.com/microsoft/UFO
OS-Copilot [149]	Linux and MacOS computer	Unified interface that includes mouse and keyboard control, API calls, and Bash or Python interpreters	Standard UI operations, Bash and Python commands, as well as API calls	GPT-4	Multi-component architecture involving a planner, configurator, actor, and critic modules	Self-directed learning capability, allowing it to adapt to new applications by autonomously generating and refining tools	https://os-copilot.github.io/
Cradle [148]	Windows computer	Complete screen videos with Grounding DINO [168] and SAM [167] for object detection and localization	Keyboard and mouse actions	GPT-4	Modular single-agent architecture	Its generalizability across various digital environments, allowing it to operate without relying on internal APIs	https://baai-agents.github.io/Cradle/
Agent S [247]	Ubuntu and Windows computer	Screenshots and accessibility tree	Standard UI operations and system-level controls	GPT-4 and Claude-3.5 Sonnet [150]	Multi-agent architecture comprising a Manager and Worker structure	Experience-augmented hierarchical planning	https://github.com/simular-ai/Agent-S
GUI Narrator [248]	Windows computer	High-resolution screenshots	Standard UI operations	GPT-4 and QwenVL-7B [363]	Two-stage architecture, detecting the cursor location and selecting keyframes, then generating action captions	Uses the cursor as a focal point to improve understanding of high-resolution GUI actions	https://showlab.github.io/GUI-Narrator
Zero-shot Agent [249]	Computer	HTML code and DOM	Standard UI operations	PaLM-2 [364]	Single-agent	Zero-shot capability in performing computer control tasks	https://github.com/google-research/google-research/tree/master/zero_shot_structured_reflection
AutoGLM [230]	Web and Mobile Android	Screenshots with SoM annotation and OCR	Standard UI operations, Native API interactions, and AI-driven actions	ChatGLM [357]	Single-agent architecture	Self-evolving online curriculum RL framework, which enables continuous improvement by interacting with real-world environments	https://xiao9905.github.io/AutoGLM/
TinyClick [231]	Web, Mobile, and Windows platforms	GUI screenshots	Standard UI operations, Native API interactions, and AI-driven actions	Florence-2-Base VLM [365]	Single-agent, with single-turn tasks	Compact size (0.27B parameters) with high performance	https://huggingface.co/Samsung/TinyClick
OSCAR [246]	Desktop and Mobile	Screenshots	Standard UI operations	GPT-4	Single-agent architecture	Ability to adapt to real-time feedback and dynamically adjust its actions	/
AgentStore [250]	Desktop and mobile environments	GUI structures and properties, accessibility trees, screenshots and terminal output *etc*	Standard UI operations, API calls	GPT-4o and InternVL2-8B [277]	Multi-agent architecture	Dynamically integrate a wide variety of heterogeneous agents, enabling both specialized and generalist capabilities	https://chengyou-jia.github.io/AgentStore-Home/
MMAC-Copilot [198]	Windows OS Desktop, mobile applications, and game environments	Screenshots	Standard UI operations, Native APIs, and Collaborative multi-agent actions	GPT-4V, SeeClick [23] and Genimi Vision for different agents	Multi-agent architecture with Planner, Programmer, Viewer, Mentor, Video Analyst, and Librarian	Collaborative multi-agent architecture where agents specialize in specific tasks	/

Similarly, **Search-Agent** [222] innovatively introduces a best-first search algorithm to enhance multi-step reasoning in interactive web environments. By exploring multiple action paths, this approach improves decision-making, achieving up to a 39% increase in success rates across benchmarks like WebArena [300]. Search-Agent's compatibility with existing multimodal LLMs demonstrates the effectiveness of search-based algorithms for complex, interactive web tasks.

Expanding on search-based strategies, **WebPilot** [223] employs a dual optimization strategy combining global and local Monte Carlo Tree Search (MCTS) [358] to improve adaptability in complex and dynamic environments. As illustrated in Figure 19, WebPilot decomposes overarching tasks into manageable sub-tasks, with each undergoing localized optimization. This approach allows WebPilot to continuously adjust its strategies in response to real-time observations, mimicking human-like decision-making and flexibility. Ex-

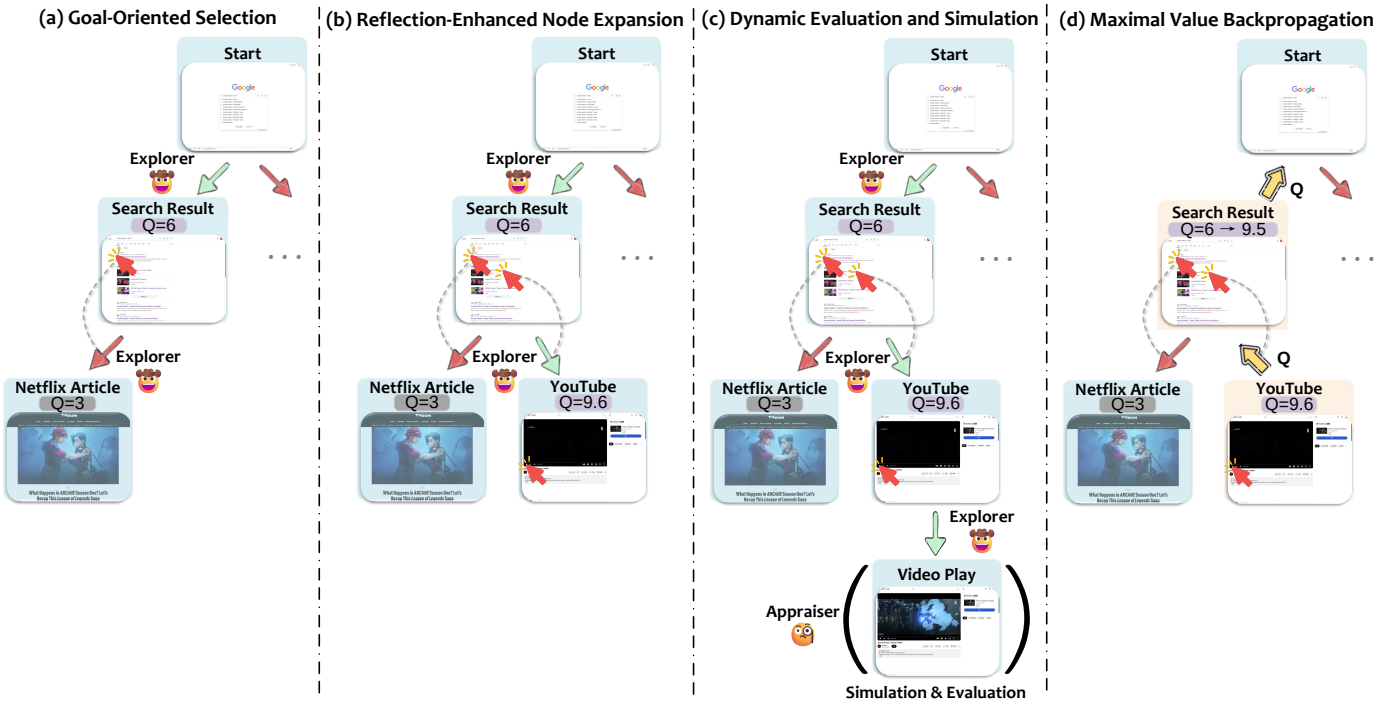


Fig. 19: An illustration of the local optimization stage in WebPilot [223] using MCTS. Figure adapted from the original paper.

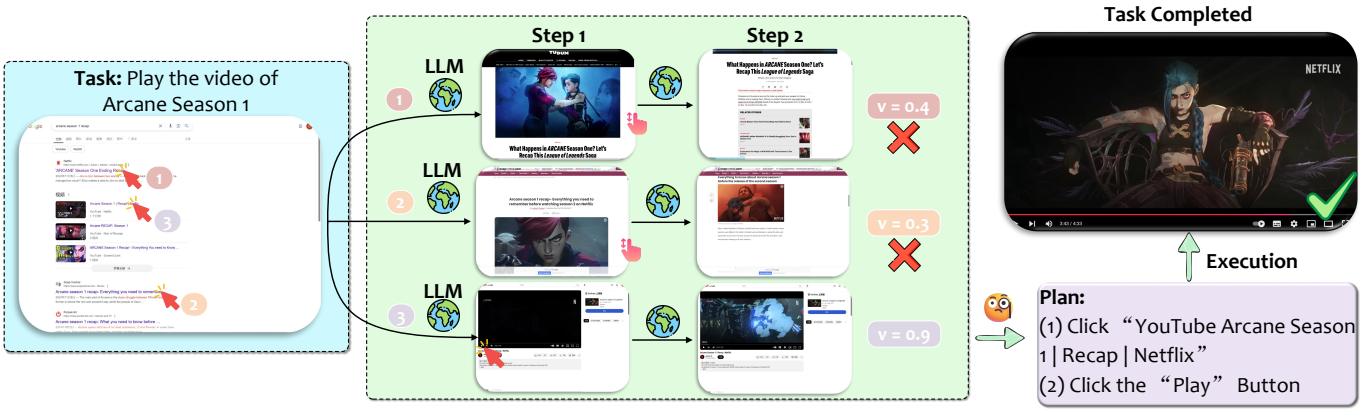


Fig. 20: An example illustrating how WebDreamer [232] uses an LLM to simulate the outcome of each action. Figure adapted from the original paper.

tensive testing on benchmarks like WebArena [300] and MiniWoB++ [133] demonstrates WebPilot's state-of-the-art performance, showcasing exceptional adaptability compared to existing methods.

Furthering the concept of predictive modeling, the **WMA** [214] introduces a world model to simulate and predict the outcomes of UI interactions. By focusing on transition-based observations, WMA allows agents to simulate action results before committing, reducing unnecessary actions and increasing task efficiency. This predictive capability is particularly effective in long-horizon tasks that require high accuracy, with WMA demonstrating strong performance on benchmarks such as WebArena [300] and Mind2Web [251].

Along similar lines, **WebDreamer** [232] introduces an innovative use of LLMs for model-based planning in web navigation, as depicted in Figure 20. WebDreamer simulates and evaluates potential actions and their multi-step outcomes using LLMs before execution [366], akin to a "dreamer" that envisions various scenarios. By preemptively assessing the potential value of different plans, WebDreamer selects and executes the plan with the highest expected value. This approach addresses critical challenges in web automation, such as safety concerns and the need for robust decision-making in complex and dynamic environments, demonstrating superiority over reactive agents in benchmarks like VisualWebArena [301] and Mind2Web-live [251].

Beyond predictive modeling, integrating API interactions into web navigation offers enhanced flexibility and efficiency. The **Hybrid Agent** [185] combines web browsing and API interactions, dynamically switching between methods based on task requirements. By utilizing API calls for structured data interaction, the Hybrid Agent reduces the time and

complexity involved in traditional web navigation, achieving higher accuracy and efficiency in task performance. This hybrid architecture underscores the benefits of integrating both structured API data and human-like browsing capabilities in AI agent systems.

Addressing the challenges of complex web structures and cross-domain interactions, **AutoWebGLM** [218] offers an efficient solution by simplifying HTML to focus on key webpage components, thereby improving task accuracy. Using reinforcement learning and rejection sampling for fine-tuning, AutoWebGLM excels in complex navigation tasks on both English and Chinese sites. Its bilingual dataset and structured action-perception modules make it practical for cross-domain web interactions, emphasizing the importance of efficient handling in diverse web tasks.

In summary, recent frameworks for web GUI agents have made substantial progress by integrating multimodal inputs, predictive models, and advanced task-specific optimizations. These innovations enable robust solutions for real-world tasks, enhancing the capabilities of web-based GUI agents and marking significant steps forward in developing intelligent, adaptive web automation.

6.2 Mobile GUI Agents

The evolution of mobile GUI agents has been marked by significant advancements, leveraging multimodal models, complex architectures, and adaptive planning to address the unique challenges of mobile environments. These agents have progressed from basic interaction capabilities to sophisticated systems capable of dynamic, context-aware operations across diverse mobile applications.

Early efforts focused on enabling human-like GUI interactions without requiring backend system access. One such pioneering framework is **AppAgent** [16], which utilizes GPT-4V's multimodal capabilities to comprehend and respond to both visual and textual information. By performing actions like tapping and swiping using real-time screenshots and structured XML data, AppAgent can interact directly with the GUI across a variety of applications, from social media to complex image editing. Its unique approach of learning through autonomous exploration and observing human demonstrations allows for rapid adaptability to new apps, highlighting the effectiveness of multimodal capabilities in mobile agents.

Building upon this foundation, **AppAgent-V2** [241] advances the framework by enhancing visual recognition and incorporating structured data parsing. This enables precise, context-aware interactions and the ability to perform complex, multi-step operations across different applications. AppAgent-V2 also introduces safety checks to handle sensitive data and supports cross-app tasks by tracking and adapting to real-time interactions. This progression underscores the importance of advanced visual recognition and structured data processing in improving task precision and safety in real-time mobile environments.

Parallel to these developments, vision-centric approaches emerged to further enhance mobile task automation without relying on app-specific data. For instance, **Mobile-Agent** [145] leverages OCR, CLIP [361], and Grounding DINO [168] for visual perception. By using screenshots and visual tools, Mobile-Agent performs operations ranging from app navigation to complex multitasking, following instructions iteratively and adjusting for errors through a self-reflective mechanism. This vision-based method positions Mobile-Agent as a versatile and adaptable assistant for mobile tasks.

To address challenges in long-sequence navigation and complex, multi-app scenarios, **Mobile-Agent-v2** [242] introduces a multi-agent architecture that separates planning, decision-making, and reflection. By distributing responsibilities among three agents, this framework optimizes task progress tracking, retains memory of task-relevant information, and performs corrective actions when errors occur. Integrated with advanced visual perception tools like Grounding DINO [168] and Qwen-VL-Int4 [275], Mobile-Agent-v2 showcases significant improvements in task completion rates on both Android and Harmony OS, highlighting the potential of multi-agent systems for handling complex mobile tasks.

In addition to vision-centric methods, some frameworks focus on translating GUI states into language to enable LLM-based action planning. **VisionTasker** [236] combines vision-based UI interpretation with sequential LLM task planning by processing mobile UI screenshots into structured natural language. Supported by YOLO-v8 [367] and PaddleOCR[28] for widget detection, VisionTasker allows the agent to automate complex tasks across unfamiliar apps, demonstrating higher accuracy than human operators on certain tasks. This two-stage design illustrates a versatile and adaptable framework, setting a strong precedent in mobile automation.

Similarly, **DroidBot-GPT** [237] showcases an innovative approach by converting GUI states into natural language prompts, enabling LLMs to autonomously decide on action sequences. By interpreting the GUI structure and translating it into language that GPT models can understand, DroidBot-GPT generalizes across various apps without requiring app-specific modifications. This adaptability underscores the transformative role of LLMs in handling complex, multi-step tasks with minimal custom data.

To enhance action prediction and context awareness, advanced frameworks integrate perception and action systems within a multimodal LLM. **CoCo-Agent** [238] exemplifies this by processing GUI elements like icons and layouts through its Comprehensive Event Perception and Comprehensive Action Planning modules. By decomposing actions into manageable steps and leveraging high-quality data from benchmarks like Android in the Wild (AITW) [262] and META-GUI [261], CoCo-Agent demonstrates its ability to automate mobile tasks reliably across varied smartphone applications.

Further advancing this integration, **CoAT** [235] introduces a chain-of-action-thought process to enhance action prediction and context awareness. Utilizing sophisticated models such as GPT-4V and set-of-mark tagging, CoAT addresses the limitations of traditional coordinate-based action recognition. By leveraging the Android-In-The-Zoo (AITZ) dataset it builds, CoAT provides deep context awareness and improves both action prediction accuracy and task completion rates, highlighting its potential for accessibility and user convenience on Android platforms.

Addressing the need for efficient handling of multi-step tasks with lower computational costs, **AutoDroid** [143] com-

28. https://github.com/PaddlePaddle/PaddleOCR

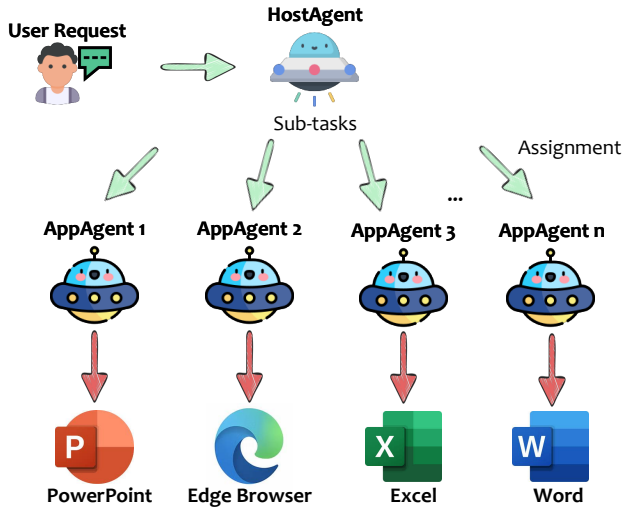


Fig. 21: The multi-agent architecture employed in UFO [17]. Figure adapted from the original paper.

bines LLM-based comprehension with app-specific knowledge. Using an HTML-style GUI representation and a memory-based approach, AutoDroid reduces dependency on extensive LLM queries. Its hybrid architecture of cloud and on-device models enhances responsiveness and accessibility, making AutoDroid a practical solution for diverse mobile tasks.

In summary, mobile GUI agents have evolved significantly, progressing from single-agent systems to complex, multi-agent frameworks capable of dynamic, context-aware operations. These innovations demonstrate that sophisticated architectures, multimodal processing, and advanced planning strategies are essential in handling the diverse challenges of mobile environments, marking significant advancements in mobile automation capabilities.

6.3 Computer GUI Agents

Computer GUI agents have evolved to offer complex automation capabilities across diverse operating systems, addressing challenges such as cross-application interaction, task generalization, and high-level task planning.

Advancements in computer GUI agents have led to the development of sophisticated frameworks capable of handling complex tasks across desktop environments. These agents have evolved from simple automation tools to intelligent systems that leverage multimodal inputs, advanced architectures, and adaptive learning to perform multi-application tasks with high efficiency and adaptability.

One significant development in this area is the introduction of multi-agent architectures that enhance task management and execution. For instance, the UI-Focused Agent, **UFO** [17] represents a pioneering framework specifically designed for the Windows operating system. UFO redefines UI-focused automation through its advanced dual-agent architecture, leveraging GPT-Vision to interpret GUI elements and execute actions autonomously across multiple applications. The framework comprises a HostAgent, responsible for global planning, task decomposition, and application selection, and an AppAgent, tasked with executing assigned subtasks within individual applications, as illustrated in Figure 21. This centralized structure enables UFO to manage complex, multi-application workflows such as aggregating information and generating reports. Similar architectural approach has also been adopted by other GUI agent frameworks [243], [245], [352]. By incorporating safeguards and customizable actions, UFO ensures efficiency and security when handling intricate commands, positioning itself as a cutting-edge assistant for Windows OS. Its architecture, exemplifies dynamic adaptability and robust task-solving capabilities across diverse applications, demonstrating the potential of multi-agent systems in desktop automation.

Building upon the theme of adaptability and generalist capabilities, **Cradle** [148] pushes the boundaries of general computer control by utilizing VLMs for interacting with various software, ranging from games to professional applications, without the need for API access. Cradle employs GPT-4o to interpret screen inputs and perform low-level actions, making it versatile across different types of software environments. Its six-module structure, covering functions such as information gathering and self-reflection, enables the agent to execute tasks, reason about actions, and utilize past interactions to inform future decisions. Cradle's capacity to function in dynamic environments, including complex software, marks it as a significant step toward creating generalist agents with broad applicability across desktop environments.

Extending the capabilities of computer GUI agents to multiple operating systems, **OS-Copilot** [149] introduces a general-purpose framework designed to operate across Linux and macOS systems. Its notable feature, **FRIDAY**, showcases the potential of self-directed learning by adapting to various applications and performing tasks without explicit training for each app. Unlike application-specific agents, FRIDAY integrates APIs, keyboard and mouse controls, and command-line operations, creating a flexible platform that can autonomously generate and refine tools as it interacts with new applications. OS-Copilot's ability to generalize across unseen applications, validated by its performance on the GAIA benchmark, provides a foundational model for OS-level agents capable of evolving in complex environments. This demonstrates promising directions for creating adaptable digital assistants that can handle diverse desktop environments and complex task requirements.

In summary, computer GUI agents have evolved significantly, progressing from single-task automation tools to advanced multi-agent systems capable of performing complex, multi-application tasks and learning from interactions. Frameworks like UFO, Cradle, and OS-Copilot illustrate the potential of adaptable, generalist agents in desktop automation, paving the way for the evolution of more intelligent and versatile AgentOS frameworks.

6.4 Cross-Platform GUI Agents

Cross-platform GUI agents have emerged as versatile solutions capable of interacting with various environments, from desktop and mobile platforms to more complex systems. These frameworks prioritize adaptability and efficiency, leveraging both lightweight models and multi-agent architectures to enhance cross-platform operability. In this subsection, we explore key frameworks that exemplify the advancements in cross-platform GUI automation.

A significant stride in this domain is represented by **AutoGLM** [230], which bridges the gap between web browsing and Android control by integrating large multimodal models for seamless GUI interactions across platforms. AutoGLM introduces an Intermediate Interface Design that separates planning and grounding tasks, improving dynamic decision-making and adaptability. By employing a self-evolving online curriculum with reinforcement learning, the agent learns incrementally from real-world feedback and can recover from errors. This adaptability and robustness make AutoGLM ideal for real-world deployment in diverse user applications, setting a new standard in cross-platform automation and offering promising directions for future research in foundation agents.

While some frameworks focus on integrating advanced models for cross-platform interactions, others emphasize efficiency and accessibility. **TinyClick** [231] addresses the need for lightweight solutions by focusing on single-turn interactions within GUIs. Utilizing the Florence-2-Base Vision-Language Model, TinyClick executes tasks based on user commands and screenshots with only 0.27 billion parameters. Despite its compact size, it achieves high accuracy—73% on Screenspot [23] and 58.3% on OmniAct [309] —outperforming larger multimodal models like GPT-4V while maintaining efficiency. Its multi-task training and MLLM-based data augmentation enable precise UI element localization, making it suitable for low-resource environments and addressing latency and resource constraints in UI grounding and action execution.

In addition to lightweight models, multi-agent architectures play a crucial role in enhancing cross-platform GUI interactions. **OSCAR** [246] exemplifies this approach by introducing a generalist GUI agent capable of autonomously navigating and controlling both desktop and mobile applications. By utilizing a state machine architecture, OSCAR dynamically handles errors and adjusts its actions based on real-time feedback, making it suitable for automating complex workflows guided by natural language. The integration of standardized OS controls, such as keyboard and mouse inputs, allows OSCAR to interact with applications in a generalized manner, improving productivity across diverse GUI environments. Its open-source design promotes broad adoption and seamless integration, offering a versatile tool for cross-platform task automation and productivity enhancement.

Expanding on the concept of multi-agent systems, **AgentStore** [250] introduces a flexible and scalable framework for integrating heterogeneous agents to automate tasks across operating systems. The key feature of AgentStore is the MetaAgent, which uses the innovative AgentToken strategy to dynamically manage a growing number of specialized agents. By enabling dynamic agent enrollment, the framework fosters adaptability and scalability, allowing both specialized and generalist capabilities to coexist. This multi-agent architecture supports diverse platforms, including desktop and mobile environments, leveraging multimodal perceptions such as GUI structures and system states. AgentStore's contributions highlight the importance of combining specialization with generalist capabilities to overcome the limitations of previous systems.

Further advancing cross-platform GUI interaction, **MMAC-Copilot** [198] employs a multi-agent, multimodal approach to handle tasks across 3D gaming, office, and mobile applications without relying on APIs. By utilizing specialized agents like Planner, Viewer, and Programmer, MMAC-Copilot collaborates to adapt to the complexities of visually rich environments. Using GPT-4V for visual recognition and OCR for text analysis, it achieves high task completion rates in visually complex environments. The framework's integration with VIBench, a benchmark for non-API applications, underscores its real-world relevance and adaptability. MMAC-Copilot's robust foundation for dynamic interaction across platforms extends applications to industries like gaming, healthcare, and productivity.

In summary, cross-platform GUI agents exemplify the future of versatile automation, offering solutions ranging from lightweight models like TinyClick to sophisticated multi-agent systems such as MMAC-Copilot. Each framework brings unique innovations, contributing to a diverse ecosystem of GUI automation tools that enhance interaction capabilities across varying platforms, and marking significant advancements in cross-platform GUI automation.

6.5 Takeaways

The landscape of GUI agent frameworks has seen notable advancements, particularly in terms of multi-agent architectures, multimodal inputs, and enhanced action sets. These developments are laying the groundwork for more versatile and powerful agents capable of handling complex, dynamic environments. Key takeaways from recent advancements include:

1) **Multi-Agent Synergy:** Multi-agent systems, such as those in UFO and MMAC-Copilot, represent a significant trend in GUI agent development. By assigning specialized roles to different agents within a framework, multi-agent systems can enhance task efficiency, adaptability, and overall performance. As agents take on more complex tasks across diverse platforms, the coordinated use of multiple agents is proving to be a powerful approach, enabling agents to handle intricate workflows with greater precision and speed.
2) **Multimodal Input Benefits:** While some agents still rely solely on text-based inputs (*e.g.*, DOM structures or HTML), incorporating visual inputs, such as screenshots, has shown clear performance advantages. Agents like WebVoyager and SeeAct highlight how visual data, combined with textual inputs, provides a richer representation of the environment state, helping agents make better-informed decisions. This integration of multimodal inputs is essential for accurate interpretation in visually complex or dynamic environments where text alone may not capture all necessary context.
3) **Expanding Action Sets Beyond UI Operations:** Recent agents have expanded their action sets beyond standard UI operations to include API calls and AI-driven actions, as seen in Hybrid Agent and AutoWebGLM. Incorporating diverse actions allows agents to achieve higher levels of interaction and task completion, particularly in environments where data can be directly retrieved or manipulated through API calls. This flexibility enhances agent capabilities, making them more efficient and adaptable across a wider range of applications.

4) **Emerging Techniques for Improved Decision-Making:** Novel approaches such as world models in WMA and search-based strategies in Search-Agent represent promising directions for more advanced decision-making. World models allow agents to simulate action outcomes, reducing unnecessary interactions and improving efficiency, especially in long-horizon tasks. Similarly, search-based algorithms like best-first and MCTS help agents explore action pathways more effectively, enhancing their adaptability in complex, real-time environments.
5) **Toward Cross-Platform Generalization:** Cross-platform frameworks, such as AutoGLM and OSCAR, underscore the value of generalizability in GUI agent design. These agents are pioneering efforts to create solutions that work seamlessly across mobile, desktop, and web platforms, moving closer to the goal of a one-stop GUI agent that can operate across multiple ecosystems. Cross-platform flexibility will be crucial for agents that aim to assist users consistently across their digital interactions.

The field of GUI agents is moving towards multi-agent architectures, multimodal capabilities, diverse action sets, and novel decision-making strategies. These innovations mark significant steps toward creating intelligent, adaptable agents capable of high performance across varied and dynamic environments. The future of GUI agents lies in the continued refinement of these trends, driving agents towards broader applicability and more sophisticated, human-like interactions across platforms.

7 Data for Optimizing LLM-Brained GUI Agents

In the previous section, we explored general frameworks for LLM-brained GUI agents, most of which rely on foundational LLMs such as GPT-4V and GPT-4o. However, to elevate these agents' performance and efficiency, optimizing their "brain", the underlying model is crucial. Achieving this often involves fine-tuning foundational models using large-scale, diverse, and high-quality contextual GUI datasets [368], which are specifically curated to enable these models to excel in GUI-specific tasks. Collecting such datasets, particularly those rich in GUI screenshots, metadata, and interactions, necessitates an elaborate process of data acquisition, filtering, and preprocessing, each requiring substantial effort and resources [369].

As GUI agents continue to gain traction, researchers have focused on assembling datasets that represent a broad spectrum of platforms and capture the diverse intricacies of GUI environments. These datasets are pivotal in training models that can generalize effectively, thanks to their coverage of varied interfaces, workflows, and user interactions. To ensure comprehensive representation, innovative methodologies have been employed to collect and structure these data assets. In the sections that follow, we detail an end-to-end pipeline for data collection and processing tailored to training GUI-specific LLMs. We also examine significant datasets from various platforms, providing insights into their unique features, the methodologies used in their creation, and their potential applications in advancing the field of LLM-brained GUI agents. To provide a structured overview, Tables 10, 11, and 12 summarize these datasets across differernt platforms, highlighting their key attributes and contributions to the evolution of GUI agent research.

7.1 Data Collection

Data is pivotal in training a purpose-built GUI agent, yet gathering it requires substantial time and effort due to the task's complexity and the varied environments involved.

7.1.1 Data Composition and Sources

The essential data components for GUI agent training closely align with the agent's perception and inference requirements discussed in Sections 5.2.2 and 5.4. At a high level, this data comprises:

1) **User Instructions:** These provide the task's overarching goal, purpose, and specific details, typically in natural language, offering a clear target for the agent to accomplish, *e.g.*, "change the font size of all text to 12".
2) **Environment Perception:** This typically includes GUI screenshots, often with various visual augmentations, as well as optional supplementary data like widget trees and UI element properties to enrich the context.
3) **Task Trajectory:** This contains the critical action sequence required to accomplish the task, along with supplementary information, such as the agent's plan. A trajectory usually involves multiple steps and actions to navigate through the task.

While user instructions and environmental perception serve as the model's input, the expected model output is the task trajectory. This trajectory's action sequence is then grounded within the environment to complete the task.

For **user instructions**, it is crucial to ensure that they are realistic and reflective of actual user scenarios. Instructions can be sourced in several ways: *(i)* directly from human designers, who can provide insights based on real-world applications; *(ii)* extracted from existing, relevant datasets if suitable data is available; *(iii)* sourcing from public materials, such as websites, application help documentation, and other publicly available resources; and *(iv)* generated by LLMs, which can simulate a broad range of user requests across different contexts. Additionally, LLMs can be employed for data augmentation [370], increasing both the quality and diversity of instructions derived from the original data.

For gathering **environment perception** data, various toolkits—such as those discussed in Section 5.2.2—can be used to capture the required GUI data. This can be done within an environment emulator (*e.g.*, Android Studio Emulator[29], Selenium WebDriver[30], Windows Sandbox[31]) or by directly interfacing with a real environment to capture the state of GUI elements, including screenshots, widget trees, and other metadata essential for the agent's operation.

Collecting **task trajectories**, which represent the agent's action sequence to complete a task, is often the most challenging aspect. Task trajectories need to be accurate, executable, and well-validated. Collection methods include *(i)*

29. https://developer.android.com/studio
30. https://www.selenium.dev/
31. https://learn.microsoft.com/en-us/windows/security/application-security/application-isolation/windows-sandbox/windows-sandbox-overview

using programmatically generated scripts, which define action sequences for predefined tasks, providing a highly controlled data source; *(ii)* employing human annotators, who complete tasks in a crowdsourced manner with each step recorded, allowing for rich, authentic action data; and *(iii)* leveraging model or agent bootstrapping [371], where an existing LLM or GUI agent attempts to complete the task and logs its actions, though this method may require additional validation due to potential inaccuracies. All these methods demand considerable effort, reflecting the complexities of gathering reliable, task-accurate data for training GUI agents.

7.1.2 Collection Pipeline

Figure 22 presents a complete pipeline for data collection aimed at training a GUI agent model. The process begins with gathering initial user instructions, which may come from various aforementioned sources. These instructions are typically prototypical, not yet tailored or grounded to a specific environment [254]. For instance, an instruction like "how to change the font size?" from a general website lacks specificity and doesn't align with the concrete requests a user might make within a particular application. To address this, an instantiation step is required [254], where instructions are contextualized within a specific environment, making them more actionable. For example, the instruction might be refined to "Change the font size of the third paragraph in a Word document of `draft.docx` to 20.", giving it a clear, environment-specific goal. This instantiation process can be conducted either manually by humans or programmatically with an LLM.

Following instantiation, instructions may undergo a filtering step to remove low-quality data, ensuring only relevant and actionable instructions remain. Additionally, data augmentation techniques can be applied to expand and diversify the dataset, improving robustness. Both of these processes can involve human validation or leverage LLMs for efficiency.

Once instruction refinement is complete, task trajectories and environment perceptions are collected simultaneously. As actions are performed within the environment, each step is logged, providing a record of the environment's state and the specific actions taken. After a full task trajectory is recorded, an evaluation phase is necessary to identify and remove any failed or inaccurate sequences, preserving the quality of the dataset. By iterating this pipeline, a high-quality dataset of GUI agent data can be compiled, which is crucial for training optimized models.

In the following sections, we review existing GUI agent datasets across various platforms, offering insights into current practices and potential areas for improvement.

7.2 Web Agent Data

Web-based GUI agents demand datasets that capture the intricate complexity and diversity of real-world web interactions. These datasets often encompass varied website structures, including DOM trees and HTML content, as well as multi-step task annotations that reflect realistic user navigation and interaction patterns. Developing agents that can generalize across different websites and perform complex tasks requires comprehensive datasets that provide rich contextual information.

Building upon this need, several significant datasets have been developed to advance web-based GUI agents. Unlike traditional datasets focusing on narrow, predefined tasks, **Mind2Web** [251] represents a significant step forward by emphasizing open-ended task descriptions, pushing agents to interpret high-level goals independently. It offers over 2,350 human-annotated tasks across 137 diverse websites, capturing complex interaction patterns and sequences typical in web navigation. This setup aids in evaluating agents' generalization across unseen domains and serves as a benchmark for language grounding in web-based GUIs, enhancing adaptability for real-world applications.

Similarly, **WebVLN** [252] expands on web GUI tasks by combining navigation with question-answering. It provides agents with text-based queries that guide them to locate relevant web pages and extract information. By leveraging both HTML and visual content from websites, WebVLN aligns with real-world challenges of web browsing. This dataset is particularly valuable for researchers aiming to develop agents capable of complex, human-like interactions in GUI-driven web spaces.

Moreover, **WebLINX** [253] focuses on conversational GUI agents, particularly emphasizing real-world web navigation through multi-turn dialogue. Featuring over 2,300 expert demonstrations across 155 real-world websites, WebLINX creates a rich environment with DOM trees and screenshots for training and evaluating agents capable of dynamic, user-guided navigation tasks. This dataset promotes agent generalization across new sites and tasks, with comprehensive action and dialogue data that provide insights into enhancing agent responsiveness in realistic web-based scenarios.

Collectively, these datasets represent essential resources that enable advancements in web agent capabilities, supporting the development of adaptable and intelligent agents for diverse web applications.

7.3 Mobile Agent Data

Mobile platforms are critical for GUI agents due to the diverse range of apps and unique user interactions they involve. To develop agents that can effectively navigate and interact with mobile interfaces, datasets must offer a mix of single and multi-step tasks, focusing on natural language instructions, UI layouts, and user interactions.

An early and foundational contribution in this domain is the **Rico** dataset [259], which provides over 72,000 unique UI screens and 10,811 user interaction traces from more than 9,700 Android apps. Rico has been instrumental for tasks such as UI layout similarity, interaction modeling, and perceptual modeling, laying the groundwork for mobile interface understanding and GUI agent development.

Building upon the need for grounding natural language instructions to mobile UI actions, **PIXELHELP** [131] introduces a dataset specifically designed for this purpose. It includes multi-step instructions, screenshots, and structured UI element data, enabling detailed analysis of how verbal instructions can be converted into mobile actions. This dataset has significant applications in accessibility and task automation, supporting agents that autonomously execute tasks based on verbal cues.

Further expanding the scope, the **Android in the Wild (AITW)** dataset [262] offers one of the most extensive

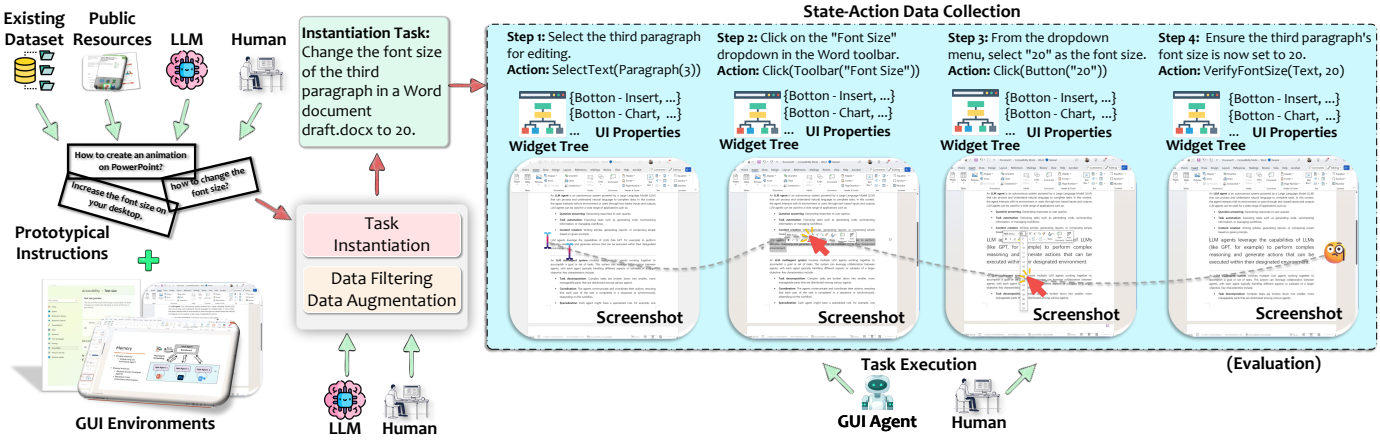


Fig. 22: A complete pipeline for data collection for training a GUI agent model.

collections of natural device interactions. Covering a broad spectrum of Android applications and diverse UI states, AITW captures multi-step tasks emulating real-world device usage. Collected through interactions with Android emulators, it includes both screenshots and action sequences, making it ideal for developing GUI agents that navigate app interfaces without relying on app-specific APIs. Due to its scale and diversity, AITW has become a widely used standard in the field.

In addition, **META-GUI** [261] provides a unique dataset for mobile task-oriented dialogue systems by enabling direct interaction with mobile GUIs, bypassing the need for API-based controls. This approach allows agents to interact across various mobile applications using multi-turn dialogues and GUI traces, broadening their capabilities in real-world applications without custom API dependencies. The dataset's support for complex interactions and multi-turn dialogue scenarios makes it valuable for building robust conversational agents.

Recently, **MobileViews** [270] emerged as the largest mobile screen dataset to date, offering over 600,000 screenshot–view hierarchy pairs from 20,000 Android apps. Collected with an LLM-enhanced app traversal tool, it provides a high-fidelity resource for mobile GUI agents in tasks such as screen summarization, tappability prediction, and UI component identification. Its scale and comprehensive coverage of screen states make MobileViews a key resource for advancing mobile GUI agent capabilities.

Collectively, mobile platforms currently boast the richest set of datasets due to their versatile tools, emulator support, and diverse use cases, reflecting the demand for high-quality, adaptive GUI agents in mobile applications.

7.4 Computer Agent Data

In contrast to mobile and web platforms, the desktop domain for GUI agents has relatively fewer dedicated datasets, despite its critical importance for applications like productivity tools and enterprise software. However, notable efforts have been made to support the development and evaluation of agents designed for complex, multi-step desktop tasks.

A significant contribution in this area is **ScreenAgent** [273], a dedicated dataset and model designed to facilitate GUI control in Linux and Windows desktop environments. ScreenAgent provides a comprehensive pipeline that enables agents to perform multi-step task execution autonomously, encompassing planning, action, and reflection phases. By leveraging annotated screenshots and detailed action sequences, it allows for high precision in UI element positioning and task completion, surpassing previous models in accuracy. This dataset is invaluable for researchers aiming to advance GUI agent capabilities in the desktop domain, enhancing agents' decision-making accuracy and user interface interactions.

Although the desktop domain has fewer datasets compared to mobile and web, efforts like ScreenAgent highlight the growing interest and potential for developing sophisticated GUI agents for computer systems.

7.5 Cross-Platform Agent Data

Cross-platform datasets play a pivotal role in developing versatile GUI agents that can operate seamlessly across mobile, computer, and web environments. Such datasets support generalizability and adaptability, enabling agents to handle varied interfaces and tasks in real-world applications.

One significant contribution is **ScreenAI** [272], which extends the scope of data collection to include both mobile and desktop interfaces. Covering tasks such as screen annotation, question-answering, and navigation, ScreenAI offers hundreds of millions of annotated samples. Its comprehensive scale and mixed-platform coverage make it a robust foundation for GUI agents that need to manage complex layouts and interactions across diverse interfaces. By emphasizing element recognition and screen summarization, ScreenAI advances the development of multi-platform GUI agents capable of handling varied visual structures.

Building upon the need for evaluating visual foundation models across environments, **VisualAgentBench** [254] is a groundbreaking cross-platform benchmark designed to assess GUI agents in both mobile and web settings. It emphasizes interaction-focused tasks, using environments like Android Virtual Device and WebArena-Lite [300] to evaluate and improve agent responses to GUI layouts and user interface actions. The dataset's innovative collection method, which combines program-based solvers and large

TABLE 10: Overview of datasets for optimizing LLMs tailored for GUI agents (Part I).

Dataset	Platform	Source	Content	Scale	Collection Method	Highlight	Link
Mind2Web [251]	Web	Crowdsourced	Task descriptions, action sequences, webpage snapshots	2,350 tasks from 137 websites	Human demonstrations	Develops generalist web agents with diverse user interactions on real-world websites	https://osu-nlp-group.github.io/Mind2Web/
WebVLN [252]	Web	Human-designed, LLM-generated	Text instructions, plans, GUI screenshots, HTML content	8,990 navigation paths, 14,825 QA pairs	WebVLN simulator, LLM-generated QA pairs	Vision-and-language navigation for human-like web browsing	https://github.com/WebVLN/WebVLN
WebLINX [253]	Web	From human experts	Conversational interactions, action sequences, DOM and screenshots	2,337 demonstrations with over 100,000 interactions	Annotated by human experts	The first large-scale dataset designed to evaluate agents in real-world conversational web navigation	https://mcgill-nlp.github.io/weblinx/
VGA [258]	Android Mobile	Rico [259]	GUI screenshots, task descriptions, action sequences, bounds, layout, and functions of GUI elements	63.8k instances, 22.3k instruction-following data pairs, 41.4k conversation data pairs	Generated by GPT-4 models	Prioritizes visual content to reduce inaccuracies	https://github.com/Linziyang1999/Vision%2DGUI%2Dassistant
Rico [259]	Android Mobile	Gathered from real Android apps on Google Play Store	Textual data, screenshots, action sequences, UI structure, annotated UI representations	72,219 unique UI screens, 10,811 user interaction traces	Crowdsourcing, automated exploration	Comprehensive dataset for mobile UI design, interaction modeling, layout generation	http://www.interactionmining.org/
PixelHelp [131]	Android Mobile	Human, web "How-to", Rico UI corpus synthetic	Natural language instructions, action sequences, GUI screenshots, structured UI data	187 multi-step instructions, 295,476 synthetic single-step commands	Human annotation and synthetic generation	Pioneering method for grounding natural language instructions to executable mobile UI actions	https://github.com/google-research/google-research/tree/master/seq2act
MoTIF [260]	Android Mobile	Human-written	Natural language instructions, action sequences, GUI screenshots, structured UI data	6,100 tasks across 125 Android apps	Human annotation	Task feasibility prediction for interactive GUI in mobile apps	https://github.com/aburns4/MoTIF
META-GUI [261]	Android Mobile	SMCalFlow [372]	Dialogues, action sequences, screenshots, Android view hierarchies	1,125 dialogues and 4,684 turns	Human annotation	Task-oriented dialogue system for mobile GUI without relying on back-end APIs	https://x-lance.github.io/META-GU
AITW [262]	Android Mobile	Human-generated instructions, LLM-generated prompts	Natural language instructions, UI screenshots, observation-action pairs	715,142 episodes and 30,378 unique instructions	Human raters using Android emulators	Large-scale dataset for device control research with extensive app and UI diversity	https://github.com/google-research/google-research/tree/master/android_in_the_wild

multimodal model bootstrapping, facilitates robust training trajectories that enhance adaptability and error recovery in GUI agent tasks.

Furthermore, **GUI-World** [271] spans multiple platforms, including desktop, mobile, and XR environments, with over 12,000 annotated videos. Designed to address the challenges of dynamic and sequential GUI tasks, GUI-World allows researchers to benchmark GUI agent capabilities across diverse interfaces. By providing detailed action sequences and QA pairs, it sets a high standard for evaluating agents in complex, real-world scenarios.

Additionally, **xLAM** [257] contributes significantly to actionable agent development by providing a unified dataset format designed to support multi-turn interactions, reasoning, and function-calling tasks. Sourced from datasets like WebShop [318], ToolBench [373], and AgentBoard [374], xLAM standardizes data formats across diverse environments, addressing the common issue of inconsistent data structures that hinder agent training and cross-environment compatibility. By offering a consistent structure, xLAM enhances the adaptability and error detection capabilities of GUI agents, allowing for more seamless integration and performance across different applications.

Collectively, these cross-platform datasets contribute to building multi-platform GUI agents, paving the way for agents that can seamlessly navigate and perform tasks across different interfaces, fostering more generalized and adaptable systems.

TABLE 11: Overview of datasets for optimizing LLMs tailored for GUI agents (Part II).

Dataset	Platform	Source	Content	Scale	Collection Method	Highlight	Link
GUI Odyssey [263]	Android Mobile	Human designers, GPT-4	Textual tasks, plans, action sequences, GUI screenshots	7,735 episodes across 201 apps	Human demonstrations	Focuses on cross-app navigation tasks on mobile devices	https://github.com/OpenGVLab/GUI-Odyssey
Amex [264]	Android Mobile	Human-designed, ChatGPT-generated	Text tasks, action sequences, high-res screenshots with multi-level annotations	104,000 screenshots, 1.6 million interactive elements, 2,946 instructions	Human annotations, autonomous scripts	Multi-level, large-scale annotations supporting complex mobile GUI tasks	https://yuxiangchai.github.io/AMEX/
Ferret-UI [265]	iOS, Android Mobile	Spotlight dataset, GPT-4	Text tasks, action plans, GUI element annotations, bounding boxes	40,000 elementary tasks, 10,000 advanced tasks	GPT-generated	Benchmark for UI-centric tasks with adjustable screen aspect ratios	https://github.com/apple/ml-ferret
AITZ [235]	Android Mobile	AITW [262]	Screen-action pairs, action descriptions	18,643 screen-action pairs across 70+ apps, 2,504 episodes	GPT-4V, icon detection models	Structured "Chain-of-Action-Thought" enhancing GUI navigation	https://github.com/IMNearth/CoAT
Octo-planner [266]	Android Mobile	GPT-4 generated	Text tasks, decomposed plans, action sequences	1,000 data points	GPT-4 generated	Optimized for task planning with GUI actions	https://huggingface.co/NexaAIDev/octopus-planning
E-ANT [267]	Android tiny-apps	Human behaviors	Task descriptions, screenshots, action sequences, page element data	40,000+ traces, 10,000 action intents	Human annotation	First large-scale Chinese dataset for GUI navigation with real human interactions	/
Mobile3M [268]	Android Mobile	Real-world interactions, simulations	UI screenshots, XML documents, action sequences	3,098,786 UI pages, 20,138,332 actions	Simulation algorithm	Large-scale Chinese mobile GUI dataset with unique navigation graph	https://github.com/Meituan-AutoML/MobileVLM
AndroidLab [269]	Android Mobile	Human design, LLM self-exploration, academic datasets	Text instructions, action sequences, XML data, screenshots	10.5k traces, 94.3k steps	Human annotation, LLM self-exploration	XML-based interaction data with unified action space	https://github.com/THUDM/Android-Lab
MobileViews [270]	Android Mobile	LLM-enhanced app traversal tool	Screenshot-view hierarchy pairs	600,000 screenshots, VH pairs from 20,000+ apps	LLM-enhanced crawler	Largest open-source mobile screen dataset	https://huggingface.co/datasets/mllmTeam/MobileViews

7.6 Takeaways

Data collection and curation for LLM-powered GUI agents is an intensive process, often requiring substantial human involvement, particularly for generating accurate action sequences and annotations. While early datasets were limited in scale and task diversity, recent advancements have led to large-scale, multi-platform datasets that support more complex and realistic GUI interactions. Key insights from these developments include:

1) **Scale and Diversity:** High-quality, large-scale data is essential for training robust GUI agents capable of handling diverse UI states and tasks. Datasets like MobileViews [270] and ScreenAI [272] illustrate the importance of vast and varied data to accommodate the dynamic nature of mobile and desktop applications, enhancing the agent's resilience across different environments.
2) **Cross-Platform Flexibility:** Cross-platform datasets such as VisualAgentBench [254] and GUI-World [271] underscore the value of generalizability, enabling agents to perform consistently across mobile, web, and desktop environments. This cross-platform adaptability is a crucial step towards creating one-stop solutions where a single GUI agent can operate seamlessly across multiple platforms.
3) **Automated Data Collection:** AI-driven data collection tools, as exemplified by OmniParser [169] and MobileViews [270], showcase the potential to significantly reduce manual efforts and accelerate scalable dataset creation. By automating the annotation process, these tools pave the way for more efficient data pipelines, moving towards a future where AI supports AI by expediting data gathering and labeling for complex GUI interactions.
4) **Unified Data Formats and Protocols:** xLAM's unified data format is an essential innovation that improves compatibility across diverse platforms [257], addressing a significant bottleneck in cross-platform GUI agent development. Establishing standardized protocols or action spaces for data collection, particularly given the varied data formats, action spaces, and environment representations across platforms, will be vital in furthering agent generalization and consistency.

In summary, the evolving landscape of datasets for LLM-powered GUI agents spans multiple platforms, with each dataset addressing unique challenges and requirements specific to its environment. These foundational resources are key to enabling agents to understand complex UIs,

TABLE 12: Overview of datasets for optimizing LLMs tailored for GUI agents (Part III).

Dataset	Platform	Source	Content	Scale	Collection Method	Highlight	Link
ScreenAgent [273]	Linux, Windows OS	Human-designed	GUI screenshots, action sequences	273 task sessions, 3,005 training screenshots, 898 test screenshots	Human annotation	VLM-based agent across multiple desktop environments	https://github.com/niuzaisheng/ScreenAgent
VisualAgentBench [254]	Android Mobile, Web	VAB-Mobile: Android Virtual Device, VAB-WebArena-Lite: WebArena [301]	Task instructions, action sequences, screen observations	VAB-Mobile: 1,213 trajectories, 10,175 steps; VAB-WebArena-Lite: 1,186 trajectories, 9,522 steps	Program-based solvers, agent bootstrapping, human demonstrations	Systematic evaluation of VLM as a visual foundation agent across multiple scenarios	https://github.com/THUDM/VisualAgentBench
GUICourse [256]	Android Mobile, Web	Web scraping, simulation, manual design	GUI screenshots, action sequences, OCR tasks, QA pairs	10 million website page-annotation pairs, 67,000 action instructions	LLM-based auto-annotation, crowdsourcing	Dataset suite for enhancing VLM GUI navigation on web and mobile platforms	https://github.com/yiye3/GUICourse
GUI-World [271]	OS, Mobile, Web, XR	Student workers, YouTube instructional videos	GUI videos, human-annotated keyframes, captions, QA data, action sequences	12,000 videos, 83,176 frames	Human annotation	Designed for dynamic, sequential GUI tasks with video data	https://gui-world.github.io/
ScreenAI [272]	Android, iOS, Desktop/Web	Crawling apps and webpages, synthetic QA	Screen annotation, screen QA, navigation, summarization	Annotation: hundreds of millions; QA: tens of millions; Navigation: millions	Model, human annotation	Comprehensive pre-training and fine-tuning for GUI tasks across platforms	https://github.com/google%2Dresearch%2Ddatasets/screen_annotation
OmniParser [169]	Web, Desktop, Mobile	Popular webpages	UI screenshots, bounding boxes, icon descriptions, OCR-derived text	67,000+ screenshots, 7,000 icon-description pairs	Finetuned detection model, OCR, human descriptions	Vision-based parsing of UI screenshots into structured elements	https://github.com/microsoft/OmniParser
Web-Hybrid [255]	Web, Android Mobile	Web-synthetic data	Screenshots, text-based referring expressions, coordinates on GUIs	10 million GUI elements, 1.3 million screenshots	Rule-based synthesis, LLMs for referring expressions	Largest dataset for GUI visual grounding	https://osu-nlp-group.github.io/UGround/
xLAM [257]	Web and tools used	Synthesized data, and existing dataset	Textual tasks, action sequences, function-calling data	60,000 data points	Collected using AI models with human verification steps	Provides a *unified format* across diverse environments, enhancing generalizability and error detection for GUI agents	https://github.com/SalesforceAIResearch/xLAM

perform nuanced interactions, and improve generalization across diverse applications. The push towards cross-platform adaptability, automated data collection, and standardized data formats will continue to shape the future of GUI agents.

8 Models for Optimizing LLM-Brained GUI Agents

LLMs act as the "brain" of GUI agents, empowering them to interpret user intents, comprehend GUI screens, and execute actions that directly impact their environments. While several existing foundation models are robust enough to serve as this core, they can be further fine-tuned and optimized to evolve into Large Action Models (LAMs)—specialized models tailored to improve the performance and efficiency of GUI agents. These LAMs bridge the gap between general-purpose capabilities and the specific demands of GUI-based interactions.

In this section, we first introduce the foundation models that currently form the backbone of GUI agents, highlighting their strengths and limitations. We then delve into the concept of LAMs, discussing how these models are fine-tuned with GUI-specific datasets to enhance their adaptability, accuracy, and action-orientation in GUI environments. Through this exploration, we illustrate the progression from general-purpose LLMs to purpose-built LAMs, laying the foundation for advanced, intelligent GUI agents.

8.1 Foundation Models

Foundation models serve as the core of LLM-powered GUI agents, providing the essential capabilities for understanding and interacting with graphical user interfaces. Recent advancements in both close-source and open-source MLLMs have significantly enhanced the potential of GUI agents, offering improvements in efficiency, scalability, and multimodal

TABLE 13: Overview of foundation models for LLM-brained GUI agents.

Model	Modality	Model Size	Architecture	Training Methods	Highlights	Open-Source	Link
GPT-4o [80]	Text, audio, image, and video	-	Multimodal autoregressive architecture	Pre-trained on a mix of public data, further trained for alignment with human preferences and safety considerations	Unified multimodal architecture that seamlessly processes and generates outputs across text, audio, image, and video, offering faster and more cost-effective operation than its predecessors	No	/
GPT-4V [274]	Text and image	-	-	Pre-trained on a large dataset of text and image data, followed by fine-tuning with reinforcement learning from human feedback (RLHF)	Notable for its multimodal capabilities, allowing it to analyze and understand images alongside text	No	/
Gemini [79]	Text, image, audio, and video	Nano versions: 1.8B/3.25B	Enhanced Transformer decoders	Large-scale pre-training on multimodal data, followed by supervised fine-tuning, reward modeling, and RLHF	Achieves state-of-the-art performance across multimodal tasks, including a groundbreaking 90% on the MMLU benchmark, and demonstrates capacity for on-device deployment with small model sizes	No	/
Claude 3.5 Sonnet (Computer Use) [150], [151]	Text and image	-	ReAct-based reasoning	-	Pioneering role in GUI automation as the first public beta model to utilize a vision-only paradigm for desktop task automation	No	/
Qwen-VL [275]	Text and image	9.6B	A Vision Transformer (ViT) [375] as the visual encoder, with a large language model based on the Qwen-7B architecture	Two stages of pre-training and a final stage of instruction fine-tuning	Achieves state-of-the-art performance on vision-language benchmarks and supports fine-grained visual understanding	Yes	httpss://github.com/QwenLM/Qwen-VL
Qwen2-VL [276]	Text, image, and video	2B/7B/72B	ViT [375] as the vision encoder, paired with the Qwen2 series of language models	The ViT is trained with image-text pairs; all parameters are unfrozen for broader multimodal learning with various datasets; fine-tuning the LLM on instruction datasets	Introduces Naive Dynamic Resolution for variable resolution image processing and Multimodal Rotary Position Embedding for enhanced multimodal integration	Yes	httpss://github.com/QwenLM/Qwen2-VL
InternVL-2 [277], [278]	Text, image, video, and medical data	1B/2B/4B/8B/26B/40B	ViT as the vision encoder and a LLM as the language component	Progressive alignment strategy, starting with coarse data and moving to fine data	Demonstrates powerful capabilities in handling complex multimodal tasks with various model sizes	Yes	httpss://internvl.github.io/blog/2024-07-02-InternVL-2.0/
CogVLM [279]	Text and image	17B	A ViT encoder, a two-layer MLP adapter, a pre-trained large language model, and a visual expert module	Stage 1 focuses on image captioning; Stage 2 combines image captioning and referring expression comprehension tasks	Achieves deep integration of visual and language features while preserving the full capabilities of large language models	Yes	httpss://github.com/THUDM/CogVLM
Ferret [280]	Text and image	7B/13B	Decoder-only architecture based on the Vicuna model, combined with a visual encoder	A combination of supervised training and additional instruction tuning	Ability to handle free-form region inputs via its hybrid region representation, enabling versatile spatial understanding and grounding	Yes	httpss://github.com/apple/ml-ferret
LLaVA [281]	Text and image	7B/13B	A vision encoder (CLIP ViT-L/14), a language decoder (Vicuna)	Pre-training using filtered image-text pairs, fine-tuning with a multimodal instruction-following dataset	Its lightweight architecture enables quick experimentation, demonstrating capabilities close to GPT-4 in multimodal reasoning	Yes	httpss://llava-vl.github.io
LLaVA-1.5 [282]	Text and image	7B/13B	A vision encoder (CLIP-ViT) and an encoder-decoder LLM architecture (*e.g.*, Vicuna or LLaMA)	Pre-training on vision-language alignment with image-text pairs; visual instruction tuning with specific task-oriented data	Notable for its data efficiency and scaling to high-resolution image inputs	Yes	httpss://llava-vl.github.io
BLIP-2 [283]	Text and image	3.4B/12.1B	A frozen image encoder, a lightweight Querying Transformer to bridge the modality gap, and a frozen large language model	Vision-language representation learning: trains the Q-Former with a frozen image encoder; Vision-to-language generative learning: connects the Q-Former to a frozen LLM to enable image-to-text generation	Achieves state-of-the-art performance on various vision-language tasks with a compute-efficient strategy by leveraging frozen pre-trained models	Yes	httpss://github.com/salesforce/LAVIS/tree/main/projects/blip2
Phi-3.5-Vision [284]	Text and image	4.2B	Image encoder: CLIP ViT-L/14 to process visual inputs, and transformer decoder based on the Phi-3.5-mini model for textual outputs	Pre-training on a combination of interleaved image-text datasets, synthetic OCR data, chart/table comprehension data, and text-only data; supervised fine-tuning using large-scale multimodal and text datasets; Direct Preference Optimization (DPO) to improve alignment, safety, and multimodal task performance	Excels in reasoning over visual and textual inputs, demonstrating competitive performance on single-image and multi-image tasks while being compact	Yes	httpss://github.com/microsoft/Phi-3CookBook/tree/main

reasoning. This subsection explores these foundation models, highlighting their innovations, contributions, and suitability for GUI agent applications. For a quick reference, Table 13 presents an overview of the key models and their characteristics.

8.1.1 Close-Source Models

While proprietary models are not openly available for customization, they offer powerful capabilities that can be directly utilized as the "brain" of GUI agents.

Among these, GPT-4V [274] and GPT-4o [80] are most

commonly used in existing GUI agent frameworks due to their strong abilities, as discussed in Section 6. GPT-4V represents a significant advancement in multimodal AI, combining text and image analysis to expand the functionality of traditional LLMs. Its ability to understand and generate responses based on both textual and visual inputs makes it well-suited for GUI agent tasks that require deep multimodal reasoning. Although its deployment is limited due to safety and ethical considerations, GPT-4V underscores the potential of foundation models to revolutionize GUI agent development with enhanced efficiency and flexibility.

Similarly, GPT-4o offers a unified multimodal autoregressive architecture capable of processing text, audio, images, and video. This model excels in generating diverse outputs efficiently, achieving faster response times at lower costs compared to its predecessors. Its rigorous safety and alignment practices make it reliable for sensitive tasks, positioning it as a robust tool for intelligent GUI agents that require comprehensive multimodal comprehension.

The **Gemini** model family [79] advances multimodal AI modeling by offering versions tailored for high-complexity tasks, scalable performance, and on-device efficiency. Notably, the Nano models demonstrate significant capability in reasoning and coding tasks despite their small size, making them suitable for resource-constrained devices. Gemini's versatility and efficiency make it a compelling choice for powering GUI agents that require both performance and adaptability.

Emphasizing industry investment in GUI automation, **Claude 3.5 Sonnet (Computer Use)** introduces a pioneering approach by utilizing a vision-only paradigm for desktop task automation [150], [151]. It leverages real-time screenshots to observe the GUI state and generate actions, eliminating the need for metadata or underlying GUI structure. This model effectively automates GUI tasks by interpreting the screen, moving the cursor, clicking buttons, and typing text. Its unique architecture integrates a ReAct-based [201] reasoning paradigm with selective observation, reducing computational overhead by observing the environment only when necessary. Additionally, Claude 3.5 maintains a history of GUI screenshots, enhancing task adaptability and enabling dynamic interaction with software environments in a human-like manner. Despite challenges in handling dynamic interfaces and error recovery, this model represents a significant step forward in creating general-purpose GUI agents. Its development highlights substantial industry investment in this area, indicating a growing focus on leveraging LLMs for advanced GUI automation.

8.1.2 Open-Source Models

Open-source models provide flexibility for customization and optimization, allowing developers to tailor GUI agents with contextual data and deploy them on devices with limited resources.

The **Qwen-VL** series [275] is notable for its fine-grained visual understanding and multimodal capabilities. With a Vision Transformer-based visual encoder and the Qwen-7B language model [363], it achieves state-of-the-art results on vision-language benchmarks while supporting multilingual interactions. Its efficiency and open-source availability, along with quantized versions for resource efficiency, make it suitable for developing GUI agents that require precise visual comprehension.

Building upon this, **Qwen2-VL** [276] introduces innovations like Naive Dynamic Resolution and Multimodal Rotary Position Embedding, enabling efficient processing of diverse modalities including extended-length videos. The scalable versions of Qwen2-VL balance computational efficiency and performance, making them adaptable for both on-device applications and complex multimodal tasks in GUI environments.

InternVL-2 [277], [278] combines a Vision Transformer with a Large Language Model to handle text, images, video, and medical data inputs. Its progressive alignment strategy and availability in various sizes allow for flexibility in deployment. By achieving state-of-the-art performance in complex multimodal tasks, InternVL-2 demonstrates powerful capabilities that are valuable for GUI agents requiring comprehensive multimodal understanding.

Advancing efficient integration of visual and linguistic information, **CogVLM** [279] excels in cross-modal tasks with a relatively small number of trainable parameters. Its ability to deeply integrate visual and language features while preserving the full capabilities of large language models makes it a cornerstone for GUI agent development, especially in applications where resource efficiency is critical.

Enhancing spatial understanding and grounding, **Ferret** [280] offers an innovative approach tailored for GUI agents. By unifying referring and grounding tasks within a single framework and employing a hybrid region representation, it provides precise interaction with graphical interfaces. Its robustness against object hallucinations and efficient architecture make it ideal for on-device deployment in real-time GUI applications.

The **LLaVA** model [281] integrates a visual encoder with a language decoder, facilitating efficient alignment between modalities. Its lightweight projection layer and modular design enable quick experimentation and adaptation, making it suitable for GUI agents that require fast development cycles and strong multimodal reasoning abilities. Building on this, **LLaVA-1.5** [282] introduces a novel MLP-based cross-modal connector and scales to high-resolution image inputs, achieving impressive performance with minimal training data. Its data efficiency and open-source availability pave the way for widespread use in GUI applications requiring detailed visual reasoning.

BLIP-2 [283] employs a compute-efficient strategy by leveraging frozen pre-trained models and introducing a lightweight Querying Transformer. This design allows for state-of-the-art performance on vision-language tasks with fewer trainable parameters. BLIP-2's modularity and efficiency make it suitable for resource-constrained environments, highlighting its potential for on-device GUI agents.

Finally, **Phi-3.5-Vision** [284] achieves competitive performance in multimodal reasoning within a compact model size. Its innovative training methodology and efficient integration of image and text understanding make it a robust candidate for GUI agents that require multimodal reasoning and on-device inference without the computational overhead of larger models.

In summary, both close-source and open-source foundation models have significantly advanced the capabilities of LLM-powered GUI agents. While proprietary models offer

powerful out-of-the-box performance, open-source models provide flexibility for customization and optimization, enabling tailored solutions for diverse GUI agent applications. The innovations in multimodal reasoning, efficiency, and scalability across these models highlight the evolving landscape of foundation models, paving the way for more intelligent and accessible GUI agents.

8.2 Large Action Models

While general-purpose foundation LLMs excel in capabilities like multimodal understanding, task planning, and tool utilization, they often lack the specialized optimizations required for GUI-oriented tasks. To address this, researchers have introduced *Large Action Models* (LAMs)—foundation LLMs fine-tuned with contextual, GUI-specific datasets (as outlined in Section 7) to enhance their action-driven capabilities. These models represent a significant step forward in refining the "brain" of GUI agents for superior performance.

In the realm of GUI agents, LAMs provide several transformative advantages:

1) **Enhanced Action Orientation:** By specializing in action-oriented tasks, LAMs enable accurate interpretation of user intentions and generation of precise action sequences. This fine-tuning ensures that LAMs can seamlessly align their outputs with GUI operations, delivering actionable steps tailored to user requests.
2) **Specialized Planning for Long, Complex Tasks:** LAMs excel in devising and executing intricate, multi-step workflows. Whether the tasks span multiple applications or involve interdependent operations, LAMs leverage their training on extensive action sequence datasets to create coherent, long-term plans. This makes them ideal for productivity-focused tasks requiring sophisticated planning across various tools.
3) **Improved GUI Comprehension and Visual Grounding:** Training on datasets that incorporate GUI screenshots allows LAMs to advance their abilities in detecting, localizing, and interpreting UI components such as buttons, menus, and forms. By utilizing visual cues instead of relying solely on structured UI metadata, LAMs become highly adaptable, performing effectively across diverse software environments.
4) **Efficiency through Model Size Reduction:** Many LAMs are built on smaller foundational models—typically around 7 billion parameters—that are optimized for GUI-specific tasks. This compact, purpose-driven design reduces computational overhead, enabling efficient operation even in resource-constrained environments, such as on-device inference.

As illustrated in Figure 23, the process of developing a purpose-built LAM for GUI agents begins with a robust, general-purpose foundation model, ideally with VLM capabilities. Fine-tuning these models on comprehensive, specialized GUI datasets—including user instructions, widget trees, UI properties, action sequences, and annotated screenshots—transforms them into optimized LAMs, effectively equipping them to serve as the "brain" of GUI agents.

This optimization bridges the gap between planning and execution. A general-purpose LLM might provide only textual plans or abstract instructions in response to user queries, which may lack precision. In contrast, a LAM-empowered GUI agent moves beyond planning to actively and intelligently execute tasks on GUIs. By interacting directly with application interfaces, these agents perform tasks with remarkable precision and adaptability. This paradigm shift marks the evolution of GUI agents from passive task planners to active, intelligent executors.

In the following sections, we present an analysis of LAMs tailored for GUI agents across different platforms, summarized in Tables 14, 15, and 16, followed by an in-depth discussion in the subsequent subsections.

8.3 LAMs for Web GUI Agents

In the domain of web-based GUI agents, researchers have developed specialized LAMs that enhance interaction and navigation within web environments. These models are tailored to understand the complexities of web GUIs, including dynamic content and diverse interaction patterns.

Building upon the need for multimodal understanding, **WebGUM** [140] integrates HTML understanding with visual perception through temporal and local tokens. It leverages Flan-T5 [356] for instruction fine-tuning and ViT [375] for visual inputs, enabling it to process both textual and visual information efficiently. This multimodal grounding allows WebGUM to generalize tasks effectively, significantly outperforming prior models on benchmarks like MiniWoB++ [133] and WebShop [318]. With its data-efficient design and capacity for multi-step reasoning, WebGUM underscores the importance of combining multimodal inputs in enhancing GUI agent performance.

Addressing the challenge of multi-step reasoning and planning in GUI environments, researchers have introduced frameworks that incorporate advanced search and learning mechanisms. For instance, **Agent Q** [229] employs MCTS combined with self-critique mechanisms and Direct Preference Optimization (DPO) [386] to improve success rates in complex tasks such as product search and reservation booking. By fine-tuning the LLaMA-3 70B model [78] to process HTML DOM representations and generate structured action plans, thoughts, and environment-specific commands, this framework showcases the power of integrating reasoning, search, and iterative fine-tuning for autonomous agent development.

Leveraging smaller models for efficient web interaction, **GLAINTEL** [285] demonstrates that high performance can be achieved without large computational resources. Utilizing the Flan-T5 [356] model with 780M parameters, it focuses on dynamic web environments like simulated e-commerce platforms. The model incorporates RL to optimize actions such as query formulation and navigation, effectively integrating human demonstrations and unsupervised learning. Achieving results comparable to GPT-4-based methods at a fraction of the computational cost, GLAINTEL underscores the potential of reinforcement learning in enhancing web-based GUI agents for task-specific optimization.

To enable continuous improvement and generalization across varied web domains, **OpenWebVoyager** [287] combines imitation learning with an iterative exploration-feedback-optimization cycle. Leveraging large multimodal models like Idefics2-8B [376], it performs autonomous web navigation

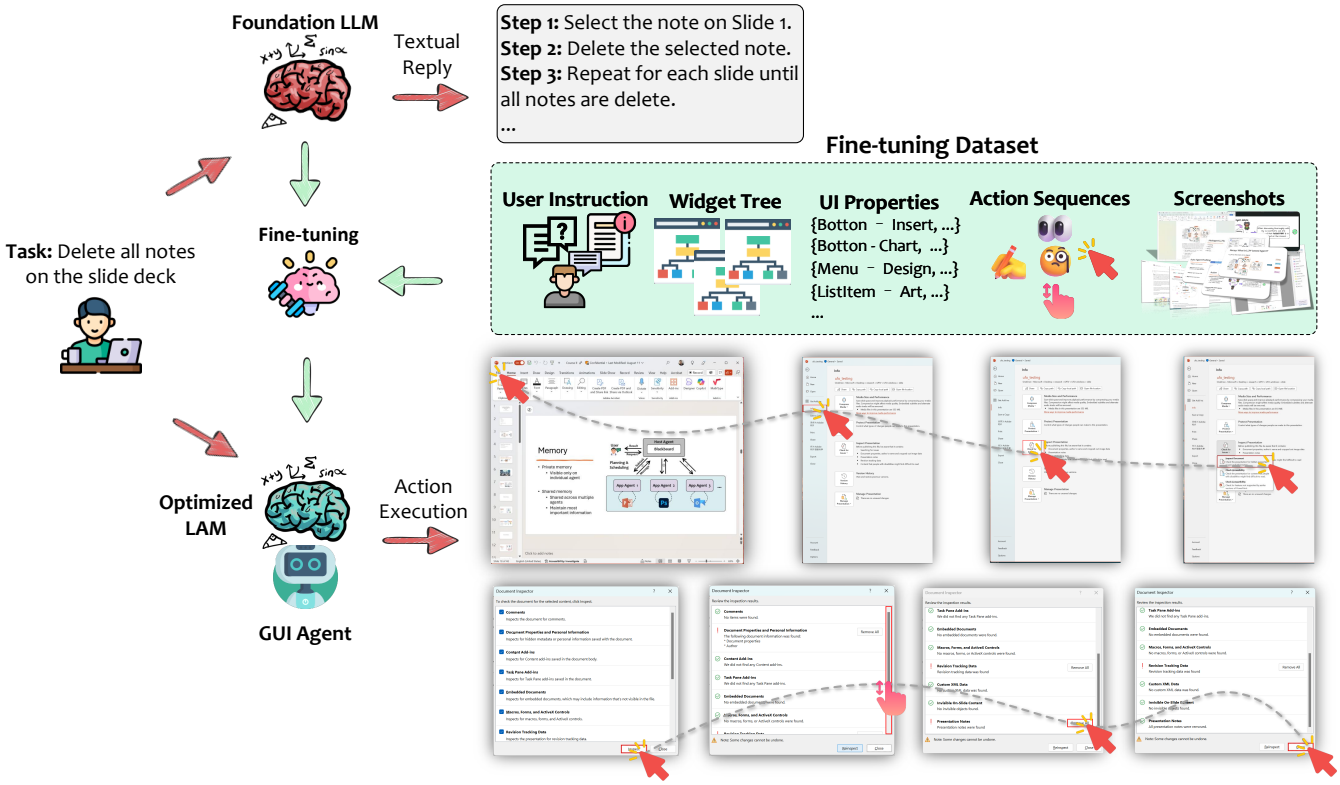


Fig. 23: The evolution from foundation LLMs to GUI agent-optimized LAM with fine-tuning.

tasks. By training on diverse datasets and fine-tuning using trajectories validated by GPT-4 feedback, the agent addresses real-world complexities without relying on synthetic environments. This approach significantly advances GUI agent frameworks by demonstrating the capability to generalize across varied web domains and tasks.

Moreover, tackling challenges such as sparse training data and policy distribution drift, **WebRL** [288] introduces a self-evolving curriculum and robust reward mechanisms for training LLMs as proficient web agents. By dynamically generating tasks based on the agent's performance, WebRL fine-tunes models like Llama-3.1 [78] and GLM-4 [357], achieving significant success rates in web-based tasks within the WebArena environment. This framework outperforms both proprietary APIs and other open-source models, highlighting the effectiveness of adaptive task generation and sustained learning improvements in developing advanced GUI agents.

These advancements in LAMs for web GUI agents illustrate the importance of integrating multimodal inputs, efficient model designs, and innovative training frameworks to enhance agent capabilities in complex web environments.

8.4 LAMs for Mobile GUI Agents

Mobile platforms present unique challenges for GUI agents, including diverse screen sizes, touch interactions, and resource constraints. Researchers have developed specialized LAMs to address these challenges, enhancing interaction and navigation within mobile environments.

Focusing on detailed UI understanding, **MobileVLM** [268] introduces an advanced vision-language model designed specifically for mobile UI manipulation tasks. Built on Qwen-VL-Chat [275], it incorporates mobile-specific pretraining tasks for intra- and inter-UI comprehension. By leveraging the Mobile3M dataset—a comprehensive corpus of 3 million UI pages and interaction traces organized into directed graphs—the model excels in action prediction and navigation tasks. MobileVLM's novel two-stage pretraining framework significantly enhances its adaptability to mobile UIs, outperforming existing VLMs in benchmarks like ScreenQA [387] and Auto-UI [239]. This work highlights the effectiveness of tailored pretraining in improving mobile GUI agent performance.

Addressing the need for robust interaction in dynamic environments, **DigiRL** [212] presents a reinforcement learning-based framework tailored for training GUI agents in Android environments. By leveraging offline-to-online RL, DigiRL adapts to real-world stochasticity, making it suitable for diverse, multi-step tasks. Unlike prior models reliant on imitation learning, DigiRL autonomously learns from interaction data, refining itself to recover from errors and adapt to new scenarios. The use of a pre-trained Vision-Language Model with 1.3 billion parameters enables efficient processing of GUI screenshots and navigation commands. Its performance on the AITW dataset demonstrates a significant improvement over baseline methods, positioning DigiRL as a benchmark in the development of intelligent agents optimized for complex GUI interactions.

To enhance GUI comprehension and reduce reliance on textual data, **VGA** [258] employs fine-tuned vision-language models that prioritize image-based cues such as shapes, colors, and positions. Utilizing the RICO [259] dataset for training, VGA is tailored for Android GUIs and employs a two-

TABLE 14: An overview of GUI-optimized models (Part I).

Model	Platform	Foundation Model	Size	Input	Output	Dataset	Highlights	Link
Agent Q [229]	Web	LLaMA-3 70B [78]	70B	HTML DOM representations	Plans, thoughts, actions, and action explanations	WebShop benchmark and OpenTable dataset	Combines Monte Carlo Tree Search (MCTS) with self-critique mechanisms, leveraging reinforcement learning to achieve exceptional performance	https://github.com/sentient%2Dengineering/agent-q
GLAINTEL [285]	Web	Flan-T5 [356]	780M	User instructions and observations of webpage state	GUI actions	1.18M real-world products, 12,087 crowd-sourced natural language intents, 1,010 human demonstrations	Efficient use of smaller LLMs, and integration of RL and human demonstrations for robust performance	/
WebN-T5 [286]	Web	T5 [74]	-	HTML and DOM with screenshots	Hierarchical navigation plans and GUI interactions	MiniWoB++, 13,000 human-made demonstrations	Combines supervised learning and reinforcement learning to address limitations of previous models in memorization and generalization	/
OpenWeb-Voyager [287]	Web	Idefics2-8b-instruct [376]	8B	GUI screenshots, accessibility trees	Actions on GUI, planning and thought, answers to queries	Mind2Web and WebVoyager datasets, and generated queries for real-world web navigation	Combining imitation learning with a feedback loop for continuous improvement	https://github.com/MinorJerry/OpenWebVoyager
WebRL [288]	Web	Llama-3.1 [78] and GLM-4 [377]	8B/9B/70B	Task instructions, action history, HTML content	Actions, element identifiers, explanations or notes	WebArena-Lite	Introduces a self-evolving online curriculum reinforcement learning framework, which dynamically generates tasks based on past failures and adapts to the agent's skill level	https://github.com/THUDM/WebRL
WebGUM [140]	Web	Flan-T5 [356] and Vision Transformer (ViT) [375]	3B	HTML, screenshots, interaction history, instructions	Web navigation actions and free-form text	MiniWoB++ and WebShop benchmarks	Integrates temporal and local multimodal perception, combining HTML and visual tokens, and uses an instruction-finetuned language model for enhanced reasoning and task generalization	https://console.cloud.google.com/storage/browser/gresearch/webllm
Mobile-VLM [268]	Mobile Android	Qwen-VL-Chat [275]	9.8B	Screenshots and structured XML documents	Action predictions, navigation steps, and element locations	Mobile3M, includes 3 million UI pages, 20+ million actions, and XML data structured as directed graphs	Mobile-specific pretraining tasks that enhance intra- and inter-UI understanding, with a uniquely large and graph-structured Chinese UI dataset (Mobile3M)	https://github.com/XiaoMi/mobilevlm
Octo-planner [266]	Mobile devices	Phi-3 Mini [284]	3.8B	User queries and available function descriptions	Execution steps	1,000 data samples generated using GPT-4	Optimized for resource-constrained devices to ensure low latency, privacy, and offline functionality	https://huggingface.co/NexaAIDev/octopus-planning
DigiRL [212]	Mobile Android	AutoUI [239]	1.3B	Screenshots	GUI actions	AiTW	Offline-to-online reinforcement learning, bridging gaps in static and dynamic environments	https://github.com/DigiRL-agent/digirl
LVG [292]	Mobile Android	Swin Transformer [378] and BERT [69]	-	UI screenshots and free-form language expressions	Bounding box coordinates	UIBert dataset and synthetic dataset	Unifies detection and grounding tasks through layout-guided contrastive learning	/

stage fine-tuning process to align responses with both visual data and human intent. The model excels in understanding GUI layouts, predicting design intents, and facilitating precise user interactions. By outperforming existing models like GPT-4V in GUI comprehension benchmarks, VGA sets a new standard for accuracy and efficiency in mobile GUI agents.

In the context of lightweight and efficient models, **UINav** [297] demonstrates a practical system for training neural agents to automate UI tasks on mobile devices. It balances accuracy, generalizability, and computational efficiency through macro actions and an error-driven demonstration collection process. UINav uses a compact encoder-decoder architecture and SmallBERT [382] for text and screen element encoding, making it suitable for on-device inference. A key innovation is its ability to generalize across diverse tasks and apps with minimal demonstrations, addressing key challenges in UI automation with a versatile framework.

These models collectively advance the field of mobile GUI agents by addressing platform-specific challenges through innovative training methods, efficient model architectures, and specialized datasets.

8.5 LAMs for Computer GUI Agents

For desktop and laptop environments, GUI agents must handle complex applications, multitasking, and varied interaction modalities. Specialized LAMs for computer GUI agents enhance capabilities in these settings, enabling more sophisticated task execution.

Integrating planning, acting, and reflecting phases, **ScreenAgent** [273] is designed for autonomous interaction with computer screens. Based on CogAgent [14], it is fine-tuned using the ScreenAgent Dataset, providing comprehensive GUI interaction data across diverse tasks. With inputs as screenshots and outputs formatted in JSON for mouse and keyboard actions, ScreenAgent achieves precise UI element localization and handles continuous multi-step tasks. Its capability to process real-time GUI interactions using a foundation model sets a new benchmark for LLM-powered GUI agents, making it an ideal reference for future research in building more generalized intelligent agents.

Bridging high-level planning with real-world manipulation, **Octopus** [299] represents a pioneering step in embodied vision-language programming. Leveraging the MPT-7B [383]

TABLE 15: An overview of GUI-optimized models (Part II).

Model	Platform	Foundation Model	Size	Input	Output	Dataset	Highlights	Link
Ferret-UI [265]	Android and iPhone platforms	Ferret [280]	7B/13B	Raw screen pixels, sub-images divided for finer resolution, bounding boxes and regional annotations	Widget bounding boxes, text from OCR tasks, descriptions of UI elements or overall screen functionality, UI interaction actions	Generated from RICO (for Android) and AMP (for iPhone)	Multi-platform support with high-resolution adaptive image encoding	https://github.com/apple/ml-ferret/tree/main/ferretui
Octopus [293]	Mobile devices	CodeLlama-7B [379], Google Gemma 2B [380]	7B, 2B	API documentation examples	Function names with arguments for API calls	RapidAPI Hub	Use of conditional masking to enforce correct output formatting	/
Octopus v2 [294]	Edge devices	Gemma-2B [380]	2B	User queries and descriptions of available functions	Function calls with precise parameters	20 Android APIs, with up to 1,000 data points generated for training	Functional tokenization strategy, which assigns unique tokens to function calls, significantly reducing the context length required for accurate prediction	/
Octopus v3 [295]	Edge devices	CLIP-based model and a causal language model	Less than 1 billion parameters	Queries and commands, images and functional tokens	Functional tokens for actions	Leveraged from Octopus v2 [294]	Introduction of functional tokens for multimodal applications enables the representation of any function as a token, enhancing the model's flexibility	/
Octopus v4 [296]	Serverless cloud-based platforms and edge devices	17 models	Varies	User queries	Domain-specific answers, actions	Synthetic datasets similar to Octopus v2	Graph-based framework integrating multiple specialized models for optimized performance	https://github.com/NexaAI/octopus-v4
VGA [258]	Mobile Android	LLaVA-v1.6-mistral-7B [281]	7B	GUI screenshots with positional, visual, and hierarchical data	Actions and function calls, descriptions of GUI components, navigation and task planning	63.8k-image dataset constructed from the RICO	Minimizes hallucinations in GUI comprehension by employing an image-centric fine-tuning approach, ensuring balanced attention between text and visual content	https://github.com/Linziyang1999/VGA%2Dvisual%2DGUI%2Dassistant
MobileFlow [146]	Mobile phones	Qwen-VL-Chat [275]	21B	GUI screenshots with OCR textual information and bounding boxes	GUI actions and question answering	70k manually labeled business-specific data spanning 10 business sectors, and datasets like RefCOCO, ScreenQA, Flickr30K	Hybrid visual encoder capable of variable-resolution input and Mixture of Experts (MoE) [381] for enhanced performance and efficiency	/
UINav [297]	Mobile Android	SmallBERT [382]	Agent model: 320k, Referee model: 430k, SmallBERT model: 17.6MB	UI elements, utterance, screen representation	Predicted actions and element to act upon	43 tasks across 128 Android apps and websites, collecting 3,661 demonstrations	Introduces a macro action framework and an error-driven demonstration collection process, significantly reducing training effort while enabling robust task performance with small, efficient models suitable for mobile devices	/
Screen-Agent [273]	Linux and Windows desktop	CogAgent [14]	18B	GUI screenshots	Mouse and keyboard actions	273 task sessions	Comprehensive pipeline of planning, acting, and reflecting to handle real computer screen operations autonomously	https://github.com/niuzaisheng/ScreenAgent
Octopus [299]	Desktop	MPT-7B [383] and CLIP ViT-L/14 [361]	7B	Visual images, scene graphs containing objects and relations, environment messages	Executable action code and plans	OctoGibson: 476 tasks with structured initial and goal states; OctoMC: 40 tasks across biomes; OctoGTA: 25 crafted tasks spanning different game settings	Incorporates reinforcement learning with environmental feedback	https://choiszt.github.io/Octopus/

and CLIP ViT-L/14 [361], Octopus integrates egocentric and bird's-eye views for visual comprehension, generating executable action code. Trained using the OctoVerse suite, its datasets encompass richly annotated environments like OmniGibson, Minecraft, and GTA-V, covering routine and reasoning-intensive tasks. Notably, Octopus innovates through Reinforcement Learning with Environmental Feedback, ensuring adaptive planning and execution. Its vision-dependent functionality offers seamless task generalization in unseen scenarios, underscoring its capability as a unified model for embodied agents operating in complex GUI environments.

These developments in computer GUI agents highlight the integration of advanced visual comprehension, planning, and action execution, paving the way for more sophisticated and capable desktop agents.

TABLE 16: An overview of GUI-optimized models (Part III).

Model	Platform	Foundation Model	Size	Input	Output	Dataset	Highlights	Link
RUIG [298]	Mobile and desktop	Swin Transformer [378] and BART [75]	4 decoder layers	UI screenshots and text instructions	Bounding box predictions in linguistic form	MoTIF dataset and RicoSCA dataset for mobile UI data and Common Crawl for desktop UI data	Innovatively uses policy gradients to improve the spatial decoding in the pixel-to-sequence paradigm	/
CogAgent [14]	PC, web, and Android platforms	CogVLM-17B [279]	18B	GUI screenshots combined with OCR-derived text	Task plans, action sequences, and textual descriptions	CCS400K, text recognition datasets: 80M synthetic text images, visual grounding datasets and GUI dataset Mind2Web and AiTW	High-resolution cross-module to balance computational efficiency and high-resolution input processing	https://github.com/THUDM/CogVLM
SeeClick [23]	iOS, Android, macOS, Windows, and web	Qwen-VL [275]	9.6B	GUI screenshots and textual instructions	GUI actions and element locations for interaction	300k webpages with text and icons, RICO, and data from LLaVA	Ability to perform GUI tasks purely from screenshots and its novel GUI grounding pre-training approach	https://github.com/njucckevin/SeeClick
ScreenAI [272]	Mobile, desktop, and tablet UIs	PaLI-3 [384]	5B	GUI screenshots with OCR text, image captions, and other visual elements	Text-based answers for questions, screen annotations with bounding box coordinates and labels, navigation instructions, summaries of screen content	262M mobile web screenshots and 54M mobile app screenshots	Unified representation of UIs and infographics, combining visual and textual elements	https://github.com/kyegomez/ScreenAI
Ferret-UI 2 [289]	iPhone, Android, iPad, Web, AppleTV	Vicuna-13B [362], Gemma-2B [380], Llama3-8B [78]	Vicuna-13B [362], Gemma-2B [380], Llama3-8B [78]	UI screenshots, annotated bounding boxes and labels for UI widgets, OCR-detected text and bounding boxes for text elements, source HTML hierarchy trees for web data	Descriptions of UI elements, widget classification, OCR, tapability, and text/widget location, interaction instructions and multi-round interaction-based QA	Core-set, GroundUI-18k, GUIDE, Spotlight	Multi-platform support with high-resolution adaptive image encoding	/
OmniParser [169]	Mobile, desktop, and webpage	BLIP-2 [283], YOLOv8 [367]	-	UI screenshots with bounding boxes and numeric IDs, and structured representations including text from OCR and descriptions of functionality	IDs, bounding boxes, and descriptions for interactable elements	67,000 UI screenshots labeled with bounding boxes and 7,185 icon-description pairs using GPT-4	Introduces a vision-only screen parsing framework, enabling general UI understanding without reliance on external information, significantly improving action prediction accuracy for LLM-powered agents	https://github.com/microsoft/OmniParser
ShowUI [290]	Websites, desktops, and mobile phones	Phi-3.5-Vision [284]	4.2B	GUI screenshots with OCR for text-based UI elements and visual grounding for icons and widgets	GUI actions, navigation, element location	ScreenSpot, RICO, GUIEnv, GUIAct, AiTW, AiTZ, GUI-World	Interleaved Vision-Language-Action approach, allowing seamless navigation, grounding, and understanding of GUI environments	https://github.com/showlab/ShowUI
OS-ATLAS [291]	Windows, macOS, Linux, Android, and the web	InternVL-2 [277] and Qwen2-VL [275]	4B/7B	GUI screenshots	GUI actions	AndroidControl, SeeClick, and others annotated with GPT-4, over 13 million GUI elements and 2.3 million screenshots	The first foundation action model designed for generalist GUI agents, supporting cross-platform GUI tasks, and introducing a unified action space	https://osatlas.github.io/
xLAM [257]	Diverse environments	Mistral-7B [359] and DeepSeek-Coder-7B [385]	Range from 1B to 8×22B	Unified function-calling data formats	Function calls, thought processes	Synthetic and augmented data, including over 60,000 high-quality samples generated using APIGen from 3,673 APIs across 21 categories	Excels in function-calling tasks by leveraging unified and scalable data pipelines	https://github.com/SalesforceAIResearch/xLAM

8.6 Cross-Platform Large Action Models

To achieve versatility across various platforms, cross-platform LAMs have been developed, enabling GUI agents to operate seamlessly in multiple environments such as mobile devices, desktops, and web interfaces.

CogAgent [14] stands out as an advanced visual language model specializing in GUI understanding and navigation across PC, web, and Android platforms. Built on CogVLM [279], it incorporates a novel high-resolution cross-module to process GUI screenshots efficiently, enabling detailed comprehension of GUI elements and their spatial relationships. Excelling in tasks requiring OCR and GUI grounding, CogAgent achieves state-of-the-art performance on benchmarks like Mind2Web [251] and AITW [262]. Its ability to generate accurate action plans and interface with GUIs positions it as a pivotal step in developing intelligent

agents optimized for GUI environments.

Focusing on universal GUI understanding, **Ferret-UI 2** [289] from Apple is a state-of-the-art multimodal large language model designed to master UI comprehension across diverse platforms, including iPhones, Android devices, iPads, web, and AppleTV. By employing dynamic high-resolution image encoding, adaptive gridding, and high-quality multimodal training data generated through GPT-4, it outperforms its predecessor and other competing models in UI referring, grounding, and interaction tasks. Ferret-UI 2's advanced datasets and innovative training techniques ensure high accuracy in spatial understanding and user-centered interactions, setting a new benchmark for cross-platform UI adaptability and performance.

Advancing GUI automation, **ShowUI** [290] introduces a pioneering Vision-Language-Action model that integrates high-resolution visual inputs with textual understanding to perform grounding, navigation, and task planning. Optimized for web, desktop, and mobile environments, ShowUI leverages the Phi-3.5-vision-instruct backbone and comprehensive datasets to achieve robust results across benchmarks like ScreenSpot [23] and GUI-Odyssey [263]. Its ability to process multi-frame and dynamic visual inputs alongside JSON-structured output actions highlights its versatility. With innovations in interleaved image-text processing and function-calling capabilities, ShowUI sets a new standard for LLM-powered GUI agents.

Addressing the need for a unified action space, **OS-ATLAS** [291] introduces a foundational action model specifically designed for GUI agents across platforms like Windows, macOS, Linux, Android, and the web. By leveraging a massive multi-platform dataset and implementing a unified action space, OS-ATLAS achieves state-of-the-art performance in GUI grounding and out-of-distribution generalization tasks. Its scalable configurations adapt to varying computational needs while maintaining versatility in handling natural language instructions and GUI elements. As a powerful open-source alternative to commercial solutions, OS-ATLAS marks a significant step toward democratizing access to advanced GUI agents.

These cross-platform LAMs demonstrate the potential of unified models that can adapt to diverse environments, enhancing the scalability and applicability of GUI agents in various contexts.

8.7 Takeaways

The exploration of LAMs for GUI agents has revealed several key insights that are shaping the future of intelligent interaction with graphical user interfaces:

1) **Smaller Models for On-Device Inference:** Many of the optimized LAMs are built from smaller foundational models, often ranging from 1 billion to 7 billion parameters. This reduction in model size enhances computational efficiency, making it feasible to deploy these models on resource-constrained devices such as mobile phones and edge devices. The ability to perform on-device inference without relying on cloud services addresses privacy concerns and reduces latency, leading to a more responsive user experience.
2) **Enhanced GUI Comprehension Reduces Reliance on Structured Data:** Models like VGA [258] and OmniParser [169] emphasize the importance of visual grounding and image-centric fine-tuning to reduce dependency on structured UI metadata. By improving GUI comprehension directly from visual inputs, agents become more adaptable to different software environments, including those where structured data may be inaccessible or inconsistent.
3) **Reinforcement Learning Bridges Static and Dynamic Environments:** The application of reinforcement learning in models like DigiRL [212] demonstrates the effectiveness of bridging static training data with dynamic real-world environments. This approach allows agents to learn from interactions, recover from errors, and adapt to changes, enhancing their robustness and reliability in practical applications.
4) **Unified Function-Calling Enhances Interoperability:** Efforts to standardize data formats and function-calling mechanisms, as seen in models like xLAM [257], facilitate multi-turn interactions and reasoning across different platforms. This unification addresses compatibility issues and enhances the agent's ability to perform complex tasks involving multiple APIs and services.

The advancements in LAMs for GUI agents highlight a trend toward specialized, efficient, and adaptable models capable of performing complex tasks across various platforms. By focusing on specialization, multimodal integration, and innovative training methodologies, researchers are overcoming the limitations of general-purpose LLMs. These insights pave the way for more intelligent, responsive, and user-friendly GUI agents that can transform interactions with software applications.

9 Evaluation for LLM-Brained GUI Agents

In the domain of GUI agents, evaluation is crucial for enhancing both functionality and user experience [52], [53] and should be conducted across multiple aspects. By systematically assessing these agents' effectiveness across various tasks, evaluation not only gauges their performance in different dimensions but also provides a framework for their continuous improvement [388]. Furthermore, it encourages innovation by identifying areas for potential development, ensuring that GUI agents evolve in tandem with advancements in LLMs and align with user expectations.

As illustrated in Figure 24, when a GUI agent completes a task, it produces an action sequence, captures screenshots, extracts UI structures, and logs the resulting environment states. These outputs serve as the foundation for evaluating the agent's performance through various metrics and measurements across diverse platforms. In the subsequent sections, we delve into these evaluation methodologies, discussing the metrics and measurements used to assess GUI agents comprehensively. We also provide an overview of existing benchmarks tailored for GUI agents across different platforms, highlighting their key features and the challenges they address.

9.1 Evaluation Metrics

Evaluating GUI agents requires robust and multidimensional metrics to assess their performance across various dimensions, including accuracy, efficiency, and compliance (*e.g.*,

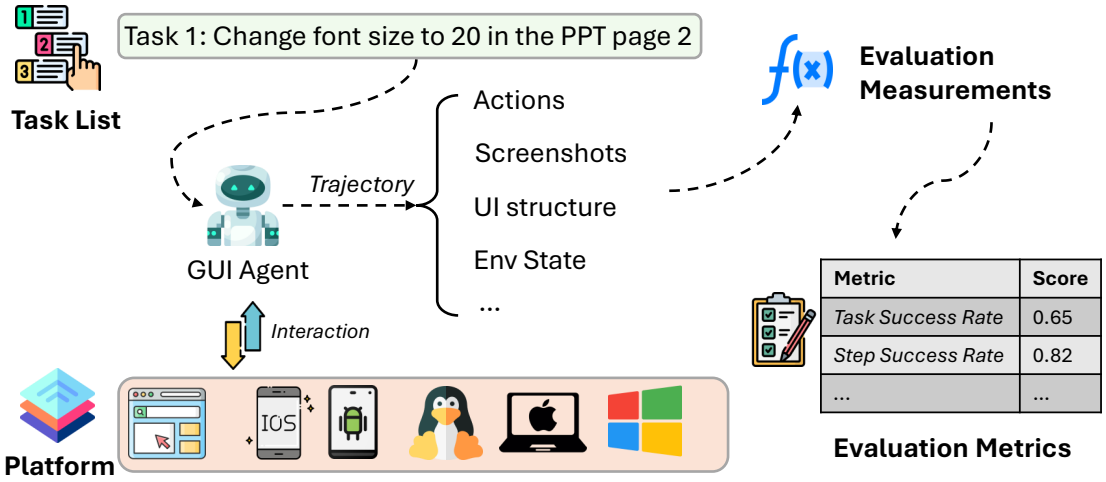


Fig. 24: An illustrative example of evaluation of task completion by a GUI agent.

safety). In a typical benchmarking setup, the GUI agent is provided with a natural language instruction as input and is expected to autonomously execute actions until the task is completed. During this process, various assets can be collected, such as the sequence of actions taken by the agent, step-wise observations (*e.g.*, DOM or HTML structures), screenshots, runtime logs, final states, and execution time. These assets enable evaluators to determine whether the task has been completed successfully and to analyze the agent's performance. In this section, we summarize the key evaluation metrics commonly used for benchmarking GUI agents. Note that different research works may use different names for these metrics, but with similar calculations. We align their names in this section.

1) **Step Success Rate:** Completing a task may require multiple steps. This metric measures the ratio of the number of steps that are successful over the total steps within a task. A high step success rate indicates precise and accurate execution of granular steps, which is essential for the reliable performance of tasks involving multiple steps [251], [262], [304].
2) **Turn Success Rate:** A *turn* indicates a single interaction between the user and the agent. A turn may consist of multiple steps, and completing a task may consist of multiple turns. This metric measures the ratio of turns that successfully address the request in that interaction over all turns. It focuses on the agent's ability to understand and fulfill user expectations during interactive or dialog-based tasks, ensuring the agent's responsiveness and reliability across iterative interactions, particularly in tasks requiring dynamic user-agent communication [142], [253].
3) **Task Success Rate:** Task success rate measures the successful task completion over all tasks set in the benchmark. It evaluates whether the final task completion state is achieved while ignoring the intermediate steps. This metric provides an overall measure of end-to-end task completion, reflecting the agent's ability to handle complex workflows holistically [308], [318], [320].
4) **Efficiency Score:** Efficiency score evaluates how effectively the agent completes tasks while considering resource consumption, execution time, or total steps the agent might take. This metric can be broken down into the following sub-metrics:
   - **Time Cost:** Measures the time taken to complete tasks.
   - **Resource Cost:** Measures the memory/CPU/GPU usage to complete tasks.
   - **LLM Cost:** Evaluates the computational or monetary cost of LLM calls used during task execution.
   - **Step Cost:** Measures the total steps required to complete tasks.

   Depending on the specific metrics used, the efficiency score can be interpreted differently in different papers [324], [326].
5) **Completion under Policy:** This metric measures the rate at which tasks are completed successfully while adhering to policy constraints. It ensures that the agent complies with user-defined or organizational rules, such as security, ethical, safety, privacy, or business guidelines, during task execution. This metric is particularly relevant for applications where compliance is as critical as task success [305].
6) **Risk Ratio:** Similar to the previous metric, the risk ratio evaluates the potential risk associated with the agent's actions during task execution. It identifies vulnerabilities, errors, or security concerns that could arise during task handling. A lower risk ratio indicates higher trustworthiness and reliability, while a higher ratio may suggest areas needing improvement to minimize risks and enhance robustness [305].

The implementation of metrics in each GUI agent benchmark might vary depending on the platform and the task formulation. In Tables 17 to 21, we mapped the original metrics used in the benchmarks, which may possess different names, to the categories that we defined above.

9.2 Evaluation Measurements

To effectively evaluate GUI agents, various measurement techniques are employed to assess their accuracy and alignment with expected outputs. These measurements validate different aspects of agent performance, ranging from textual

and visual correctness to interaction accuracy and system state awareness, using code, models, and even agents as evaluators [24]. Below, we summarize key measurement approaches used in benchmarking GUI agents. Based on these measurements, the evaluation metrics defined beforehand can be calculated accordingly.

1) **Text Match:** This measurement evaluates whether the text-based outputs of the agent match the expected results. For example, whether a target product name is reached when the agent is browsing an e-commerce website. It can involve different levels of strictness, including:
   - **Exact Match:** Ensures the output perfectly matches the expected result.
   - **Partial or Fuzzy Match:** Allows for approximate matches, which are useful for handling minor variations such as typos or synonyms.
   - **Semantic Similarity:** Measures deeper alignment in semantic meaning using techniques like cosine similarity of text embeddings or other semantic similarity measures.

   Text Match is widely applied in tasks involving textual selections, data entry, or natural language responses.
2) **Image Match:** Image Match focuses on validating whether the agent acts or stops on the expected page (*e.g.*, webpage, app UI), or selects the right image. It involves comparing screenshots, selected graphical elements, or visual outcomes against ground truth images using image similarity metrics or visual question answering (VQA) methods. This measurement is particularly crucial for tasks requiring precise visual identification.
3) **Element Match:** This measurement checks whether specific widget elements (*e.g.*, those in HTML, DOM, or application UI hierarchies) interacted with by the agent align with the expected elements. These may include:
   - **HTML Tags and Attributes:** Ensuring the agent identifies and interacts with the correct structural elements.
   - **URLs and Links:** Validating navigation-related elements.
   - **DOM Hierarchies:** Confirming alignment with expected DOM structures in dynamic or complex web interfaces.
   - **UI Controls and Widgets:** Verifying interactions with platform-specific controls such as buttons, sliders, checkboxes, dropdown menus, or other GUI components in desktop and mobile applications.
   - **Accessibility Identifiers:** Utilizing accessibility identifiers or resource IDs in mobile platforms like Android and iOS to ensure correct element selection.
   - **View Hierarchies:** Assessing alignment with expected view hierarchies in mobile applications, similar to DOM hierarchies in web applications.
   - **System Controls and APIs:** Ensuring correct interaction with operating system controls or APIs, such as file dialogs, system menus, or notifications in desktop environments.

   Element Match ensures robust interaction with user interface components across different platforms during task execution.
4) **Action Match:** This measurement assesses the accuracy of the agent's actions, such as clicks, scrolls, or keystrokes, by comparing them against an expected sequence. It involves:
   - **Action Accuracy:** Validates that each action (including action type and its arguments) is performed correctly (*e.g.*, clicking the correct button, typing the right input).
   - **Action Sequence Alignment:** Ensures actions occur in the correct order to meet task requirements.
   - **Location Prediction:** Checks that spatial actions, such as mouse clicks or touch gestures, target the intended regions of the interface.

   Action Match is vital for evaluating step-wise correctness in task completion.
5) **State Information:** State Information captures runtime data related to the system's environment during task execution. It provides insights into contextual factors that may influence the agent's behavior, such as:
   - **Application State:** Information about the state of the application being interacted with (*e.g.*, open files, active windows, saved files in given locations).
   - **System Logs:** Detailed logs recording the agent's decisions and interactions.
   - **Environment Variables:** Contextual data about the operating system or runtime environment.

   This measurement is valuable for debugging, performance analysis, and ensuring reliability under diverse conditions.

Each of these measurement techniques contributes to a comprehensive evaluation framework, ensuring that the agent not only completes tasks but does so with precision, efficiency, and adaptability. Together, they help build trust in the agent's ability to perform reliably in real-world scenarios while maintaining compliance with policy constraints.

9.3 Evaluation Platforms

Evaluating GUI agents requires diverse platforms to capture the varying environments in which these agents operate. The platforms span web, mobile, and desktop environments, each with unique characteristics, challenges, and tools for evaluation. This section summarizes the key aspects of these platforms and their role in benchmarking GUI agents.

1) **Web:** Web platforms are among the most common environments for GUI agents, reflecting their prevalence in everyday tasks such as browsing, form filling, and data scraping. Key characteristics of web platforms for evaluation include:
   - **Dynamic Content:** Web applications often involve dynamic elements generated through JavaScript, AJAX, or similar technologies, requiring agents to handle asynchronous updates effectively.
   - **Diverse Frameworks:** The variety of web technologies (*e.g.*, HTML, CSS, JavaScript frameworks) demands robust agents capable of interacting with a range of interface designs and structures.
   - **Tools and Libraries:** Evaluation often uses tools such as Selenium, Puppeteer, or Playwright to emulate

browser interactions, collect runtime information, and compare outcomes against expected results.
- **Accessibility Compliance:** Metrics like WCAG (Web Content Accessibility Guidelines) adherence can also be evaluated to ensure inclusivity.

2) **Mobile:** Mobile platforms, particularly Android and iOS, pose unique challenges for GUI agents due to their constrained interfaces and touch-based interactions. Evaluating agents on mobile platforms involves:
   - **Screen Size Constraints:** Agents must adapt to limited screen real estate, ensuring interactions remain accurate and efficient.
   - **Touch Gestures:** Evaluating the agent's ability to simulate gestures such as taps, swipes, and pinches is essential.
   - **Platform Diversity:** Android devices vary significantly in terms of screen sizes, resolutions, and system versions, while iOS offers more standardized conditions.
   - **Evaluation Tools:** Tools like Appium and Espresso (for Android) or XCTest (for iOS) and emulators are commonly used for testing and evaluation.
3) **Desktop:** Desktop platforms provide a richer and more complex environment for GUI agents, spanning multiple operating systems such as Windows, macOS, and Linux. Evaluations on desktop platforms often emphasize:
   - **Application Diversity:** Agents must handle a wide range of desktop applications, including productivity tools, web browsers, and custom enterprise software.
   - **Interaction Complexity:** Desktop interfaces often include advanced features such as keyboard shortcuts, drag-and-drop, and context menus, which agents must handle correctly.
   - **Cross-Platform Compatibility:** Evaluations may involve ensuring agents can operate across multiple operating systems and versions.
   - **Automation Frameworks:** Tools such as Windows UI Automation, macOS Accessibility APIs, and Linux's AT-SPI are used to automate and monitor agent interactions.
   - **Resource Usage:** Memory and CPU usage are significant metrics, particularly for long-running tasks or resource-intensive applications.

Each platform presents distinct challenges and opportunities for evaluating GUI agents. Web platforms emphasize scalability and dynamic interactions, mobile platforms focus on touch interfaces and performance, and desktop platforms require handling complex workflows and cross-application tasks. Some benchmarks are cross-platform, requiring agents to be robust, adaptable, and capable of generalizing across different environments.

All the metrics, measurements, and platforms discussed are essential for a comprehensive evaluation of GUI agents across multiple aspects. Most existing benchmarks rely on them for evaluation. In what follows, we first provide an overview of these benchmarks for GUI agents in Tables 17 to 21, and then detail them selectively in the following subsections.

9.4 Web Agent Benchmarks

Evaluating GUI agents in web environments necessitates benchmarks that capture the complexities and nuances of web-based tasks. Over the years, several benchmarks have been developed, each contributing unique perspectives and challenges to advance the field.

One of the pioneering efforts in this domain is **MiniWoB++** [132], [133], focusing on assessing reinforcement learning agents on web-based GUI tasks. It introduces realistic interaction scenarios, including clicking, typing, and navigating web elements, and leverages workflow-guided exploration (WGE) to improve efficiency in environments with sparse rewards. Agents are evaluated based on success rates, determined by their ability to achieve final goal states, highlighting adaptability and robustness across various complexities.

Building upon the need for more realistic environments, **Mind2Web** [251] represents a significant advancement by enabling agents to handle real-world HTML environments rather than simplified simulations. Established after the advent of LLMs [144], it offers a large dataset of over 2,000 tasks spanning multiple domains, presenting challenges from basic actions to complex multi-page workflows. The benchmark emphasizes end-to-end task performance through metrics like Element Accuracy and Task Success Rate, encouraging rigorous evaluation of agents.

Extending Mind2Web's capabilities, **MT-Mind2Web** [302] introduces conversational web navigation, requiring sophisticated interactions that span multiple turns with both users and the environment. This advanced benchmark includes 720 web navigation conversation sessions with 3,525 instruction and action sequence pairs, averaging five user-agent interactions per session, thereby testing agents' conversational abilities and adaptability.

To further enhance realism, **WebArena** [300] sets a new standard with its realistic web environment that mimics genuine human interactions. Featuring 812 tasks across multiple domains, it requires agents to perform complex, long-horizon interactions over multi-tab web interfaces. By focusing on functional correctness rather than surface-level matches, WebArena promotes thorough assessment of agents' practical abilities.

Recognizing the importance of multimodal capabilities, **VisualWebArena**, an extension of WebArena [300], was designed to assess agents on realistic visually grounded web tasks. Comprising 910 diverse tasks in domains like Classifieds, Shopping, and Reddit, it adds new visual functions for measuring open-ended tasks such as visual question answering and fuzzy image matching, thereby challenging agents in multimodal understanding.

Similarly, **VideoWebArena** [312] focuses on evaluating agents' abilities to comprehend and interact with video content on the web. It presents 74 videos across 2,021 tasks, challenging agents in video-based information retrieval, contextual reasoning, and skill application. This benchmark highlights critical deficiencies in current models, emphasizing the need for advancements in agentic reasoning and video comprehension.

Complementing this, **VisualWebBench** [314] offers a multimodal benchmark that assesses understanding, OCR,

TABLE 17: Overview of GUI agent benchmarks (Part I).

Benchmark	Platform	Year	Highlight	Data Size	Metric	Measurement	Link
MiniWoB++ [132], [133]	Web	2017	Evaluates agents on basic web interactions like clicking, typing, and form navigation.	100 web interaction tasks	Task Success Rate	Element Match	https://github.com/Farama%2DFoundation/miniwob%2Dplusplus
RUSS [129]	Web	2021	Uses ThingTalk for mapping natural language to web actions, enabling precise web-based task execution in real HTML environments.	741 instructions	Task Success Rate	Text Match, Element Match	https://github.com/xnancy/russ
WebShop [318]	Web	2022	Simulates e-commerce navigation with real-world products, challenging agents with instruction comprehension, multi-page navigation, and strategic exploration.	12,087 instructions	Task Success Rate, Step Success Rate"	Text Match	https://webshop-pnlp.github.io/
Mind2Web [251]	Web	2023	Tests adaptability on real-world, dynamic websites across domains.	2,000 tasks	Step Success Rate, Task Success Rate	Element Match, Action Match	https://github.com/OSU-NLP-Group/Mind2Web
WebArena [300]	Web	2023	Simulates realistic, multi-tab browsing on Docker-hosted websites, focusing on complex, long-horizon tasks that mirror real online interactions.	812 long-horizon tasks	Step Success Rate	Text Match	https://webarena.dev/
VisualWebArena [301]	Web	2024	Assesses multimodal agents on visually grounded tasks, requiring both visual and textual interaction capabilities in web environments.	910 tasks	Step Success Rate	Text Match, Image Match	https://jykoh.com/vwa
MT-Mind2Web [302]	Web	2024	Introduces conversational web navigation with multi-turn interactions, supported by a specialized multi-turn web dataset.	720 sessions/3525 instructions	Step Success Rate, Turn Success Rate	Element Match, Action Match	https://github.com/magicgh/self-map
MMInA [303]	Web	2024	Tests multihop, multimodal tasks on real-world websites, requiring agents to handle cross-page information extraction and reasoning for complex tasks.	1,050 tasks	Step Success Rate, Task Success Rate	Text Match, Element Match	https://mmina.cliangyu.com/
WebCanvas [304]	Web	2024	Provides intermediate action tracking for realistic task assessment, along with an updated Mind2Web-Live dataset and tools for annotation.	542 tasks	Step Success Rate, Task Success Rate, Efficiency Score	Element Match, Text Match, trajectory length	https://huggingface.co/datasets/iMeanAI/Mind2Web-Live
AutoWebBench [218]	Web	2024	Bilingual web browsing benchmark with 10,000 browsing traces, supporting evaluation across language-specific environments.	10,000 traces	Step Success Rate, Efficiency Score	Element Match, Action Match, Time	https://github.com/THUDM/AutoWebGLM
WorkArena [311]	Web	2024	Focuses on real-world enterprise software interactions, targeting tasks frequently performed by knowledge workers	19,912 unique task instances	Task Success Rate, Efficiency Score, Completion under Policy, Turn Success Rate	Element Match, Text Match, Execution-based Validation	https://github.com/ServiceNow/WorkArena

grounding, and reasoning across website, element, and action levels. Spanning 1.5K samples from real-world websites, it identifies challenges such as poor grounding and subpar OCR with low-resolution inputs, providing a crucial evaluation perspective distinct from general multimodal benchmarks.

Beyond the challenges of multimodality, understanding agents' resilience to environmental distractions is crucial. **EnvDistraction** [313] introduces a benchmark that evaluates the faithfulness of multimodal GUI agents under non-malicious distractions, such as pop-ups and recommendations. The study demonstrates that even advanced agents are prone to such distractions, revealing vulnerabilities that necessitate robust multimodal perception for reliable automation.

Focusing on safety and trustworthiness, **ST-WebAgentBench** [305] takes a unique approach by emphasizing the management of unsafe behaviors in enterprise settings. It features a human-in-the-loop system and a policy-driven hierarchy, introducing the Completion under Policy (CuP) metric to evaluate agents' compliance with organizational, user, and task-specific policies. This benchmark operates in web environments using BrowserGym [311] and includes 235 tasks with policies addressing various safety dimensions, providing a comprehensive framework for evaluating agents in enterprise scenarios.

TABLE 18: Overview of GUI agent benchmarks (Part II).

Benchmark	Platform	Year	Highlight	Data Size	Metric	Measurement	Link
VideoWebArena [312]	Web	2024	Focuses on long-context multimodal agents using video tutorials for task completion	74 videos amounting to approximately 4 hours, with 2,021 tasks in total	Task Success Rate, Intermediate Intent Success Rate, Efficiency Scores	Element Match, State Information, Exact and Fuzzy Text Matches	https://github.com/ljang0/videowebarena
EnvDistraction [313]	Web	2024	Evaluates the "faithfulness" of multimodal GUI agents by assessing their susceptibility to environmental distractions, such as pop-ups, fake search results, or misleading recommendations	1,198 tasks	Task Success Rate	Text Match, Element Match, State Information	https://github.com/xbmxb/EnvDistraction
WebVLN-v1 [252]	Web	2024	Combines navigation and question-answering on shopping sites, integrating visual and textual content for unified web interaction evaluation.	8,990 paths and and 14,825 QA pairs	Task Success Rate, Efficiency Score	Element Match, Path Length, Trajectory Length	https://github.com/WebVLN/WebVLN
WEBLINX [253]	Web	2024	Focuses on conversational navigation, requiring agents to follow multi-turn user instructions in realistic, dialogue-based web tasks.	100k interactions	Turn Success Rate	Element Match, Text Match, Action Match	https://mcgill-nlp.github.io/weblinx/
ST-WebAgentBench [305]	Web	2024	Evaluates policy-driven safety in web agents, using the Completion under Policy metric to ensure compliance in enterprise-like environments.	235 tasks	Task Success Rate, Completion under Policy (CuP), Risk Ratio	Element Match, Action Match, Text Match	https://sites.google.com/view/st-webagentbench/home
CompWoB [306]	Web	2023	Tests agents on sequential, compositional tasks that require state management across multiple steps, simulating real-world automation scenarios.	50 compositional tasks	Task Success Rate	Element Match	https://github.com/google-research/google-research/tree/master/compositional_rl/compwob
TURKING BENCH [307]	Web	2024	Uses natural HTML tasks from crowdsourcing to assess interaction skills with real-world web layouts and elements.	32.2K instances	Task Success Rate	Text Match, Element Match, Image Match	https://turkingbench.github.io
Mind2Web-Live [226]	Web	2024	Frames web navigation as functionality-driven question-answering, allowing agents to generalize across diverse web tasks and optimize navigation efficiency.	104 samples	Task Success Rate, Efficiency Score	Text Match, Image Match, Element Match, Path Length	https://anonymous.4open.science/r/naviqate
VisualWebBench [314]	Web	2024	Provides a fine-grained assessment of multimodal large language models (MLLMs) on web-specific tasks	1,534 instances from 139 real websites across 87 subdomains	Task Success Rate, Turn Success Rate, Efficiency Metrics	Text Match, Image Match, Element Match, Action Match	https://visualwebbench.github.io/
WONDERBREAD [315]	Web	2024	Focuses on business process management (BPM) tasks like documentation, knowledge transfer, and process improvement	2,928 human demonstrations across 598 distinct workflows	Task Success Rate, Step Success Rate, Efficiency Score, Completion under Policy	Text Match, Action Match, State Information	https://github.com/HazyResearch/wonderbread
WebOlympus [316]	Web	2024	An open platform for web agents that simplifies running demos, evaluations, and data collection for web agents on live websites	50 tasks	Task Success Rate, Step Success Rate	Action Match	/

Addressing the automation of enterprise software tasks, **WorkArena** [311] offers a benchmark emphasizing tasks commonly performed within the ServiceNow platform. With 19,912 unique instances across 33 tasks, it highlights the significant performance gap between current state-of-the-art agents and human capabilities in enterprise UI automation, setting a trajectory for future innovation.

In the realm of interacting with live websites, **WebOlympus** [316] introduces an open platform that enables web agents to interact with live websites through a Chrome extension-based interface. Supporting diverse tasks and integrating a safety monitor to prevent harmful actions, it

TABLE 19: Overview of GUI agent benchmarks (Part III).

Benchmark	Platform	Year	Highlight	Data Size	Metric	Measurement	Link
AndroidEnv [211]	Android	2021	Provides an open-source platform based on the Android ecosystem with over 100 tasks across approximately 30 apps, focusing on reinforcement learning for various Android interactions.	100+ tasks	NA	NA	https://github.com/google-deepmind/android_env
PIXELHELP [131]	Android	2020	Includes a corpus of natural language instructions paired with UI actions across four task categories, aiding in grounding language to UI interactions.	187 multi-step instructions	Step Success Rate	Element Match, Action Match	https://github.com/google-research/google-research/tree/master/seq2act
Mobile-Env [320]	Android	2024	Comprehensive toolkit for Android GUI benchmarks to enable controlled evaluations of real-world app interactions.	224 tasks	Task Success Rate, Step Success Rate	Text Match, Element Match, Image Match, State Information	https://github.com/X-LANCE/Mobile-Env
B-MOCA [321]	Android	2024	Benchmarks mobile device control agents on realistic tasks, incorporating UI layout and language randomization to evaluate generalization capabilities.	131 tasks	Task Success Rate	Element Match, State Information	https://b-moca.github.io/
AndroidWorld [322]	Android	2024	Offers a dynamic Android environment, allowing for diverse natural language instruction testing.	116 tasks	Task Success Rate	State Information	https://github.com/google-research/android_world
Mobile-Eval [145]	Android	2024	Benchmark based on mainstream Android apps, and designed to test common mobile interactions.	30 instructions	Task Success Rate, Step Success Rate, Efficiency Score	Text Match, Path Length	https://github.com/X-PLUG/MobileAgent
DroidTask [143]	Android	2024	Android Task Automation benchmark supports exploration and task recording in real apps with corresponding GUI action traces.	158 tasks	Step Success Rate, Task Success Rate	Element Match, Action Match	https://github.com/MobileLLM/AutoDroid
AITW [262]	Android	2023	A large-scale dataset, which is partly inspired by PIXELHELP, covering diverse Android interactions.	715,142 episodes	Task Success Rate, Step Success Rate	Action Match	https://github.com/google-research/google-research/tree/master/android_in_the_wild
AndroidArena [323]	Android	2024	Focuses on daily cross-app and constrained tasks within the Android ecosystem, providing single-app and multi-app interaction scenarios.	221 tasks	Task Success Rate, Step Success Rate, Efficiency Score	Action Match, Path Length	https://github.com/AndroidArenaAgent/AndroidArena
ANDROIDLAB [269]	Android	2024	Provides a structured evaluation framework with 138 tasks across nine apps, supporting both text-only and multimodal agent evaluations on Android.	138 tasks	Task Success Rate, Step Success Rate, Efficiency Score	Element Match, Image Match	https://github.com/THUDM/Android-Lab

promotes safer automation of web-based tasks and provides a critical tool for evaluating agent performance in realistic scenarios.

Collectively, these benchmarks have significantly contributed to advancing the evaluation of web-based GUI agents, each addressing different aspects such as realism, multimodality, safety, and enterprise applicability. Their developments reflect the evolving challenges and requirements in creating sophisticated agents capable of complex web interactions.

9.5 Mobile Agent Benchmarks

Evaluating GUI agents on mobile platforms presents unique challenges due to the diversity of interactions and the complexity of mobile applications. Several benchmarks have been developed to address these challenges, each contributing to the advancement of mobile agent evaluation.

An early effort in this domain is **PIXELHELP** [131], which focuses on grounding natural language instructions to actions on mobile user interfaces. Addressing the significant challenge of interpreting and executing complex, multi-step tasks, PIXELHELP provides a comprehensive dataset pairing English instructions with human-performed actions on a mobile UI emulator. It comprises 187 multi-step instructions across four task categories, offering a robust resource for evaluating models on task accuracy through metrics like Complete Match and Partial Match.

Building upon the need for systematic evaluation, **ANDROIDLAB** [269] establishes a comprehensive framework for Android-based autonomous agents. It introduces both an

TABLE 20: Overview of GUI agent benchmarks (Part IV).

Benchmark	Platform	Year	Highlight	Data Size	Metric	Measurement	Link
LlamaTouch [327]	Mobile Android	2024	Enables faithful and scalable evaluations for mobile UI task automation by matching task execution traces against annotated essential states	496 tasks covering 57 unique Android applications	Task Success Rate, Step Success Rate, Efficiency Score	Text Match, Action Match, State Information Match	https://github.com/LlamaTouch/LlamaTouch
MobileAgentBench [328]	Mobile Android	2024	Provides a fully autonomous evaluation process on real Android devices and flexibility in judging success conditions across multiple paths to completion	100 tasks across 10 open-source Android applications	Task Success Rate, Efficiency Score, Latency, Token Cost	State Information (UI State Matching)	https://mobileagentbench.github.io/
Mobile-Bench [324]	Android	2024	Supports both UI and API-based actions in multi-app scenarios, testing agents on single and multi-task structures with a checkpoint-based evaluation approach.	832 entries (200+ tasks)	Task Success Rate, Step Success Rate, Efficiency Score	Action Match, Path Length	https://github.com/XiaoMi/MobileBench
Mobile Safety Bench [325]	Android	2024	Prioritizes safety evaluation in mobile control tasks, with distinct tasks focused on helpfulness, privacy, and legal compliance.	100 tasks	Task Success Rate, Risk Mitigation Success	Action Match with Safety Considered, Element Match, State Information	https://mobilesafetybench.github.io/
SPA-BENCH [326]	Android	2024	Extensive evaluation framework supporting single-app and cross-app tasks in English and Chinese, providing a plug-and-play structure for diverse task scenarios.	340 tasks	Task Success Rate, Step Success Rate, Efficiency Score	Action Match, State Information, Time Spent, API Cost	https://spa-bench.github.io
VisualAgent Bench [254]	Web, Android, Game, Virtual Embodied.	2024	First benchmark designed for visual foundation agents across GUI and multimodal tasks, focusing on vision-centric interactions in Android, web, and game environments.	4,482 trajectories	Task Success Rate	Text Match	https://github.com/THUDM/VisualAgentBench/
OSWorld [308]	Linux, Windows, macOS, Web	2024	Scalable, real computer environment for multimodal agents, supporting task setup, execution-based evaluation, and interactive learning across Ubuntu, Windows, and macOS.	369 Ubuntu tasks, 43 Windows tasks	Task Success Rate	Execution-based State Information (such as internal file interpretation, permission management)	https://os-world.github.io/
Windows Agent Arena [329]	Windows	2024	Adaptation of OSWorld focusing exclusively on the Windows OS with diverse multi-step tasks, enabling agents to use a wide range of applications and tools.	154 tasks	Task Success Rate	Same as OSWorld, scalable with cloud parallelization	https://microsoft.github.io/WindowsAgentArena
OmniACT [309]	MacOS, Linux, Windows, Web	2024	Assesses agents' capability to generate executable programs for computer tasks across desktop and web applications in various OS environments, prioritizing multimodal challenges.	9,802 data points	Task Success Rate, Step Success Rate	Action Match	https://huggingface.co/datasets/Writer/omniact

action space and operational modes that support consistent evaluations for text-only and multimodal models. By providing XML and SoM operation modes, ANDROIDLAB allows LLMs and LMMs to simulate real-world interactions in equivalent environments. The benchmark includes 138 tasks across nine apps, encompassing typical Android functionalities, and evaluates agents using metrics such as Success Rate and Reversed Redundancy.

To further challenge agents in handling both API and UI operations, **Mobile-Bench** [324] offers an innovative approach by combining these elements within a realistic Android environment. Its multi-app setup and three distinct task categories test agents' capabilities in handling simple and complex mobile interactions, pushing beyond traditional single-app scenarios. The evaluation leverages CheckPoint metrics, assessing agents at each key action step, providing insights into planning and decision-making skills.

Emphasizing safety in mobile device control, **MobileSafetyBench** [325] provides a structured evaluation framework that prioritizes both helpfulness and safety. It rigorously tests agents across common mobile tasks within an Android emulator, focusing on layered risk assessment, including legal compliance and privacy. A distinctive feature is its indirect prompt injection test to probe agent robustness. The evaluation ensures agents are scored on practical success while managing risks, advancing research in LLM reliability and secure autonomous device control.

Expanding the scope to multiple languages and applica-

TABLE 21: Overview of GUI agent benchmarks (Part V).

Benchmark	Platform	Year	Highlight	Data Size	Metric	Measurement	Link
VideoGUI [310]	Windows, Web	2024	Focuses on visual-centric tasks from instructional videos, emphasizing planning and action precision in applications like Adobe Photoshop and Premiere Pro.	178 tasks, 463 subtasks	Task Success Rate	State Information, Action Match	https://showlab.github.io/videogui
Spider2-V [330]	Linux	2024	Benchmarks agents across data science and engineering workflows in authentic enterprise software environments, covering tasks from data ingestion to visualization.	494 tasks	Task Success Rate	Action Match, State Information	https://spider2-v.github.io
Act2Cap [248]	Windows	2024	Emphasizes GUI action narration using cursor-based prompts in video format, covering a variety of GUI interactions like clicks, typing, and dragging.	4,189 samples	Step Success Rate	Element Match	https://showlab.github.io/GUI-Narrator
OFFICEBENCH [331]	Linux	2024	Tests cross-application automation in office workflows with complex multi-step tasks across applications like Word and Excel, assessing operational integration in realistic scenarios.	300 tasks	Task Success Rate	Action match, Text Match, State Information	https://github.com/zlwang-cs/OfficeBench
AssistGUI [96]	Windows Platform	2024	The first benchmark focused on task-oriented desktop GUI automation	100 tasks from 9 popular applications	Task Success Rate, Efficiency Score	Element Match, Action Match	https://showlab.github.io/assistgui/
SPR Benchmark [317]	Mobile, Web, and Operating Systems	2024	Evaluates GUI screen readers' ability to describe both content and layout information	Includes 650 screenshots annotated with 1,500 target points and regions	Task Success Rate, Efficiency Score	Text Match, Element Match	/
AgentStudio [332]	Windows, Linux, macOS	2024	Open toolkit for creating and benchmarking general-purpose virtual agents, supporting complex interactions across diverse software applications.	NA	Step Success Rate	Action Match, State Information and Image Match	https://computer-agents.github.io/agent-studio/
CRAB [319]	Linux, Android	2024	Cross-environment benchmark evaluating agents across mobile and desktop devices, using a graph-based evaluation method to handle multiple correct paths and task flexibility.	120 tasks	Step Success Rate, Effiency Score	Action Match	https://github.com/crab-benchmark
ScreenSpot [23]	iOS, Android, macOS, Windows, Web.	2024	Vision-based GUI benchmark with pre-trained GUI grounding, assessing agents' ability to interact with GUI elements across mobile, desktop, and web platforms using only screenshots.	1,200 instructions	Step Success Rate	Action Match	https://github.com/njucckevin/SeeClick

tion scenarios, **SPA-BENCH** [326] introduces an extensive benchmark for smartphone agents. It assesses both single-app and cross-app tasks in a plug-and-play framework that supports seamless agent integration. With a diverse task collection across Android apps, including system and third-party apps, SPA-BENCH offers a realistic testing environment measuring agent capabilities in understanding UIs and handling app navigation through metrics like success rate, efficiency, and resource usage.

Focusing on efficient and user-friendly evaluation, **MobileAgentBench** [328] presents a benchmark tailored for agents on Android devices. It offers a fully autonomous testing process, leveraging final UI state matching and real-time app event tracking. With 100 tasks across 10 open-source Android applications categorized by difficulty, it accommodates multiple paths to success, enhancing reliability and applicability. Comprehensive metrics, including task success rate, efficiency, latency, and token cost, provide insights into agent performance.

Complementing these efforts, **LlamaTouch** [327] introduces a benchmark and testbed for mobile UI task automation in real-world Android environments. Emphasizing essential state annotation, it enables precise evaluation of tasks regardless of execution path variability or dynamic UI elements.

With 496 tasks spanning 57 unique applications, LlamaTouch demonstrates scalability and fidelity through advanced matching techniques, integrating pixel-level screenshots and textual screen hierarchies, reducing false negatives and supporting diverse task complexities.

Collectively, these benchmarks have significantly advanced the evaluation of mobile-based GUI agents, addressing challenges in task complexity, safety, efficiency, and scalability. Their contributions are instrumental in developing more capable and reliable agents for mobile platforms.

9.6 Computer Agent Benchmarks

Evaluating GUI agents on desktop computers involves diverse applications and complex workflows. Several benchmarks have been developed to assess agents' capabilities in these environments, each addressing specific challenges and advancing the field.

An early benchmark in this domain is **Act2Cap** [248], which focuses on capturing and narrating GUI actions in video formats using a cursor as a pivotal visual guide. Act2Cap emphasizes the detailed nuances of GUI interactions, particularly cursor-based actions like clicks and drags, essential for advancing automation capabilities in GUI-intensive tasks. It includes a substantial dataset of 4,189 samples across various Windows GUI environments, employing metrics based on element-wise Intersection over Union to evaluate semantic accuracy and temporal and spatial precision.

To provide a scalable and genuine computer environment for multimodal agents, **OSWorld** [308] introduces a pioneering framework that supports task setup, execution-based evaluation, and interactive learning across multiple operating systems, including Ubuntu, Windows, and macOS. OSWorld serves as a unified environment that mirrors the complexity and diversity of real-world computer use, accommodating arbitrary applications and open-ended computer tasks. It includes a comprehensive suite of 369 tasks on Ubuntu and 43 tasks on Windows, utilizing execution-based evaluation metrics like success rate for rigorous assessment.

Building on OSWorld, **WindowsArena** [329] adapts the framework to create over 150 diverse tasks specifically for the Windows operating system. Focusing on multi-modal, multi-step tasks, it requires agents to demonstrate abilities in planning, screen understanding, and tool usage within a real Windows environment. Addressing the challenge of slow evaluation times, WindowsArena enables parallelized deployment in the Azure cloud, drastically reducing evaluation time and allowing for comprehensive testing across various applications and web domains.

Focusing on office automation tasks, **OFFICEBENCH** [331] introduces a groundbreaking framework for benchmarking LLM agents in realistic office workflows. Simulating intricate workflows across multiple office applications like Word, Excel, and Email within a Linux Docker environment, it evaluates agents' proficiency in cross-application automation. The benchmark challenges agents with complex tasks at varying difficulty levels, demanding adaptability to different complexities and use cases. Customized metrics assess operation accuracy and decision-making, providing critical insights into agents' capabilities in managing multi-application office scenarios.

Addressing the automation of data science and engineering workflows, **Spider2-V** [330] offers a distinctive benchmark. It features 494 real-world tasks across 20 enterprise-level applications, spanning the entire data science workflow from data warehousing to visualization. Assessing agents' abilities to handle both code generation and complex GUI interactions within authentic enterprise software environments on Ubuntu, it employs a multifaceted evaluation method that includes information-based validation, file-based comparison, and execution-based verification.

In the realm of productivity software, **AssistGUI** [96] provides a pioneering framework for evaluating agents' capabilities. It introduces an Actor-Critic Embodied Agent framework capable of complex hierarchical task planning, GUI parsing, and action generation. The dataset includes diverse tasks across design, office work, and system settings, supported by project files for reproducibility. By emphasizing outcome-driven evaluation with pixel-level precision and procedural adherence, AssistGUI highlights the potential and limitations of current LLM-based agents in managing intricate desktop software workflows.

Collectively, these benchmarks provide comprehensive evaluation frameworks for GUI agents on desktop platforms, addressing challenges in task complexity, cross-application automation, scalability, and fidelity. Their contributions are instrumental in advancing the development of sophisticated agents capable of complex interactions in desktop environments.

9.7 Cross-Platform Agent Benchmarks

To develop GUI agents capable of operating across multiple platforms, cross-platform benchmarks are essential. These benchmarks challenge agents to adapt to different environments and interfaces, evaluating their versatility and robustness.

Addressing this need, **VisualAgentBench** (VAB) [254] represents a pioneering benchmark for evaluating GUI and multimodal agents across a broad spectrum of realistic, interactive tasks. Encompassing platforms such as Web (WebArena-Lite [300]), Android (VAB-Mobile [269]), and game environments, VAB focuses on vision-based interaction and high-level decision-making tasks. The benchmark employs a multi-level data collection strategy involving human demonstrations, program-based solvers, and model bootstrapping. Evaluation metrics concentrate on success rates, ensuring comprehensive performance assessments in tasks like navigation and content modification, thereby filling a significant gap in benchmarking standards for GUI-based LLM agents.

Complementing this, **CRAB** [319] introduces an innovative benchmark by evaluating multimodal language model agents in cross-environment interactions. It uniquely supports seamless multi-device task execution, evaluating agents in scenarios where tasks span both Ubuntu Linux and Android environments. By introducing a graph-based evaluation method that breaks down tasks into sub-goals and accommodates multiple correct paths to completion, CRAB provides a nuanced assessment of planning, decision-making, and adaptability. Metrics such as Completion Ratio, Execution Efficiency, Cost Efficiency, and Success Rate offer comprehensive insights into agent performance.

Focusing on GUI grounding for cross-platform visual agents, **ScreenSpot** [23] offers a comprehensive benchmark emphasizing tasks that rely on interpreting screenshots rather than structured data. ScreenSpot includes over 600 screenshots and 1,200 diverse instructions spanning mobile (iOS, Android), desktop (macOS, Windows), and web platforms. It evaluates click accuracy and localization precision by measuring how effectively agents can identify and interact with GUI elements through visual cues alone. By challenging models with a wide variety of UI elements, ScreenSpot addresses real-world complexities, making it an essential resource for evaluating visual GUI agents across varied environments.

These cross-platform benchmarks collectively advance the development of GUI agents capable of operating seamlessly across multiple platforms. By providing comprehensive evaluation frameworks, they are instrumental in assessing and enhancing the versatility and adaptability of agents in diverse environments.

9.8 Takeaways

The evolution of GUI agent benchmarks reflects a broader shift towards more realistic, interactive, and comprehensive evaluation environments. This section highlights key trends and future directions in the benchmarking of LLM-brained GUI agents.

1) **Towards More Interactive and Realistic Environments:** Recent advancements in GUI agent benchmarking emphasize the transition from synthetic scenarios to more interactive and realistic environments. This shift is evident in the use of simulators, Docker containers, and real-world applications to create "live" environments that better mimic genuine user interactions. Such environments not only provide a more accurate assessment of agent capabilities but also pose new challenges in terms of performance and robustness.
2) **Cross-Platform Benchmarks:** The emergence of cross-platform benchmarks that encompass mobile, web, and desktop environments represents a significant step towards evaluating the generalizability of GUI agents. However, these benchmarks introduce fundamental challenges unique to each platform. A unified interface for accessing platform-specific information, such as HTML and DOM structures, could substantially streamline the benchmarking process and reduce implementation efforts. Future work should focus on standardizing these interfaces to facilitate seamless agent evaluation across diverse environments.
3) **Increased Human Interaction and Realism:** There is a growing trend towards incorporating more human-like interactions in benchmarks, as seen in multi-turn and conversational scenarios. These setups mirror real-world use cases more closely, thereby providing a rigorous test of an agent's ability to handle dynamic, iterative interactions. As GUI agents become more sophisticated, benchmarks must continue to evolve to include these nuanced interaction patterns, ensuring agents can operate effectively in complex, human-centric environments.
4) **Scalability and Automation Challenges:** Scalability remains a significant concern in benchmarking GUI agents. The creation of realistic tasks and the development of evaluation methods for individual cases often require substantial human effort. Automation of these processes could alleviate some of the scalability issues, enabling more extensive and efficient benchmarking. Future research should explore automated task generation and evaluation techniques to enhance scalability.
5) **Emphasis on Safety, Privacy, and Compliance:** There is a notable trend towards evaluating GUI agents on safety, privacy, and compliance metrics. These considerations are increasingly important as agents are integrated into sensitive and regulated domains. Encouraging this trend will help ensure that agents not only perform tasks effectively but also adhere to necessary legal and ethical standards. Future benchmarks should continue to expand on these dimensions, incorporating evaluations that reflect real-world compliance and data security requirements.

The landscape of GUI agent benchmarking is rapidly evolving to meet the demands of increasingly complex and interactive environments. By embracing cross-platform evaluations, fostering human-like interactions, addressing scalability challenges, and prioritizing safety and compliance, the community can pave the way for the next generation of sophisticated GUI agents. Continued innovation and collaboration will be essential in refining benchmarks to ensure they accurately capture the multifaceted capabilities of modern agents, ultimately leading to more intuitive and effective human-computer interactions.

10 Applications of LLM-Brained GUI Agents

As LLM-brained GUI agents continue to mature, a growing number of applications leverage this concept to create more intelligent, user-friendly, and natural language-driven interfaces. These advancements are reflected in research papers, open-source projects, and industry solutions. Typical applications encompass *(i)* **GUI testing**, which has transitioned from traditional script-based approaches to more intuitive, natural language-based interactions, and *(ii)* **virtual assistants**, which automate users' daily tasks in a more adaptive and responsive manner through natural language interfaces.

10.1 GUI Testing

GUI testing evaluates a software application's graphical user interface to ensure compliance with specified requirements, functionality, and user experience standards. It verifies interface elements like buttons, menus, and windows, as well as their responses to user interactions. Initially conducted manually, GUI testing evolved with the advent of automation tools such as Selenium and Appium, enabling testers to automate repetitive tasks, increase coverage, and reduce testing time [33], [390]. However, LLM-powered GUI agents have introduced a paradigm shift, allowing non-experts to test GUIs intuitively through natural language interfaces. These agents cover diverse scenarios, including general testing, input generation, and bug reproduction, without the need for traditional scripting [390].

TABLE 22: Overview of GUI-testing with LLM-powered GUI agents.

Project	Category	Platform	Model	Perception	Action	Scenario	Highlight	Link
Daniel and Anne [389]	General testing	General-purpose platforms	GPT-3	GUI structure and state	Standard UI operations	Automates the software testing process using natural language test cases	Applies GPT-3's language understanding capabilities to GUI-based software testing, enabling natural interaction through text-based test case descriptions.	https://github.com/neuroevolution%2Dai/SoftwareTestingLanguageModel
Daniel and Anne [333]	General testing	Web platforms	GPT-4	HTML DOM structure	Standard UI operations	Automated GUI testing to enhance branch coverage and efficiency	Performs end-to-end GUI testing using GPT-4's natural language understanding and reasoning capabilities.	https://github.com/SoftwareTestingLLMs/WebtestingWithLLMs
GPTDroid [112]	General testing	Mobile Android	GPT-3.5	UI view hierarchy files	Standard UI operations and compound actions	Automates GUI testing to improve testing coverage and detect bugs efficiently	Formulates GUI testing as a Q& A task, utilizing LLM capabilities to provide human-like interaction.	https://github.com/franklinbill/GPTDroid
DROID-AGENT [334]	General testing	Mobile Android	GPT-3.5, GPT-4	JSON representation of the GUI state	Standard UI operations, higher-level APIs, and custom actions	Semantic, intent-driven automation of GUI testing	Autonomously generates and executes high-level, realistic tasks for Android GUI testing based on app-specific functionalities.	https://github.com/coinse/droidagent
AUITest-Agent [335]	General testing	Mobile Android	GPT-4	GUI screenshots, UI hierarchy files, and CV-enhanced techniques like Vision-UI	Standard UI operations	Automated functional testing of GUIs	Features dynamic agent organization for step-oriented testing and a multi-source data extraction strategy for precise function verification.	https://github.com/bz-lab/AUITestAgent
VisionDroid [336]	General testing	Mobile Android	GPT-4	GUI screenshots with annotated bounding boxes, View hierarchy files	Standard UI operations	Identifies non-crash bugs	Integrates vision-driven prompts and GUI text alignment with vision-language models to enhance understanding of GUI contexts and app logic.	https://github.com/testtestA6/VisionDroid
AXNav [337]	Accessibility testing	iOS mobile devices	GPT-4	GUI screenshots, UI element detection model, and OCR	Gestures, capturing screenshots, and highlighting potential accessibility issues	Automates accessibility testing workflows, including testing features like VoiceOver, Dynamic Type, Bold Text, and Button Shapes	Adapts to natural language test instructions and generates annotated videos to visually and interactively review accessibility test results.	/
Cui *et al.*, [338]	Test input generation	Mobile Android	GPT-3.5, GPT-4	GUI structures and contextual information	Entering text inputs	Generating and validating text inputs for Android applications	Demonstrates the effectiveness of various LLMs in generating context-aware text inputs, improving UI test coverage, and identifying previously unreported bugs.	/
QTypist [339]	Test input generation	Mobile Android	GPT-3	UI hierarchy files	Generates semantic text inputs	Automates mobile GUI testing by generating appropriate text inputs	Formulates text input generation as a cloze-style fill-in-the-blank language task.	/
CrashTranslator [340]	Bug replay	Mobile Android	GPT-3	Crash-related stack trace information and GUI structure	Standard UI operations	Automates the reproduction of mobile application crashes	Leverages LLMs for iterative GUI navigation and crash reproduction from stack traces, integrating a reinforcement learning-based scoring system to optimize exploration steps.	https://github.com/wuchiuwong/CrashTranslator
AdbGPT [341]	Bug replay	Mobile Android	GPT-3.5	GUI structure and hierarchy	Standard UI operations	Automates bug reproduction by extracting S2R (Steps to Reproduce) entities	Combines prompt engineering with few-shot learning and chain-of-thought reasoning to leverage LLMs for GUI-based tasks.	https://github.com/sidongfeng/AdbGPT
MagicWand [342]	Verrification	Mobile Android	GPT-4V	UI screenshots and hierarchical UI control tree	Standard UI operations	Automates the verification of "How-to" instructions from a search engine	Features a three-stage process: extracting instructions, executing them in a simulated environment, and reranking search results based on execution outcomes.	/

Figure 25 illustrates the use of an LLM-powered GUI agent to test font size adjustment on Windows OS. With only a natural language test case description, the agent autonomously performs the testing by executing UI operations, navigating through the settings menu, and leveraging its screen understanding capabilities to verify the final outcome



Fig. 25: An example of testing font size adjustment using an LLM-powered GUI agent.

of font size adjustment. This approach dramatically reduces the effort required for human or script-based testing. Next, we detail the GUI testing works powered by GUI agents, and first provide an overview Table 22.

10.1.1 General Testing

Early explorations demonstrated how LLMs like GPT-3 could automate GUI testing by interpreting natural language test cases and programmatically executing them. For example, one approach integrates GUI states with GPT-3 prompts, leveraging tools like Selenium and OpenCV to reduce manual scripting and enable black-box testing [389]. Building on this, a subsequent study employed GPT-4 and Selenium WebDriver for web application testing, achieving superior branch coverage compared to traditional methods like monkey testing [333]. These advances highlight how LLMs simplify GUI testing workflows while significantly enhancing coverage and efficiency.

Further pushing boundaries, **GPTDroid** reframed GUI testing as an interactive Q&A task. By extracting structured semantic information from GUI pages and leveraging memory mechanisms for long-term exploration, it increased activity coverage by 32%, uncovering critical bugs with remarkable precision [112]. This approach underscores the potential of integrating conversational interfaces with memory for comprehensive app testing. For Android environments, **DROIDAGENT** introduced an intent-driven testing framework. It automates task generation and execution by perceiving GUI states in JSON format and using LLMs for realistic task planning. Its ability to set high-level goals and achieve superior feature coverage demonstrates how intent-based testing can transform functional verification in GUI applications [334].

AUITestAgent extended the capabilities of LLM-powered GUI testing by bridging natural language-driven requirements and GUI functionality [335]. Employing multi-modal analysis and dynamic agent organization, it efficiently executes both simple and complex testing instructions. This framework highlights the value of combining multi-source data extraction with robust language models to automate functional testing in commercial apps. Incorporating vision-based methods, **VisionDroid** redefined GUI testing by aligning screenshots with textual contexts to detect non-crash bugs [336]. This innovation ensures application reliability by identifying logical inconsistencies and exploring app functionalities that conventional methods often overlook.

Accessibility testing has also benefited from LLM-powered agents. **AXNav** addresses challenges in iOS accessibility workflows, automating tests for features like VoiceOver and Dynamic Type using natural language instructions and pixel-based models. Its ability to generate annotated videos for interactive review positions AXNav as a scalable and user-friendly solution for accessibility testing [337].

10.1.2 Text Input generation

In the realm of text input generation, Cui *et al.*, demonstrated how GPT-3.5 and GPT-4 could enhance Android app testing by generating context-aware text inputs for UI fields [338]. By systematically evaluating these inputs across multiple apps, they revealed the potential of LLMs in improving test coverage and detecting unique bugs with minimal manual intervention. Similarly, **QTypist** formulated text input generation as a fill-in-the-blank task, leveraging LLMs to improve activity and page coverage by up to 52% [339].

10.1.3 Bug Replay

For bug reproduction, **CrashTranslator** automated the reproduction of crashes from stack traces by integrating reinforcement learning with LLMs. Its iterative navigation and crash prediction steps significantly reduced debugging time and outperformed state-of-the-art methods [340]. Meanwhile,

AdbGPT demonstrated how few-shot learning and chain-of-thought reasoning could transform textual bug reports into actionable GUI operations. By dynamically inferring GUI actions, AdbGPT provided an efficient and lightweight solution for bug replay [341].

10.1.4 Verification

Finally, as a novel application in testing, **MagicWand** showcased the potential of LLMs in automating "How-to" verifications. By extracting, executing, and refining instructions from search engines, it addressed critical challenges in user-centric task automation, improving the reliability of GUI-driven workflows [342].

In summary, LLM-powered GUI agents have revolutionized GUI testing by introducing natural language-driven methods, vision-based alignment, and automated crash reproduction. These innovations have enhanced test coverage, efficiency, and accessibility, setting new benchmarks for intelligent GUI testing frameworks.

10.2 Virtual Assistants

Virtual assistants, such as Siri[32], are AI-driven applications that help users by performing tasks, answering questions, and executing commands across various platforms, including web browsers, mobile phones, and computers. Initially, these assistants were limited to handling simple commands via voice or text input, delivering rule-based responses or running fixed workflows similar to RPA. They focused on basic tasks, such as setting alarms or checking the weather.

With advancements in LLMs and agents, virtual assistants have evolved significantly. They now support more complex, context-aware interactions on device GUIs through textual or voice commands and provide personalized responses, catering to diverse applications and user needs on various platforms. This progression has transformed virtual assistants from basic utilities into intelligent, adaptive tools capable of managing intricate workflows and enhancing user productivity across platforms. Figure 26 presents a conceptual example of a GUI agent-powered virtual assistant on a smartphone[33]. In this scenario, the agent enables users to interact through chat, handling tasks such as setting up a screenshot shortcut on their behalf. This feature is particularly beneficial for users unfamiliar with the phone's functionalities, simplifying complex tasks into conversational commands.

To explore more real-world applications of virtual assistants powered by GUI agents, we provide an overview of advancements across research, open-source initiatives, and production-level applications, as summarized in Table 23.

10.2.1 Research

Recent research efforts have significantly advanced the capabilities of virtual assistants by integrating LLM-powered GUI agents, enabling more intelligent and adaptable interactions within various applications.

Firstly, the integration of LLMs into GUI-based automation has been explored to enhance business process automation. For instance, [344] introduces Agentic Process Automation through the development of **ProAgent**, which automates both the creation and execution of workflows in GUI environments. By utilizing agents like ControlAgent and DataAgent, it supports complex actions such as dynamic branching and report generation in applications like Slack and Google Sheets. This approach transcends traditional RPA by enabling flexible, intelligent workflows, significantly reducing the need for manual intervention and highlighting the transformative potential of LLM-powered agents in virtual assistants.

Building upon the idea of integrating LLMs with GUI environments, researchers have focused on mobile platforms to automate complex tasks. **LLMPA** [345] is a pioneering framework that leverages LLMs to automate multi-step tasks within mobile applications like Alipay. It interacts directly with app GUIs, mimicking human actions such as clicks and typing, and employs UI tree parsing and object detection for precise environment understanding. A unique controllable calibration module ensures logical action execution, demonstrating the potential of LLM-powered virtual assistants to handle intricate workflows and real-world impact in assisting users with diverse tasks.

Similarly, the automation of smartphone tasks through natural language prompts has been addressed by **PromptRPA** [349]. Utilizing a multi-agent framework, it automates tasks within smartphone GUI environments, tackling challenges like interface updates and user input variability. Advanced perception methods, including OCR and hierarchical GUI analysis, are employed to understand and interact with mobile interfaces. By supporting real-time feedback and iterative improvements, PromptRPA underscores the importance of user-centered design in LLM-driven virtual assistants.

In the realm of accessibility, LLM-powered GUI agents have been instrumental in enhancing user experience for individuals with disabilities. For example, **VizAbility** [343] enhances the accessibility of data visualizations for blind and low-vision users. By combining structured chart navigation with LLM-based conversational interactions, users can ask natural language queries and receive insights on chart content and trends. Leveraging frameworks like Olli[34] and chart specifications such as Vega-Lite[35], VizAbility allows exploration of visual data without direct visual perception, addressing real-world accessibility challenges in GUIs.

Furthermore, addressing the needs of older adults, **EasyAsk** [350] serves as a context-aware in-app assistant that enhances usability for non-technical users. By integrating multi-modal inputs, combining natural voice queries and touch interactions with GUI elements, it generates accurate and contextual tutorial searches. EasyAsk demonstrates how GUI agents can enhance accessibility by integrating contextual information and interactive tutorials, empowering users to navigate smartphone functions effectively.

Voice interaction has also been a focus area, with tools like **GPTVoiceTasker** [346] facilitating hands-free interaction with Android GUIs through natural language commands.

32. https://www.apple.com/siri/

33. The application and scenario depicted in the figure are conceptual and fabricated. They do not reflect the actual functionality of any specific smartphone. Readers should consult the phone manual or official guidance for accurate information on AI assistant capabilities.

34. https://mitvis.github.io/olli/

35. https://vega.github.io/

TABLE 23: Overview of virtual assistants with LLM-powered GUI agents.

Project	Type	Platform	Model	Perception	Action	Scenario	Highlight	Link
ProAgent [344]	Research	Web and Desktop	GPT-4	Task descriptions and structured application data	Standard UI operations and dynamic branching	Automates business processes such as data analysis, report generation, and notifications via GUI-based tools	Introduces dynamic workflows where agents interpret and execute tasks flexibly, surpassing traditional RPA systems	https://github.com/OpenBMB/ProAgent
LLMPA [345]	Research	Mobile (Android)	AntLLM-10b	UI tree structures, visual modeling, and text extraction modules	Standard UI operations	Automates user interactions within mobile apps, such as ticket booking	Integrates LLM reasoning capabilities with a modular design that supports task decomposition, object detection, and robust action prediction in GUI environments	/
VizAbility [343]	Research	Desktop	GPT-4V	Keyboard-navigable tree views	Navigates chart structures and generates answers	Assists blind and low-vision users in exploring and understanding data visualizations	Integrates structured chart navigation with LLM-powered conversational capabilities, enabling visually impaired users to query in natural language	https://dwr.bc.edu/vizability/
GPTVoiceTasker [346]	Research	Mobile (Android)	GPT-4	Android Accessibility Tree	Standard UI operations	Automates user interactions on mobile devices through voice commands	Integrates LLMs for natural command interpretation and real-time GUI interactions, using a graph-based local database to record and replicate interactions	https://github.com/vuminhduc796/GPTVoiceTasker
AutoTask [347]	Research	Mobile (Android)	GPT-4	Android Accessibility Tree	Standard UI operations	Automates multi-step tasks on mobile devices	Operates without predefined scripts or configurations, autonomously exploring GUI environments	https://github.com/BowenBryanWang/AutoTask
AssistEditor [348]	Research	Windows	UniVTG [391]	GUI elements, user requirements, and video data	Standard UI operations	Automates video editing workflows	Employs a multi-agent collaboration framework where agents specialize in roles to integrate user requirements into video editing workflows	/
PromptRPA [349]	Research	Mobile (Android)	GPT-4 and GPT-3.5 Turbo	Layout hierarchy and screenshots with OCR	Standard UI operations and application-level functionalities	Automates smartphone tasks and creates interactive tutorials	Integrates user feedback loops for continuous improvement, addressing interface evolution and task variability	/
EasyAsk [350]	Research	Mobile (Android)	GPT-4	Android Accessibility Tree	Highlights specific UI elements for user interaction	Assists older adults in learning and navigating smartphone functions through in-app interactive tutorials	Combines voice and touch inputs, supplementing incomplete or ambiguous queries with in-app contextual information	/
OpenAdapt [351]	Open-source	Desktop	LLM, VLM (e.g., GPT-4, ACT-1)	Screenshots with CV tools for GUI parsing	Standard UI operations	Automates repetitive tasks across industries	Learns task automation by observing user interactions, eliminating manual scripting	https://github.com/OpenAdaptAI/OpenAdapt
AgentSea [352]	Open-source	Desktop and Web	LLM, VLM	Screenshots with CV tools for GUI parsing	Standard UI operations	Automates tasks within GUI environments	Offers a modular toolkit adhering to the UNIX philosophy, allowing developers to create custom AI agents for diverse GUI environments	https://www.agentsea.ai/
Open Interpreter [353]	Open-source	Desktop, Web, Mobile (Android)	LLM	System perception via command-line	Shell commands, code, and native APIs	Automates tasks, conducts data analysis, manages files, and controls web browsers for research	Executes code locally, providing full access to system resources and libraries, overcoming limitations of cloud-based services	https://github.com/OpenInterpreter/open-interpreter
MultiOn [354]	Production	Web	LLM	/	Standard UI operations	Automates web-based tasks	Performs autonomous web actions via natural language commands	https://www.multion.ai/
YOYO Agent in MagicOS [355]	Production	Mobile (MagicOS 9.0)	MagicLM	GUI context	Executes in-app and cross-app operations	Automates daily tasks, enhancing productivity	Leverages MagicLM to understand and execute complex tasks across applications, learning user habits to provide personalized assistance	/
Power Automate [119]	Production	Windows	LLM, VLM	Records user interactions with the GUI	Standard UI operations	Automates repetitive tasks and streamlines workflows	Translates natural language descriptions of desired automations into executable workflows	https://learn.microsoft.com/en-us/power-automate/desktop-flows/create%2Dflow-using%2Dai-recorder

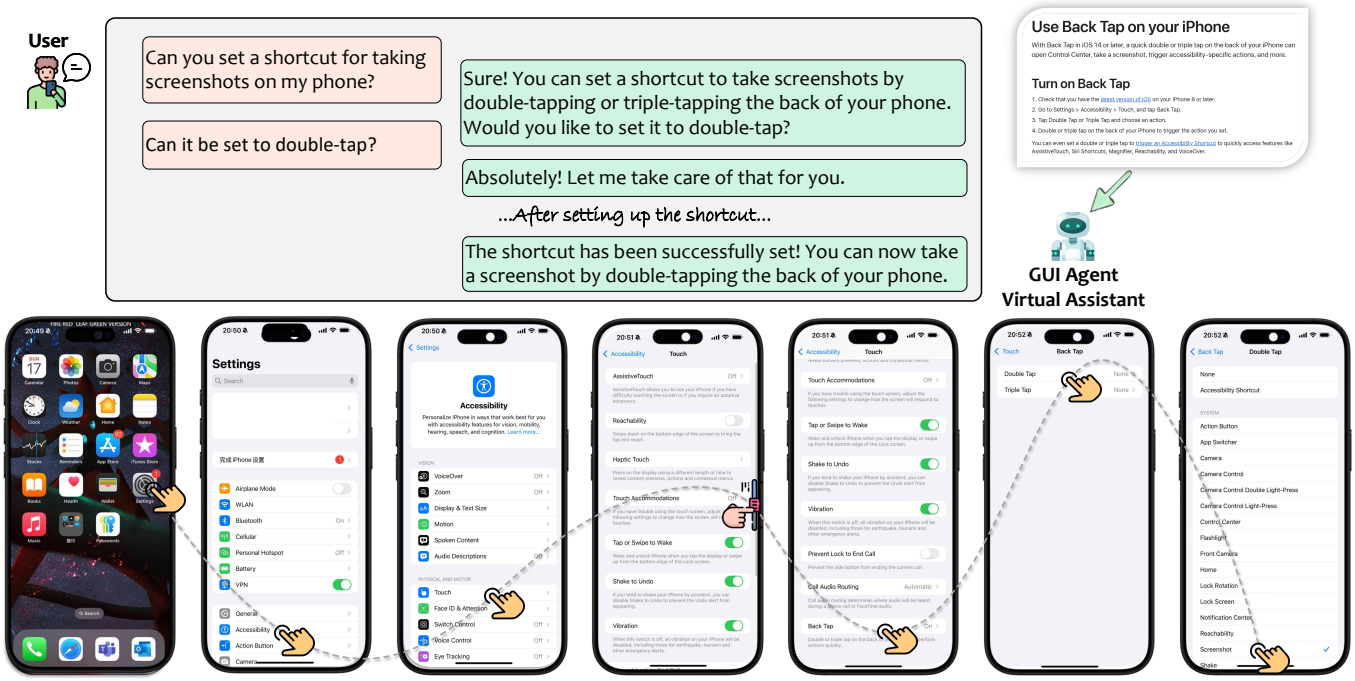


Fig. 26: A conceptual example of a GUI agent-powered virtual assistant on a smartphone.

It bridges the gap between voice commands and GUI-based actions using real-time semantic extraction and a hierarchical representation of UI elements. By automating multi-step tasks and learning from user behavior, it enhances task efficiency and reduces cognitive load, highlighting the transformative potential of LLMs in improving accessibility and user experience in mobile environments.

Expanding on voice-powered interactions, **AutoTask** [347] enables virtual assistants to execute multi-step tasks in GUI environments without predefined scripts. It autonomously explores and learns from mobile GUIs, effectively combining voice command interfaces with dynamic action engines to interact with GUI elements. Utilizing trial-and-error and experience-driven learning, AutoTask adapts to unknown tasks and environments, showcasing its potential in enhancing voice-driven virtual assistants for hands-free interactions.

Finally, in the domain of creative workflows, **AssistEditor** [348] exemplifies a multi-agent framework for automating video editing tasks. By interacting with GUI environments, it autonomously performs complex workflows using dialogue systems and video understanding models to bridge user intent with professional editing tasks. The innovative use of specialized agents ensures efficient task distribution and execution, demonstrating the practical application of LLM-powered GUI agents in real-world scenarios and expanding automation into creative domains.

These research endeavors collectively showcase significant advancements in LLM-powered GUI agents, highlighting their potential to transform virtual assistants into intelligent, adaptable tools capable of handling complex tasks across various platforms and user needs.

10.2.2 Open-Source Projects

In addition to research prototypes, open-source projects have contributed substantially to the development and accessibility of LLM-brained GUI agents, enabling wider adoption and customization.

One such project is **OpenAdapt** [351], an open-source framework that utilizes large multimodal models to automate tasks by observing and replicating user interactions within GUI environments. It captures screenshots and records user inputs, employing computer vision techniques to understand and execute standard UI operations. Designed to streamline workflows across various industries, OpenAdapt learns from user demonstrations, thereby reducing the need for manual scripting and showcasing adaptability in GUI-based task automation.

Similarly, **AgentSea** [352] offers a comprehensive and modular toolkit for creating intelligent agents that can navigate and interact with various GUI environments across multiple platforms. Its flexibility is particularly beneficial for developing virtual assistants capable of automating complex tasks within applications, enhancing user productivity. By adhering to the UNIX philosophy, AgentSea ensures that each tool is specialized, promoting ease of use and extensibility. Its open-source nature fosters community collaboration and innovation in AI-driven GUI automation.

Open Interpreter [353] further exemplifies the potential of open-source contributions by leveraging large language models to execute code locally. Users can interact with their computer's GUI through natural language commands, supporting multiple programming languages and operating across various platforms. By facilitating tasks such as data analysis, web automation, and system management, Open Interpreter provides unrestricted access to system resources and libraries, enhancing flexibility and control. Its customiza-

tion capabilities make it a valuable asset for users aiming to streamline operations through AI-powered virtual assistance.

These open-source projects not only advance the state of LLM-powered GUI agents but also democratize access to intelligent virtual assistants, enabling developers and users to tailor solutions to specific needs and applications.

10.2.3 Production

The integration of LLM-brained GUI agents into production environments demonstrates their practical viability and impact on enhancing user experiences in commercial applications.

Power Automate [119] exemplifies an AI-powered GUI agent that enhances user interaction with desktop applications. By allowing users to describe tasks in natural language while recording actions, it translates these descriptions into automated workflows, effectively bridging the gap between user intent and execution. Its ability to record and replicate user actions within the GUI streamlines the automation of repetitive tasks, making it a valuable tool for increasing efficiency and highlighting advancements in user-friendly automation solutions.

In the realm of web interactions, **MultiOn** [354] serves as a personal AI agent that autonomously interacts with web-based GUIs to execute user-defined tasks. Leveraging large language models, it interprets natural language commands and translates them into precise web actions, effectively automating complex or repetitive tasks. MultiOn's approach to perceiving and manipulating web elements enables seamless functioning across various web platforms, enhancing user productivity and streamlining web interactions.

On mobile platforms, the **YOYO Agent** in *MagicOS* [355] exemplifies an LLM-powered GUI agent operating within the MagicOS 9.0 interface. Utilizing Honor's MagicLM, it comprehends and executes user commands across various applications, learning from user behavior to offer personalized assistance. This integration demonstrates how large language models can enhance virtual assistants, enabling them to perform complex tasks within GUI environments and improving user experience and productivity on mobile devices.

These production-level implementations highlight the practical applications and benefits of LLM-brained GUI agents in enhancing automation, productivity, and user engagement across different platforms and industries.

10.3 Takeaways

The application of LLM-brained GUI agents has ushered in new capabilities and interfaces for tasks such as GUI testing and virtual assistance, introducing natural language interactions, enhanced automation, and improved accessibility across platforms. These agents are transforming the way users interact with software applications by simplifying complex tasks and making technology more accessible. However, despite these advancements, LLM-brained GUI agents are still in their infancy, and several challenges need to be addressed for them to reach maturity. Key insights from recent developments include:

1) **Natural Language-Driven Interactions:** LLM-powered GUI agents have enabled users to interact with applications using natural language, significantly lowering the barrier to entry for non-expert users. In GUI testing, tools like GPTDroid [112] and AUITestAgent [335] allow testers to specify test cases and requirements in plain language, automating the execution and verification processes. Similarly, virtual assistants like LLMPA [345] and ProAgent [344] interpret user commands to perform complex tasks, showcasing the potential of natural language interfaces in simplifying user interactions across platforms.
2) **Enhanced Automation of Complex Tasks:** These agents have demonstrated the ability to automate multi-step and intricate workflows without the need for manual scripting. Projects like AutoTask [347] and GPTVoiceTasker [346] autonomously explore and interact with GUI environments, executing tasks based on high-level goals or voice commands. In GUI testing, agents have improved coverage and efficiency by automating the generation of test inputs and reproducing bugs from textual descriptions, as seen in CrashTranslator [340] and AdbGPT [341].
3) **Multimodal Perception and Interaction:** Integrating visual and textual inputs has enhanced the agents' understanding of GUI contexts, leading to better decision-making and interaction accuracy. Agents like VizAbility [343] and OpenAdapt [351] utilize screenshots, UI trees, and OCR to perceive the environment more comprehensively. This multimodal approach is crucial for applications that require precise identification and manipulation of GUI elements, especially in dynamic or visually complex interfaces.
4) **Improved Accessibility and User Experience:** LLM-brained GUI agents have contributed to making technology more accessible to users with disabilities or limited technical proficiency. Tools like VizAbility [343] aid blind and low-vision users in understanding data visualizations, while EasyAsk [350] assists older adults in navigating smartphone functions. By tailoring interactions to the needs of diverse user groups, these agents enhance inclusivity and user experience.

LLM-brained GUI agents are transforming the landscape of GUI interaction and automation by introducing natural language understanding, enhanced automation capabilities, and improved accessibility. While they are still in the early stages of development, the ongoing advancements and emerging applications hold great promise for the future. Continued research and innovation are essential to overcome current challenges and fully realize the potential of these intelligent agents across diverse domains and platforms.

11 Limitations, Challenges and Future Roadmap

Despite significant advancements in the development of LLM-brained GUI agents, it is important to acknowledge that this field is still in its infancy. Several technical challenges and limitations hinder their widespread adoption in real-world applications. Addressing these issues is crucial to enhance the agents' effectiveness, safety, and user acceptance. In this section, we outline key limitations and propose future research directions to overcome these challenges, providing concrete examples to illustrate each point.

11.1 Privacy Concerns

LLM-powered GUI agents often require access to sensitive user data, including screenshots, personal credentials, interaction logs, and confidential documents, which may need to be transmitted to remote servers for processing. This cloud-based deployment raises significant privacy risks, such as data breaches, unauthorized access, and potential misuse of personal information [392]–[394]. For instance, consider an agent that automates email management. To sort emails or compose responses, the agent needs access to the user's email content, which may include sensitive information. Transmitting this data to a cloud server for processing could expose the user to privacy risks. Users may be hesitant to adopt such agents due to concerns over data security and privacy violations [395], [396].

Potential Solutions: To mitigate privacy concerns, future research should focus on enabling *on-device inference*, where the language model operates directly on the user's device without uploading personal data [397], [398]. Achieving this requires advancements in model compression techniques [399], on-device optimization [400], and efficient inference algorithms [401] to accommodate the computational limitations of user devices. Additionally, implementing privacy-preserving techniques like federated learning [402], differential privacy [403], and homomorphic encryption [404] can enhance data security while allowing the model to learn from user data.

Furthermore, developers of GUI agents should collaborate with privacy policymakers to ensure that user data and privacy are appropriately protected [405]. They should make the data handling processes transparent to users, clearly informing them about what data are being transmitted and how they are used, and obtain explicit user consent [406].

11.2 Latency, Performance, and Resource Constraints

Since GUI agents rely on LLMs, which are large models with substantial computational demands, this can result in high latency and slow response times, negatively affecting the user experience [407]. In time-sensitive applications, delays in action execution can lead to user frustration or even system failure. This issue becomes more pronounced in long-term tasks, where latency can accumulate at every step, exacerbating the problem. Furthermore, when using on-device inference with resource-constrained devices, the situation can be even more severe [408]. For example, a mobile app incorporating an LLM-powered agent may experience slow performance or excessive battery drain, detracting from the user experience [409].

Potential Solutions: Future work should aim to reduce inference latency by optimizing model architectures for speed and efficiency [410]. Techniques such as model distillation can create smaller, faster models without substantially compromising performance [411]. Leveraging hardware accelerators like GPUs, TPUs, or specialized AI chips, and exploring parallel processing methods can enhance computational efficiency [412]. Implementing incremental inference and caching mechanisms may also improve responsiveness by reusing computations where applicable [413]. Additionally, research into model optimization and compression techniques, such as pruning [414] and quantization [399] can produce lightweight models suitable for deployment on resource-constrained devices. Exploring edge computing [398] and distributed inference [415] can help distribute the computational load effectively.

Moreover, GUI agents should collaborate with application developers to encourage them to expose high-level native APIs for different functionalities [184], [185], which combine several UI operations into single API calls. By integrating these APIs into the GUI agent, tasks can be completed with fewer steps, making the process much faster and reducing cumulative latency.

11.3 Safety and Reliability

The ability of GUI agents to perform real-world actions within software environments could introduce safety [394], [416] and reliability [417] concerns. Erroneous actions could result in unintended consequences, such as data corruption, application crashes, or security vulnerabilities [418]. The probabilistic nature of LLM outputs means that agents may occasionally generate incorrect, inconsistent or hallucinated actions [419], [420]. For example, an agent tasked with automating financial transactions might misinterpret a command and transfer funds to the wrong account, leading to financial losses. The agent may also be vulnerable to black-box attacks, which could compromise its functionality and security [421]. Additionally, integrating GUI agents into existing software ecosystems involves compatibility issues and security considerations, and may encounter resistance from users who distrust automated systems.

Potential Solutions: Ensuring safety and reliability necessitates robust error detection and handling mechanisms [422]. Future research should focus on integrating validation steps that verify the correctness of inferred actions before execution [423]. Developing formal verification methods [424], implementing exception handling routines [425], and establishing rollback procedures [426] are essential for preventing and mitigating the impact of errors. Additionally, incorporating permission management [427]–[430] to limit the agent's access rights can prevent unauthorized or harmful operations.

Furthermore, creating standardized interaction protocols can facilitate smoother and safer integration with various applications and systems [431]. Ensuring that agents comply with security best practices, such as secure authentication and authorization protocols [432], is essential.

11.4 Human-Agent Interaction

When using a GUI agent, any interruption or interaction by the user in the environment may interfere with the agent. Resolving such conflicts and designing the relationship between the human user and the GUI agent becomes challenging [433], [434]. In addition, users may provide vague or ambiguous requests, leading agents to misunderstand the intended task. Furthermore, agents may encounter situations where they lack sufficient information or face unexpected obstacles, or need confirmation from the user for certain action [17]. Determining when and how the agent should seek human assistance or clarification is crucial for effective collaboration [435]. This is common in daily usage when the agent does not have sufficient context [436]. Furthermore, users may need to intervene in the agent's behavior if it does not meet their expectations.

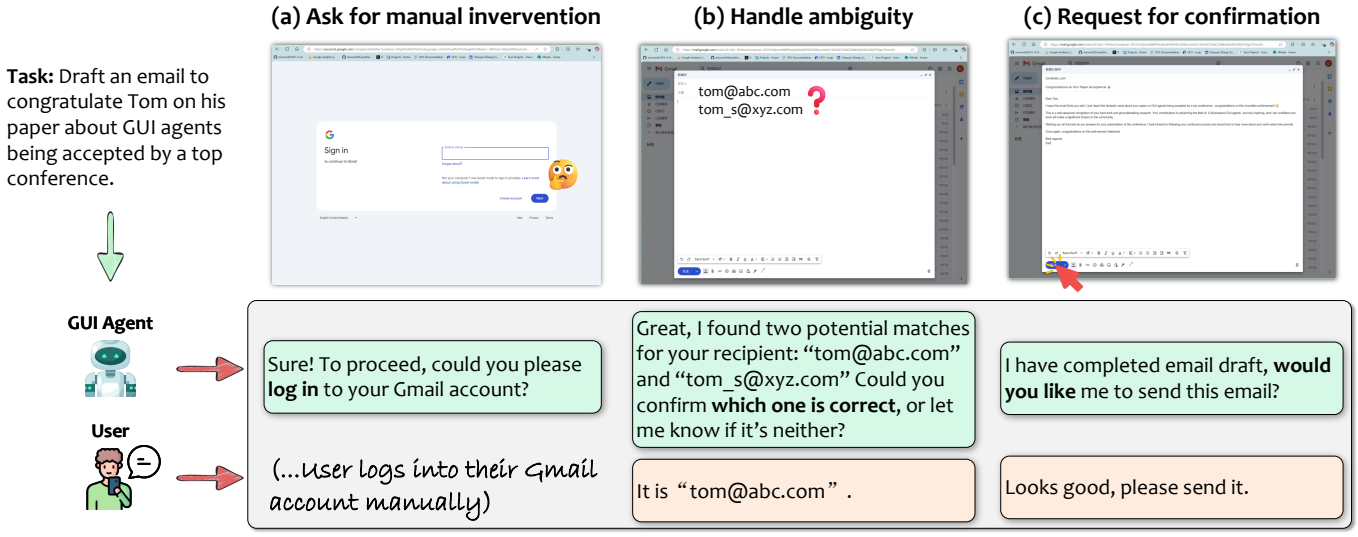


Fig. 27: An illustrative example of human-agent interaction for completing an email sending request.

As illustrated in the fabricated example shown in Figure 27, when the agent is tasked with sending an email to Tom, it engages in several steps to ensure privacy, accuracy, and user intent. First, the agent requests the user to manually log in, preserving sensitive credentials like usernames and passwords. Next, when multiple matches are found for the recipient "Tom", the agent resolves the ambiguity by prompting the user to select the correct contact. Finally, before sending the email, the agent seeks confirmation from the user, acknowledging that sending an email is a sensitive action since it cannot be retracted [17]. This seemingly simple request demonstrates the complexity of human-agent interaction, requiring careful design to handle privacy, ambiguity, and confirmation efficiently [437]. It highlights the challenges and intricacies involved in developing robust and user-friendly GUI agents.

Potential Solutions: Emphasizing *user-centered design* [438] principles can address user needs and concerns, providing options for customization and control over the agent's behavior [435]. Equipping agents with the ability to engage in *clarification dialogues* when user instructions are unclear can enhance task accuracy [439]. Natural language understanding components can detect ambiguity and prompt users for additional information. For instance, the agent could ask, "There are two contacts named John. Do you mean John Smith or John Doe?" Incorporating *human-in-the-loop* systems allows for human intervention during task execution, enabling users to guide or correct the agent's decisions when necessary [440]. Developing adaptive interaction models that facilitate seamless collaboration between humans and agents is essential. Additionally, providing transparency and explainability in the agent's reasoning processes can build user trust and improve cooperation [441], [442].

11.5 Customization and Personalization

For GUI agents to be truly effective, they need to understand individual user preferences and adapt their behavior accordingly [43], [443]. Users have unique habits, workflows, and preferences, and a one-size-fits-all approach may not provide an optimal user experience [444]. For example, an agent that assists with document editing may need to learn a user's preferred writing style, formatting choices, and commonly used phrases. Without personalization, the agent may suggest edits that the user finds unhelpful or inconsistent with their style. Customization requires the agent to learn and adapt to user-specific preferences [445], which can be challenging due to variability among users. Balancing personalization with privacy concerns is also critical, as collecting and utilizing personal data must be handled responsibly.

Potential Solutions: Future research should focus on developing mechanisms for *user modeling* [446] and *preference learning* [447], enabling agents to tailor their actions to individual users. Techniques such as reinforcement learning from user feedback [448], collaborative filtering [449], and context-aware computing [450] can help agents learn user preferences over time. Ensuring that personalization is achieved without compromising privacy is essential [451], potentially through on-device learning and anonymized data processing.

11.6 Ethical and Regulatory Challenges

The deployment of LLM-powered GUI agents raises ethical questions regarding accountability, transparency, fairness, and user consent [394], [452]–[454]. There is a risk of biased behavior inherited from training data, leading to unfair or discriminatory actions [455], [456]. For example, an agent used in hiring processes might inadvertently exhibit bias by filtering out resumes based on gender or ethnicity if such biases are present in the training data. Additionally, regulatory compliance varies across industries and regions, complicating deployment.

Potential Solutions: Addressing these concerns requires establishing clear ethical guidelines and regulatory frameworks for the development and use of GUI agents [457]. Future work should focus on creating mechanisms for auditing and monitoring agent behavior [458] to ensure

compliance with ethical standards and legal requirements [459]. Incorporating bias detection and mitigation strategies in language models can help prevent discriminatory or unfair actions [460]. Providing users with control over data usage and clear information about the agent's capabilities can enhance transparency and trust.

11.7 Scalability and Generalization

Many existing GUI agents are tailored to specific applications or environments, which limits their scalability and generalizability. The diversity of software interfaces, with each application featuring unique designs, layouts, and interaction patterns, poses a significant challenge for developing agents capable of operating seamlessly across multiple platforms, even for common pop-up windows [461]. For instance, an agent developed to automate tasks in a specific version of a word processor may fail when the application updates its interface or when used with a different word processor that has a different UI layout. This issue becomes even more pronounced when the agent encounters applications or environments that it is unfamiliar with or has not encountered during training. Even if these new environments share some similarities with previously seen GUIs, the agent may still make mistakes and require exploration to fully understand their functionality. The lack of generalization limits the agent's applicability and necessitates continuous updates or retraining, which can be resource-intensive [462], [463].

Potential Solutions: To enhance scalability and generalization, one solution from the dataset perspective is to create comprehensive GUI agent datasets that cover a wide range of environments, user requests, GUI designs, platforms, and interaction patterns. By exposing the LLM to diverse data sources during training, the model can learn common patterns and develop a more generalized understanding, enabling it to adapt to infer the functionality of new interfaces based on learned similarities [464].

To further enhance adaptability, research can focus on techniques such as *transfer learning* [465] and *meta-learning* [466]. *Transfer learning* involves pre-training a model on a large, diverse dataset and then fine-tuning it on a smaller, task-specific dataset. In the context of GUI agents, this means training the LLM on a wide array of GUI interactions before customizing it for a particular application or domain. *Meta-learning*, enables the model to rapidly adapt to new tasks with minimal data by identifying underlying structures and patterns across different tasks. These approaches enable agents to generalize from limited data and adapt to new environments with minimal retraining.

However, even with these measures, the agent may still encounter difficulties in unfamiliar environments. To address this, we advocate for developers to provide helpful knowledge bases, such as guidance documents, application documentation, searchable FAQs, and even human demonstrations on how to use the application [467]–[469]. Techniques like RAG [174] can be employed, where the agent retrieves relevant information from a knowledge base at runtime to inform its decisions [470]. For instance, if the agent encounters an unknown interface element, it can query the documentation to understand its purpose and how to interact with it. This approach enhances the agent's capabilities without requiring extensive retraining. Implementing these solutions requires collaborative efforts not only from agent developers but also from application or environment providers.

11.8 Summary

LLM-brained GUI agents hold significant promise for automating complex tasks and enhancing user productivity across various applications. However, realizing this potential requires addressing the outlined limitations through dedicated research and development efforts. By addressing these challenges, the community can develop more robust and widely adopted GUI agents.

Collaboration among researchers, industry practitioners, policymakers, and users is essential to navigate these challenges successfully. Establishing interdisciplinary teams can foster innovation and ensure that GUI agents are developed responsibly, with a clear understanding of technical, ethical, and societal implications. As the field progresses, continuous evaluation and adaptation will be crucial to align technological advancements with user needs and expectations, ultimately leading to more intelligent, safe, and user-friendly GUI agents.

12 Conclusion

The combination of LLMs and GUI automation marks a transformative moment in human-computer interaction. LLMs provide the "brain" for natural language processing, comprehension, and GUI understanding, while GUI automation tools serve as the "hands", translating the agent's cognitive abilities into actionable commands within software environments. Together, they form LLM-powered GUI agents that introduce a new paradigm in user interaction, allowing users to control applications through straightforward natural language commands instead of complex, platform-specific UI operations. This synergy has shown remarkable potential, with applications flourishing in both research and industry.

In this survey, we provide a comprehensive, systematic, and timely overview of the field of LLM-powered GUI agents. Our work introduces the core components and advanced techniques that underpin these agents, while also examining critical elements such as data collection, model development, frameworks, evaluation methodologies, and real-world applications. Additionally, we explore the current limitations and challenges faced by these agents and outline a roadmap for future research directions. We hope this survey serves as a valuable handbook for those learning about LLM-powered GUI agents and as a reference point for researchers aiming to stay at the forefront of developments in this field.

As we look to the future, the concept of LLM-brained GUI agents promises to become increasingly tangible, fundamentally enhancing productivity and accessibility in daily life. With ongoing research and development, this technology stands poised to reshape how we interact with digital systems, transforming complex workflows into seamless, natural interactions.

References

[1] B. J. Jansen, "The graphical user interface," *ACM SIGCHI Bull.*, vol. 30, pp. 22–26, 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:18416305

[2] H. Sampath, A. Merrick, and A. P. Macvean, "Accessibility of command line interfaces," *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:233987139

[3] R. Michalski, J. Grobelny, and W. Karwowski, "The effects of graphical interface design characteristics on human-computer interaction task efficiency," *ArXiv*, vol. abs/1211.6712, 2006. [Online]. Available: https://api.semanticscholar.org/CorpusID:14695409

[4] T. D. Hellmann and F. Maurer, "Rule-based exploratory testing of graphical user interfaces," in *2011 Agile Conference*. IEEE, 2011, pp. 107–116.

[5] J. Steven, P. Chandra, B. Fleck, and A. Podgurski, "jrapture: A capture/replay tool for observation-based testing," *SIGSOFT Softw. Eng. Notes*, vol. 25, no. 5, p. 158–167, Aug. 2000. [Online]. Available: https://doi.org/10.1145/347636.348993

[6] L. Ivančić, D. Suša Vugec, and V. Bosilj Vukšić, "Robotic process automation: systematic literature review," in *Business Process Management: Blockchain and Central and Eastern Europe Forum: BPM 2019 Blockchain and CEE Forum, Vienna, Austria, September 1–6, 2019, Proceedings 17*. Springer, 2019, pp. 280–295.

[7] W. contributors, "Large language model — wikipedia, the free encyclopedia," 2024, accessed: 2024-11-25. [Online]. Available: https://en.wikipedia.org/wiki/Large_language_model

[8] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[9] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *arXiv preprint arXiv:2307.06435*, 2023.

[10] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, "A survey on multimodal large language models," *arXiv preprint arXiv:2306.13549*, 2023.

[11] T. Wu, S. He, J. Liu, S. Sun, K. Liu, Q.-L. Han, and Y. Tang, "A brief overview of chatgpt: The history, status quo and potential future development," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 5, pp. 1122–1136, 2023.

[12] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.

[13] Z. Shen, "Llm with tools: A survey," *arXiv preprint arXiv:2409.18807*, 2024.

[14] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Zhang, J. Li, B. Xu, Y. Dong, M. Ding, and J. Tang, "Cogagent: A visual language model for gui agents," 2023. [Online]. Available: https://arxiv.org/abs/2312.08914

[15] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "Gpt-4v(ision) is a generalist web agent, if grounded," 2024. [Online]. Available: https://arxiv.org/abs/2401.01614

[16] C. Zhang, Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "Appagent: Multimodal agents as smartphone users," 2023. [Online]. Available: https://arxiv.org/abs/2312.13771

[17] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "UFO: A UI-Focused Agent for Windows OS Interaction," *arXiv preprint arXiv:2402.07939*, 2024.

[18] Y. Guan, D. Wang, Z. Chu, S. Wang, F. Ni, R. Song, L. Li, J. Gu, and C. Zhuang, "Intelligent virtual assistants with llm-based process automation," *ArXiv*, vol. abs/2312.06677, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:266174422

[19] Y. Zhang, X. Zhao, J. Yin, L. Zhang, and Z. Chen, "Operating system and artificial intelligence: A systematic review," *arXiv preprint arXiv:2407.14567*, 2024.

[20] K. Mei, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "Aios: Llm agent operating system," *arXiv e-prints, pp. arXiv–2403*, 2024.

[21] W. Aljedaani, A. Habib, A. Aljohani, M. M. Eler, and Y. Feng, "Does chatgpt generate accessible code? investigating accessibility challenges in llm-generated source code," in *International Cross-Disciplinary Conference on Web Accessibility*, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:273550267

[22] D. Chin, Y. Wang, and G. G. Xia, "Human-centered llm-agent user interface: A position paper," *ArXiv*, vol. abs/2405.13050, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:269982753

[23] K. Cheng, Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang, and Z. Wu, "Seeclick: Harnessing gui grounding for advanced visual gui agents," 2024. [Online]. Available: https://arxiv.org/abs/2401.10935

[24] M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, "Agent-as-a-judge: Evaluate agents with agents," 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:273350802

[25] K. Li and M. Wu, *Effective GUI testing automation: Developing an automated GUI testing tool*. John Wiley & Sons, 2006.

[26] O. Rodríguez-Valdés, T. E. Vos, P. Aho, and B. Marín, "30 years of automated gui testing: a bibliometric analysis," in *Quality of Information and Communications Technology: 14th International Conference, QUATIC 2021, Algarve, Portugal, September 8–11, 2021, Proceedings 14*. Springer, 2021, pp. 473–488.

[27] Y. L. Arnatovich and L. Wang, "A systematic literature review of automated techniques for functional gui testing of mobile applications," *arXiv preprint arXiv:1812.11470*, 2018.

[28] K. S. Said, L. Nie, A. A. Ajibode, and X. Zhou, "Gui testing for mobile applications: objectives, approaches and challenges," in *Proceedings of the 12th Asia-Pacific Symposium on Internetware*, 2020, pp. 51–60.

[29] X. Li, "Gui testing for android applications: a survey," in *2023 7th International Conference on Computer, Software and Modeling (ICCSM)*. IEEE, 2023, pp. 6–10.

[30] J.-J. Oksanen, "Test automation for windows gui application," 2023.

[31] P. S. Deshmukh, S. S. Date, P. N. Mahalle, and J. Barot, "Automated gui testing for enhancing user experience (ux): A survey of the state of the art," in *International Conference on ICT for Sustainable Development*. Springer, 2023, pp. 619–628.

[32] M. Bajammal, A. Stocco, D. Mazinanian, and A. Mesbah, "A survey on the use of computer vision to improve software engineering tasks," *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1722–1742, 2020.

[33] S. Yu, C. Fang, Z. Tuo, Q. Zhang, C. Chen, Z. Chen, and Z. Su, "Vision-based mobile app gui testing: A survey," *arXiv preprint arXiv:2310.13518*, 2023.

[34] R. Syed, S. Suriadi, M. Adams, W. Bandara, S. J. Leemans, C. Ouyang, A. H. Ter Hofstede, I. Van De Weerd, M. T. Wynn, and H. A. Reijers, "Robotic process automation: contemporary themes and challenges," *Computers in Industry*, vol. 115, p. 103162, 2020.

[35] T. Chakraborti, V. Isahagian, R. Khalaf, Y. Khazaeni, V. Muthusamy, Y. Rizk, and M. Unuvar, "From robotic process automation to intelligent process automation: –emerging trends–," in *Business Process Management: Blockchain and Robotic Process Automation Forum: BPM 2020 Blockchain and RPA Forum, Seville, Spain, September 13–18, 2020, Proceedings 18*. Springer, 2020, pp. 215–228.

[36] J. G. Enríquez, A. Jiménez-Ramírez, F. J. Domínguez-Mayo, and J. A. García-García, "Robotic process automation: a scientific and industrial systematic mapping study," *IEEE Access*, vol. 8, pp. 39 113–39 129, 2020.

[37] J. Ribeiro, R. Lima, T. Eckhardt, and S. Paiva, "Robotic process automation and artificial intelligence in industry 4.0–a literature review," *Procedia Computer Science*, vol. 181, pp. 51–58, 2021.

[38] M. Nass, E. Alégroth, and R. Feldt, "Why many challenges with gui test automation (will) remain," *Information and Software Technology*, vol. 138, p. 106625, 2021.

[39] S. Agostinelli, A. Marrella, and M. Mecella, "Research challenges for intelligent robotic process automation," in *Business Process Management Workshops: BPM 2019 International Workshops, Vienna, Austria, September 1–6, 2019, Revised Selected Papers 17*. Springer, 2019, pp. 12–18.

[40] A. Wali, S. Mahamad, and S. Sulaiman, "Task automation intelligent agents: A review," *Future Internet*, vol. 15, no. 6, p. 196, 2023.

[41] P. Zhao, Z. Jin, and N. Cheng, "An in-depth survey of large language model-based artificial intelligence agents," *arXiv preprint arXiv:2309.14365*, 2023.

[42] Y. Cheng, C. Zhang, Z. Zhang, X. Meng, S. Hong, W. Li, Z. Wang, Z. Wang, F. Yin, J. Zhao *et al.*, "Exploring large language model based intelligent agents: Definitions, methods, and prospects," *arXiv preprint arXiv:2401.03428*, 2024.

[43] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun *et al.*, "Personal llm agents: Insights and survey about the capability, efficiency and security," *arXiv preprint arXiv:2401.05459*, 2024.

[44] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[45] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[46] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," *arXiv preprint arXiv:2402.01680*, 2024.

[47] S. Han, Q. Zhang, Y. Yao, W. Jin, Z. Xu, and C. He, "Llm multi-agent systems: Challenges and open problems," *arXiv preprint arXiv:2402.03578*, 2024.

[48] C. Sun, S. Huang, and D. Pompili, "Llm-based multi-agent reinforcement learning: Current and future directions," *arXiv preprint arXiv:2405.11106*, 2024.

[49] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of llm agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024.

[50] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," *arXiv preprint arXiv:2404.13501*, 2024.

[51] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1–45, 2024.

[52] L. Li, G. Chen, H. Shi, J. Xiao, and L. Chen, "A survey on multimodal benchmarks: In the era of large ai models," *arXiv preprint arXiv:2409.18142*, 2024.

[53] J. Huang and J. Zhang, "A survey on evaluation of multimodal large language models," *arXiv preprint arXiv:2408.15769*, 2024.

[54] J. Xie, Z. Chen, R. Zhang, X. Wan, and G. Li, "Large multimodal agents: A survey," *arXiv preprint arXiv:2402.15116*, 2024.

[55] Z. Durante, Q. Huang, N. Wake, R. Gong, J. S. Park, B. Sarkar, R. Taori, Y. Noda, D. Terzopoulos, Y. Choi *et al.*, "Agent ai: Surveying the horizons of multimodal interaction," *arXiv preprint arXiv:2401.03568*, 2024.

[56] B. Wu, Y. Li, M. Fang, Z. Song, Z. Zhang, Y. Wei, and L. Chen, "Foundations and recent trends in multimodal mobile agents: A survey," *arXiv preprint arXiv:2411.02006*, 2024.

[57] S. Wang, W. Liu, J. Chen, W. Gan, X. Zeng, S. Yu, X. Hao, K. Shao, Y. Wang, and R. Tang, "Gui agents with foundation models: A comprehensive survey," 2024. [Online]. Available: https://arxiv.org/abs/2411.04890

[58] M. Gao, W. Bu, B. Miao, Y. Wu, Y. Li, J. Li, S. Tang, Q. Wu, Y. Zhuang, and M. Wang, "Generalist virtual agents: A survey on autonomous agents across digital platforms," *arXiv preprint arXiv:2411.10943*, 2024.

[59] T. S. d. Moura, E. L. Alves, H. F. d. Figueirêdo, and C. d. S. Baptista, "Cytestion: Automated gui testing for web applications," in *Proceedings of the XXXVII Brazilian Symposium on Software Engineering*, 2023, pp. 388–397.

[60] T. Yeh, T.-H. Chang, and R. C. Miller, "Sikuli: using gui screenshots for search and automation," in *Proceedings of the 22nd annual ACM symposium on User interface software and technology*, 2009, pp. 183–192.

[61] C. E. Shannon, "Prediction and entropy of printed english," *Bell system technical journal*, vol. 30, no. 1, pp. 50–64, 1951.

[62] W. B. Cavnar, J. M. Trenkle *et al.*, "N-gram-based text categorization," in *Proceedings of SDAIR-94, 3rd annual symposium on document analysis and information retrieval*, vol. 161175. Ann Arbor, Michigan, 1994, p. 14.

[63] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[64] B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, vol. 1, 2020.

[65] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2021.

[66] L. R. Medsker, L. Jain *et al.*, "Recurrent neural networks," *Design and Applications*, vol. 5, no. 64-67, p. 2, 2001.

[67] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.

[68] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[69] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[70] Y. Liu, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, vol. 364, 2019.

[71] Z. Lan, "Albert: A lite bert for self-supervised learning of language representations," *arXiv preprint arXiv:1909.11942*, 2019.

[72] A. Radford, "Improving language understanding by generative pre-training," 2018.

[73] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[74] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[75] M. Lewis, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.

[76] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.

[77] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[78] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[79] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican *et al.*, "Gemini: a family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[80] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.

[81] Y. Jiang, C. Zhang, S. He, Z. Yang, M. Ma, S. Qin, Y. Kang, Y. Dang, S. Rajmohan, Q. Lin *et al.*, "Xpert: Empowering incident management with query recommendations via large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[82] C. Zhang, Z. Ma, Y. Wu, S. He, S. Qin, M. Ma, X. Qin, Y. Kang, Y. Liang, X. Gou *et al.*, "Allhands: Ask me anything on large-scale verbatim feedback via large language models," *arXiv preprint arXiv:2403.15157*, 2024.

[83] J. Liu, C. Zhang, J. Qian, M. Ma, S. Qin, C. Bansal, Q. Lin, S. Rajmohan, and D. Zhang, "Large language models can deliver accurate and interpretable time series anomaly detection," *arXiv preprint arXiv:2405.15370*, 2024.

[84] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, T. Liu *et al.*, "A survey on in-context learning," *arXiv preprint arXiv:2301.00234*, 2022.

[85] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu *et al.*, "Instruction tuning for large language models: A survey," *arXiv preprint arXiv:2308.10792*, 2023.

[86] J. Huang and K. C.-C. Chang, "Towards reasoning in large language models: A survey," *arXiv preprint arXiv:2212.10403*, 2022.

[87] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[88] R. Ding, C. Zhang, L. Wang, Y. Xu, M. Ma, W. Zhang, S. Qin, S. Rajmohan, Q. Lin, and D. Zhang, "Everything of thoughts: Defying the law of penrose triangle for thought generation," *arXiv preprint arXiv:2311.04254*, 2023.

[89] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[90] T. D. White, G. Fraser, and G. J. Brown, "Improving random gui testing with image-based widget detection," in *Proceedings of the*

28th ACM SIGSOFT international symposium on software testing and analysis, 2019, pp. 307–317.
[91] G. Kim, P. Baldi, and S. McAleer, "Language models can solve computer tasks," 2023. [Online]. Available: https://arxiv.org/abs/2303.17491
[92] B. Qiao, L. Li, X. Zhang, S. He, Y. Kang, C. Zhang, F. Yang, H. Dong, J. Zhang, L. Wang *et al.*, "Taskweaver: A code-first agent framework," *arXiv preprint arXiv:2311.17541*, 2023.
[93] M. A. Boshart and M. J. Kosa, "Growing a gui from an xml tree," *ACM SIGCSE Bulletin*, vol. 35, no. 3, pp. 223–223, 2003.
[94] Y. Li and O. Hilliges, *Artificial intelligence for human computer interaction: a modern approach*. Springer, 2021.
[95] H. Y. Abuaddous, A. M. Saleh, O. Enaizan, F. Ghabban, and A. B. Al-Badareen, "Automated user experience (ux) testing for mobile application: Strengths and limitations." *International Journal of Interactive Mobile Technologies*, vol. 16, no. 4, 2022.
[96] D. Gao, L. Ji, Z. Bai, M. Ouyang, P. Li, D. Mao, Q. Wu, W. Zhang, P. Wang, X. Guo *et al.*, "Assistgui: Task-oriented pc graphical user interface automation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 13 289–13 298.
[97] J. Qian, Z. Shang, S. Yan, Y. Wang, and L. Chen, "Roscript: A visual script driven truly non-intrusive robotic testing system for touch screen applications," in *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 297–308.
[98] A. Bruns, A. Kornstadt, and D. Wichmann, "Web application tests with selenium," *IEEE software*, vol. 26, no. 5, pp. 88–91, 2009.
[99] N. Rupp, K. Peschke, M. Köppl, D. Drissner, and T. Zuchner, "Establishment of low-cost laboratory automation processes using autoit and 4-axis robots," *SLAS technology*, vol. 27, no. 5, pp. 312–318, 2022.
[100] M. F. Granda, O. Parra, and B. Alba-Sarango, "Towards a model-driven testing framework for gui test cases generation from user stories." in *ENASE*, 2021, pp. 453–460.
[101] J. Xu, W. Du, X. Liu, and X. Li, "Llm4workflow: An llm-based automated workflow model generation tool," *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:273465368
[102] R. Gove and J. Faytong, "Machine learning and event-based software testing: classifiers for identifying infeasible gui event sequences," in *Advances in computers*. Elsevier, 2012, vol. 86, pp. 109–135.
[103] T. J.-J. Li, L. Popowski, T. Mitchell, and B. A. Myers, "Screen2vec: Semantic embedding of gui screens and gui components," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021, pp. 1–15.
[104] T.-H. Chang, T. Yeh, and R. C. Miller, "Gui testing using computer vision," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2010, pp. 1535–1544.
[105] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023.
[106] J. Ye, K. Chen, X. Xie, L. Ma, R. Huang, Y. Chen, Y. Xue, and J. Zhao, "An empirical study of gui widget detection for industrial mobile games," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 1427–1437.
[107] J. Chen, M. Xie, Z. Xing, C. Chen, X. Xu, L. Zhu, and G. Li, "Object detection for graphical user interface: Old fashioned or deep learning or a combination?" in *proceedings of the 28th ACM joint meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1202–1214.
[108] J. Qian, Y. Ma, C. Lin, and L. Chen, "Accelerating ocr-based widget localization for test automation of gui applications," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.
[109] O. Gambino, L. Rundo, V. Cannella, S. Vitabile, and R. Pirrone, "A framework for data-driven adaptive gui generation based on dicom," *Journal of biomedical informatics*, vol. 88, pp. 37–52, 2018.
[110] J. He, I.-L. Yen, T. Peng, J. Dong, and F. Bastani, "An adaptive user interface generation framework for web services," in *2008 IEEE Congress on Services Part II (services-2 2008)*. IEEE, 2008, pp. 175–182.
[111] Z. Stefanidi, G. Margetis, S. Ntoa, and G. Papagiannakis, "Real-time adaptation of context-aware intelligent user interfaces, for enhanced situational awareness," *IEEE Access*, vol. 10, pp. 23 367–23 393, 2022.
[112] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
[113] P. Brie, N. Burny, A. Sluÿters, and J. Vanderdonckt, "Evaluating a large language model on searching for gui layouts," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, no. EICS, pp. 1–37, 2023.
[114] T. Wetzlmaier, R. Ramler, and W. Putschögl, "A framework for monkey gui testing," in *2016 IEEE international conference on software testing, verification and validation (ICST)*. IEEE, 2016, pp. 416–423.
[115] X. Zeng, D. Li, W. Zheng, F. Xia, Y. Deng, W. Lam, W. Yang, and T. Xie, "Automated test input generation for android: are we really there yet in an industrial case?" in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 987–992. [Online]. Available: https://doi.org/10.1145/2950290.2983958
[116] A. M. Memon, M. E. Pollack, and M. L. Soffa, "Hierarchical gui test case generation using automated planning," *IEEE transactions on software engineering*, vol. 27, no. 2, pp. 144–155, 2001.
[117] S. Agostinelli, M. Lupia, A. Marrella, and M. Mecella, "Automated generation of executable rpa scripts from user interface logs," in *Business Process Management: Blockchain and Robotic Process Automation Forum: BPM 2020 Blockchain and RPA Forum, Seville, Spain, September 13–18, 2020, Proceedings 18*. Springer, 2020, pp. 116–131.
[118] A. Memon, I. Banerjee, N. Hashmi, and A. Nagarajan, "Dart: a framework for regression testing "nightly/daily builds" of gui applications," in *International Conference on Software Maintenance, 2003. ICSM 2003. Proceedings.*., 2003, pp. 410–419.
[119] Microsoft, "Create desktop flows using record with copilot (preview)," 2024, accessed: 2024-11-16. [Online]. Available: https://learn.microsoft.com/en-us/power-automate/desktop-flows/create-flow-using-ai-recorder
[120] selenium. (2024) Selenium: Browser automation. Accessed: 2024-11-05. [Online]. Available: https://www.selenium.dev/
[121] appium. (2024) Appium: Cross-platform automation framework for all kinds of apps. Accessed: 2024-11-05. [Online]. Available: https://appium.io/docs/en/latest/
[122] smartbear. (2024) Testcomplete: Automated ui testing tool. Accessed: 2024-11-05. [Online]. Available: https://smartbear.com/product/testcomplete/
[123] katalon. (2024) Katalon studio: Easy test automation for web, api, mobile, and desktop. Accessed: 2024-11-05. [Online]. Available: https://katalon.com/katalon-studio
[124] ranorex. (2024) Ranorex studio: Test automation for gui testing. Accessed: 2024-11-05. [Online]. Available: https://www.ranorex.com/
[125] G. Hu, L. Zhu, and J. Yang, "Appflow: using machine learning to synthesize robust, reusable ui tests," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 269–282. [Online]. Available: https://doi.org/10.1145/3236024.3236055
[126] Y. Li, Z. Yang, Y. Guo, and X. Chen, "Humanoid: A deep learning-based approach to automated black-box android app testing," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1070–1073.
[127] F. YazdaniBanafsheDaragh and S. Malek, "Deep gui: Black-box gui input generation with deep learning," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 905–916.
[128] M. Xie, S. Feng, Z. Xing, J. Chen, and C. Chen, "Uied: a hybrid tool for gui element detection," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1655–1659.
[129] N. Xu, S. Masling, M. Du, G. Campagna, L. Heck, J. Landay, and M. S. Lam, "Grounding open-domain instructions to automate web support tasks," 2021. [Online]. Available: https://arxiv.org/abs/2103.16057

[130] S. Mazumder and O. Riva, "Flin: A flexible natural language interface for web navigation," *arXiv preprint arXiv:2010.12844*, 2020.

[131] Y. Li, J. He, X. Zhou, Y. Zhang, and J. Baldridge, "Mapping natural language instructions to mobile ui action sequences," 2020. [Online]. Available: https://arxiv.org/abs/2005.03776

[132] T. Shi, A. Karpathy, L. Fan, J. Hernandez, and P. Liang, "World of bits: An open-domain platform for web-based agents," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 3135–3144. [Online]. Available: https://proceedings.mlr.press/v70/shi17a.html

[133] E. Z. Liu, K. Guu, P. Pasupat, T. Shi, and P. Liang, "Reinforcement learning on web interfaces using workflow-guided exploration," 2018. [Online]. Available: https://arxiv.org/abs/1802.08802

[134] Y. Lan, Y. Lu, Z. Li, M. Pan, W. Yang, T. Zhang, and X. Li, "Deeply reinforcing android gui testing with deep reinforcement learning," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[135] D. Toyama, P. Hamel, A. Gergely, G. Comanici, A. Glaese, Z. Ahmed, T. Jackson, S. Mourad, and D. Precup, "Androidenv: A reinforcement learning platform for android," *arXiv preprint arXiv:2105.13231*, 2021.

[136] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "Webshop: Towards scalable real-world web interaction with grounded language agents," *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 744–20 757, 2022.

[137] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Communications surveys & tutorials*, vol. 21, no. 3, pp. 2224–2287, 2019.

[138] P. Martins, F. Sá, F. Morgado, and C. Cunha, "Using machine learning for cognitive robotic process automation (rpa)," in *2020 15th Iberian Conference on Information Systems and Technologies (CISTI)*. IEEE, 2020, pp. 1–6.

[139] I. Gur, H. Furuta, A. Huang, M. Safdari, Y. Matsuo, D. Eck, and A. Faust, "A real-world webagent with planning, long context understanding, and program synthesis," *arXiv preprint arXiv:2307.12856*, 2023.

[140] H. Furuta, K.-H. Lee, O. Nachum, Y. Matsuo, A. Faust, S. S. Gu, and I. Gur, "Multimodal web navigation with instruction-finetuned foundation models," *arXiv preprint arXiv:2305.11854*, 2023.

[141] K. Ma, H. Zhang, H. Wang, X. Pan, W. Yu, and D. Yu, "Laser: Llm agent with state-space exploration for web navigation," *arXiv preprint arXiv:2309.08172*, 2023.

[142] Y. Deng, X. Zhang, W. Zhang, Y. Yuan, S.-K. Ng, and T.-S. Chua, "On the multi-turn instruction following for conversational web agents," *arXiv preprint arXiv:2402.15057*, 2024.

[143] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, "Autodroid: Llm-powered task automation in android," in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, 2024, pp. 543–557.

[144] A. Yan, Z. Yang, W. Zhu, K. Lin, L. Li, J. Wang, J. Yang, Y. Zhong, J. McAuley, J. Gao *et al.*, "Gpt-4v in wonderland: Large multimodal models for zero-shot smartphone gui navigation," *arXiv preprint arXiv:2311.07562*, 2023.

[145] J. Wang, H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang, "Mobile-agent: Autonomous multi-modal mobile device agent with visual perception," 2024. [Online]. Available: https://arxiv.org/abs/2401.16158

[146] S. Nong, J. Zhu, R. Wu, J. Jin, S. Shan, X. Huang, and W. Xu, "Mobileflow: A multimodal llm for mobile gui agent," 2024. [Online]. Available: https://arxiv.org/abs/2407.04346

[147] J. Zhang, J. Wu, Y. Teng, M. Liao, N. Xu, X. Xiao, Z. Wei, and D. Tang, "Android in the zoo: Chain-of-action-thought for gui agents," *arXiv preprint arXiv:2403.02713*, 2024.

[148] W. Tan, W. Zhang, X. Xu, H. Xia, Z. Ding, B. Li, B. Zhou, J. Yue, J. Jiang, Y. Li, R. An, M. Qin, C. Zong, L. Zheng, Y. Wu, X. Chai, Y. Bi, T. Xie, P. Gu, X. Li, C. Zhang, L. Tian, C. Wang, X. Wang, B. F. Karlsson, B. An, S. Yan, and Z. Lu, "Cradle: Empowering foundation agents towards general computer control," 2024. [Online]. Available: https://arxiv.org/abs/2403.03186

[149] Z. Wu, C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao, T. Yu, and L. Kong, "Os-copilot: Towards generalist computer agents with self-improvement," 2024. [Online]. Available: https://arxiv.org/abs/2402.07456

[150] Anthropic. (2024) Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku. Accessed: 2024-10-26. [Online]. Available: https://www.anthropic.com/news/3-5-models-and-computer-use

[151] S. Hu, M. Ouyang, D. Gao, and M. Z. Shou, "The dawn of gui agent: A preliminary case study with claude 3.5 computer use," 2024. [Online]. Available: https://arxiv.org/abs/2411.10323

[152] A. M. Memon, I. Banerjee, and A. Nagarajan, "Gui ripping: reverse engineering of graphical user interfaces for testing." in *WCRE*, vol. 3, 2003, p. 260.

[153] J. Wang, Z. Liu, L. Zhao, Z. Wu, C. Ma, S. Yu, H. Dai, Q. Yang, Y. Liu, S. Zhang *et al.*, "Review of large vision models and visual prompt engineering," *Meta-Radiology*, p. 100047, 2023.

[154] R. Hardy and E. Rukzio, "Touch & interact: touch-based interaction of mobile phones with displays," in *Proceedings of the 10th international conference on Human computer interaction with mobile devices and services*, 2008, pp. 245–254.

[155] H. Lee, J. Park, and U. Lee, "A systematic survey on android api usage for data-driven analytics with smartphones," *ACM Computing Surveys*, vol. 55, no. 5, pp. 1–38, 2022.

[156] S. Mitra and T. Acharya, "Gesture recognition: A survey," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 37, no. 3, pp. 311–324, 2007.

[157] K. Jokinen, "User interaction in mobile navigation applications," in *Map-based Mobile Services: Design, Interaction and Usability*. Springer, 2008, pp. 168–197.

[158] W. Enck, D. Octeau, P. D. McDaniel, and S. Chaudhuri, "A study of android application security." in *USENIX security symposium*, vol. 2, no. 2, 2011.

[159] M. Egele, C. Kruegel, E. Kirda, and G. Vigna, "Pios: Detecting privacy leaks in ios applications." in *NDSS*, vol. 2011, 2011, p. 18th.

[160] B. Sierkowski, "Achieving web accessibility," in *Proceedings of the 30th annual ACM SIGUCCS conference on User services*, 2002, pp. 288–291.

[161] N. Fernandes, R. Lopes, and L. Carriço, "On web accessibility evaluation environments," in *Proceedings of the International Cross-Disciplinary Conference on Web Accessibility*, 2011, pp. 1–10.

[162] J. J. Garrett *et al.*, "Ajax: A new approach to web applications," 2005.

[163] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v," *arXiv preprint arXiv:2310.11441*, 2023.

[164] X. Wu, J. Ye, K. Chen, X. Xie, Y. Hu, R. Huang, L. Ma, and J. Zhao, "Widget detection-based testing for industrial mobile games," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2023, pp. 173–184.

[165] E. Gamma, "Design patterns: elements of reusable object-oriented software," *Person Education Inc*, 1995.

[166] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, Q. He, Y. Ma, M. Huang, and S. Wang, "A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness," 2024. [Online]. Available: https://arxiv.org/abs/2411.03350

[167] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo *et al.*, "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4015–4026.

[168] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su *et al.*, "Grounding dino: Marrying dino with grounded pre-training for open-set object detection," *arXiv preprint arXiv:2303.05499*, 2023.

[169] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, "Omniparser for pure vision based gui agent," 2024. [Online]. Available: https://arxiv.org/abs/2408.00203

[170] K. Moran, C. Watson, J. Hoskins, G. Purnell, and D. Poshyvanyk, "Detecting and summarizing gui changes in evolving mobile apps," in *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*, 2018, pp. 543–553.

[171] F. P. Ricós, R. Neeft, B. Marín, T. E. Vos, and P. Aho, "Using gui change detection for delta testing," in *International Conference on Research Challenges in Information Science*. Springer, 2023, pp. 509–517.

[172] Y. Du, F. Wei, and H. Zhang, "Anytool: Self-reflective, hierarchical agents for large-scale api calls," *arXiv preprint arXiv:2402.04253*, 2024.

[173] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[174] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[175] S. Zhang, Z. Zhang, K. Chen, X. Ma, M. Yang, T. Zhao, and M. Zhang, "Dynamic planning for llm-based graphical user interface automation," *arXiv preprint arXiv:2410.00467*, 2024.

[176] J. Cho, J. Kim, D. Bae, J. Choo, Y. Gwon, and Y.-D. Kwon, "Caap: Context-aware action planning prompting to solve computer tasks with front-end ui only," *arXiv preprint arXiv:2406.06947*, 2024.

[177] G. Dagan, F. Keller, and A. Lascarides, "Dynamic planning with a llm," *arXiv preprint arXiv:2308.06391*, 2023.

[178] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, "Decomposed prompting: A modular approach for solving complex tasks," *arXiv preprint arXiv:2210.02406*, 2022.

[179] Y. Chen, A. Pesaranghader, T. Sadhu, and D. H. Yi, "Can we rely on llm agents to draft long-horizon plans? let's take travelplanner as an example," *arXiv preprint arXiv:2408.06318*, 2024.

[180] A. Sweigart, "Pyautogui: A cross-platform gui automation python module," GitHub repository, 2024, accessed: 2024-10-27. [Online]. Available: https://github.com/asweigart/pyautogui

[181] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," in *International conference on machine learning*. Pmlr, 2021, pp. 8821–8831.

[182] X. Gu, H. Zhang, D. Zhang, and S. Kim, "Deep api learning," in *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*, 2016, pp. 631–642.

[183] T. Masterman, S. Besen, M. Sawtell, and A. Chao, "The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey," *arXiv preprint arXiv:2404.11584*, 2024.

[184] J. Lu, Z. Zhang, F. Yang, J. Zhang, L. Wang, C. Du, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "Turn every application into an agent: Towards efficient human-agent-computer interaction with api-first llm-based agents," *arXiv preprint arXiv:2409.17140*, 2024.

[185] Y. Song, F. Xu, S. Zhou, and G. Neubig, "Beyond browsing: Api-based web agents," *arXiv preprint arXiv:2410.16464*, 2024.

[186] S. Lee, J. Choi, J. Lee, M. H. Wasi, H. Choi, S. Y. Ko, S. Oh, and I. Shin, "Explore, select, derive, and recall: Augmenting llm with human-like memory for mobile task automation," *arXiv preprint arXiv:2312.03003*, 2023.

[187] J. Lu, S. An, M. Lin, G. Pergola, Y. He, D. Yin, X. Sun, and Y. Wu, "Memochat: Tuning llms to use memos for consistent long-range open-domain conversation," *arXiv preprint arXiv:2308.08239*, 2023.

[188] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei, "Augmenting language models with long-term memory," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[189] J. Tack, J. Kim, E. Mitchell, J. Shin, Y. W. Teh, and J. R. Schwarz, "Online adaptation of language models with a memory of amortized contexts," *arXiv preprint arXiv:2403.04317*, 2024.

[190] X. Zhu, Y. Chen, H. Tian, C. Tao, W. Su, C. Yang, G. Huang, B. Li, L. Lu, X. Wang *et al.*, "Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory," *arXiv preprint arXiv:2305.17144*, 2023.

[191] L. Zheng, R. Wang, X. Wang, and B. An, "Synapse: Trajectory-as-exemplar prompting with memory for computer control," 2024. [Online]. Available: https://arxiv.org/abs/2306.07863

[192] X. Zhan, T. Liu, L. Fan, L. Li, S. Chen, X. Luo, and Y. Liu, "Research on third-party libraries in android apps: A taxonomy and systematic literature review," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 4181–4213, 2021.

[193] Y. Li, G. Li, L. He, J. Zheng, H. Li, and Z. Guan, "Widget captioning: Generating natural language description for mobile user interface elements," *arXiv preprint arXiv:2010.04295*, 2020.

[194] B. Wang, G. Li, X. Zhou, Z. Chen, T. Grossman, and Y. Li, "Screen2words: Automatic mobile ui summarization with multimodal learning," in *The 34th Annual ACM Symposium on User Interface Software and Technology*, 2021, pp. 498–510.

[195] C. Bai, X. Zang, Y. Xu, S. Sunkara, A. Rastogi, J. Chen *et al.*, "Uibert: Learning generic multimodal representations for ui understanding," *arXiv preprint arXiv:2107.13731*, 2021.

[196] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for "mind" exploration of large language model society," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[197] W. Chen, Z. You, R. Li, Y. Guan, C. Qian, C. Zhao, C. Yang, R. Xie, Z. Liu, and M. Sun, "Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence," 2024. [Online]. Available: https://arxiv.org/abs/2407.07061

[198] Z. Song, Y. Li, M. Fang, Z. Chen, Z. Shi, Y. Huang, and L. Chen, "Mmac-copilot: Multi-modal agent collaboration operating system copilot," *arXiv preprint arXiv:2404.18074*, 2024.

[199] M. Renze and E. Guven, "Self-reflection in llm agents: Effects on problem-solving performance," *arXiv preprint arXiv:2405.06682*, 2024.

[200] J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, and A. Suhr, "Autonomous evaluation and refinement of digital agents," in *First Conference on Language Modeling*, 2024.

[201] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[202] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[203] Z. Tao, T.-E. Lin, X. Chen, H. Li, Y. Wu, Y. Li, Z. Jin, F. Huang, D. Tao, and J. Zhou, "A survey on self-evolution of large language models," *arXiv preprint arXiv:2404.14387*, 2024.

[204] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642.

[205] Z. Zhu, Y. Xue, X. Chen, D. Zhou, J. Tang, D. Schuurmans, and H. Dai, "Large language models can learn rules," *arXiv preprint arXiv:2310.07064*, 2023.

[206] Y. Zhang, P. Xiao, L. Wang, C. Zhang, M. Fang, Y. Du, Y. Puzyrev, R. Yao, S. Qin, Q. Lin, M. Pechenizkiy, D. Zhang, S. Rajmohan, and Q. Zhang, "Ruag: Learned-rule-augmented generation for large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.03349

[207] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.

[208] Y. Wang, W. Zhong, L. Li, F. Mi, X. Zeng, W. Huang, L. Shang, X. Jiang, and Q. Liu, "Aligning large language models with human: A survey," *arXiv preprint arXiv:2307.12966*, 2023.

[209] Y. Zhai, H. Bai, Z. Lin, J. Pan, S. Tong, Y. Zhou, A. Suhr, S. Xie, Y. LeCun, Y. Ma *et al.*, "Fine-tuning large vision-language models as decision-making agents via reinforcement learning," *arXiv preprint arXiv:2405.10292*, 2024.

[210] M. L. Puterman, "Markov decision processes," *Handbooks in operations research and management science*, vol. 2, pp. 331–434, 1990.

[211] D. Toyama, P. Hamel, A. Gergely, G. Comanici, A. Glaese, Z. Ahmed, T. Jackson, S. Mourad, and D. Precup, "Androidenv: A reinforcement learning platform for android," 2021. [Online]. Available: https://arxiv.org/abs/2105.13231

[212] H. Bai, Y. Zhou, M. Cemri, J. Pan, A. Suhr, S. Levine, and A. Kumar, "Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2406.11896

[213] T. Wang, Z. Wu, J. Liu, J. Hao, J. Wang, and K. Shao, "Distrl: An asynchronous distributed reinforcement learning framework for on-device control agents," *arXiv preprint arXiv:2410.14803*, 2024.

[214] H. Chae, N. Kim, K. T. iunn Ong, M. Gwak, G. Song, J. Kim, S. Kim, D. Lee, and J. Yeo, "Web agents with world models: Learning and leveraging environment dynamics in web navigation," 2024. [Online]. Available: https://arxiv.org/abs/2410.13232

[215] I. Gur, H. Furuta, A. Huang, M. Safdari, Y. Matsuo, D. Eck, and A. Faust, "A real-world webagent with planning, long context

understanding, and program synthesis," 2024. [Online]. Available: https://arxiv.org/abs/2307.12856
[216] K. Ma, H. Zhang, H. Wang, X. Pan, W. Yu, and D. Yu, "Laser: Llm agent with state-space exploration for web navigation," 2024. [Online]. Available: https://arxiv.org/abs/2309.08172
[217] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "Webvoyager: Building an end-to-end web agent with large multimodal models," 2024. [Online]. Available: https://arxiv.org/abs/2401.13919
[218] H. Lai, X. Liu, I. L. Iong, S. Yao, Y. Chen, P. Shen, H. Yu, H. Zhang, X. Zhang, Y. Dong, and J. Tang, "Autowebglm: Bootstrap and reinforce a large language model-based web navigating agent," 2024. [Online]. Available: https://arxiv.org/abs/2404.03648
[219] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, L. Z. Liu, Y. Xu, H. Su, D. Shin, C. Xiong, and T. Yu, "Openagents: An open platform for language agents in the wild," 2023. [Online]. Available: https://arxiv.org/abs/2310.10634
[220] J. Kil, C. H. Song, B. Zheng, X. Deng, Y. Su, and W.-L. Chao, "Dual-view visual contextualization for web navigation," 2024. [Online]. Available: https://arxiv.org/abs/2402.04476
[221] T. Abuelsaad, D. Akkil, P. Dey, A. Jagmohan, A. Vempaty, and R. Kokku, "Agent-e: From autonomous web navigation to foundational design principles in agentic systems," 2024. [Online]. Available: https://arxiv.org/abs/2407.13032
[222] J. Y. Koh, S. McAleer, D. Fried, and R. Salakhutdinov, "Tree search for language model agents," *arXiv preprint arXiv:2407.01476*, 2024.
[223] Y. Zhang, Z. Ma, Y. Ma, Z. Han, Y. Wu, and V. Tresp, "Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration," *arXiv preprint arXiv:2408.15978*, 2024.
[224] K. Yang, Y. Liu, S. Chaudhary, R. Fakoor, P. Chaudhari, G. Karypis, and H. Rangwala, "Agentoccam: A simple yet strong baseline for llm-based web agents," 2024. [Online]. Available: https://arxiv.org/abs/2410.13825
[225] S. Murty, D. Bahdanau, and C. D. Manning, "Nnetscape navigator: Complex demonstrations for web agents without a demonstrator," *arXiv preprint arXiv:2410.02907*, 2024.
[226] M. Shahbandeh, P. Alian, N. Nashid, and A. Mesbah, "Naviqate: Functionality-guided web application navigation," *arXiv preprint arXiv:2409.10741*, 2024.
[227] I. L. Iong, X. Liu, Y. Chen, H. Lai, S. Yao, P. Shen, H. Yu, Y. Dong, and J. Tang, "Openwebagent: An open toolkit to enable web agents on large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, 2024, pp. 72–81.
[228] B. Tang and K. G. Shin, "Steward: Natural language web automation," *arXiv preprint arXiv:2409.15441*, 2024.
[229] P. Putta, E. Mills, N. Garg, S. Motwani, C. Finn, D. Garg, and R. Rafailov, "Agent q: Advanced reasoning and learning for autonomous ai agents," *arXiv preprint arXiv:2408.07199*, 2024.
[230] X. Liu, B. Qin, D. Liang, G. Dong, H. Lai, H. Zhang, H. Zhao, I. L. Iong, J. Sun, J. Wang *et al.*, "Autoglm: Autonomous foundation agents for guis," *arXiv preprint arXiv:2411.00820*, 2024.
[231] P. Pawlowski, K. Zawistowski, W. Lapacz, M. Skorupa, A. Wiacek, S. Postansque, and J. Hoscilowicz, "Tinyclick: Single-turn agent for empowering gui automation," *arXiv preprint arXiv:2410.11871*, 2024.
[232] Y. Gu, B. Zheng, B. Gou, K. Zhang, C. Chang, S. Srivastava, Y. Xie, P. Qi, H. Sun, and Y. Su, "Is your llm secretly a world model of the internet? model-based planning for web agents," *arXiv preprint arXiv:2411.06559*, 2024.
[233] G. Verma, R. Kaur, N. Srishankar, Z. Zeng, T. Balch, and M. Veloso, "Adaptagent: Adapting multimodal web agents with few-shot learning from human demonstrations," *arXiv preprint arXiv:2411.13451*, 2024.
[234] J. Kim, D.-K. Kim, L. Logeswaran, S. Sohn, and H. Lee, "Autointent: Automated intent discovery and self-exploration for large language model web agents," *arXiv preprint arXiv:2410.22552*, 2024.
[235] J. Zhang, J. Wu, Y. Teng, M. Liao, N. Xu, X. Xiao, Z. Wei, and D. Tang, "Android in the zoo: Chain-of-action-thought for gui agents," 2024. [Online]. Available: https://arxiv.org/abs/2403.02713
[236] Y. Song, Y. Bian, Y. Tang, G. Ma, and Z. Cai, "Visiontasker: Mobile task automation using vision based ui understanding and llm task planning," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '24. ACM, Oct. 2024, p. 1–17. [Online]. Available: http://dx.doi.org/10.1145/3654777.3676386
[237] H. Wen, H. Wang, J. Liu, and Y. Li, "Droidbot-gpt: Gpt-powered ui automation for android," 2024. [Online]. Available: https://arxiv.org/abs/2304.07061
[238] X. Ma, Z. Zhang, and H. Zhao, "Coco-agent: A comprehensive cognitive mllm agent for smartphone gui automation," 2024. [Online]. Available: https://arxiv.org/abs/2402.11941
[239] Z. Zhang and A. Zhang, "You only look at screens: Multimodal chain-of-action agents," 2024. [Online]. Available: https://arxiv.org/abs/2309.11436
[240] A. Yan, Z. Yang, W. Zhu, K. Lin, L. Li, J. Wang, J. Yang, Y. Zhong, J. McAuley, J. Gao, Z. Liu, and L. Wang, "Gpt-4v in wonderland: Large multimodal models for zero-shot smartphone gui navigation," 2023. [Online]. Available: https://arxiv.org/abs/2311.07562
[241] Y. Li, C. Zhang, W. Yang, B. Fu, P. Cheng, X. Chen, L. Chen, and Y. Wei, "Appagent v2: Advanced agent for flexible mobile interactions," 2024. [Online]. Available: https://arxiv.org/abs/2408.11824
[242] J. Wang, H. Xu, H. Jia, X. Zhang, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang, "Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration," 2024. [Online]. Available: https://arxiv.org/abs/2406.01014
[243] J. Zhang, C. Zhao, Y. Zhao, Z. Yu, M. He, and J. Fan, "Mobileexperts: A dynamic tool-enabled agent team in mobile devices," 2024. [Online]. Available: https://arxiv.org/abs/2407.03913
[244] F. Christianos, G. Papoudakis, T. Coste, H. B. Ammar, H. Jianye, J. Wang, and K. Shao, "Lightweight multi-modal android app control," in *NeurIPS 2024 Workshop on Open-World Agents*.
[245] Z. Zhu, H. Tang, Y. Li, K. Lan, Y. Jiang, H. Zhou, Y. Wang, S. Zhang, L. Sun, L. Chen *et al.*, "Moba: A two-level agent system for efficient mobile task automation," *arXiv preprint arXiv:2410.13757*, 2024.
[246] X. Wang and B. Liu, "Oscar: Operating system control via state-aware reasoning and re-planning," *arXiv preprint arXiv:2410.18963*, 2024.
[247] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent s: An open agentic framework that uses computers like a human," 2024. [Online]. Available: https://arxiv.org/abs/2410.08164
[248] Q. Wu, D. Gao, K. Q. Lin, Z. Wu, X. Guo, P. Li, W. Zhang, H. Wang, and M. Z. Shou, "Gui action narrator: Where and when did that action take place?" 2024. [Online]. Available: https://arxiv.org/abs/2406.13719
[249] T. Li, G. Li, Z. Deng, B. Wang, and Y. Li, "A zero-shot language agent for computer control with structured reflection," *arXiv preprint arXiv:2310.08740*, 2023.
[250] C. Jia, M. Luo, Z. Dang, Q. Sun, F. Xu, J. Hu, T. Xie, and Z. Wu, "Agentstore: Scalable integration of heterogeneous agents as specialized generalist computer assistant," *arXiv preprint arXiv:2410.18603*, 2024.
[251] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2web: Towards a generalist agent for the web," 2023. [Online]. Available: https://arxiv.org/abs/2306.06070
[252] Q. Chen, D. Pitawela, C. Zhao, G. Zhou, H.-T. Chen, and Q. Wu, "Webvln: Vision-and-language navigation on websites," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 2, pp. 1165–1173, Mar. 2024. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/27878
[253] X. H. Lù, Z. Kasner, and S. Reddy, "Weblinx: Real-world website navigation with multi-turn dialogue," 2024. [Online]. Available: https://arxiv.org/abs/2402.05930
[254] X. Liu, T. Zhang, Y. Gu, I. L. Iong, Y. Xu, X. Song, S. Zhang, H. Lai, X. Liu, H. Zhao, J. Sun, X. Yang, Y. Yang, Z. Qi, S. Yao, X. Sun, S. Cheng, Q. Zheng, H. Yu, H. Zhang, W. Hong, M. Ding, L. Pan, X. Gu, A. Zeng, Z. Du, C. H. Song, Y. Su, Y. Dong, and J. Tang, "Visualagentbench: Towards large multimodal models as visual foundation agents," 2024. [Online]. Available: https://arxiv.org/abs/2408.06327
[255] B. Gou, R. Wang, B. Zheng, Y. Xie, C. Chang, Y. Shu, H. Sun, and Y. Su, "Navigating the digital world as humans do: Universal visual grounding for gui agents," 2024. [Online]. Available: https://arxiv.org/abs/2410.05243
[256] W. Chen, J. Cui, J. Hu, Y. Qin, J. Fang, Y. Zhao, C. Wang, J. Liu, G. Chen, Y. Huo, Y. Yao, Y. Lin, Z. Liu, and M. Sun, "Guicourse: From general vision language models to versatile gui agents," 2024. [Online]. Available: https://arxiv.org/abs/2406.11317

[257] J. Zhang, T. Lan, M. Zhu, Z. Liu, T. Hoang, S. Kokane, W. Yao, J. Tan, A. Prabhakar, H. Chen *et al.*, "xlam: A family of large action models to empower ai agent systems," *arXiv preprint arXiv:2409.03215*, 2024.

[258] Z. Meng, Y. Dai, Z. Gong, S. Guo, M. Tang, and T. Wei, "Vga: Vision gui assistant – minimizing hallucinations through image-centric fine-tuning," 2024. [Online]. Available: https://arxiv.org/abs/2406.14056

[259] B. Deka, Z. Huang, C. Franzen, J. Hibschman, D. Afergan, Y. Li, J. Nichols, and R. Kumar, "Rico: A mobile app dataset for building data-driven design applications," in *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 845–854. [Online]. Available: https://doi.org/10.1145/3126594.3126651

[260] A. Burns, D. Arsan, S. Agrawal, R. Kumar, K. Saenko, and B. A. Plummer, "A dataset for interactive vision-language navigation with unknown command feasibility," 2022. [Online]. Available: https://arxiv.org/abs/2202.02312

[261] L. Sun, X. Chen, L. Chen, T. Dai, Z. Zhu, and K. Yu, "Meta-gui: Towards multi-modal conversational agents on mobile gui," 2022. [Online]. Available: https://arxiv.org/abs/2205.11029

[262] C. Rawles, A. Li, D. Rodriguez, O. Riva, and T. Lillicrap, "Android in the wild: A large-scale dataset for android device control," 2023. [Online]. Available: https://arxiv.org/abs/2307.10088

[263] Q. Lu, W. Shao, Z. Liu, F. Meng, B. Li, B. Chen, S. Huang, K. Zhang, Y. Qiao, and P. Luo, "Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices," 2024. [Online]. Available: https://arxiv.org/abs/2406.08451

[264] Y. Chai, S. Huang, Y. Niu, H. Xiao, L. Liu, D. Zhang, P. Gao, S. Ren, and H. Li, "Amex: Android multi-annotation expo dataset for mobile gui agents," 2024. [Online]. Available: https://arxiv.org/abs/2407.17490

[265] K. You, H. Zhang, E. Schoop, F. Weers, A. Swearngin, J. Nichols, Y. Yang, and Z. Gan, "Ferret-ui: Grounded mobile ui understanding with multimodal llms," in *European Conference on Computer Vision*. Springer, 2025, pp. 240–255.

[266] W. Chen, Z. Li, Z. Guo, and Y. Shen, "Octo-planner: On-device language model for planner-action agents," 2024. [Online]. Available: https://arxiv.org/abs/2406.18082

[267] K. Wang, T. Xia, Z. Gu, Y. Zhao, S. Shen, C. Meng, W. Wang, and K. Xu, "E-ant: A large-scale dataset for efficient automatic gui navigation," 2024. [Online]. Available: https://arxiv.org/abs/2406.14250

[268] Q. Wu, W. Xu, W. Liu, T. Tan, J. Liu, A. Li, J. Luan, B. Wang, and S. Shang, "Mobilevlm: A vision-language model for better intra- and inter-ui understanding," 2024. [Online]. Available: https://arxiv.org/abs/2409.14818

[269] Y. Xu, X. Liu, X. Sun, S. Cheng, H. Yu, H. Lai, S. Zhang, D. Zhang, J. Tang, and Y. Dong, "Androidlab: Training and systematic benchmarking of android autonomous agents," 2024. [Online]. Available: https://arxiv.org/abs/2410.24024

[270] L. Gao, L. Zhang, S. Wang, S. Wang, Y. Li, and M. Xu, "Mobileviews: A large-scale mobile gui dataset," *arXiv preprint arXiv:2409.14337*, 2024.

[271] D. Chen, Y. Huang, S. Wu, J. Tang, L. Chen, Y. Bai, Z. He, C. Wang, H. Zhou, Y. Li, T. Zhou, Y. Yu, C. Gao, Q. Zhang, Y. Gui, Z. Li, Y. Wan, P. Zhou, J. Gao, and L. Sun, "Gui-world: A dataset for gui-oriented multimodal llm-based agents," 2024. [Online]. Available: https://arxiv.org/abs/2406.10819

[272] G. Baechler, S. Sunkara, M. Wang, F. Zubach, H. Mansoor, V. Etter, V. Cărbune, J. Lin, J. Chen, and A. Sharma, "Screenai: A vision-language model for ui and infographics understanding," 2024. [Online]. Available: https://arxiv.org/abs/2402.04615

[273] R. Niu, J. Li, S. Wang, Y. Fu, X. Hu, X. Leng, H. Kong, Y. Chang, and Q. Wang, "Screenagent: A vision language model-driven computer control agent," 2024. [Online]. Available: https://arxiv.org/abs/2402.07945

[274] OpenAI, "Gpt-4v(ision) system card," OpenAI, Tech. Rep., September 2023. [Online]. Available: https://cdn.openai.com/papers/GPTV_System_Card.pdf

[275] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, "Qwen-vl: A frontier large vision-language model with versatile abilities," *arXiv preprint arXiv:2308.12966*, 2023.

[276] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, Y. Fan, K. Dang, M. Du, X. Ren, R. Men, D. Liu, C. Zhou, J. Zhou, and J. Lin, "Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution," 2024. [Online]. Available: https://arxiv.org/abs/2409.12191

[277] Z. Chen, J. Wu, W. Wang, W. Su, G. Chen, S. Xing, M. Zhong, Q. Zhang, X. Zhu, L. Lu *et al.*, "Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 24 185–24 198.

[278] Z. Chen, W. Wang, H. Tian, S. Ye, Z. Gao, E. Cui, W. Tong, K. Hu, J. Luo, Z. Ma *et al.*, "How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites," *arXiv preprint arXiv:2404.16821*, 2024.

[279] W. Wang, Q. Lv, W. Yu, W. Hong, J. Qi, Y. Wang, J. Ji, Z. Yang, L. Zhao, X. Song, J. Xu, B. Xu, J. Li, Y. Dong, M. Ding, and J. Tang, "Cogvlm: Visual expert for pretrained language models," 2024. [Online]. Available: https://arxiv.org/abs/2311.03079

[280] H. You, H. Zhang, Z. Gan, X. Du, B. Zhang, Z. Wang, L. Cao, S.-F. Chang, and Y. Yang, "Ferret: Refer and ground anything anywhere at any granularity," *arXiv preprint arXiv:2310.07704*, 2023.

[281] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *Advances in neural information processing systems*, vol. 36, 2024.

[282] H. Liu, C. Li, Y. Li, and Y. J. Lee, "Improved baselines with visual instruction tuning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 26 296–26 306.

[283] J. Li, D. Li, S. Savarese, and S. Hoi, "Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *International conference on machine learning*. PMLR, 2023, pp. 19 730–19 742.

[284] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.

[285] M. Fereidouni and A. B. Siddique, "Search beyond queries: Training smaller language models for web interactions via reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2404.10887

[286] L.-A. Thil, M. Popa, and G. Spanakis, "Navigating webai: Training agents to complete web tasks with large language models and reinforcement learning," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC '24, vol. 30. ACM, Apr. 2024, p. 866–874. [Online]. Available: http://dx.doi.org/10.1145/3605098.3635903

[287] H. He, W. Yao, K. Ma, W. Yu, H. Zhang, T. Fang, Z. Lan, and D. Yu, "Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization," 2024. [Online]. Available: https://arxiv.org/abs/2410.19609

[288] Z. Qi, X. Liu, I. L. Iong, H. Lai, X. Sun, X. Yang, J. Sun, Y. Yang, S. Yao, T. Zhang, W. Xu, J. Tang, and Y. Dong, "Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2411.02337

[289] Z. Li, K. You, H. Zhang, D. Feng, H. Agrawal, X. Li, M. P. S. Moorthy, J. Nichols, Y. Yang, and Z. Gan, "Ferret-ui 2: Mastering universal user interface understanding across platforms," *arXiv preprint arXiv:2410.18967*, 2024.

[290] K. Q. Lin, L. Li, D. Gao, Z. Yang, Z. Bai, W. Lei, L. Wang, and M. Z. Shou, "Training a vision-language-action model as generalist gui agent," in *NeurIPS 2024 Workshop on Open-World Agents*.

[291] Z. Wu, Z. Wu, F. Xu, Y. Wang, Q. Sun, C. Jia, K. Cheng, Z. Ding, L. Chen, P. P. Liang *et al.*, "Os-atlas: A foundation action model for generalist gui agents," *arXiv preprint arXiv:2410.23218*, 2024.

[292] Y. Qian, Y. Lu, A. Hauptmann, and O. Riva, "Visual grounding for user interfaces," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, Y. Yang, A. Davani, A. Sil, and A. Kumar, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 97–107. [Online]. Available: https://aclanthology.org/2024.naacl-industry.9

[293] W. Chen, Z. Li, and M. Ma, "Octopus: On-device language model for function calling of software apis," 2024. [Online]. Available: https://arxiv.org/abs/2404.01549

[294] W. Chen and Z. Li, "Octopus v2: On-device language model for super agent," 2024. [Online]. Available: https://arxiv.org/abs/2404.01744

[295] ——, "Octopus v3: Technical report for on-device sub-

billion multimodal ai agent," 2024. [Online]. Available: https://arxiv.org/abs/2404.11459

[296] ——, "Octopus v4: Graph of language models," 2024. [Online]. Available: https://arxiv.org/abs/2404.19296

[297] W. Li, F.-L. Hsu, W. Bishop, F. Campbell-Ajala, M. Lin, and O. Riva, "Uinav: A practical approach to train on-device automation agents," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, 2024, pp. 36–51.

[298] Z. Zhang, W. Xie, X. Zhang, and Y. Lu, "Reinforced ui instruction grounding: Towards a generic ui task automation api," 2023. [Online]. Available: https://arxiv.org/abs/2310.04716

[299] J. Yang, Y. Dong, S. Liu, B. Li, Z. Wang, H. Tan, C. Jiang, J. Kang, Y. Zhang, K. Zhou *et al.*, "Octopus: Embodied vision-language programmer from environmental feedback," in *European Conference on Computer Vision*. Springer, 2025, pp. 20–38.

[300] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," 2024. [Online]. Available: https://arxiv.org/abs/2307.13854

[301] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. C. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried, "Visualwebarena: Evaluating multimodal agents on realistic visual web tasks," 2024. [Online]. Available: https://arxiv.org/abs/2401.13649

[302] Y. Deng, X. Zhang, W. Zhang, Y. Yuan, S.-K. Ng, and T.-S. Chua, "On the multi-turn instruction following for conversational web agents," 2024. [Online]. Available: https://arxiv.org/abs/2402.15057

[303] Z. Zhang, S. Tian, L. Chen, and Z. Liu, "Mmina: Benchmarking multihop multimodal internet agents," 2024. [Online]. Available: https://arxiv.org/abs/2404.09992

[304] Y. Pan, D. Kong, S. Zhou, C. Cui, Y. Leng, B. Jiang, H. Liu, Y. Shang, S. Zhou, T. Wu, and Z. Wu, "Webcanvas: Benchmarking web agents in online environments," 2024. [Online]. Available: https://arxiv.org/abs/2406.12373

[305] I. Levy, B. Wiesel, S. Marreed, A. Oved, A. Yaeli, and S. Shlomov, "St-webagentbench: A benchmark for evaluating safety and trustworthiness in web agents," *arXiv preprint arXiv:2410.06703*, 2024.

[306] H. Furuta, Y. Matsuo, A. Faust, and I. Gur, "Exposing limitations of language model agents in sequential-task compositions on the web," in *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.

[307] K. Xu, Y. Kordi, T. Nayak, A. Asija, Y. Wang, K. Sanders, A. Byerly, J. Zhang, B. Van Durme, and D. Khashabi, "Tur [k] ingbench: A challenge benchmark for web agents," *arXiv preprint arXiv:2403.11905*, 2024.

[308] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments," 2024. [Online]. Available: https://arxiv.org/abs/2404.07972

[309] R. Kapoor, Y. P. Butala, M. Russak, J. Y. Koh, K. Kamble, W. Alshikh, and R. Salakhutdinov, "Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web," 2024. [Online]. Available: https://arxiv.org/abs/2402.17553

[310] K. Q. Lin, L. Li, D. Gao, Q. WU, M. Yan, Z. Yang, L. Wang, and M. Z. Shou, "Videogui: A benchmark for gui automation from instructional videos," 2024. [Online]. Available: https://arxiv.org/abs/2406.10227

[311] A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, L. Boisvert, M. Thakkar, Q. Cappart, D. Vazquez *et al.*, "Workarena: How capable are web agents at solving common knowledge work tasks?" *arXiv preprint arXiv:2403.07718*, 2024.

[312] L. Jang, Y. Li, C. Ding, J. Lin, P. P. Liang, D. Zhao, R. Bonatti, and K. Koishida, "Videowebarena: Evaluating long context multimodal agents with video understanding web tasks," *arXiv preprint arXiv:2410.19100*, 2024.

[313] X. Ma, Y. Wang, Y. Yao, T. Yuan, A. Zhang, Z. Zhang, and H. Zhao, "Caution for the environment: Multimodal agents are susceptible to environmental distractions," 2024. [Online]. Available: https://arxiv.org/abs/2408.02544

[314] J. Liu, Y. Song, B. Y. Lin, W. Lam, G. Neubig, Y. Li, and X. Yue, "Visualwebbench: How far have multimodal llms evolved in web page understanding and grounding?" 2024. [Online]. Available: https://arxiv.org/abs/2404.05955

[315] M. Wornow, A. Narayan, B. Viggiano, I. S. Khare, T. Verma, T. Thompson, M. A. F. Hernandez, S. Sundar, C. Trujillo, K. Chawla *et al.*, "Wonderbread: A benchmark for evaluating multimodal foundation models on business process management tasks," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[316] B. Zheng, B. Gou, S. Salisbury, Z. Du, H. Sun, and Y. Su, "Webolympus: An open platform for web agents on live websites," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2024, pp. 187–197.

[317] Y. Fan, L. Ding, C.-C. Kuo, S. Jiang, Y. Zhao, X. Guan, J. Yang, Y. Zhang, and X. E. Wang, "Read anywhere pointed: Layout-aware gui screen reading with tree-of-lens grounding," 2024. [Online]. Available: https://arxiv.org/abs/2406.19263

[318] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "Webshop: Towards scalable real-world web interaction with grounded language agents," 2023. [Online]. Available: https://arxiv.org/abs/2207.01206

[319] T. Xu, L. Chen, D.-J. Wu, Y. Chen, Z. Zhang, X. Yao, Z. Xie, Y. Chen, S. Liu, B. Qian, P. Torr, B. Ghanem, and G. Li, "Crab: Cross-environment agent benchmark for multimodal language model agents," 2024. [Online]. Available: https://arxiv.org/abs/2407.01511

[320] D. Zhang, Z. Shen, R. Xie, S. Zhang, T. Xie, Z. Zhao, S. Chen, L. Chen, H. Xu, R. Cao, and K. Yu, "Mobile-env: Building qualified evaluation benchmarks for llm-gui interaction," 2024. [Online]. Available: https://arxiv.org/abs/2305.08144

[321] J. Lee, T. Min, M. An, C. Kim, and K. Lee, "Benchmarking mobile device control agents across diverse configurations," 2024. [Online]. Available: https://arxiv.org/abs/2404.16660

[322] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. Bishop, W. Li, F. Campbell-Ajala, D. Toyama, R. Berry, D. Tyamagundlu, T. Lillicrap, and O. Riva, "Androidworld: A dynamic benchmarking environment for autonomous agents," 2024. [Online]. Available: https://arxiv.org/abs/2405.14573

[323] M. Xing, R. Zhang, H. Xue, Q. Chen, F. Yang, and Z. Xiao, "Understanding the weakness of large language model agents within a complex android environment," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6061–6072.

[324] S. Deng, W. Xu, H. Sun, W. Liu, T. Tan, J. Liu, A. Li, J. Luan, B. Wang, R. Yan *et al.*, "Mobile-bench: An evaluation benchmark for llm-based mobile agents," *arXiv preprint arXiv:2407.00993*, 2024.

[325] J. Lee, D. Hahm, J. S. Choi, W. B. Knox, and K. Lee, "Mobilesafetybench: Evaluating safety of autonomous agents in mobile device control," *arXiv preprint arXiv:2410.17520*, 2024.

[326] J. Chen, D. Yuen, B. Xie, Y. Yang, G. Chen, Z. Wu, L. Yixing, X. Zhou, W. Liu, S. Wang *et al.*, "Spa-bench: A comprehensive benchmark for smartphone agent evaluation," in *NeurIPS 2024 Workshop on Open-World Agents*, 2024.

[327] L. Zhang, S. Wang, X. Jia, Z. Zheng, Y. Yan, L. Gao, Y. Li, and M. Xu, "Llamatouch: A faithful and scalable testbed for mobile ui task automation," 2024. [Online]. Available: https://arxiv.org/abs/2404.16054

[328] L. Wang, Y. Deng, Y. Zha, G. Mao, Q. Wang, T. Min, W. Chen, and S. Chen, "Mobileagentbench: An efficient and user-friendly benchmark for mobile llm agents," 2024. [Online]. Available: https://arxiv.org/abs/2406.08184

[329] R. Bonatti, D. Zhao, F. Bonacci, D. Dupont, S. Abdali, Y. Li, Y. Lu, J. Wagle, K. Koishida, A. Bucker, L. Jang, and Z. Hui, "Windows agent arena: Evaluating multi-modal os agents at scale," 2024. [Online]. Available: https://arxiv.org/abs/2409.08264

[330] R. Cao, F. Lei, H. Wu, J. Chen, Y. Fu, H. Gao, X. Xiong, H. Zhang, Y. Mao, W. Hu, T. Xie, H. Xu, D. Zhang, S. Wang, R. Sun, P. Yin, C. Xiong, A. Ni, Q. Liu, V. Zhong, L. Chen, K. Yu, and T. Yu, "Spider2-v: How far are multimodal agents from automating data science and engineering workflows?" 2024. [Online]. Available: https://arxiv.org/abs/2407.10956

[331] Z. Wang, Y. Cui, L. Zhong, Z. Zhang, D. Yin, B. Y. Lin, and J. Shang, "Officebench: Benchmarking language agents across multiple applications for office automation," 2024. [Online]. Available: https://arxiv.org/abs/2407.19056

[332] L. Zheng, Z. Huang, Z. Xue, X. Wang, B. An, and S. Yan,

"Agentstudio: A toolkit for building general virtual agents," 2024. [Online]. Available: https://arxiv.org/abs/2403.17918

[333] D. Zimmermann and A. Koziolek, "Gui-based software testing: An automated approach using gpt-4 and selenium webdriver," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. IEEE, 2023, pp. 171–174.

[334] J. Yoon, R. Feldt, and S. Yoo, "Intent-driven mobile gui testing with autonomoufs large language model agents," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2024, pp. 129–139.

[335] Y. Hu, X. Wang, Y. Wang, Y. Zhang, S. Guo, C. Chen, X. Wang, and Y. Zhou, "Auitestagent: Automatic requirements oriented gui function testing," 2024. [Online]. Available: https://arxiv.org/abs/2407.09018

[336] Z. Liu, C. Li, C. Chen, J. Wang, B. Wu, Y. Wang, J. Hu, and Q. Wang, "Vision-driven automated mobile gui testing via multimodal large language model," 2024. [Online]. Available: https://arxiv.org/abs/2407.03037

[337] M. Taeb, A. Swearngin, E. Schoop, R. Cheng, Y. Jiang, and J. Nichols, "Axnav: Replaying accessibility tests from natural language," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–16.

[338] C. Cui, T. Li, J. Wang, C. Chen, D. Towey, and R. Huang, "Large language models for mobile gui text input generation: An empirical study," *arXiv preprint arXiv:2404.08948*, 2024.

[339] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, "Fill in the blank: Context-aware automated text input generation for mobile gui testing," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1355–1367.

[340] Y. Huang, J. Wang, Z. Liu, Y. Wang, S. Wang, C. Chen, Y. Hu, and Q. Wang, "Crashtranslator: Automatically reproducing mobile application crashes directly from stack trace," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[341] S. Feng and C. Chen, "Prompting is all you need: Automated android bug replay with large language models," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[342] L. Ding, J. Bheemanpally, and Y. Zhang, "Improving technical" how-to" query accuracy with automated search results verification and reranking," *arXiv preprint arXiv:2404.08860*, 2024.

[343] J. Gorniak, Y. Kim, D. Wei, and N. W. Kim, "Vizability: Enhancing chart accessibility with llm-based conversational interaction," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, 2024, pp. 1–19.

[344] Y. Ye, X. Cong, S. Tian, J. Cao, H. Wang, Y. Qin, Y. Lu, H. Yu, H. Wang, Y. Lin *et al.*, "Proagent: From robotic process automation to agentic process automation," *arXiv preprint arXiv:2311.10751*, 2023.

[345] Y. Guan, D. Wang, Z. Chu, S. Wang, F. Ni, R. Song, and C. Zhuang, "Intelligent agents with llm-based process automation," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 5018–5027.

[346] M. D. Vu, H. Wang, Z. Li, J. Chen, S. Zhao, Z. Xing, and C. Chen, "Gptvoicetasker: Llm-powered virtual assistant for smartphone," *arXiv preprint arXiv:2401.14268*, 2024.

[347] L. Pan, B. Wang, C. Yu, Y. Chen, X. Zhang, and Y. Shi, "Autotask: Executing arbitrary voice commands by exploring and learning from mobile gui," *arXiv preprint arXiv:2312.16062*, 2023.

[348] D. Gao, S. Hu, Z. Bai, Q. Lin, and M. Z. Shou, "Assisteditor: Multi-agent collaboration for gui workflow automation in video creation," in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, pp. 11 255–11 257.

[349] T. Huang, C. Yu, W. Shi, Z. Peng, D. Yang, W. Sun, and Y. Shi, "Promptrpa: Generating robotic process automation on smartphones from textual prompts," *arXiv preprint arXiv:2404.02475*, 2024.

[350] W. Gao, K. Du, Y. Luo, W. Shi, C. Yu, and Y. Shi, "Easyask: An in-app contextual tutorial search assistant for older adults with voice and touch inputs," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 8, no. 3, pp. 1–27, 2024.

[351] OpenAdapt AI, "OpenAdapt: Open Source Generative Process Automation," 2024, accessed: 2024-10-26. [Online]. Available: https://github.com/OpenAdaptAI/OpenAdapt

[352] AgentSea*f* AI. (2024) Introduction to agentsea platform. Accessed: 2024-10-26. [Online]. Available: https://www.agentsea.ai/

[353] O. Interpreter, "Open interpreter: A natural language interface for computers," GitHub repository, 2024, accessed: 2024-10-27. [Online]. Available: https://github.com/OpenInterpreter/open-interpreter

[354] MultiOn AI. (2024) Multion ai: Ai agents that act on your behalf. Accessed: 2024-10-26. [Online]. Available: https://www.multion.ai/

[355] HONOR, "Honor introduces magicos 9.0," 2024, accessed: 2024-11-16. [Online]. Available: https://www.fonearena.com/blog/438680/honor-magicos-9-0-features.html

[356] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, "Scaling instruction-finetuned language models," *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.

[357] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai *et al.*, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," *arXiv preprint arXiv:2406.12793*, 2024.

[358] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in games*, vol. 4, no. 1, pp. 1–43, 2012.

[359] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[360] Baidu Research, "ERNIE Bot: Baidu's Knowledge-Enhanced Large Language Model Built on Full AI Stack Technology," 2024, [Online; accessed 9-November-2024]. [Online]. Available: https://research.baidu.com/Blog/index-view?id=183

[361] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.

[362] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality," March 2023. [Online]. Available: https://lmsys.org/blog/2023-03-30-vicuna/

[363] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," 2023. [Online]. Available: https://arxiv.org/abs/2309.16609

[364] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu, "Palm 2 technical report," 2023. [Online]. Available: https://arxiv.org/abs/2305.10403

[365] B. Xiao, H. Wu, W. Xu, X. Dai, H. Hu, Y. Lu, M. Zeng, C. Liu, and L. Yuan, "Florence-2: Advancing a unified representation for a variety of vision tasks," 2023. [Online]. Available: https://arxiv.org/abs/2311.06242

[366] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and

K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[367] D. Reis, J. Kupec, J. Hong, and A. Daoudi, "Real-time flying object detection with yolov8," *arXiv preprint arXiv:2305.09972*, 2023.

[368] W. Li, W. Bishop, A. Li, C. Rawles, F. Campbell-Ajala, D. Tyamagundlu, and O. Riva, "On the effects of data scale on computer control agents," *arXiv preprint arXiv:2406.03679*, 2024.

[369] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao *et al.*, "Data-juicer: A one-stop data processing system for large language models," in *Companion of the 2024 International Conference on Management of Data*, 2024, pp. 120–134.

[370] B. Ding, C. Qin, R. Zhao, T. Luo, X. Li, G. Chen, W. Xia, J. Hu, L. A. Tuan, and S. Joty, "Data augmentation using llms: Data perspectives, learning paradigms and challenges," in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 1679–1705.

[371] Z. Tan, D. Li, S. Wang, A. Beigi, B. Jiang, A. Bhattacharjee, M. Karami, J. Li, L. Cheng, and H. Liu, "Large language models for data annotation: A survey," *arXiv preprint arXiv:2402.13446*, 2024.

[372] J. Andreas, J. Bufe, D. Burkett, C. Chen, J. Clausman, J. Crawford, K. Crim, J. DeLoach, L. Dorner, J. Eisner *et al.*, "Task-oriented dialogue as dataflow synthesis," *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 556–571, 2020.

[373] Z. Guo, S. Cheng, H. Wang, S. Liang, Y. Qin, P. Li, Z. Liu, M. Sun, and Y. Liu, "Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models," 2024.

[374] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, "Agentboard: An analytical evaluation board of multi-turn llm agents," *arXiv preprint arXiv:2401.13178*, 2024.

[375] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[376] H. Laurençon, L. Tronchon, M. Cord, and V. Sanh, "What matters when building vision-language models?" *arXiv preprint arXiv:2405.02246*, 2024.

[377] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "Glm: General language model pretraining with autoregressive blank infilling," *arXiv preprint arXiv:2103.10360*, 2021.

[378] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 012–10 022.

[379] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024. [Online]. Available: https://arxiv.org/abs/2308.12950

[380] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love *et al.*, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.

[381] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A survey on mixture of experts," *arXiv preprint arXiv:2407.06204*, 2024.

[382] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019.

[383] MosaicML, "Mosaicml: Mpt-7b," 2023, accessed: 2024-11-19. [Online]. Available: https://www.mosaicml.com/blog/mpt-7b

[384] X. Chen, X. Wang, L. Beyer, A. Kolesnikov, J. Wu, P. Voigtlaender, B. Mustafa, S. Goodman, I. Alabdulmohsin, P. Padlewski *et al.*, "Pali-3 vision language models: Smaller, faster, stronger," *arXiv preprint arXiv:2310.09199*, 2023.

[385] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming–the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[386] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[387] Y.-C. Hsiao, F. Zubach, G. Baechler, V. Carbune, J. Lin, M. Wang, S. Sunkara, Y. Zhu, and J. Chen, "Screenqa: Large-scale question-answer pairs over mobile app screenshots," 2024. [Online]. Available: https://arxiv.org/abs/2209.08199

[388] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, "Agentbench: Evaluating llms as agents," *arXiv preprint arXiv:2308.03688*, 2023.

[389] D. Zimmermann and A. Koziolek, "Automating gui-based software testing with gpt-3," in *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2023, pp. 62–65.

[390] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, 2024.

[391] K. Q. Lin, P. Zhang, J. Chen, S. Pramanick, D. Gao, A. J. Wang, R. Yan, and M. Z. Shou, "Univtg: Towards unified video-language temporal grounding," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 2794–2804.

[392] Z. Liao, L. Mo, C. Xu, M. Kang, J. Zhang, C. Xiao, Y. Tian, B. Li, and H. Sun, "Eia: Environmental injection attack on generalist web agents for privacy leakage," *arXiv preprint arXiv:2409.11295*, 2024.

[393] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, "The emerged security and privacy of llm agent: A survey with case studies," *arXiv preprint arXiv:2407.19354*, 2024.

[394] Y. Gan, Y. Yang, Z. Ma, P. He, R. Zeng, Y. Wang, Q. Li, C. Zhou, S. Li, T. Wang, Y. Gao, Y. Wu, and S. Ji, "Navigating the risks: A survey of security, privacy, and ethics threats in llm-based agents," 2024. [Online]. Available: https://arxiv.org/abs/2411.09523

[395] Y. Yang, X. Yang, S. Li, C. Lin, Z. Zhao, C. Shen, and T. Zhang, "Security matrix for multimodal agents on mobile devices: A systematic and proof of concept study," *arXiv preprint arXiv:2407.09295*, 2024.

[396] X. Zhang, H. Xu, Z. Ba, Z. Wang, Y. Hong, J. Liu, Z. Qin, and K. Ren, "Privacyasst: Safeguarding user privacy in tool-using large language model agents," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[397] J. Xu, Z. Li, W. Chen, Q. Wang, X. Gao, Q. Cai, and Z. Ling, "On-device language models: A comprehensive review," *arXiv preprint arXiv:2409.00088*, 2024.

[398] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *arXiv preprint arXiv:2407.18921*, 2024.

[399] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.

[400] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi *et al.*, "Mobilellm: Optimizing sub-billion parameter language models for on-device use cases," *arXiv preprint arXiv:2402.14905*, 2024.

[401] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li *et al.*, "A survey on efficient inference for large language models," *arXiv preprint arXiv:2404.14294*, 2024.

[402] W. Kuang, B. Qian, Z. Li, D. Chen, D. Gao, X. Pan, Y. Xie, Y. Li, B. Ding, and J. Zhou, "Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 5260–5271.

[403] P. Mai, R. Yan, Z. Huang, Y. Yang, and Y. Pang, "Split-and-denoise: Protect large language model inference with local differential privacy," *arXiv preprint arXiv:2310.09130*, 2023.

[404] L. de Castro, A. Polychroniadou, and D. Escudero, "Privacy-preserving large language model inference via gpu-accelerated fully homomorphic encryption," in *Neurips Safe Generative AI Workshop 2024*.

[405] J. Wolff, W. Lehr, and C. S. Yoo, "Lessons from gdpr for ai policymaking," *Virginia Journal of Law & Technology*, vol. 27, no. 4, p. 2, 2024.

[406] Z. Zhang, M. Jia, H.-P. Lee, B. Yao, S. Das, A. Lerner, D. Wang, and T. Li, ""it's a fair game", or is it? examining how users navigate disclosure risks and benefits when using llm-based conversational agents," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–26.

[407] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari, "Llm inference serving: Survey of recent advances and opportunities," *arXiv preprint arXiv:2407.12391*, 2024.

[408] M. Xu, W. Yin, D. Cai, R. Yi, D. Xu, Q. Wang, B. Wu, Y. Zhao, C. Yang, S. Wang *et al.*, "A survey of resource-efficient llm and multimodal foundation models," *arXiv preprint arXiv:2401.08092*, 2024.

[409] D. Chen, Y. Liu, M. Zhou, Y. Zhao, H. Wang, S. Wang, X. Chen, T. F. Bissyandé, J. Klein, and L. Li, "Llm for mobile: An initial roadmap," *arXiv preprint arXiv:2407.06573*, 2024.

[410] Z. Wan, X. Wang, C. Liu, S. Alam, Y. Zheng, J. Liu, Z. Qu, S. Yan, Y. Zhu, Q. Zhang *et al.*, "Efficient large language models: A survey," *arXiv preprint arXiv:2312.03863*, 2023.

[411] X. Xu, M. Li, C. Tao, T. Shen, R. Cheng, J. Li, C. Xu, D. Tao, and T. Zhou, "A survey on knowledge distillation of large language models," *arXiv preprint arXiv:2402.13116*, 2024.

[412] C. Kachris, "A survey on hardware accelerators for large language models," *arXiv preprint arXiv:2401.09890*, 2024.

[413] W. Lee, J. Lee, J. Seo, and J. Sim, "{InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 155–172.

[414] Z. Wang, J. Wohlwend, and T. Lei, "Structured pruning of large language models," *arXiv preprint arXiv:1910.04732*, 2019.

[415] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, "Fast distributed inference serving for large language models," *arXiv preprint arXiv:2305.05920*, 2023.

[416] U. Anwar, A. Saparov, J. Rando, D. Paleka, M. Turpin, P. Hase, E. S. Lubana, E. Jenner, S. Casper, O. Sourbut *et al.*, "Foundational challenges in assuring alignment and safety of large language models," *arXiv preprint arXiv:2404.09932*, 2024.

[417] L. Zhong and Z. Wang, "A study on robustness and reliability of large language model code generation," *arXiv preprint arXiv:2308.10335*, 2023.

[418] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang *et al.*, "R-judge: Benchmarking safety risk awareness for llm agents," *arXiv preprint arXiv:2401.10019*, 2024.

[419] L. Zhang, Q. Jin, H. Huang, D. Zhang, and F. Wei, "Respond in my language: Mitigating language inconsistency in response generation based on large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 4177–4192.

[420] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's song in the ai ocean: A survey on hallucination in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2309.01219

[421] C. Xu, M. Kang, J. Zhang, Z. Liao, L. Mo, M. Yuan, H. Sun, and B. Li, "Advweb: Controllable black-box attacks on vlm-powered web agents," *arXiv preprint arXiv:2410.17401*, 2024.

[422] L. Pan, M. S. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically correcting large language models: Surveying the landscape of diverse self-correction strategies," *ArXiv*, vol. abs/2308.03188, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:260682695

[423] X. Huang, W. Ruan, W. Huang, G. Jin, Y. Dong, C. Wu, S. Bensalem, R. Mu, Y. Qi, X. Zhao, K. Cai, Y. Zhang, S. Wu, P. Xu, D. Wu, A. Freitas, and M. A. Mustafa, "A survey of safety and trustworthiness of large language models through the lens of verification and validation," *Artif. Intell. Rev.*, vol. 57, p. 175, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:258823083

[424] S. Jha, S. K. Jha, P. Lincoln, N. D. Bastian, A. Velasquez, and S. Neema, "Dehallucinating large language models using formal methods guided iterative prompting," *2023 IEEE International Conference on Assured Autonomy (ICAA)*, pp. 149–152, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:260810131

[425] Q. Zhang, T. Zhang, J. Zhai, C. Fang, B.-C. Yu, W. Sun, and Z. Chen, "A critical review of large language model on software engineering: An example from chatgpt and automated program repair," *ArXiv*, vol. abs/2310.08879, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:264127977

[426] R. Koo and S. Toueg, "Checkpointing and rollback-recovery for distributed systems," *IEEE Transactions on Software Engineering*, vol. SE-13, pp. 23–31, 1986. [Online]. Available: https://api.semanticscholar.org/CorpusID:206777989

[427] Y. Luo, Q. Zhang, Q. Shen, H. Liu, and Z. Wu, "Android multi-level system permission management approach," *ArXiv*, vol. abs/1712.02217, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:20909985

[428] H. Hao, V. Singh, and W. Du, "On the effectiveness of api-level access control using bytecode rewriting in android," in *Proceedings of the 8th ACM SIGSAC symposium on Information, computer and communications security*, 2013, pp. 25–36.

[429] A. P. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner, "Android permissions demystified," in *Proceedings of the 18th ACM conference on Computer and communications security*, 2011, pp. 627–638.

[430] M. Lutaaya, "Rethinking app permissions on ios," in *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, pp. 1–6.

[431] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang *et al.*, "Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning," *arXiv preprint arXiv:2406.09187*, 2024.

[432] S. Berkovits, J. D. Guttman, and V. Swarup, "Authentication for mobile agents," in *Mobile Agents and Security*, 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:13987376

[433] J. Gao, S. A. Gebreegziabher, K. T. W. Choo, T. J.-J. Li, S. T. Perrault, and T. W. Malone, "A taxonomy for human-llm interaction modes: An initial exploration," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–11.

[434] J. M. Bradshaw, P. J. Feltovich, and M. Johnson, "Human–agent interaction," in *The handbook of human-machine interaction*. CRC Press, 2017, pp. 283–300.

[435] X. Feng, Z.-Y. Chen, Y. Qin, Y. Lin, X. Chen, Z. Liu, and J.-R. Wen, "Large language model-based human-agent collaboration for complex task solving," *arXiv preprint arXiv:2402.12914*, 2024.

[436] A. Amayuelas, L. Pan, W. Chen, and W. Wang, "Knowledge of knowledge: Exploring known-unknowns uncertainty with large language models," *arXiv preprint arXiv:2305.13712*, 2023.

[437] C. Y. Kim, C. P. Lee, and B. Mutlu, "Understanding large-language model (llm)-powered human-robot interaction," in *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, 2024, pp. 371–380.

[438] Y. Lu, Y. Yang, Q. Zhao, C. Zhang, and T. J.-J. Li, "Ai assistance for ux: A literature review through human-centered ai," *arXiv preprint arXiv:2402.06089*, 2024.

[439] J. Wester, T. Schrills, H. Pohl, and N. van Berkel, ""as an ai language model, i cannot": Investigating llm denials of user requests," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–14.

[440] J. Wang, W. Ma, P. Sun, M. Zhang, and J.-Y. Nie, "Understanding user experience in large language model interactions," *arXiv preprint arXiv:2401.08329*, 2024.

[441] E. Cambria, L. Malandri, F. Mercorio, N. Nobani, and A. Seveso, "XAI meets llms: A survey of the relation between explainable ai and large language models," 2024. [Online]. Available: https://arxiv.org/abs/2407.15248

[442] X. Wu, H. Zhao, Y. Zhu, Y. Shi, F. Yang, T. Liu, X. Zhai, W. Yao, J. Li, M. Du *et al.*, "Usable xai: 10 strategies towards exploiting explainability in the llm era," *arXiv preprint arXiv:2403.08946*, 2024.

[443] H. Cai, Y. Li, W. Wang, F. Zhu, X. Shen, W. Li, and T.-S. Chua, "Large language models empowered personalized web agents," *arXiv preprint arXiv:2410.17236*, 2024.

[444] H. Li, C. Yang, A. Zhang, Y. Deng, X. Wang, and T.-S. Chua, "Hello again! llm-powered personalized agent for long-term dialogue," *arXiv preprint arXiv:2406.05925*, 2024.

[445] H. Li, H. Jiang, T. Zhang, Z. Yu, A. Yin, H. Cheng, S. Fu, Y. Zhang, and W. He, "Traineragent: Customizable and efficient model training through llm-powered multi-agent system," *arXiv preprint arXiv:2311.06622*, 2023.

[446] Z. Tan and M. Jiang, "User modeling in the era of large language models: Current research and future directions," *arXiv preprint arXiv:2312.11518*, 2023.

[447] G. Gao, A. Taymanov, E. Salinas, P. Mineiro, and D. Misra, "Aligning llm agents by learning latent preference from user edits," *arXiv preprint arXiv:2404.15269*, 2024.

[448] T. Kaufmann, P. Weng, V. Bengs, and E. Hüllermeier, "A survey of reinforcement learning from human feedback," *arXiv preprint arXiv:2312.14925*, 2023.

[449] S. Kim, H. Kang, S. Choi, D. Kim, M. Yang, and C. Park, "Large language models meet collaborative filtering: An efficient all-round llm-based recommender system," in *Proceedings of the 30th ACM*

SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 1395–1406.
[450] W. Talukdar and A. Biswas, "Improving large language model (llm) fidelity through context-aware grounding: A systematic approach to reliability and veracity," *arXiv preprint arXiv:2408.04023*, 2024.
[451] X. Xiao and Y. Tao, "Personalized privacy preservation," in *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, 2006, pp. 229–240.
[452] I. H. Sarker, "Llm potentiality and awareness: a position paper from the perspective of trustworthy and responsible ai modeling," *Discover Artificial Intelligence*, vol. 4, no. 1, p. 40, 2024.
[453] A. Biswas and W. Talukdar, "Guardrails for trust, safety, and ethical development and deployment of large language models (llm)," *Journal of Science & Technology*, vol. 4, no. 6, pp. 55–82, 2023.
[454] Y. Li, M. Du, R. Song, X. Wang, and Y. Wang, "A survey on fairness in large language models," *arXiv preprint arXiv:2308.10149*, 2023.
[455] E. Ferrara, "Should chatgpt be biased? challenges and risks of bias in large language models," *arXiv preprint arXiv:2304.03738*, 2023.
[456] Y. Yu, Y. Zhuang, J. Zhang, Y. Meng, A. J. Ratner, R. Krishna, J. Shen, and C. Zhang, "Large language model as attributed training data generator: A tale of diversity and bias," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
[457] A. Piñeiro-Martín, C. García-Mateo, L. Docío-Fernández, and M. D. C. Lopez-Perez, "Ethical challenges in the development of virtual assistants powered by large language models," *Electronics*, vol. 12, no. 14, p. 3170, 2023.
[458] B. Zheng, Z. Liu, S. Salisbury, Z. Du, X. Huang, Q. Zheng, L. Davis, M. Lin, X. Jin, H. Sun *et al.*, "Agentmonitor: Towards a generalist guardrail for web agent."
[459] C.-M. Chan, J. Yu, W. Chen, C. Jiang, X. Liu, W. Shi, Z. Liu, W. Xue, and Y. Guo, "Agentmonitor: A plug-and-play framework for predictive and secure multi-agent systems," *arXiv preprint arXiv:2408.14972*, 2024.
[460] L. Lin, L. Wang, J. Guo, and K.-F. Wong, "Investigating bias in llm-based bias detection: Disparities between llms and human perception," *arXiv preprint arXiv:2403.14896*, 2024.
[461] Y. Zhang, T. Yu, and D. Yang, "Attacking vision-language computer agents via pop-ups," 2024. [Online]. Available: https://arxiv.org/abs/2411.02391
[462] R. Grosse, J. Bae, C. Anil, N. Elhage, A. Tamkin, A. Tajdini, B. Steiner, D. Li, E. Durmus, E. Perez *et al.*, "Studying large language model generalization with influence functions," *arXiv preprint arXiv:2308.03296*, 2023.
[463] X. Zhang, J. Li, W. Chu, J. Hai, R. Xu, Y. Yang, S. Guan, J. Xu, and P. Cui, "On the out-of-distribution generalization of multimodal large language models," *arXiv preprint arXiv:2402.06599*, 2024.
[464] Y. Song, W. Xiong, X. Zhao, D. Zhu, W. Wu, K. Wang, C. Li, W. Peng, and S. Li, "Agentbank: Towards generalized llm agents via fine-tuning on 50000+ interaction trajectories," *arXiv preprint arXiv:2410.07706*, 2024.
[465] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big data*, vol. 3, pp. 1–40, 2016.
[466] Y. Chen, R. Zhong, S. Zha, G. Karypis, and H. He, "Meta-learning via language model in-context tuning," *arXiv preprint arXiv:2110.07814*, 2021.
[467] Y. Zhu, S. Qiao, Y. Ou, S. Deng, N. Zhang, S. Lyu, Y. Shen, L. Liang, J. Gu, and H. Chen, "Knowagent: Knowledge-augmented planning for llm-based agents," *arXiv preprint arXiv:2403.03101*, 2024.
[468] Y. Guan, D. Wang, Y. Wang, H. Wang, R. Sun, C. Zhuang, J. Gu, and Z. Chu, "Explainable behavior cloning: Teaching large language model agents through learning by demonstration," *arXiv preprint arXiv:2410.22916*, 2024.
[469] C.-Y. Hsieh, S.-A. Chen, C.-L. Li, Y. Fujii, A. Ratner, C.-Y. Lee, R. Krishna, and T. Pfister, "Tool documentation enables zero-shot tool-usage with large language models," *arXiv preprint arXiv:2308.00675*, 2023.
[470] T. Kagaya, T. J. Yuan, Y. Lou, J. Karlekar, S. Pranata, A. Kinose, K. Oguri, F. Wick, and Y. You, "Rap: Retrieval-augmented planning with contextual memory for multimodal llm agents," *arXiv preprint arXiv:2402.03610*, 2024.